2.3.5 Материалы для изучения по научной специальности Математическое и программное обеспечение вычислительных систем, комплексов и компьютерных сетей Гусев Виталий Евгеньевич 2.3.5 2026

Содержание

1 Раздел 2. Дискретная математика и математическая логика. 3

1.1 Алгебра логики. Булевы функции, канонические формы задания булевых функций. Понятие полной системы. Критерий полноты Поста. Минимизация булевых функций в классах нормальных форм 3

1.1.1 Алгебра логики: высказывания, операции, формулы и равносильности 3

1.1.2 Булевы функции 9

1.1.3 Канонические формы задания булевых функций 14

1.1.4 Понятие полной системы. Критерий полноты Поста 20

1.1.5 Минимизация булевых функций в классах нормальных форм . . 25

1.1.6 Краткое резюме 31

1.2 Исчисление предикатов первого порядка. Понятие интерпретации. Выполнимость и общезначимость формулы первого порядка. Понятие модели. Теорема о полноте исчисления предикатов первого порядка . . . 32

1.2.1 Исчисление предикатов первого порядка 32

1.2.2 Понятие интерпретации 38

1.2.3 Выполнимость и общезначимость формулы первого порядка . . 42

1.2.4 Понятие модели 46

1.2.5 Теорема о полноте исчисления предикатов первого порядка . . . 50

1.2.6 Краткое резюме 54

1.3 Элементы универсальной алгебры: отношения, функции, фактормножества, упорядоченные множества и решётки 55

1.3.1 Отношения, соответствия и функции 55

1.3.2 Отношение эквивалентности, разбиения и фактормножество . . 58

1.3.3 Отношения частичного порядка 61

1.3.4 Решётки: теоретико-множественное и алгебраическое определения 68

1.3.5 Свойства решёток 70

1.3.6 Полные решётки и системы замыканий 76

1.3.7 Булевы решётки 82
1.3.8 Краткое резюме 86
1.4 Формальные языки и способы их описания. Классификация формальных грамматик. Их использование в лексическом и синтаксическом анализе 87
1.4.1 Формальные языки: алфавит, цепочка, язык 87
1.4.2 Порождающие грамматики и способы их задания 90
1.4.3 Классификация грамматик и языков по Хомскому 96
1.4.4 Разбор цепочек и неоднозначность грамматик 100
1.4.5 Распознаватели и их связь с типами языков 104
1.4.6 Использование формальных грамматик в лексическом анализе . 108
1.4.7 Использование формальных грамматик в синтаксическом анализе 116
1.4.8 Краткое резюме 137
1.5 Основы комбинаторного анализа. Метод производящих функций, метод включений и исключений. Примеры применения 138
1.5.1 Основные понятия комбинаторики: размещения, перестановки, сочетания 138
1.5.2 Разбиения множеств и чисел, числа Стирлинга 145
1.5.3 Метод производящих функций 150
1.5.4 Метод включений и исключений 159
1.5.5 Краткое резюме 163
1.6 Коды с исправлением ошибок. Алфавитное кодирование. Методы сжатия информации 163
1.6.1 Алфавитное кодирование и взаимная однозначность 163
1.6.2 Префиксные коды и их построение 169
1.6.3 Оптимальное кодирование (сжатие информации) 174
1.6.4 Помехоустойчивое кодирование 178
1.6.5 Краткое резюме 185

1 Раздел 2. Дискретная математика и математическая логика.

1.1 Алгебра логики. Булевы функции, канонические формы задания булевых функций. Понятие полной системы. Критерий полноты Поста. Минимизация булевых функций в классах нормальных форм

Основные источники: Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.1–2.5, с. 96–128; Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. I, §1–6, с. 16–65; гл. II, §8–11, с. 90–111.

1.1.1 Алгебра логики: высказывания, операции, формулы и равносильности

Высказывания и их истинностные значения. Алгебра логики изучает высказывания и способы построения из них новых высказываний. Под *высказыванием* понимается повествовательное предложение, о котором можно определённо сказать, истинно оно или ложно. Высказывание не может быть одновременно истинным и ложным: это выражает *закон противоречия*. Кроме того, принимается *закон исключённого третьего*: высказывание либо истинно, либо ложно, третьего не дано. *(Источник: Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. I, §1, с. 16–17; Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.1, с. 96.)*

Не всякое предложение является высказыванием. Вопросы, побуждения, команды, предложения с неопределёнными переменными («$x + 1 = 2$») сами по себе не имеют определённого истинностного значения и высказываниями не являются. Например, предложения

«Москва — столица России», «7<4», «Волга впадает в Каспийское море»

являются высказываниями, а предложения «Решите уравнение $x+1 = 2$» и «Который час?» — нет. *(Источник: Игошин, гл. I, §1, с. 16–17; Макоха и др., гл. 2, §2.1, с. 96.)*

Истинность будем обозначать символом 1, ложность — символом 0. Если P — высказывание, то его логическое значение обозначается $\lambda(P)$:

$$\lambda(P) = \begin{cases} 1, & \text{если } P \text{ истинно,} \\ 0, & \text{если } P \text{ ложно.} \end{cases}$$

Функция λ называется *функцией истинности*. Например, для высказываний

A_1 : «Москва — столица России», A_2 : «Саратов находится на берегу Невы»

имеем $\lambda(A_1) = 1$, $\lambda(A_2) = 0$. *(Источник: Игошин, гл. I, §1, с. 17; Макоха и др., гл. 2, §2.1, с. 96–97.)*

Логические операции над высказываниями. Из простых высказываний с помощью логических связок строятся составные высказывания. Основными операциями алгебры логики являются отрицание, конъюнкция, дизъюнкция, импликация и эквивалентность. Каждая операция определяется таблицей истинности, то есть правилом, по которому значения истинности исходных высказываний определяют значение истинности составного высказывания. *(Источник: Макоха и др., гл. 2, §2.1, с. 97–101; Игошин, гл. I, §1, с. 17–22.)*

Отрицание. Отрицанием высказывания P называется новое высказывание, обозначаемое $\neg P$ (читается «не P» или «неверно, что P»), которое истинно тогда и только тогда, когда P ложно, и ложно тогда и только тогда, когда P истинно. Таблица истинности:

$\lambda(P)$	$\lambda(\neg P)$
0	1
1	0

Иными словами, $\lambda(\neg P) = \neg\lambda(P)$. *(Источник: Макоха и др., гл. 2, §2.1, определение 2.1 и табл. 2.1, с. 97; Игошин, гл. I, §1, с. 17–18.)*

Например, если A_6 — высказывание «Волга впадает в Каспийское море», то $\neg A_6$ — высказывание «Волга не впадает в Каспийское море». Так как $\lambda(A_6) = 1$, то $\lambda(\neg A_6) = 0$. *(Источник: Макоха и др., гл. 2, §2.1, пример 1.2, с. 98.)*

Конъюнкция. Конъюнкцией высказываний P и Q называется новое высказывание, обозначаемое $P \wedge Q$ (читается «P и Q»), которое истинно в том и только в том случае, когда истинны оба высказывания P и Q, и ложно во всех остальных случаях. Таблица истинности:

$\lambda(P)$	$\lambda(Q)$	$\lambda(P \wedge Q)$
0	0	0
0	1	0
1	0	0
1	1	1

Имеем $\lambda(P \wedge Q) = \lambda(P) \wedge \lambda(Q)$. *(Источник: Макоха и др., гл. 2, §2.1, определение 2.2 и табл. 2.1, с. 97; Игошин, гл. I, §1, с. 18–19.)*

Например, высказывание «Саратов находится на берегу Невы, и все люди смертны» является конъюнкцией $A_2 \wedge A_3$. Поскольку $\lambda(A_2) = 0$, а $\lambda(A_3) = 1$, получаем

$$\lambda(A_2 \wedge A_3) = 0 \wedge 1 = 0.$$

(Источник: Макоха и др., гл. 2, §2.1, пример 1.4, с. 99.)

Дизъюнкция. Дизъюнкцией высказываний P и Q называется новое высказывание, обозначаемое $P \vee Q$ (читается «P или Q»), которое истинно тогда и только тогда, когда хотя бы одно из высказываний P, Q истинно, и ложно тогда и только тогда, когда оба высказывания ложны. Таблица истинности:

$\lambda(P)$	$\lambda(Q)$	$\lambda(P \vee Q)$
0	0	0
0	1	1
1	0	1
1	1	1

Имеем $\lambda(P \vee Q) = \lambda(P) \vee \lambda(Q)$. *(Источник: Макоха и др., гл. 2, §2.1, определение 2.3 и табл. 2.1, с. 97; Игошин, гл. I, §1, с. 19–20.)*

Союз «или» здесь употребляется в неисключающем смысле: допускается, что истинны оба высказывания. Например, высказывание «Все люди смертны, или $7 < 4$» истинно, так как первый член дизъюнкции истинен. *(Источник: Макоха и др., гл. 2, §2.1, пример 1.6, с. 99; Игошин, гл. I, §1, с. 19–20.)*

Импликация. Импликацией высказываний P и Q называется новое высказывание, обозначаемое $P \to Q$ (читается «если P, то Q»), которое ложно в том и только в том случае, когда P истинно, а Q ложно, и истинно во всех остальных случаях. В импликации $P \to Q$ высказывание P называется *посылкой* (антецедентом), а Q — *следствием* (консеквентом). Таблица истинности:

$\lambda(P)$	$\lambda(Q)$	$\lambda(P \to Q)$
0	0	1
0	1	1
1	0	0
1	1	1

Имеем $\lambda(P \to Q) = \lambda(P) \to \lambda(Q)$. *(Источник: Макоха и др., гл. 2, §2.1, определение 2.4 и табл. 2.1, с. 97; Игошин, гл. I, §1, с. 20–21.)*

Особенность импликации состоит в том, что она истинна при ложной посылке. Это соответствует логическому смыслу условного высказывания: если условие не выполнено, то обещание «если P, то Q» не нарушено. Например, высказывание «Если Саратов находится на берегу Невы, то А.С. Пушкин — великий русский математик» истинно, так как посылка и следствие ложны:

$$\lambda(A_2 \to A_7) = 0 \to 0 = 1.$$

(Источник: Макоха и др., гл. 2, §2.1, пример 1.8, с. 101; Игошин, гл. I, §1, с. 20–21.)

Эквивалентность. Эквивалентностью высказываний P и Q называется новое высказывание, обозначаемое $P \leftrightarrow Q$ (читается «P тогда и только тогда, когда Q»), которое истинно в том и только в том случае, когда оба высказывания одновременно истинны или одновременно ложны. Таблица истинности:

$\lambda(P)$	$\lambda(Q)$	$\lambda(P \leftrightarrow Q)$
0	0	1
0	1	0
1	0	0
1	1	1

Имеем $\lambda(P \leftrightarrow Q) = \lambda(P) \leftrightarrow \lambda(Q)$. *(Источник: Макоха и др., гл. 2, §2.1, определение 2.5 и табл. 2.1, с. 97; Игошин, гл. I, §1, с. 21–22.)*

Производные операции. Кроме пяти основных операций, в алгебре логики используются производные операции:

- *сложение по модулю* 2 $P \oplus Q$, которое истинно тогда и только тогда, когда истинно ровно одно из высказываний P, Q;
- *штрих Шеффера* $P \mid Q = \neg(P \wedge Q)$;

- *стрелка Пирса* $P \downarrow Q = \neg(P \vee Q)$;
- *исключающее «или»* (альтернатива), совпадающее с $P \oplus Q$.

(Источник: Макоха и др., гл. 2, §2.1, с. 98–99; Игошин, гл. I, §1, с. 21–22.)

Сводная таблица истинности основных и некоторых производных операций:

x	y	$\neg x$	$x \wedge y$	$x \vee y$	$x \to y$	$x \leftrightarrow y$	$x \oplus y$	$x \mid y$
0	0	1	0	0	1	1	0	1
0	1	1	0	1	1	0	1	1
1	0	0	0	1	0	0	1	1
1	1	0	1	1	1	1	0	0

(Источник: таблицы операций: Макоха и др., гл. 2, §2.1, табл. 2.1–2.2, с. 97–98; Игошин, гл. I, §1, с. 17–22.)

Формулы алгебры логики. Для построения сложных высказываний используется язык формул. Алфавит алгебры логики содержит:

1) пропозициональные переменные $P, Q, R, \ldots, X, Y, Z, \ldots, X_1, X_2, \ldots$;
2) символы логических операций $\neg, \wedge, \vee, \to, \leftrightarrow$;
3) технические знаки: скобки (и).

(Источник: Игошин, гл. I, §2, с. 24; Макоха и др., гл. 2, §2.2, с. 101.)

Формула алгебры логики определяется индуктивно:

1) каждая пропозициональная переменная есть формула;
2) если F_1 и F_2 — формулы, то $\neg F_1$, $(F_1 \wedge F_2)$, $(F_1 \vee F_2)$, $(F_1 \to F_2)$, $(F_1 \leftrightarrow F_2)$ также являются формулами;
3) никаких других формул, кроме получающихся по пунктам 1 и 2, нет.

(Источник: Игошин, гл. I, §2, определение 2.1, с. 24; Макоха и др., гл. 2, §2.2, с. 101.)

Внешние скобки обычно опускают. При записи формул принимается соглашение о старшинстве операций: сначала выполняется отрицание, затем конъюнкция, затем дизъюнкция, затем импликация, затем эквивалентность. Например, формула

$$\neg X \wedge Y \to Z \vee X$$

понимается как

$$(((\neg X) \wedge Y) \to (Z \vee X)).$$

(Источник: Игошин, гл. I, §2, с. 26–27; Макоха и др., гл. 2, §2.2, с. 101.)

Интерпретация и таблица истинности. Если в формулу $F(X_1, \ldots, X_n)$ вместо пропозициональных переменных подставить конкретные высказывания $A_1, \ldots, A_n$, получится новое высказывание $F(A_1, \ldots, A_n)$, называемое *конкретизацией* формулы. Процесс подстановки конкретных высказываний называется *интерпретацией* формулы. Обратный переход от высказывания естественного языка к формуле называется *формализацией*. *(Источник: Игошин, гл. I, §2, с. 25–33.)*

Для вычисления логического значения формулы составляют таблицу истинности. В первых n столбцах перечисляют все 2^n наборов значений переменных, в промежуточных столбцах — значения подформул, в последнем — значение всей формулы. Справедлива теорема:

$$\lambda(F(A_1, \ldots, A_n)) = F(\lambda(A_1), \ldots, \lambda(A_n)).$$

Иными словами, логическое значение составного высказывания равно значению формулы на соответствующем наборе значений истинности исходных высказываний. *(Источник: Игошин, гл. I, §2, теорема 2.2, с. 26; Макоха и др., гл. 2, §2.2, с. 101–102.)*

Пример. Для формулы

$$F = (\neg X \vee Y) \to (X \wedge \neg Y)$$

таблица истинности имеет вид:

X	Y	$\neg X$	$\neg X \vee Y$	$\neg Y$	$X \wedge \neg Y$	F
0	0	1	1	1	0	0
0	1	1	1	0	0	0
1	0	0	0	1	1	1
1	1	0	1	0	0	0

(Источник: пример составлен по образцу примера 2.1: Макоха и др., гл. 2, §2.1, табл. 2.3, с. 98–99.)

Классификация формул. Формула называется:

- *тождественно истинной* (или *тавтологией*), если она принимает значение 1 на всех наборах значений переменных;
- *тождественно ложной* (или *противоречием*), если она принимает значение 0 на всех наборах;
- *выполнимой*, если она принимает значение 1 хотя бы на одном наборе;
- *опровержимой*, если она принимает значение 0 хотя бы на одном наборе.

(Источник: Игошин, гл. I, §2, с. 31–32; Макоха и др., гл. 2, §2.2, с. 101–104.)

Например, формула $X \vee \neg X$ является тавтологией, формула $X \wedge \neg X$ — тождественно ложной, а формула $X \wedge Y$ — выполнимой и опровержимой. *(Источник: Игошин, гл. I, §2, с. 31–32; Макоха и др., гл. 2, §2.1, с. 99–101.)*

Равносильность формул. Две формулы F и H называются *равносильными* (или эквивалентными), если при любых значениях входящих в них пропозициональных переменных логические значения получающихся высказываний совпадают. Обозначение: $F \equiv H$. *(Источник: Игошин, гл. I, §4, определение 4.1, с. 40; Макоха и др., гл. 2, §2.1, с. 99–101.)*

Справедлив *признак равносильности*: две формулы F и H равносильны тогда и только тогда, когда формула $F \leftrightarrow H$ является тавтологией:

$$F \equiv H \quad \Longleftrightarrow \quad \models (F \leftrightarrow H).$$

(Источник: Игошин, гл. I, §4, теорема 4.2, с. 40–41.)

Отношение равносильности рефлексивно, симметрично и транзитивно, то есть является отношением эквивалентности. Оно разбивает множество всех формул на классы равносильных формул. *(Источник: Игошин, гл. I, §4, следствие 4.3, с. 41.)*

Основные равносильности. Приведём важнейшие законы алгебры логики:

1) $x \wedge x \equiv x, \qquad x \vee x \equiv x$ (идемпотентность);

2) $x \wedge y \equiv y \wedge x, \qquad x \vee y \equiv y \vee x$ (коммутативность);

3) $x \wedge (y \wedge z) \equiv (x \wedge y) \wedge z, \qquad x \vee (y \vee z) \equiv (x \vee y) \vee z$ (ассоциативность);

4) $x \wedge (y \vee z) \equiv (x \wedge y) \vee (x \wedge z), \qquad x \vee (y \wedge z) \equiv (x \vee y) \wedge (x \vee z)$ (дистрибутивность);

5) $x \vee (x \wedge y) \equiv x, \qquad x \wedge (x \vee y) \equiv x$ (поглощение);

6) $\neg(x \wedge y) \equiv \neg x \vee \neg y, \qquad \neg(x \vee y) \equiv \neg x \wedge \neg y$ (законы де Моргана);

7) $x \vee \neg x \equiv 1, \qquad x \wedge \neg x \equiv 0;$

8) $\neg\neg x \equiv x$ (двойное отрицание);

9) $x \wedge 1 \equiv x, \quad x \vee 0 \equiv x, \quad x \wedge 0 \equiv 0, \quad x \vee 1 \equiv 1;$

10) $x \to y \equiv \neg x \vee y;$

11) $x \leftrightarrow y \equiv (x \to y) \wedge (y \to x);$

12) $x \to y \equiv \neg y \to \neg x$ (контрапозиция).

(Источник: Макоха и др., гл. 2, §2.1, с. 99–101; Игошин, гл. I, §4, теорема 4.4, с. 42–44.)

Лемма о замене. Если $F \equiv G$ и в формуле H заменить некоторую подформулу F на G, то полученная формула H' будет равносильна H. Это позволяет проводить *равносильные преобразования* формул: заменять подформулы на равносильные им формулы, не меняя значения всей формулы. *(Источник: Игошин, гл. I, §4, с. 43–44; Макоха и др., гл. 2, §2.2, с. 105–106.)*

Пример равносильного преобразования. Упростим формулу

$$F = \neg(X_1 \to \neg X_2) \wedge \neg(X_2 \to \neg X_1).$$

Используя $P \to Q \equiv \neg P \vee Q$ и закон де Моргана, получаем:

$$\neg(X_1 \to \neg X_2) \equiv \neg(\neg X_1 \vee \neg X_2) \equiv X_1 \wedge X_2.$$

Аналогично

$$\neg(X_2 \to \neg X_1) \equiv X_1 \wedge X_2.$$

Следовательно,

$$F \equiv (X_1 \wedge X_2) \wedge (X_1 \wedge X_2) \equiv X_1 \wedge X_2.$$

(Источник: Игошин, гл. I, §4, пример 4.6, с. 44.)

Принцип двойственности. Функция $f^*(x_1, \ldots, x_n) = \neg f(\neg x_1, \ldots, \neg x_n)$ называется *двойственной* к функции f. Если $f^* = f$, функция называется *самодвойственной*. Для формул principle двойственности формулируется так: двойственная формула получается из исходной заменой $\wedge$ на $\vee$, $\vee$ на $\wedge$, 0 на 1, 1 на 0 с сохранением структуры. Если $F \equiv H$, то $F^* \equiv H^*$. В частности, если формула является тавтологией, то двойственная ей формула также является тавтологией. *(Источник: Макоха и др., гл. 2, §2.2, с. 106–107; Игошин, гл. I, §4, с. 44–45.)*

Связь формул с булевыми функциями. Каждая формула алгебры логики от переменных $X_1, \ldots, X_n$ задаёт некоторую булеву функцию $f : \{0,1\}^n \to \{0,1\}$: значение $f(\alpha_1, \ldots, \alpha_n)$ равно значению формулы при $X_1 = \alpha_1, \ldots, X_n = \alpha_n$. Разные формулы могут задавать одну и ту же булеву функцию; это возможно тогда и только тогда, когда формулы равносильны. Таким образом, равносильность формул есть не что иное, как совпадение соответствующих им булевых функций. *(Источник: Игошин, гл. II, §10, с. 104–106; Макоха и др., гл. 2, §2.2–2.3, с. 101–110.)*

Итог пункта. В пункте 1.1.1 введены основные понятия алгебры логики: высказывание, истинностное значение, логические операции, формула, интерпретация, таблица истинности, тавтология, противоречие, выполнимость, равносильность. Установлены основные законы алгебры логики и принцип двойственности. Показано, что равносильные преобразования позволяют упрощать формулы, а каждая формула задаёт булеву функцию. Эти понятия служат основой для дальнейшего изучения булевых функций, канонических форм, полноты систем и минимизации. *(Обобщение по: Макоха и др., гл. 2, §2.1–2.2, с. 96–107; Игошин, гл. I, §1–4, с. 16–45.)*

1.1.2 Булевы функции

Основные источники: Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.2, с. 101–107; §2.3, теорема 2.1, с. 108. Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. II, §8–10, с. 90–106.

Понятие булевой функции. Пусть n — неотрицательное целое число. Рассмотрим множество всех упорядоченных наборов длины n, составленных из нулей и единиц:

$$\{0,1\}^n = \{(x_1, \ldots, x_n) \mid x_i \in \{0,1\},\ i = 1, \ldots, n\}.$$

Это множество содержит ровно 2^n элементов: число наборов длины n из двух символов равно 2^n. *(Источник: Игошин, гл. II, §8, с. 92–94; §10, с. 100–101; Макоха и др., гл. 2, §2.2, с. 101–102.)*

Определение. *Булевой функцией от n переменных* называется произвольное отображение

$$f : \{0,1\}^n \to \{0,1\}.$$

Иными словами, булева функция каждому набору $(x_1, \ldots, x_n) \in \{0,1\}^n$ ставит в соответствие единственное значение $f(x_1, \ldots, x_n) \in \{0,1\}$. *(Источник: Макоха и др., гл. 2, §2.2, с. 101–102; Игошин, гл. II, §10, определение 10.1, с. 100–101.)*

Множество всех булевых функций от n переменных обозначается $P_2(n)$. Множество всех булевых функций (при всевозможных n) обозначается P_2:

$$P_2 = \bigcup_{n=0}^{\infty} P_2(n).$$

При $n = 0$ получаем функции от нуля переменных — это константы 0 и 1. Формально $\{0,1\}^0$ состоит из одного пустого набора, поэтому функций от нуля переменных ровно две. *(Источник: Макоха и др., гл. 2, §2.2, с. 101–102; Игошин, гл. II, §9, с. 94–95.)*

Табличное задание булевой функции. Булеву функцию от n переменных удобно задавать таблицей истинности. В первых n столбцах перечисляются все наборы из $\{0,1\}^n$ в лексикографическом порядке, в последнем столбце — значения функции. Например, для $n = 3$ наборы располагаются так:

$$000,\ 001,\ 010,\ 011,\ 100,\ 101,\ 110,\ 111.$$

Эти наборы можно рассматривать как двоичные записи чисел $0, 1, 2, \ldots, 7$. *(Источник: Игошин, гл. II, §9, с. 94–95; Макоха и др., гл. 2, §2.2, с. 102–103.)*

Часто функцию задают *вектором значений* длины 2^n:

$$(f(0,\ldots,0),\ f(0,\ldots,0,1),\ \ldots,\ f(1,\ldots,1)).$$

Например, мажоритарная функция от трёх переменных $f(x,y,z)$ равна 1, если хотя бы две переменные равны 1. Её таблица:

x	y	z	$f(x,y,z)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Вектор значений этой функции:

$$(0,0,0,1,0,1,1,1).$$

Можно также записать множество номеров наборов, на которых функция равна 1:

$$\{3,5,6,7\}.$$

(Источник: Макоха и др., гл. 2, §2.2, с. 103–104; Игошин, гл. II, §9, с. 95–97.)

Число булевых функций от n переменных. Справедлива теорема: число всех различных булевых функций от n переменных равно

$$|P_2(n)| = 2^{2^n}.$$

(Источник: Макоха и др., гл. 2, §2.2, с. 102; Игошин, гл. II, §10, теорема 10.3, с. 102–104.)

Доказательство. Булева функция от n переменных полностью определяется своим вектором значений длины 2^n. Каждая из 2^n координат этого вектора может независимо принимать одно из двух значений: 0 или 1. Поэтому число различных векторов, а значит и число различных функций, равно

$$\underbrace{2 \cdot 2 \cdot \ldots \cdot 2}_{2^n \text{ раз}} = 2^{2^n}.$$

(Источник: Игошин, гл. II, §10, с. 102–104.)

Для малых n получаем:

$$|P_2(0)| = 2, \qquad |P_2(1)| = 4, \qquad |P_2(2)| = 16, \qquad |P_2(3)| = 256.$$

Уже при $n = 4$ число функций равно $2^{16} = 65536$, а при $n = 5$ — 2^{32}, то есть более четырёх миллиардов. Поэтому табличное задание при больших n становится необозримым. *(Источник: Макоха и др., гл. 2, §2.2, с. 102–103; Игошин, гл. II, §9, с. 95–97; §10, с. 102–104.)*

Равенство булевых функций. Две булевы функции f и g называются *равными*, если они принимают одинаковые значения на всех наборах значений их существенных переменных. Обозначение: $f = g$. *(Источник: Макоха и др., гл. 2, §2.2, с. 104–105; Игошин, гл. II, §9, с. 96–97.)*

При этом функции могут быть формально записаны от разного числа переменных. Например, функцию $f(x, y) = x \wedge (y \vee \neg y)$ можно рассматривать как функцию от одной существенной переменной x, поскольку $y \vee \neg y \equiv 1$, и $f(x, y) \equiv x$. Переменная y здесь фактически не влияет на значение функции. *(Источник: Макоха и др., гл. 2, §2.2, с. 104–105; Игошин, гл. II, §9, с. 96–97.)*

Фиктивные и существенные переменные. Переменная x_i называется *фиктивной* для функции $f(x_1, \ldots, x_n)$, если для любых значений остальных переменных выполняется равенство

$$f(\ldots, x_i = 0, \ldots) = f(\ldots, x_i = 1, \ldots).$$

Иными словами, значение функции не зависит от x_i. Переменная, не являющаяся фиктивной, называется *существенной*; говорят также, что функция *существенно зависит* от этой переменной. *(Источник: Макоха и др., гл. 2, §2.2, с. 104–105; Игошин, гл. II, §10, с. 101–102.)*

Пример. Функция

$$f(x, y) = x \wedge (y \vee \neg y)$$

не зависит от y, так как $y \vee \neg y = 1$ при $y = 0$ и при $y = 1$. Следовательно, y — фиктивная переменная, а x — существенная. *(Источник: Макоха и др., гл. 2, §2.2, с. 104–105.)*

Пример. Мажоритарная функция

$$\mathrm{maj}(x, y, z) = xy \vee xz \vee yz$$

существенно зависит от всех трёх переменных. Действительно, при $x = 0$:

$$\mathrm{maj}(0, 0, 1) = 0, \qquad \mathrm{maj}(0, 1, 1) = 1,$$

значит, x существенна. Аналогично проверяются y и z. *(Источник: Макоха и др., гл. 2, §2.2, с. 103–104; Игошин, гл. II, §9, с. 95–97.)*

Булевы функции от одного аргумента. При $n = 1$ существует ровно $2^{2^1} = 4$ булевы функции. Их таблицы:

x	$f_0(x)$	$f_1(x)$	$f_2(x)$	$f_3(x)$
0	0	0	1	1
1	0	1	0	1

Это соответственно:

$$f_0(x) \equiv 0, \qquad f_1(x) \equiv x, \qquad f_2(x) \equiv \neg x, \qquad f_3(x) \equiv 1.$$

(Источник: Игошин, гл. II, §9, с. 94–95; Макоха и др., гл. 2, §2.2, с. 101–103.)

Булевы функции от двух аргументов. При $n = 2$ существует $2^{2^2} = 16$ булевых функций. Основные из них приведены в таблице:

x	y	0	$x \wedge y$	$x \vee y$	$x \to y$	$x \leftrightarrow y$	$x \oplus y$	$x \mid y$	$x \downarrow y$	1
0	0	0	0	0	1	1	0	1	1	1
0	1	0	0	1	1	0	1	1	0	1
1	0	0	0	1	0	0	1	1	0	1
1	1	0	1	1	1	1	0	0	0	1

Здесь:

- $x \wedge y$ — конъюнкция;
- $x \vee y$ — дизъюнкция;
- $x \to y$ — импликация;
- $x \leftrightarrow y$ — эквивалентность;
- $x \oplus y$ — сложение по модулю 2, или исключающее «или»;
- $x \mid y = \neg(x \wedge y)$ — штрих Шеффера;
- $x \downarrow y = \neg(x \vee y)$ — стрелка Пирса.

(Источник: Игошин, гл. II, §9, с. 95–100; Макоха и др., гл. 2, §2.2, с. 101–103.)

Функция $x \oplus y$ может быть выражена формулой

$$x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge y).$$

Штрих Шеффера и стрелка Пирса будут важны в дальнейшем при изучении полных систем булевых функций. *(Источник: Макоха и др., гл. 2, §2.1, с. 98–99; §2.4, с. 110–114; Игошин, гл. II, §11, с. 107–111.)*

Формулы и суперпозиция. Пусть заданы булевы функции $g_1(y_1, \ldots, y_m), \ldots, g_n(y_1, \ldots, y_m)$ и функция $f(x_1, \ldots, x_n)$. *Суперпозицией* называется функция

$$h(y_1, \ldots, y_m) = f(g_1(y_1, \ldots, y_m), \ldots, g_n(y_1, \ldots, y_m)).$$

Иными словами, вместо переменных функции f подставляются функции g_i. *(Источник: Макоха и др., гл. 2, §2.2, с. 105–106; Игошин, гл. II, §10, определение 10.2, с. 101–102.)*

Например, если $f(x, y) = x \vee y$, $g_1(u, v) = u \wedge v$, $g_2(u, v) = \neg u$, то

$$f(g_1(u, v), g_2(u, v)) = (u \wedge v) \vee \neg u.$$

(Источник: Макоха и др., гл. 2, §2.2, с. 105–106.)

Каждая формула алгебры логики от переменных $X_1, \ldots, X_n$ задаёт булеву функцию от n переменных: значение функции на наборе $(\alpha_1, \ldots, \alpha_n)$ равно значению формулы при $X_1 = \alpha_1, \ldots, X_n = \alpha_n$. При этом разным формулам может соответствовать одна и та же булева функция; это происходит тогда и только тогда, когда формулы равносильны. *(Источник: Игошин, гл. II, §10, с. 104–106; Макоха и др., гл. 2, §2.2–2.3, с. 101–110.)*

Теорема о представлении булевых функций формулами. Справедлива следующая важная теорема.

Теорема. Всякая булева функция от n переменных может быть представлена в виде формулы через отрицание, конъюнкцию и дизъюнкцию. *(Источник: Макоха и др., гл. 2, §2.3, теорема 2.1, с. 108; Игошин, гл. II, §10, теорема 10.5, с. 104–105.)*

Для доказательства используется *лемма о разложении функции по переменной*:

$$f(x_1, x_2, \ldots, x_n) = \big(x_1 \wedge f(1, x_2, \ldots, x_n)\big) \vee \\ \vee \big(\neg x_1 \wedge f(0, x_2, \ldots, x_n)\big).$$

Аналогичное разложение возможно по любой переменной. *(Источник: Игошин, гл. II, §10, лемма 10.4, с. 103–104.)*

Идея доказательства теоремы. Применим индукцию по числу переменных n. При $n = 1$ все четыре функции от одной переменной выражаются через $\neg, \wedge, \vee$:

$$0 = x \wedge \neg x, \qquad 1 = x \vee \neg x, \qquad x = x, \qquad \neg x = \neg x.$$

Предположим, что теорема верна для функций от $n - 1$ переменной. Для функции $f(x_1, \ldots, x_n)$ запишем разложение по x_1:

$$f(x_1, \ldots, x_n) = \big(x_1 \wedge f(1, x_2, \ldots, x_n)\big) \vee \big(\neg x_1 \wedge f(0, x_2, \ldots, x_n)\big).$$

Функции $f(1, x_2, \ldots, x_n)$ и $f(0, x_2, \ldots, x_n)$ зависят уже от $n-1$ переменной. По предположению индукции они выражаются через $\neg, \wedge, \vee$. Подставляя эти выражения в правую часть, получаем формулу для $f(x_1, \ldots, x_n)$. *(Источник: Игошин, гл. II, §10, с. 104–105; Макоха и др., гл. 2, §2.3, с. 108.)*

Следствием является возможность задания любой булевой функции формулой над базисом $\{\neg, \wedge, \vee\}$. В дальнейшем будут рассмотрены и другие базисы, а также критерий полноты произвольной системы булевых функций. *(Источник: Макоха и др., гл. 2, §2.3–2.4, с. 108–114; Игошин, гл. II, §10–11, с. 104–111.)*

Пример: мажоритарная функция. Мажоритарная функция от трёх переменных

$$\mathrm{maj}(x, y, z) = xy \vee xz \vee yz$$

равна 1 тогда и только тогда, когда хотя бы две из переменных равны 1. Её таблица была приведена выше. Эта функция не имеет фиктивных переменных и существенно зависит от всех трёх аргументов. *(Источник: Макоха и др., гл. 2, §2.2, с. 103–104; Игошин, гл. II, §9, с. 95–97.)*

Связь с алгеброй высказываний. Булевы функции являются математической моделью логических операций. Пропозициональной переменной X соответствует булева переменная $x \in \{0, 1\}$. Логическим связкам соответствуют булевы функции:

$$\neg \leftrightarrow \text{отрицание}, \qquad \wedge \leftrightarrow \text{конъюнкция}, \qquad \vee \leftrightarrow \text{дизъюнкция},$$

$$\to\leftrightarrow \text{импликация}, \qquad \leftrightarrow\leftrightarrow \text{эквивалентность}.$$

Формуле алгебры логики соответствует булева функция; равносильным формулам соответствует одна и та же булева функция. Обратно, каждая булева функция может быть задана формулой, и таких формул бесконечно много. *(Источник: Игошин, гл. II, §10, с. 104–106; Макоха и др., гл. 2, §2.2–2.3, с. 101–110.)*

Итог пункта. В пункте 1.1.2 введено понятие булевой функции как отображения $\{0, 1\}^n \to \{0, 1\}$, рассмотрены табличный и векторный способы её задания, доказана формула для числа булевых функций 2^{2^n}, введены понятия фиктивной и существенной переменной, равенства функций, суперпозиции. Показано, что каждая формула алгебры логики задаёт булеву функцию и, обратно, всякая булева функция выражается формулой через $\neg, \wedge, \vee$. Эти результаты служат основой для изучения канонических форм, полных систем и минимизации булевых функций. *(Обобщение по: Макоха и др., гл. 2, §2.2–2.3, с. 101–110; Игошин, гл. II, §8–10, с. 90–106.)*

1.1.3 Канонические формы задания булевых функций

Основные источники: Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.3, с. 107–110; §2.2, с. 101–107. Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. I, §5, с. 46–53.

Элементарные конъюнкции и дизъюнкции. При построении нормальных форм булевых функций используются особые виды формул — элементарные конъюнкции и элементарные дизъюнкции.

Определение. *Элементарной конъюнкцией* от переменных $x_1, \ldots, x_n$ называется конъюнкция переменных или их отрицаний:

$$x_1^{\alpha_1} \wedge x_2^{\alpha_2} \wedge \cdots \wedge x_n^{\alpha_n},$$

где $\alpha_i \in \{0, 1\}$ и

$$x_i^{\alpha_i} = \begin{cases} x_i, & \text{если } \alpha_i = 1, \\ \neg x_i, & \text{если } \alpha_i = 0. \end{cases}$$

Аналогично, *элементарной дизъюнкцией* называется дизъюнкция переменных или их отрицаний:

$$x_1^{\beta_1} \vee x_2^{\beta_2} \vee \cdots \vee x_n^{\beta_n}.$$

(Источник: Макоха и др., гл. 2, §2.3, с. 107; Игошин, гл. I, §5, с. 46–47.)

Заметим, что в элементарную конъюнкцию или дизъюнкцию каждая переменная может входить не более одного раза. Если переменная входит в конъюнкцию и сама, и со своим отрицанием, то такая конъюнкция тождественно равна нулю; если в дизъюнкцию входят и x_i, и $\neg x_i$, то она тождественно равна единице. В дальнейшем такие вырожденные случаи обычно исключаются. *(Источник: Игошин, гл. I, §5, с. 46–47; Макоха и др., гл. 2, §2.3, с. 107–108.)*

ДНФ и КНФ. Определение. *Дизъюнктивной нормальной формой* (ДНФ) называется дизъюнкция элементарных конъюнкций:

$$\bigvee_{i=1}^{k} K_i,$$

где K_i — элементарные конъюнкции. Аналогично, *конъюнктивной нормальной формой* (КНФ) называется конъюнкция элементарных дизъюнкций:

$$\bigwedge_{i=1}^{k} D_i,$$

где D_i — элементарные дизъюнкции. *(Источник: Макоха и др., гл. 2, §2.3, с. 107–108; Игошин, гл. I, §5, с. 46–47.)*

Примерами ДНФ служат формулы

$$(x \wedge y) \vee (\neg x \wedge z), \qquad (x \wedge y \wedge \neg z) \vee (\neg x \wedge y) \vee x.$$

Примерами КНФ — формулы

$$(x \vee y) \wedge (\neg x \vee z), \qquad (x \vee y \vee z) \wedge (x \vee \neg y) \wedge z.$$

(Источник: Игошин, гл. I, §5, с. 47; Макоха и др., гл. 2, §2.3, с. 107–108.)

Любая формула алгебры логики может быть приведена к ДНФ и к КНФ с помощью равносильных преобразований, основанных на законах дистрибутивности, де Моргана, двойного отрицания и выражении импликации и эквивалентности через $\neg, \wedge, \vee$. *(Источник: Макоха и др., гл. 2, §2.3, с. 107–108; Игошин, гл. I, §5, с. 46–47.)*

Совершенные ДНФ и КНФ. Среди всех ДНФ и КНФ, равносильных данной функции, особую роль играют *совершенные* формы, обладающие свойством единственности.

Определение. *Совершенной ДНФ* (СДНФ) называется ДНФ, удовлетворяющая следующим условиям:

1) каждая элементарная конъюнкция содержит все переменные $x_1, \ldots, x_n$ (с отрицанием или без него);

2) ни одна элементарная конъюнкция не содержит одновременно переменную и её отрицание;

3) все элементарные конъюнкции различны.

Аналогично определяется *совершенная КНФ* (СКНФ): это КНФ, в которой каждая элементарная дизъюнкция содержит все переменные (с отрицанием или без него), не содержит одновременно переменную и её отрицание, и все дизъюнкции различны. *(Источник: Макоха и др., гл. 2, §2.3, с. 107–108; Игошин, гл. I, §5, определение 5.1, с. 47.)*

Каждая элементарная конъюнкция, входящая в СДНФ, называется *минтермом* (или *конституентой единицы*). Каждая элементарная дизъюнкция, входящая в СКНФ, называется *макстермом* (или *конституентой нуля*). *(Источник: Макоха и др., гл. 2, §2.3, с. 108–109; Игошин, гл. I, §5, с. 48–51.)*

Существование и единственность СДНФ. Справедлива следующая фундаментальная теорема.

Теорема (о СДНФ). Для всякой булевой функции $f(x_1, \ldots, x_n)$, не равной тождественно нулю, существует и единственна (с точностью до порядка членов дизъюнкции) её совершенная дизъюнктивная нормальная форма. *(Источник: Макоха и др., гл. 2, §2.3, с. 108–109; Игошин, гл. I, §5, лемма 5.2 и теорема 5.3, с. 48–51.)*

Доказательство существования опирается на разложение функции по всем переменным. Именно, для любой булевой функции $f(x_1, \ldots, x_n)$ справедливо представление

$$f(x_1, \ldots, x_n) = \bigvee_{\substack{(\alpha_1,\ldots,\alpha_n)\in\{0,1\}^n \\ f(\alpha_1,\ldots,\alpha_n)=1}} x_1^{\alpha_1} \wedge x_2^{\alpha_2} \wedge \cdots \wedge x_n^{\alpha_n}. \tag{2.1}$$

В этой формуле дизъюнкция берётся по всем наборам $(\alpha_1, \ldots, \alpha_n)$, на которых функция равна 1. Каждое слагаемое является минтермом, а вся дизъюнкция — СДНФ. *(Источник: Макоха и др., гл. 2, §2.3, формула (2.1), с. 108; Игошин, гл. I, §5, с. 48–50.)*

Доказательство единственности состоит в следующем. Предположим, что некоторая функция имеет две различные СДНФ. Пусть K — минтерм, входящий в первую СДНФ, но не входящий во вторую. Рассмотрим набор $(\alpha_1, \ldots, \alpha_n)$, на котором K обращается в 1. На этом наборе первая СДНФ равна 1. Если вторая СДНФ не содержит минтерма $x_1^{\alpha_1} \wedge \cdots \wedge x_n^{\alpha_n}$, то на данном наборе все её минтермы обращаются в 0, и вторая СДНФ равна 0. Противоречие. Следовательно, все минтермы обеих форм совпадают, и СДНФ единственна. *(Источник: Игошин, гл. I, §5, с. 49–51; Макоха и др., гл. 2, §2.3, с. 108–109.)*

Если функция тождественно равна нулю, то её СДНФ не существует (её нельзя представить в виде дизъюнкции минтермов, так как на всех наборах функция равна 0). Иногда условно полагают СДНФ нулевой функции равной 0, но формально это не является совершенной формой. *(Источник: Игошин, гл. I, §5, с. 50–51; Макоха и др., гл. 2, §2.3, с. 108–109.)*

Существование и единственность СКНФ. Двойственным образом формулируется теорема о СКНФ.

Теорема (о СКНФ). Для всякой булевой функции $f(x_1, \ldots, x_n)$, не равной тождественно единице, существует и единственна (с точностью до порядка членов конъюнкции) её совершенная конъюнктивная нормальная форма. *(Источник: Макоха и др., гл. 2, §2.3, с. 109–110; Игошин, гл. I, §5, лемма 5.4 и теорема 5.5, с. 51–53.)*

СКНФ имеет вид

$$f(x_1,\ldots,x_n)=\bigwedge_{\substack{(\beta_1,\ldots,\beta_n)\in\{0,1\}^n\\ f(\beta_1,\ldots,\beta_n)=0}}\left(x_1^{1-\beta_1}\vee x_2^{1-\beta_2}\vee\cdots\vee x_n^{1-\beta_n}\right), \tag{2.3}$$

где $x^1=x$, $x^0=\neg x$. Конъюнкция берётся по всем наборам, на которых функция равна 0. Каждый сомножитель является макстермом, а всё произведение — СКНФ. *(Источник: Макоха и др., гл. 2, §2.3, формула (2.3), с. 109–110; Игошин, гл. I, §5, с. 51–53.)*

Если функция тождественно равна единице, то её СКНФ не существует (формально можно было бы считать СКНФ единичной функции равной 1, но это не совершенная форма). *(Источник: Игошин, гл. I, §5, с. 52–53; Макоха и др., гл. 2, §2.3, с. 109–110.)*

Построение СДНФ и СКНФ по таблице истинности. Пусть функция $f(x_1,\ldots,x_n)$ задана таблицей истинности. Алгоритм построения СДНФ:

1) отметить в таблице все строки, в которых значение функции равно 1;
2) для каждой такой строки $(\alpha_1,\ldots,\alpha_n)$ выписать минтерм $x_1^{\alpha_1}\wedge\cdots\wedge x_n^{\alpha_n}$, где $x_i^1=x_i$, $x_i^0=\neg x_i$;
3) соединить все выписанные минтермы знаками дизъюнкции.

Алгоритм построения СКНФ:

1) отметить в таблице все строки, в которых значение функции равно 0;
2) для каждой такой строки $(\beta_1,\ldots,\beta_n)$ выписать макстерм $x_1^{1-\beta_1}\vee\cdots\vee x_n^{1-\beta_n}$;
3) соединить все выписанные макстермы знаками конъюнкции.

(Источник: Макоха и др., гл. 2, §2.3, с. 108–110; Игошин, гл. I, §5, с. 48–53.)

Пример построения СДНФ и СКНФ. Пусть функция $f(x,y,z)$ задана таблицей:

x	y	z	$f(x,y,z)$
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Единицы стоят на наборах с номерами $0,1,3,5,7$. Строим СДНФ:

$$f_{\text{СДНФ}}=\neg x\neg y\neg z\vee\neg x\neg yz\vee\neg xyz\vee x\neg yz\vee xyz.$$

Нули стоят на наборах $2, 4, 6$. Строим СКНФ. Для набора $2 = (0, 1, 0)$ макстерм имеет вид $x \vee \neg y \vee z$. Для набора $4 = (1, 0, 0)$ — $\neg x \vee y \vee z$. Для набора $6 = (1, 1, 0)$ — $\neg x \vee \neg y \vee z$. Следовательно,

$$f_{\text{СКНФ}} = (x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee \neg y \vee z).$$

В числовой форме:

$$f = \Sigma(0, 1, 3, 5, 7) = \Pi(2, 4, 6).$$

(Источник: Макоха и др., гл. 2, §2.3, примеры 2.2–2.3, с. 109–110; Игошин, гл. I, §5, с. 48–53.)

Построение СДНФ и СКНФ равносильными преобразованиями. Если функция задана не таблицей, а формулой, то СДНФ и СКНФ можно получить равносильными преобразованиями. Для этого нужно:

1) выразить все импликации и эквивалентности через $\neg, \wedge, \vee$;
2) с помощью законов де Моргана и двойного отрицания пронести отрицания до переменных;
3) с помощью дистрибутивности привести формулу к ДНФ или КНФ;
4) если полученная ДНФ (КНФ) не совершенна, то «размножить» недостающие переменные, используя равенства

$$x \vee \neg x \equiv 1, \qquad x \wedge \neg x \equiv 0;$$

5) удалить повторяющиеся минтермы (макстермы) и поглотить менее полные более полными.

(Источник: Игошин, гл. I, §5, с. 53; Макоха и др., гл. 2, §2.3, с. 108–110.)

Пример. Приведём к СДНФ формулу

$$f = (x \to y) \wedge z.$$

Выразим импликацию:

$$f = (\neg x \vee y) \wedge z.$$

Применим дистрибутивность:

$$f = (\neg x \wedge z) \vee (y \wedge z).$$

Получена ДНФ, но не СДНФ, так как не хватает переменной y в первом члене и x во втором. «Размножим»:

$$\neg x \wedge z = \neg x \wedge (y \vee \neg y) \wedge z = \neg xyz \vee \neg x\neg yz,$$

$$y \wedge z = (x \vee \neg x) \wedge y \wedge z = xyz \vee \neg xyz.$$

Объединяя и удаляя повторения, получаем

$$f_{\text{СДНФ}} = \neg x\neg yz \vee \neg xyz \vee xyz.$$

(Источник: пример составлен по образцу задач из: Игошин, гл. I, §5, с. 53; Макоха и др., гл. 2, §2.3, с. 108–110.)

Связь между СДНФ и СКНФ. СДНФ и СКНФ одной и той же функции двойственны друг другу: если в СДНФ заменить $\wedge$ на $\vee$, $\vee$ на $\wedge$, x_i на $\neg x_i$, $\neg x_i$ на x_i, то получится СКНФ. Точнее, справедливо равенство

$$(f) = \neg\,(\neg f).$$

Это означает, что для построения СКНФ можно построить СДНФ отрицания функции, а затем взять отрицание и применить законы де Моргана. *(Источник: Макоха и др., гл. 2, §2.3, с. 110; Игошин, гл. I, §5, с. 51–53.)*

Пример. Для функции из предыдущего примера

$$f = \Sigma(0, 1, 3, 5, 7)$$

отрицание имеет единицы на наборах $2, 4, 6$, поэтому

$$\neg f = \Sigma(2, 4, 6).$$

СДНФ отрицания:

$$\neg f = \neg x y \neg z \vee x \neg y \neg z \vee x y \neg z.$$

Беря отрицание и применяя де Моргана, получаем

$$f = (x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee \neg y \vee z),$$

что совпадает с ранее найденной СКНФ. *(Источник: Макоха и др., гл. 2, §2.3, с. 110; Игошин, гл. I, §5, с. 51–53.)*

Числовая форма записи. СДНФ и СКНФ удобно записывать в числовой форме. Если функция равна 1 на наборах с номерами $i_1, \ldots, i_k$, то пишут

$$f = \Sigma(i_1, \ldots, i_k).$$

Если функция равна 0 на наборах с номерами $j_1, \ldots, j_m$, то пишут

$$f = \Pi(j_1, \ldots, j_m).$$

Например, для рассмотренной функции

$$f = \Sigma(0, 1, 3, 5, 7) = \Pi(2, 4, 6).$$

(Источник: Макоха и др., гл. 2, §2.3, с. 109–110; Игошин, гл. I, §5, с. 48–53.)

Замечание о несовершенных формах. СДНФ и СКНФ являются каноническими: они единственны для данной функции (если функция не равна тождественно 0 или 1). Однако они, как правило, не являются минимальными: содержат много литералов и конъюнкций/дизъюнкций. Например, функция

$$f(x, y, z) = xy \vee xz \vee yz$$

в СДНФ имеет вид

$$f_{\text{СДНФ}} = \neg x y z \vee x \neg y z \vee x y \neg z \vee x y z,$$

то есть содержит четыре минтерма, тогда как минимальная ДНФ $xy \vee xz \vee yz$ содержит три конъюнкции. Поэтому после приведения к совершенной форме часто возникает задача минимизации, рассматриваемая в пункте 1.1.5. *(Источник: Макоха и др., гл. 2, §2.5, с. 114–115; Игошин, гл. I, §5, с. 53.)*

Единственность и мощность. Число минтермов в СДНФ равно числу единиц функции, а число макстермов в СКНФ — числу нулей. Для функции от n переменных максимальное число минтермов равно 2^n. Всего различных СДНФ от n переменных (не считая нулевой функции) равно $2^{2^n} - 1$, поскольку каждая функция, кроме тождественного нуля, имеет единственную СДНФ. *(Источник: Игошин, гл. II, §10, с. 102–104; гл. I, §5, с. 48–53.)*

Итог пункта. В пункте 1.1.3 введены элементарные конъюнкции и дизъюнкции, понятия ДНФ и КНФ, а также их совершенные формы — СДНФ и СКНФ. Доказаны теоремы о существовании и единственности совершенных форм. Описаны два способа построения СДНФ и СКНФ: по таблице истинности и равносильными преобразованиями. Показана двойственность СДНФ и СКНФ и введена числовая форма записи. Отмечено, что совершенные формы, будучи каноническими, не всегда минимальны, что мотивирует дальнейшее изучение минимизации булевых функций. *(Обобщение по: Макоха и др., гл. 2, §2.3, с. 107–110; Игошин, гл. I, §5, с. 46–53.)*

1.1.4 Понятие полной системы. Критерий полноты Поста

Основные источники: Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.4, с. 110–114. Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. II, §11, с. 107–111.

Замыкание и полнота системы булевых функций. На множестве булевых функций P_2 определена операция *суперпозиции*: подстановка одних функций вместо переменных других и переименование переменных. Суперпозиции позволяют из исходного набора функций получать новые функции. Возникает вопрос: какие системы функций обладают свойством, что через их функции можно выразить *любую* булеву функцию?

Определение. Пусть $M \subseteq P_2$ — некоторое множество булевых функций. *Замыканием* множества M называется множество $[M]$, состоящее из всех булевых функций, которые можно получить из функций множества M с помощью конечного числа операций суперпозиции (включая переименование переменных). Иными словами, $[M]$ — это наименьшее множество, содержащее M и замкнутое относительно суперпозиции. *(Источник: Макоха и др., гл. 2, §2.4, с. 110–111; Игошин, гл. II, §11, определение 11.1, с. 107.)*

Определение. Множество M называется *функционально замкнутым* (или просто *замкнутым*), если $[M] = M$. Множество M называется *полным*, если $[M] = P_2$, то есть через функции из M выражается любая булева функция. *(Источник: Макоха и др., гл. 2, §2.4, определения 2.13–2.15, с. 111–112; Игошин, гл. II, §11, определение 11.1, с. 107.)*

Определение. *Базисом* называется полная система M, никакая собственная подсистема которой не является полной. Базис — это минимальная по включению полная система. *(Источник: Макоха и др., гл. 2, §2.4, с. 114; Игошин, гл. II, §11, с. 107–108.)*

Примеры полных систем. Простейшие примеры полных систем булевых функций:

1) $\{\neg, \wedge, \vee\}$ — избыточная полная система (теорема о представлении булевых функций формулами);

2) $\{\neg, \wedge\}$ — базис (конъюнкция и отрицание);

3) $\{\neg, \vee\}$ — базис (дизъюнкция и отрицание);

4) $\{|\}$ — базис, состоящий из одной функции (штрих Шеффера);

5) $\{\downarrow\}$ — базис, состоящий из одной функции (стрелка Пирса);

6) $\{\neg, \to\}$ — базис;

7) $\{\oplus, \wedge, 1\}$ — базис (полином Жегалкина).

(Источник: Макоха и др., гл. 2, §2.4, с. 114; Игошин, гл. II, §11, теорема 11.2, с. 107.)

Проверим, например, полноту системы $\{|\}$. Штрих Шеффера выражается через отрицание и конъюнкцию: $x \mid y = \neg(x \wedge y)$. Следовательно, из $x \mid y$ можно получить $\neg x = x \mid x$. Далее,

$$x \wedge y = \neg(x \mid y) = (x \mid y) \mid (x \mid y).$$

Таким образом, через штрих Шеффера выражаются $\neg$ и $\wedge$, а система $\{\neg, \wedge\}$ полна; значит, полна и $\{|\}$. Аналогично проверяется полнота $\{\downarrow\}$. *(Источник: Игошин, гл. II, §11, с. 107; Макоха и др., гл. 2, §2.4, с. 114.)*

Возникает задача: как в общем случае определить, является ли данная система функций полной? Ответ даёт *критерий полноты Поста*, для формулировки которого вводятся пять специальных классов булевых функций.

Класс P_0: функции, сохраняющие нуль. Определение. Булева функция $f(x_1, \ldots, x_n)$ называется *сохраняющей нуль*, если

$$f(0, 0, \ldots, 0) = 0.$$

Множество всех таких функций обозначается P_0. *(Источник: Макоха и др., гл. 2, §2.4, определение 2.16, с. 112; Игошин, гл. II, §11, с. 107–108.)*

Примеры. Функции 0, x, $x \wedge y$, $x \vee y$, $x \oplus y$ принадлежат P_0; функции 1, $\neg x$, $x \to y$ не принадлежат P_0. *(Источник: Макоха и др., гл. 2, §2.4, с. 112; Игошин, гл. II, §11, с. 108.)*

Класс P_1: функции, сохраняющие единицу. Определение. Булева функция $f(x_1, \ldots, x_n)$ называется *сохраняющей единицу*, если

$$f(1, 1, \ldots, 1) = 1.$$

Множество всех таких функций обозначается P_1. *(Источник: Макоха и др., гл. 2, §2.4, определение 2.16, с. 112; Игошин, гл. II, §11, с. 107–108.)*

Примеры. Функции 1, x, $x \wedge y$, $x \vee y$, $x \to y$, $x \leftrightarrow y$ принадлежат P_1; функции 0, $\neg x$, $x \oplus y$ не принадлежат P_1. *(Источник: Макоха и др., гл. 2, §2.4, с. 112; Игошин, гл. II, §11, с. 108.)*

Класс L: линейные функции. **Определение.** Булева функция $f(x_1, \ldots, x_n)$ называется *линейной*, если её можно представить в виде

$$f(x_1, \ldots, x_n) = a_0 \oplus a_1 x_1 \oplus a_2 x_2 \oplus \cdots \oplus a_n x_n,$$

где $a_0, a_1, \ldots, a_n \in \{0, 1\}$, а операции $\oplus$ и $\cdot$ понимаются как сложение и умножение по модулю 2. Множество всех линейных функций обозначается L. Выражение в правой части называется *полиномом Жегалкина* степени не выше первой. *(Источник: Макоха и др., гл. 2, §2.4, определение 2.17, с. 112; Игошин, гл. II, §11, с. 108.)*

Примеры. Функции 0, 1, x, $\neg x = 1 \oplus x$, $x \oplus y$, $x \leftrightarrow y = 1 \oplus x \oplus y$ принадлежат L. Функции $x \wedge y$, $x \vee y = \neg(x \wedge y)$, $x \to y = \neg x \vee y$ не являются линейными. *(Источник: Макоха и др., гл. 2, §2.4, с. 112–113; Игошин, гл. II, §11, с. 108.)*

Класс S: самодвойственные функции. **Определение.** Булева функция $f(x_1, \ldots, x_n)$ называется *самодвойственной*, если она совпадает со своей двойственной функцией:

$$f^*(x_1, \ldots, x_n) = f(x_1, \ldots, x_n),$$

где

$$f^*(x_1, \ldots, x_n) = \neg f(\neg x_1, \ldots, \neg x_n).$$

Множество всех самодвойственных функций обозначается S. *(Источник: Макоха и др., гл. 2, §2.4, определение 2.18, с. 112–113; Игошин, гл. II, §11, с. 108.)*

Примеры. Функции x, $\neg x$ являются самодвойственными. Функции 0, 1, $x \wedge y$, $x \vee y$ не являются самодвойственными. Например, для $x \wedge y$ двойственная функция есть $x \vee y$, и $x \wedge y \not\equiv x \vee y$. *(Источник: Макоха и др., гл. 2, §2.4, с. 113; Игошин, гл. II, §11, с. 108.)*

Класс M: монотонные функции. **Определение.** Введём на множестве $\{0, 1\}$ отношение порядка: $0 \leq 0$, $0 \leq 1$, $1 \leq 1$. Булева функция $f(x_1, \ldots, x_n)$ называется *монотонной*, если для любых наборов $(\alpha_1, \ldots, \alpha_n)$ и $(\beta_1, \ldots, \beta_n)$, таких что $\alpha_i \leq \beta_i$ для всех $i = 1, \ldots, n$, выполняется неравенство

$$f(\alpha_1, \ldots, \alpha_n) \leq f(\beta_1, \ldots, \beta_n).$$

Множество всех монотонных функций обозначается M. *(Источник: Макоха и др., гл. 2, §2.4, определения 2.19–2.20, с. 113; Игошин, гл. II, §11, с. 108.)*

Примеры. Функции 0, 1, x, $x \wedge y$, $x \vee y$ монотонны. Функции $\neg x$, $x \oplus y$, $x \to y$ не монотонны. Например, для $\neg x$: $0 \leq 1$, но $\neg 0 = 1 > 0 = \neg 1$, что нарушает монотонность. *(Источник: Макоха и др., гл. 2, §2.4, с. 113; Игошин, гл. II, §11, с. 108.)*

Сводная таблица принадлежности основных функций классам:

Функция	P_0	P_1	L	S	M
0	да	нет	да	нет	да
1	нет	да	да	нет	да
x	да	да	да	да	да
$\neg x$	нет	нет	да	да	нет
$x \wedge y$	да	да	нет	нет	да
$x \vee y$	да	да	нет	нет	да
$x \to y$	нет	да	нет	нет	нет
$x \leftrightarrow y$	да	да	да	нет	нет
$x \oplus y$	да	нет	да	нет	нет
$x \mid y$	нет	нет	нет	нет	нет
$x \downarrow y$	нет	нет	нет	нет	нет

(Источник: таблица составлена по определениям классов: Макоха и др., гл. 2, §2.4, с. 112–113; Игошин, гл. II, §11, с. 107–108.)

Критерий полноты Поста. Центральный результат теории булевых функций — теорема Поста (1921 г.).

Теорема (критерий полноты Поста). Система булевых функций $M \subseteq P_2$ является полной тогда и только тогда, когда она не содержится целиком ни в одном из пяти замкнутых классов P_0, P_1, L, S, M. Иными словами, M полна тогда и только тогда, когда выполняются все пять условий:

1) в M существует функция, не сохраняющая нуль ($f \notin P_0$);

2) в M существует функция, не сохраняющая единицу ($f \notin P_1$);

3) в M существует нелинейная функция ($f \notin L$);

4) в M существует несамодвойственная функция ($f \notin S$);

5) в M существует немонотонная функция ($f \notin M$).

(Источник: Макоха и др., гл. 2, §2.4, теорема 2.2, с. 113; Игошин, гл. II, §11, теорема 11.4, с. 109.)

Необходимость. Предположим, что M содержится целиком в одном из классов, например в P_0. Каждый из пяти классов P_0, P_1, L, S, M замкнут: суперпозиция функций из класса снова принадлежит этому классу. Следовательно, и все функции, полученные из M суперпозицией, лежат в P_0. Значит, $[M] \subseteq P_0$, и поскольку $P_0 \neq P_2$ (например, $\neg x \notin P_0$), система M не является полной. Аналогично для остальных классов. *(Источник: Игошин, гл. II, §11, с. 109; Макоха и др., гл. 2, §2.4, с. 113.)*

Достаточность. Предположим, что в M есть функции $f_0 \notin P_0$, $f_1 \notin P_1$, $f_2 \notin L$, $f_3 \notin S$, $f_4 \notin M$. Показывается, что тогда из них с помощью суперпозиций можно получить полную систему, например $\{\neg, \wedge\}$ или штрих Шеффера. Доказательство конструктивно, но довольно громоздко; оно проводится в несколько шагов и опирается на последовательное «уничтожение» принадлежности классам. Полное доказательство имеется в учебнике Игошина (гл. II, §11, с. 109–111) и в книге Макохи и др. (гл. 2, §2.4, с. 113–114). *(Источник: Игошин, гл. II, §11, с. 109–111; Макоха и др., гл. 2, §2.4, с. 113–114.)*

Свойства классов Поста. Классы P_0, P_1, L, S, M называются *предполными*: каждый из них не является полным, но добавление к нему любой функции, не принадлежащей этому классу, даёт полную систему. Например, P_0 не полон, но $P_0 \cup \{\neg x\}$ полон, так как $\neg x \notin P_0$. *(Источник: Макоха и др., гл. 2, §2.4, с. 114; Игошин, гл. II, §11, с. 109.)*

Пост показал, что кроме этих пяти классов других предполных классов в P_2 нет. Всего существует счётное множество замкнутых классов булевых функций; они разбиваются на семь групп, из которых шесть конечны, а одна бесконечна. Пять классов P_0, P_1, L, S, M — важнейшие из них, так как через них формулируется критерий полноты. *(Источник: Макоха и др., гл. 2, §2.4, с. 111; Игошин, гл. II, §11, с. 109.)*

Пример проверки полноты. Рассмотрим систему $M = \{\neg, \wedge\}$. Проверим условия критерия Поста:

1) $\neg \notin P_0$, так как $\neg 0 = 1 \neq 0$;

2) $\neg \notin P_1$, так как $\neg 1 = 0 \neq 1$;

3) $\wedge \notin L$, так как $x \wedge y$ нелинейна (её полином Жегалкина содержит член xy);

4) $\wedge \notin S$, так как двойственная к $x \wedge y$ функция есть $x \vee y$, и $x \wedge y \not\equiv x \vee y$;

5) $\neg \notin M$, так как $0 \leq 1$, но $\neg 0 = 1 > 0 = \neg 1$.

Все пять условий выполнены, следовательно, система $\{\neg, \wedge\}$ полна. *(Источник: Макоха и др., гл. 2, §2.4, с. 113–114; Игошин, гл. II, §11, с. 109.)*

Рассмотрим систему $M = \{x \wedge y, x \vee y\}$. Она содержится в P_0 (обе функции сохраняют нуль), в P_1 (обе сохраняют единицу) и в M (обе монотонны). Условия критерия не выполнены, система не полна. Действительно, через монотонные функции нельзя выразить $\neg x$, так как отрицание немонотонно. *(Источник: Игошин, гл. II, §11, с. 109; Макоха и др., гл. 2, §2.4, с. 113.)*

Система $M = \{x \to y\}$ не полна: она содержится в P_1 ($1 \to 1 = 1$). Система $M = \{x \oplus y, 1\}$ содержится в L, поэтому не полна. Система $M = \{|\}$ проверяется так: $x \mid y \notin P_0$ ($0 \mid 0 = 1$), $x \mid y \notin P_1$ ($1 \mid 1 = 0$), $x \mid y \notin L$ (так как $\neg(x \wedge y)$ нелинейна), $x \mid y \notin S$ ($0 \mid 0 = 1 \neq 0 = 1 \mid 1$), $x \mid y \notin M$. Все условия выполнены, система $\{|\}$ полна. *(Источник: Макоха и др., гл. 2, §2.4, с. 114; Игошин, гл. II, §11, с. 107–109.)*

Замечание о числе классов. Пост установил, что всякий замкнутый класс булевых функций имеет конечный базис, и дал полное описание решётки замкнутых классов. Пять классов P_0, P_1, L, S, M — предполные; любой другой замкнутый класс содержится в одном из них. Поэтому для проверки полноты достаточно убедиться, что система не лежит ни в одном из этих пяти классов. *(Источник: Макоха и др., гл. 2, §2.4, с. 111–114; Игошин, гл. II, §11, с. 109–111.)*

Итог пункта. В пункте 1.1.4 введены понятия замыкания, замкнутой и полной системы, базиса. Описаны пять классов Поста P_0, P_1, L, S, M, приведены их определения и примеры принадлежности функций этим классам. Сформулирован критерий полноты Поста: система булевых функций полна тогда и только тогда, когда она не содержится целиком ни в одном из пяти классов Поста. Отмечено, что эти классы

являются предполными, и приведены примеры проверки полноты конкретных систем. *(Обобщение по: Макоха и др., гл. 2, §2.4, с. 110–114; Игошин, гл. II, §11, с. 107–111.)*

1.1.5 Минимизация булевых функций в классах нормальных форм

Основные источники: Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.5, с. 114–128. Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. I, §4, с. 40–45; §5, с. 46–53; §6, с. 54–65.

Постановка задачи минимизации. Совершенные ДНФ и КНФ, будучи каноническими формами, единственны для данной функции, но, как правило, содержат избыточное число литералов и членов. Техническая реализация такой формы приводит к громоздким логическим схемам. Поэтому возникает задача *минимизации* булевых функций: для данной функции найти равносильную ей ДНФ или КНФ, содержащую наименьшее возможное число литералов (переменных и их отрицаний), а при равенстве числа литералов — наименьшее число членов. *(Источник: Макоха и др., гл. 2, §2.5, с. 114–115.)*

Определение. *Минимальной ДНФ* (МДНФ) функции f называется ДНФ, равносильная f и содержащая наименьшее возможное число литералов среди всех ДНФ, равносильных f. Аналогично определяется *минимальная КНФ* (МКНФ). *(Источник: Макоха и др., гл. 2, §2.5, с. 114–115.)*

Точное решение задачи минимизации включает следующие этапы:

1) представление функции в СДНФ (или СКНФ);

2) получение сокращённой ДНФ (КНФ);

3) нахождение всех тупиковых ДНФ (КНФ);

4) выбор среди них минимальной формы.

(Источник: Макоха и др., гл. 2, §2.5, с. 118.)

Импликанты и простые импликанты. Определение. *Импликантой* функции $f(x_1, \ldots, x_n)$ называется булева функция g, принимающая значение 1 только на тех наборах, на которых f равна 1. Иными словами, $g \leq f$ поточечно: если $g(\alpha) = 1$, то и $f(\alpha) = 1$. *(Источник: Макоха и др., гл. 2, §2.5, определение 2.23, с. 117.)*

Определение. Импликанта g называется *простой*, если никакая её собственная часть (то есть результат удаления хотя бы одного литерала) уже не является импликантой f. *(Источник: Макоха и др., гл. 2, §2.5, с. 117–118.)*

Пример. Пусть

$$f(x, y, z) = xyz \vee xy\neg z \vee x\neg yz.$$

Конъюнкция xyz является импликантой, но не простой: её часть xy тоже является импликантой, так как

$$xy = xy(z \vee \neg z) = xyz \vee xy\neg z,$$

и $xy \leq f$. В то же время xy — простая импликанта, поскольку ни x, ни y не являются импликантами f. Конъюнкция $x\neg yz$ также является простой импликантой. *(Источник: Макоха и др., гл. 2, §2.5, с. 117–118.)*

Определение. *Сокращённой ДНФ* называется дизъюнкция всех простых импликант функции. *(Источник: Макоха и др., гл. 2, §2.5, определение 2.24, с. 118.)*

Определение. *Тупиковой ДНФ* называется дизъюнкция простых импликант, из которой нельзя удалить ни одну импликанту без потери равносильности исходной функции. *Минимальная ДНФ* — это тупиковая ДНФ с наименьшим числом литералов. *(Источник: Макоха и др., гл. 2, §2.5, определения 2.25–2.26, с. 118.)*

Логическая функция может иметь несколько тупиковых ДНФ и несколько минимальных ДНФ (если тупиковые формы имеют одинаковое число литералов). *(Источник: Макоха и др., гл. 2, §2.5, с. 118.)*

Метод тождественных преобразований. Простейший метод минимизации состоит в применении законов алгебры логики:

- *склеивания*: $xy \vee x\neg y \equiv x$;
- *поглощения*: $x \vee xy \equiv x$;
- *дистрибутивности*: $x(y \vee z) \equiv xy \vee xz$;
- законов де Моргана и двойного отрицания.

(Источник: Макоха и др., гл. 2, §2.5, с. 115–117; Игошин, гл. I, §4, с. 42–44.)

Пример. Минимизируем функцию, заданную в СДНФ:

$$f(x,y,z) = x\neg yz \vee xyz \vee x\neg y\neg z.$$

Вынесем x за скобки:

$$f = x(\neg yz \vee yz \vee \neg y\neg z).$$

Применим склеивание к первым двум членам:

$$\neg yz \vee yz = z.$$

Тогда

$$f = x(z \vee \neg y\neg z).$$

Применим дистрибутивность:

$$f = xz \vee x\neg y\neg z.$$

Далее можно применить поглощение: $xz \vee x\neg y\neg z = x(z \vee \neg y\neg z)$. Окончательно:

$$f_{\min} = x(z \vee \neg y\neg z).$$

(Источник: Макоха и др., гл. 2, §2.5, пример 2.10, с. 116.)

Метод тождественных преобразований удобен для простых функций, но при большом числе переменных становится трудоёмким. Кроме того, он не гарантирует получения минимальной формы, так как группировать минтермы можно по-разному. *(Источник: Макоха и др., гл. 2, §2.5, с. 115–117.)*

Метод Квайна. Метод Квайна (У. Куайн, 1950-е гг.) позволяет систематически находить сокращённую ДНФ, а затем все тупиковые и минимальные формы. Он состоит из следующих шагов.

Шаг 1. Записать функцию в СДНФ.

Шаг 2. Выполнить все возможные операции *неполного склеивания*:

$$xy \vee x\neg y \equiv x.$$

Склеивать можно только минтермы, отличающиеся ровно одним литералом. Результат склеивания называется импликантой ранга $n-1$, если исходные минтермы имели ранг n (ранг — число литералов).

Шаг 3. Выполнить все возможные операции *поглощения*:

$$x \vee xy \equiv x.$$

Поглощается импликанта, которая является частью другой импликанты.

Шаг 4. Если после шагов 2–3 появились новые импликанты, к ним снова применяют склеивание и поглощение. Процесс продолжается до тех пор, пока это возможно.

Шаг 5. Полученная дизъюнкция простых импликант есть *сокращённая ДНФ*.

Шаг 6. Построить *импликантную матрицу*. Строки соответствуют простым импликантам, столбцы — минтермам СДНФ. На пересечении строки и столбца ставится метка (например, крестик), если данная простая импликанта покрывает данный минтерм.

Шаг 7. Найти все *тупиковые ДНФ*. Тупиковая ДНФ — это минимальный по включению набор простых импликант, покрывающих все минтермы. Если импликанта покрывает некоторый минтерм, который не покрывается никакой другой импликантой, то она *существенна* и обязана войти во все тупиковые ДНФ.

Шаг 8. Среди всех тупиковых ДНФ выбрать форму с наименьшим числом литералов — это и есть МДНФ. *(Источник: Макоха и др., гл. 2, §2.5, с. 119–124.)*

Пример 1. Минимизировать функцию

$$f(x,y,z) = \neg x\neg y\neg z \vee \neg x\neg yz \vee \neg xyz \vee x\neg yz \vee xyz.$$

СДНФ содержит пять минтермов: $000, 001, 011, 101, 111$. Выполним склеивание. Удобно записать результаты в таблицу:

Склеиваемые минтермы	Импликанта
$000, 001$	$\neg x\neg y$
$001, 011$	$\neg xz$
$001, 101$	$\neg yz$
$011, 111$	yz
$101, 111$	xz
$001, 011, 101, 111$	z

Импликанта z получается двукратным склеиванием (сначала $001, 011 \to \neg xz$ и $101, 111 \to xz$, затем $\neg xz \vee xz \to z$; аналогично для $\neg yz$ и yz). Импликанты $\neg xz, \neg yz, yz, xz$ поглощаются импликантой z. Остаются простые импликанты $\neg x\neg y$ и z. Сокращённая ДНФ:

$$f_{\text{сокр}} = \neg x\neg y \vee z.$$

Импликантная матрица:

	000	001	011	101	111
$\neg x\neg y$	+	+			
z		+	+	+	+

Импликанта $\neg x\neg y$ единственная покрывает минтерм 000, значит, она существенна. Импликанта z единственная покрывает минтермы $011, 101, 111$, значит, она тоже существенна. Обе входят в единственную тупиковую ДНФ, она же минимальна:

$$f_{\min} = \neg x\neg y \vee z.$$

(Источник: Макоха и др., гл. 2, §2.5, с. 119–123; пример адаптирован.)

Пример 2. Минимизировать функцию

$$f(x, y, z) = xyz \vee x\neg yz \vee \neg xyz \vee \neg x\neg yz.$$

Все четыре минтерма склеиваются в пары:

$$xyz \vee x\neg yz = xz, \qquad \neg xyz \vee \neg x\neg yz = \neg xz,$$

$$xyz \vee \neg xyz = yz, \qquad x\neg yz \vee \neg x\neg yz = \neg yz.$$

Далее $xz \vee \neg xz = z$, $yz \vee \neg yz = z$. Простая импликанта — только z. Минимальная ДНФ:

$$f_{\min} = z.$$

(Источник: Макоха и др., гл. 2, §2.5, с. 119–124; пример адаптирован.)

Замечание о сложности. Метод Квайна требует перебора пар минтермов на каждом шаге. При числе переменных $n \geq 5$ и большом числе единиц функции число пар быстро растёт, и метод становится громоздким. Тем не менее он даёт точное решение задачи минимизации в классе ДНФ. *(Источник: Макоха и др., гл. 2, §2.5, с. 121–124.)*

Карты Карно. Более удобным для ручной минимизации при $n \leq 5$–6 является графический метод карт Карно (диаграмм Вейча). Карта Карно — это таблица из 2^n клеток, расположенных так, что любые две соседние клетки (по горизонтали или вертикали) отличаются значением ровно одной переменной. Клетки на противоположных краях карты также считаются соседними («заворачиваются»).

В клетки записываются значения функции. Единицы объединяются в прямоугольные группы, содержащие $1, 2, 4, 8, \ldots$ клеток (степени двойки). Группа может быть прямоугольником или квадратом; она должна быть максимальной, то есть её нельзя расширить, не захватив нули. Одна и та же единица может входить в несколько групп.

Каждая группа даёт импликанту, в которой сохраняются только те переменные, которые не меняют значения внутри группы. Переменные, меняющие значение, из импликанты исключаются. Минимальная ДНФ получается как дизъюнкция импликант, соответствующих выбранному покрытию всех единиц.

Карта Карно для трёх переменных. Столбцы соответствуют наборам yz: $00, 01, 11, 10$; строки — значениям x: $0, 1$. Именно такой порядок обеспечивает соседство клеток, отличающихся на одну переменную.

$x\backslash yz$	00	01	11	10
0				
1				

Карта Карно для четырёх переменных. Строки соответствуют наборам xy: $00, 01, 11, 10$; столбцы — наборам zw: $00, 01, 11, 10$:

$xy\backslash zw$	00	01	11	10
00				
01				
11				
10				

(Источник: Макоха и др., гл. 2, §2.5, с. 124–127.)

Правила группировки.

1) Группируются только единицы.
2) Группа должна содержать $1, 2, 4, 8, \ldots$ клеток.
3) Группа должна быть прямоугольной (или квадратной).
4) Группы могут пересекаться.
5) Группы могут «заворачиваться» через края карты.
6) Каждая группа должна быть максимальной по размеру.
7) Число групп должно быть минимальным, а суммарное число литералов в соответствующих импликантах — наименьшим.

(Источник: Макоха и др., гл. 2, §2.5, с. 124–127.)

Пример. Для функции

$$f = \Sigma(0, 1, 3, 5, 7)$$

построим карту Карно для трёх переменных:

$x\backslash yz$	00	01	11	10
0	1	1	1	0
1	0	1	1	0

Группа из четырёх единиц в столбцах 01 и 11 (при $x = 0$ и $x = 1$) даёт импликанту z (переменная x меняется, переменная y меняется, переменная z остаётся равной 1). Оставшаяся единица в клетке $x = 0, yz = 00$ объединяется с единицей $x = 0, yz = 01$ в группу из двух клеток, которая даёт $\neg x \neg y$. В результате

$$f_{\min} = z \vee \neg x \neg y.$$

(Источник: Макоха и др., гл. 2, §2.5, с. 124–127; пример адаптирован.)

Пример с четырьмя переменными. Пусть

$$f = \Sigma(0, 1, 2, 3, 5, 7, 8, 10, 13, 15).$$

Карта Карно:

$xy\backslash zw$	00	01	11	10
00	1	1	1	1
01	0	1	1	0
11	0	1	1	0
10	1	0	1	0

Группировка:

- четыре единицы в первой строке (00) дают $\neg x\neg y$;
- четыре единицы в столбцах $01, 11$ при $y = 1$ (клетки $01, 11$ для $xy = 00, 01, 11$) дают yz (здесь x меняется, w меняется, $y = 1$, $z = 1$);
- две единицы в клетках 1000 и 1011? Проверим: клетка $xy = 10, zw = 00$ содержит 1; клетка $xy = 10, zw = 11$ содержит 1. Они не соседние по одной переменной. Нужно объединять иначе. Клетки 1000 и 1010 соседние (отличаются w), дают $x\neg y\neg z$. Клетки 1011 и 1111 соседние (отличаются x), дают yz.

В результате

$$f_{\min} = \neg x\neg y \vee yz \vee x\neg y\neg z.$$

(Источник: Макоха и др., гл. 2, §2.5, с. 124–127; пример адаптирован.)

Минимизация в классе КНФ. Минимизация КНФ выполняется двойственным образом. Вместо единиц группируются *нули* функции. Каждая группа нулей даёт элементарную дизъюнкцию, в которой сохраняются переменные, не меняющиеся внутри группы, но с противоположными значениями: если переменная внутри группы равна 0, она входит в дизъюнкцию без отрицания; если равна 1 — с отрицанием. Минимальная КНФ получается как конъюнкция таких дизъюнкций.

Пример. Для функции $f = \Sigma(0, 1, 3, 5, 7)$ нули стоят на наборах $2, 4, 6$. Карта Карно с нулями:

$x\backslash yz$	00	01	11	10
0	1	1	1	0
1	0	1	1	0

Нуль в клетке $x = 0, yz = 10$ и нуль в клетке $x = 1, yz = 10$ объединяются в группу из двух клеток. Внутри группы x меняется, $y = 1$, $z = 0$. Дизъюнкция: $\neg y \vee z$. Нуль в клетке $x = 1, yz = 00$ объединяется с нулём $x = 1, yz = 10$? Они не соседние (отличаются y и z одновременно). Проверим: 100 и 110 — соседние (отличаются y); 100 и 000 — соседние (отличаются x). Группа из двух нулей 100 и 000 даёт дизъюнкцию $\neg x \vee z$ (внутри группы x меняется, $y = 0$, $z = 0$; значит, $\neg y$? Нет: для нулей правило другое). Аккуратно: для нулей в клетках 000 и 100 переменная x меняется, $y = 0$, $z = 0$. Дизъюнкция должна обращаться в 0 на этих наборах, то есть при $x = 0, y = 0, z = 0$ и $x = 1, y = 0, z = 0$. Этому условию удовлетворяет $y \vee z$. Значит, группа даёт $y \vee z$. Аналогично группа нулей 100 и 110 даёт $\neg x \vee z$. Объединяя, получаем

$$f_{\min\,\text{КНФ}} = (y \vee z) \wedge (\neg x \vee z) = (\neg x \vee z)(y \vee z).$$

Это равносильно найденной ранее МДНФ $z \vee \neg x\neg y$:

$$z \vee \neg x\neg y = (z \vee \neg x)(z \vee \neg y).$$

(Источник: Макоха и др., гл. 2, §2.5, с. 124–128; Игошин, гл. I, §5, с. 51–53.)

Сравнение методов.

- *Метод тождественных преобразований* прост, но не гарантирует минимальности и плохо работает при большом числе переменных.
- *Метод Квайна* даёт точное решение, систематичен, но требует перебора и громоздок при $n \geq 5$.
- *Карты Карно* удобны для $n \leq 5$–6, дают наглядное решение, но плохо поддаются автоматизации. Для $n \geq 6$ карты становятся труднообозримыми.

Для большего числа переменных применяются машинные методы: метод Квайна–Мак-Класки, эвристические алгоритмы, основанные на импликантных матрицах, и другие. Разработаны также точные алгоритмы, гарантирующие минимальность, но они имеют экспоненциальную сложность. *(Источник: Макоха и др., гл. 2, §2.5, с. 127–128.)*

Замечание о минимальности. Следует различать *сокращённую* ДНФ (дизъюнкция всех простых импликант), *тупиковые* ДНФ (минимальные по включению наборы простых импликант, покрывающие все минтермы) и *минимальную* ДНФ (тупиковая ДНФ с наименьшим числом литералов). Минимальная ДНФ всегда является тупиковой, но не наоборот. Число тупиковых ДНФ может быть большим, и выбор минимальной требует перебора. *(Источник: Макоха и др., гл. 2, §2.5, с. 118–124.)*

Итог пункта. В пункте 1.1.5 рассмотрена задача минимизации булевых функций в классах ДНФ и КНФ. Введены понятия импликанты, простой импликанты, сокращённой, тупиковой и минимальной ДНФ. Изложены три метода минимизации: метод тождественных преобразований (склеивание и поглощение), метод Квайна (систематическое построение сокращённой ДНФ и импликантной матрицы) и метод карт Карно (графическая группировка единиц или нулей). Показано, что минимальная ДНФ не всегда совпадает с сокращённой и что задача минимизации может иметь несколько решений. Отмечены ограничения методов и необходимость машинных алгоритмов при большом числе переменных. *(Обобщение по: Макоха и др., гл. 2, §2.5, с. 114–128; Игошин, гл. I, §4–6, с. 40–65.)*

1.1.6 Краткое резюме

Алгебра логики изучает высказывания и операции над ними. Булевы функции являются математической моделью логических операций и принимают значения в $\{0, 1\}$. Любая булева функция может быть задана таблицей истинности, формулой, СДНФ или СКНФ; СДНФ и СКНФ являются каноническими формами и единственны для данной функции (кроме тождественных констант). Полнота системы булевых функций означает возможность выразить через неё любую булеву функцию; критерий Поста сводит проверку полноты к пяти замкнутым классам P_0, P_1, L, S, M. Минимизация в классах ДНФ и КНФ позволяет получить более простые схемы и формулы; для её решения применяются тождественные преобразования, метод Квайна и карты Карно. *(Обобщение по: Макоха и др., гл. 2, §2.1–2.5, с. 96–128; Игошин, гл. I, §1–6, с. 16–65; гл. II, §8–11, с. 90–111.)*
Источники к разделу:

- Макоха А.Н., Сахнюк П.А., Червяков Н.И. Дискретная математика. М.: ФИЗМАТЛИТ, 2005. Гл. 2, §2.1–2.5, с. 96–128.
- Игошин В.И. Математическая логика и теория алгоритмов. М.: Академия, 2008. Гл. I, §1–6, с. 16–65; гл. II, §8–11, с. 90–111.

1.2 Исчисление предикатов первого порядка. Понятие интерпретации. Выполнимость и общезначимость формулы первого порядка. Понятие модели. Теорема о полноте исчисления предикатов первого порядка

Основной источник: Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. II, §1–5, с. 52–91; Гл. III, §1–3, с. 95–114.

1.2.1 Исчисление предикатов первого порядка

Основные источники: Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. II, §1–2, с. 52–70; Гл. III, §1–2, с. 95–108.

Общая идея аксиоматического построения. В предыдущих разделах формулы логики предикатов изучались с содержательной, семантической точки зрения: вводились интерпретации, модели, понятия истинности и общезначимости. При этом способы *доказательства* формул оставались неформализованными, они опирались на не уточнённые до конца представления о свойствах моделей. Задача исчисления предикатов — уточнить понятие доказательства: задать конечный набор аксиом и правил вывода, из которых чисто механически (без обращения к смыслу) можно получать все логические законы и только их. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 95.)*

Язык первого порядка. Исчисление предикатов строится в фиксированном *языке первого порядка* (языке первой ступени). Язык задаётся сигнатурой

$$\Omega = \{\mathrm{Srt}, \mathrm{Cnst}, \mathrm{Fn}, \mathrm{Pr}\},$$

где:

1) Srt — непустое множество *сортов*; для каждого сорта $\pi \in \mathrm{Srt}$ зафиксирован счётный набор символов переменных $x^\pi, y^\pi, z^\pi, \ldots$;

2) Cnst — множество (возможно, пустое) *констант*; каждой константе приписан сорт;

3) Fn — множество (возможно, пустое) *функциональных символов*; с каждой буквой $f \in \mathrm{Fn}$ связан её *вид* $(\pi_1, \ldots, \pi_k \to \pi)$, где $\pi_i, \pi \in \mathrm{Srt}$, $k > 0$ — *арность* f;

4) Pr — непустое множество *предикатных символов*; с каждой буквой $P \in \mathrm{Pr}$ связан её *вид* $(\pi_1, \ldots, \pi_k)$, где $\pi_i \in \mathrm{Srt}$, $k \geq 0$ — *арность* P. При $k = 0$ символ называется *пропозициональной переменной*.

Наиболее важен случай *односортного* языка: Srt состоит из одного элемента, и сорт можно не упоминать. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 52–53.)*

Термы. Понятие *терма сорта* π задаётся индуктивно:

1) каждая переменная x сорта π есть терм сорта π;

2) каждая константа c сорта π есть терм сорта π;

3) если f — функциональный символ вида $(\pi_1, \ldots, \pi_k \to \pi)$, а $t_1, \ldots, t_k$ — термы сортов $\pi_1, \ldots, \pi_k$ соответственно, то $f(t_1, \ldots, t_k)$ есть терм сорта π.

Множество всех термов сорта π обозначается Trm_π, множество всех термов языка — Trm_Ω. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 54–55.)*

Например, если g и h — двуместные функциональные символы, c — константа, x, y — переменные, то выражения

$$x, \quad c, \quad g(x,y), \quad h(c,x), \quad g(h(c,x),\, g(x,h(c,y)))$$

являются термами. Запись $g(t_1,t_2)$ означает применение двуместного функционального символа к термам t_1, t_2; в конкретных языках (например, арифметики) для g, h используют привычные обозначения t_1+t_2, $t_1\cdot t_2$. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 54–55; §4, с. 83–85.)*

Формулы. Понятие *формулы* языка Ω задаётся индуктивно:

1) если P — предикатный символ вида $(\pi_1, \ldots, \pi_k)$, а $t_1, \ldots, t_k$ — термы сортов $\pi_1, \ldots, \pi_k$, то выражение $P(t_1, \ldots, t_k)$ есть *атомарная формула*;

2) если A, B — формулы, то $\neg A$, $(A \wedge B)$, $(A \vee B)$, $(A \supset B)$ — формулы;

3) если A — формула и x — переменная, то $\forall x\, A$ и $\exists x\, A$ — формулы;

4) других формул нет.

(Источник: Колмогоров, Драгалин, гл. II, §1, с. 56–60.)

В исчислении предикатов используются связки $\neg, \wedge, \vee, \supset$ и кванторы $\forall, \exists$. Эквивалентность $A \equiv B$ рассматривается как сокращение для $(A \supset B) \wedge (B \supset A)$; в некоторых изложениях она вводится как самостоятельная связка. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 56–60.)*

Свободные и связанные переменные. Вхождение переменной x в формулу A называется *связанным*, если оно находится в области действия квантора $\forall x$ или $\exists x$. В противном случае вхождение называется *свободным*. Переменная x называется *параметром* формулы A, если она входит в A свободно хотя бы один раз. Множество параметров A обозначается $\mathrm{Fv}(A)$. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 60–62.)*

Формула без параметров называется *замкнутой* или *предложением*. Множество всех предложений языка Ω обозначается St. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 62.)*

Множество $\mathrm{Fv}(A)$ вычисляется индуктивно:

$$\begin{aligned}
&\mathrm{Fv}(P(t_1,\ldots,t_k)) = \mathrm{Fv}(t_1) \cup \cdots \cup \mathrm{Fv}(t_k),\\
&\mathrm{Fv}(A \wedge B) = \mathrm{Fv}(A \vee B) = \mathrm{Fv}(A \supset B) = \mathrm{Fv}(A) \cup \mathrm{Fv}(B),\\
&\mathrm{Fv}(\neg A) = \mathrm{Fv}(A),\\
&\mathrm{Fv}(\forall x\, A) = \mathrm{Fv}(\exists x\, A) = \mathrm{Fv}(A) \setminus \{x\}.
\end{aligned}$$

(Источник: Колмогоров, Драгалин, гл. II, §1, с. 62.)

Например, в формуле

$$\forall x\, P(x) \wedge \exists x\, Q(x,x) \supset \exists y\, R(x,y)$$

переменная x входит один раз связанно и один раз свободно, переменная y связана, параметром является только x. Формула не замкнута. *(Источник: Колмогоров, Драгалин, гл. II, §1, с. 60–62.)*

Правильная подстановка. При подстановке терма t вместо переменной x в формулу A возможна *коллизия переменных*: переменная, свободная в t, может попасть в область действия квантора по этой переменной. Например, если в формуле $\exists y\, P(x,y,z)$ заменить x на $f(x,y)$, получится $\exists y\, P(f(x,y),y,z)$, которая означает совсем не то, что нужно, и к тому же зависит от двух параметров, а не от трёх. *(Источник: Колмогоров, Драгалин, гл. II, §2, с. 65–67.)*

Правильный выход — предварительное переименование связанных переменных: формула $\exists y\, P(x,y,z)$ равносильна $\exists u\, P(x,u,z)$, и подстановка даёт $\exists u\, P(f(x,y),u,z)$, что и требуется. Формально определяется *правильная подстановка* $A(x\|t)$: если никакая связанная переменная A не совпадает со свободной переменной t, подстановка сводится к простому замещению; в противном случае связанные переменные переименовываются. *(Источник: Колмогоров, Драгалин, гл. II, §2, с. 67–70.)*

Обозначение $A(x\|t)$ часто сокращают до $A(t)$, если из контекста ясно, вместо какой переменной подставляется терм. *(Источник: Колмогоров, Драгалин, гл. II, §2, с. 70.)*

Аксиомы исчисления предикатов. Зафиксируем язык первого порядка Ω. *Аксиомами исчисления предикатов* в языке Ω называются формулы следующих четырнадцати видов:

(A1) $A \supset (B \supset A)$;
(A2) $(A \supset (B \supset C)) \supset ((A \supset B) \supset (A \supset C))$;
(A3) $A \supset (B \supset A \wedge B)$;
(A4) $A \wedge B \supset A$;
(A5) $A \wedge B \supset B$;
(A6) $(A \supset C) \supset ((B \supset C) \supset (A \vee B \supset C))$;
(A7) $A \supset A \vee B$;
(A8) $B \supset A \vee B$;
(A9) $(A \supset B) \supset ((A \supset \neg B) \supset \neg A)$;
(A10) $\neg\neg A \supset A$;
(A11) $\forall x\, A \supset A(x\|t)$;
(A12) $\forall x\, (C \supset A(x)) \supset (C \supset \forall x\, A(x))$;
(A13) $A(x\|t) \supset \exists x\, A$;
(A14) $\forall x\, (A(x) \supset C) \supset (\exists x\, A(x) \supset C)$.

Здесь A, B, C — произвольные формулы языка Ω. Каждая строка задаёт *схему аксиом*: подставляя вместо A, B, C конкретные формулы, получаем конкретные аксиомы. В схемах (A12) и (A14) формула C не содержит свободной переменной x; запись

$A(x\|t)$ означает правильную подстановку терма t вместо переменной x. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 95–96.)*

Схемы (A1)–(A10) — это аксиомы исчисления высказываний, перенесённые в язык предикатов; схемы (A11)–(A14) — собственно предикатные аксиомы, описывающие свойства кванторов. Схемы (A11) и (A13) позволяют переходить от кванторной формулы к её частному случаю и обратно; (A12) и (A14) позволяют выносить квантор за скобки, если переменная не входит свободно в другую часть формулы. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 95–96.)*

Правила вывода. В исчислении предикатов два правила вывода:

1) *modus ponens* (MP): из A и $A \supset B$ следует B;

2) *правило обобщения*: из A следует $\forall x\, A$.

(Источник: Колмогоров, Драгалин, гл. III, §1, с. 96–97.)

Структурное требование. При применении правила обобщения к формуле A, зависящей от гипотез Γ, переменная x не должна входить свободно ни в одну из гипотез Γ. Без этого ограничения можно было бы вывести ложные утверждения (например, из гипотезы $x = 0$ получить $\forall x\,(x = 0)$). Структурное требование обеспечивает корректность вывода. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 97–100.)*

Вывод и выводимость. *Выводом* формулы A из множества гипотез Γ называется конечная последовательность формул $B_1, \ldots, B_n$, в которой $B_n \equiv A$ и каждая B_i является либо аксиомой, либо гипотезой из Γ, либо получена из предыдущих формул по одному из правил вывода. Обозначение: $\Gamma \vdash A$. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 96–100.)*

Если $\Gamma = \varnothing$, пишут $\vdash A$ и говорят, что A *доказуема* (является *теоремой* исчисления предикатов). *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 96–100.)*

Элементы множества Γ называются *гипотезами* вывода. Множество Γ может быть конечным или бесконечным; в последнем случае в выводе используется лишь конечное число гипотез. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 96–100.)*

Пример вывода. Покажем, что $\vdash A \supset A$. Строим последовательность:

1) $A \supset ((A \supset A) \supset A)$	— схема (A1) при $B = (A \supset A)$;
2) $(A \supset ((A \supset A) \supset A)) \supset ((A \supset (A \supset A)) \supset (A \supset A))$	— схема (A2);
3) $(A \supset (A \supset A)) \supset (A \supset A)$	— MP: 1, 2;
4) $A \supset (A \supset A)$	— схема (A1);
5) $A \supset A$	— MP: 3, 4.

(Источник: Колмогоров, Драгалин, гл. III, §1, с. 96–97; пример построен по образцу стандартных выводов в гильбертовском исчислении.)

Теорема о дедукции. Если $\Gamma, A \vdash B$, то $\Gamma \vdash A \supset B$. В частности, если $A \vdash B$, то $\vdash A \supset B$. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 100–102.)*

Теорема о дедукции позволяет сводить доказательство импликации $A \supset B$ к выводу B из гипотезы A. В математической практике это соответствует стандартному приёму: чтобы доказать «если A, то B», предполагают A и выводят B. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 102.)*

Техника естественного вывода. Непосредственно строить выводы в аксиоматическом исчислении крайне неудобно: они получаются громоздкими и непохожими на обычные математические рассуждения. Поэтому на практике используют *производные правила*, которые можно получить из аксиом и правила MP, но которые по форме близки к обычным приёмам рассуждений. Набор таких правил называется *техникой естественного вывода. (Источник: Колмогоров, Драгалин, гл. III, §2, с. 102–103.)*

Структурные правила:

закон тождества: $A \vdash A$;

правило добавления: $\dfrac{\Gamma \vdash A}{\Gamma, B \vdash A}$;

правило перестановки: $\dfrac{\Gamma, B, C, \Delta \vdash A}{\Gamma, C, B, \Delta \vdash A}$;

правило сокращения: $\dfrac{\Gamma, B, B, \Delta \vdash A}{\Gamma, B, \Delta \vdash A}$;

правило сечения: $\dfrac{\Gamma \vdash A;\ \Delta, A \vdash B}{\Gamma, \Delta \vdash B}$.

(Источник: Колмогоров, Драгалин, гл. III, §2, с. 102.)

Правила введения и удаления логических связок:

$\supset$-введение: $\dfrac{\Gamma, A \vdash B}{\Gamma \vdash A \supset B}$;

$\supset$-удаление (MP): $\dfrac{\Gamma \vdash A;\ \Gamma \vdash A \supset B}{\Gamma \vdash B}$;

$\wedge$-введение: $\dfrac{\Gamma \vdash A;\ \Gamma \vdash B}{\Gamma \vdash A \wedge B}$;

$\wedge$-удаление: $\dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}$, $\dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}$;

$\vee$-введение: $\dfrac{\Gamma \vdash A}{\Gamma \vdash A \vee B}$, $\dfrac{\Gamma \vdash B}{\Gamma \vdash A \vee B}$;

$\vee$-удаление (разбор случаев): $\dfrac{\Gamma \vdash A \vee B;\ \Gamma, A \vdash C;\ \Gamma, B \vdash C}{\Gamma \vdash C}$;

$\neg$-введение (от противного): $\dfrac{\Gamma, A \vdash B;\ \Gamma, A \vdash \neg B}{\Gamma \vdash \neg A}$;

$\neg$-удаление: $\dfrac{\Gamma \vdash A;\ \Gamma \vdash \neg A}{\Gamma \vdash B}$.

(Источник: Колмогоров, Драгалин, гл. III, §2, с. 102–105.)

Правила введения и удаления кванторов:

$\forall$-введение: $\dfrac{\Gamma \vdash A(x)}{\Gamma \vdash \forall x\, A(x)}$ (x не свободна в Γ);

$\forall$-удаление: $\dfrac{\Gamma \vdash \forall x\, A(x)}{\Gamma \vdash A(t)}$;

$\exists$-введение: $\dfrac{\Gamma \vdash A(t)}{\Gamma \vdash \exists x\, A(x)}$;

$\exists$-удаление: $\dfrac{\Gamma \vdash \exists x\, A(x);\ \Gamma, A(y) \vdash B}{\Gamma \vdash B}$ (y не свободна в $\Gamma, B, \exists x\, A(x)$).

(Источник: Колмогоров, Драгалин, гл. III, §2, с. 103–105.)

Правило $\vee$-удаления соответствует разбору случаев: если нужно вывести C из дизъюнкции $A \vee B$, достаточно вывести C из A и из B по отдельности. Правило $\exists$-удаления (правило единичного выбора, правило C) соответствует рассуждению: если существует x с $A(x)$, возьмём такой x (обозначим его через y) и выведем B. Правило $\neg$-введения соответствует доказательству от противного: чтобы доказать $\neg A$, допустим A и выведем противоречие. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 103–104.)*

Правило подстановки. Из $\Gamma \vdash A$ следует

$$\Gamma(x_1, \ldots, x_k \| t_1, \ldots, t_k) \vdash A(x_1, \ldots, x_k \| t_1, \ldots, t_k),$$

где t_i — термы соответствующего сорта, а $\Gamma(x\|t)$ обозначает результат правильной подстановки одновременно во все формулы из Γ. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 105.)*

Пример вывода с использованием техники естественного вывода. Докажем формулу

$$\vdash A \vee B \supset \neg(\neg A \wedge \neg B).$$

По правилу $\supset$-введения достаточно установить секвенцию

$$A \vee B \vdash \neg(\neg A \wedge \neg B).$$

Слева стоит дизъюнкция, поэтому по правилу $\vee$-удаления достаточно вывести $\neg(\neg A \wedge \neg B)$ из A и из B по отдельности.

1) $A \vdash \neg(\neg A \wedge \neg B)$. По правилу $\neg$-введения достаточно, допустив $\neg A \wedge \neg B$, получить противоречие. Из $\neg A \wedge \neg B$ по правилу $\wedge$-удаления получаем $\neg A$, а по гипотезе имеем A. Это противоречие; следовательно, $A \vdash \neg(\neg A \wedge \neg B)$.

2) $B \vdash \neg(\neg A \wedge \neg B)$ — аналогично, с использованием $\wedge$-удаления $\neg B$ и гипотезы B.

Применяя $\vee$-удаление к $A \vee B \vdash A \vee B$, $A \vdash \neg(\neg A \wedge \neg B)$ и $B \vdash \neg(\neg A \wedge \neg B)$, получаем $A \vee B \vdash \neg(\neg A \wedge \neg B)$. Применяя $\supset$-введение, получаем $\vdash A \vee B \supset \neg(\neg A \wedge \neg B)$. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 104; пример построен по образцу приведённых там рассуждений.)*

Замечание об эквивалентных формулировках. Существуют и другие формулировки исчисления предикатов, эквивалентные приведённой. В частности, широко распространены *исчисления натурального вывода* и *исчисления секвенций*, начало изучению которых положил Г. Генцен в 1934 г. В этих исчислениях в основу кладутся не аксиомы, а правила введения и удаления логических связок и кванторов; они более удобны для практических выводов и для доказательств метатеорем. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 105–107.)*

Связь с семантикой. Исчисление предикатов корректно: всякая выводимая формула общезначима, то есть если $\vdash A$, то $\models A$. Это утверждение несложно доказать индукцией по длине вывода: все аксиомы общезначимы, а правила MP и обобщения сохраняют общезначимость. Обратное утверждение — что всякая общезначимая формула выводима — составляет содержание *теоремы Гёделя о полноте* (см. п. 1.2.5). *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 95–96; §2, с. 107.)*

Итог пункта. Исчисление предикатов первого порядка — это формальная аксиоматическая теория в фиксированном языке первого порядка. Язык задаётся сигнатурой (сорта, константы, функциональные и предикатные символы); из них индуктивно строятся термы и формулы. Свободные и связанные переменные, правильная подстановка, коллизия переменных — ключевые технические понятия. Исчисление имеет четырнадцать схем аксиом и два правила вывода (modus ponens и обобщение). Понятие вывода $\Gamma \vdash A$ позволяет определить доказуемую формулу. Теорема о дедукции и техника естественного вывода облегчают построение конкретных выводов. Корректность исчисления доказывается несложно; полнота — предмет отдельной теоремы. *(Обобщение по: Колмогоров, Драгалин, гл. II, §1–2, с. 52–70; гл. III, §1–2, с. 95–108.)*

1.2.2 Понятие интерпретации

Основные источники: Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. II, §3–4, с. 77–87; вводная часть, §9, с. 50–51.

Зачем нужна интерпретация. Формулы и термы языка первого порядка сами по себе — это лишь строки символов, они ничего не выражают. Чтобы придать им содержательный смысл, нужно указать, какие объекты пробегают переменные, что обозначают константы, какие функции соответствуют функциональным символам и какие отношения — предикатным. Совокупность таких указаний называется *интерпретацией* языка. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77; вводная часть, с. 50.)*

Носитель. Первый шаг интерпретации — указание области, из которой берутся значения переменных. *Носителем* для языка Ω (объектной областью) называется функция $D\colon \mathrm{Srt} \to \mathcal{V}$, где $\mathcal{V}$ — универсум всех множеств, такая, что для каждого сорта $\pi \in \mathrm{Srt}$ множество $D(\pi) = D_\pi$ непусто. В односортном случае носитель отождествляется с единственным множеством D. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77.)*

Множество D_π называется *носителем сорта* π (объектной областью сорта π). Требование непустоты существенно: квантор $\forall x$ должен пробегать непустое множество, иначе семантика стала бы вырожденной. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77.)*

Интерпретация символов. Второй шаг — приписывание значений символам сигнатуры. *Интерпретацией* языка Ω на носителе D называется приписывание:

1) каждой константе c сорта π — некоторого элемента $c^D \in D_\pi$;

2) каждому функциональному символу f вида $(\pi_1, \ldots, \pi_k \to \pi)$ — функции

$$f^D \colon D_{\pi_1} \times \cdots \times D_{\pi_k} \to D_\pi;$$

3) каждому предикатному символу P вида $(\pi_1, \ldots, \pi_k)$ — отношения

$$P^D \subseteq D_{\pi_1} \times \cdots \times D_{\pi_k},$$

которое удобно рассматривать как функцию $P^D \colon D_{\pi_1} \times \cdots \times D_{\pi_k} \to \{0, 1\}$, где $P^D(a_1, \ldots, a_k) = 1$ означает, что набор $(a_1, \ldots, a_k)$ принадлежит отношению.

(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77–83; §4, с. 83–85.)

Совокупность носителя и интерпретации символов часто обозначают одной буквой M (или $\mathfrak{M}$) и называют *интерпретацией* (моделью) языка Ω. В п. 1.2.4 это понятие будет уточнено как «модель языка». *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77–83.)*

Оценка переменных. Третий шаг — приписывание значений свободным переменным. *Оценкой* переменных на носителе D называется функция θ, сопоставляющая каждой переменной x сорта π элемент $\theta(x) \in D_\pi$. Множество всех оценок обозначается Θ_D. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 78–80.)*

Оценка нужна только для свободных переменных: связанные переменные «пробегают» носитель под действием кванторов, и их значения определяются в процессе вычисления истинности. Формально это выражается через модификацию оценки: если θ — оценка и $a \in D_\pi$, то $\theta[x \mapsto a]$ обозначает оценку, совпадающую с θ на всех переменных, кроме x, для которой $\theta[x \mapsto a](x) = a$. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 78–80.)*

Значение терма. Пусть зафиксированы интерпретация на носителе D и оценка θ. *Значение терма* t при оценке θ обозначается $|t|^D_\theta$ и определяется индуктивно по построению терма:

1) если t — переменная x сорта π, то $|x|^D_\theta = \theta(x)$;

2) если t — константа c сорта π, то $|c|^D_\theta = c^D$;

3) если $t = f(t_1, \ldots, t_k)$, то

$$|f(t_1, \ldots, t_k)|^D_\theta = f^D\big(|t_1|^D_\theta, \ldots, |t_k|^D_\theta\big).$$

(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–81.)

Таким образом, каждый терм при данной интерпретации и данной оценке обозначает конкретный элемент носителя. Например, в языке арифметики Ar при стандартной интерпретации ω и оценке $\theta(x) = 2$, $\theta(y) = 6$, $\theta(z) = 4$ терм $(x + y) \cdot z + Sx$ принимает значение

$$(2 + 6) \cdot 4 + (2 + 1) = 8 \cdot 4 + 3 = 35.$$

(Источник: Колмогоров, Драгалин, гл. II, §4, с. 83–85.)

Истинность формулы. *Значение истинности формулы* A при данной интерпретации и оценке θ обозначается $|A|^D_\theta \in \{0,1\}$ и определяется индуктивно:

1) для атомарной формулы $P(t_1,\dots,t_k)$:

$$|P(t_1,\dots,t_k)|^D_\theta = P^D\big(|t_1|^D_\theta,\dots,|t_k|^D_\theta\big);$$

2) для логических связок:

$$\begin{aligned} |\neg A|^D_\theta &= 1 - |A|^D_\theta,\\ |A \wedge B|^D_\theta &= \min\big(|A|^D_\theta, |B|^D_\theta\big),\\ |A \vee B|^D_\theta &= \max\big(|A|^D_\theta, |B|^D_\theta\big),\\ |A \supset B|^D_\theta &= \max\big(1 - |A|^D_\theta, |B|^D_\theta\big); \end{aligned}$$

3) для кванторов:

$$\begin{aligned} |\forall x\, A(x)|^D_\theta &= \min_{a \in D_\pi} |A(x)|^D_{\theta[x\mapsto a]},\\ |\exists x\, A(x)|^D_\theta &= \max_{a \in D_\pi} |A(x)|^D_{\theta[x\mapsto a]}, \end{aligned}$$

где π — сорт переменной x.

(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)

Квантор $\forall$ интерпретируется как «минимум по всем элементам носителя»: формула $\forall x\, A(x)$ истинна тогда и только тогда, когда $A(x)$ истинна при всех значениях x. Квантор $\exists$ интерпретируется как «максимум»: формула $\exists x\, A(x)$ истинна тогда и только тогда, когда $A(x)$ истинна хотя бы при одном значении x. Такая интерпретация согласуется с интуитивным смыслом кванторов и с определением истинности в конечных моделях, где $\forall$ сводится к конъюнкции, а $\exists$ — к дизъюнкции. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83; ср. вводная часть, с. 50–51.)*

Говорят, что формула A *истинна в данной интерпретации при оценке* θ, если $|A|^D_\theta = 1$. Если A замкнута (не имеет свободных переменных), её значение не зависит от оценки, и пишут просто $|A|^D$ или $M \models A$. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)*

Пример интерпретации. Рассмотрим язык элементарной арифметики Ar. Он содержит:

- константу 0;
- функциональные символы: двуместные $+$ и $\cdot$, одноместный S (прибавление единицы);
- предикатный символ $=$ (двуместный).

Стандартная интерпретация ω на носителе $\mathbb{N}$ приписывает 0 число 0, $+$ — сложение натуральных чисел, $\cdot$ — умножение, S — операцию $n \mapsto n+1$, $=$ — отношение равенства. При оценке $\theta(x)=3$, $\theta(z)=5$ формула $\exists y\,(x+y=z)$ превращается в истинное высказывание $\exists y\,(3+y=5)$; при оценке $\theta(x)=5$, $\theta(z)=3$ — в ложное $\exists y\,(5+y=3)$. *(Источник: Колмогоров, Драгалин, гл. II, §4, с. 83–85.)*

Тот же язык можно интерпретировать иначе. Модель R имеет носитель $\mathbb{R}$; константе 0 сопоставлено действительное число 0, символам $+$, $\cdot$, S — сложение, умножение и прибавление единицы в $\mathbb{R}$, символу $=$ — равенство действительных чисел. В этой интерпретации формула $\forall x \exists z\,(x + z = 0)$ истинна, а в стандартной модели ω — ложна (в $\mathbb{N}$ нет отрицательных чисел). Это показывает, что одна и та же формула может быть истинной в одной интерпретации и ложной в другой. *(Источник: Колмогоров, Драгалин, гл. II, §4, с. 85–87.)*

Свободные и связанные переменные при интерпретации. Значение формулы зависит от оценки только через значения её свободных переменных. Иными словами, если две оценки θ и θ' совпадают на всех параметрах формулы A, то $|A|^D_\theta = |A|^D_{\theta'}$. Это утверждение доказывается индукцией по построению формулы и оправдывает интуитивное представление о том, что связанные переменные — «немые», их имена не важны. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)*

В частности, для замкнутой формулы оценка вообще не нужна: значение $|A|^D$ определено однозначно. Именно поэтому в определении модели (п. 1.2.4) достаточно указать носитель и интерпретацию символов — оценка для предложений несущественна. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)*

Истинность и оценка: пример с кванторами. Покажем, как работают кванторы. Пусть язык содержит двуместный предикатный символ $<$, интерпретируемый как обычное отношение «меньше» на $\mathbb{N}$. Тогда:

$$|\forall x \exists y\,(x < y)|_\theta = \min_{a \in \mathbb{N}} \max_{b \in \mathbb{N}} |a < b| = 1,$$

поскольку для каждого натурального a найдётся большее натуральное b (например, $b = a + 1$). Напротив,

$$|\exists y \forall x\,(x < y)|_\theta = \max_{b \in \mathbb{N}} \min_{a \in \mathbb{N}} |a < b| = 0,$$

поскольку нет наибольшего натурального числа. Этот пример показывает, что порядок кванторов существен: формулы $\forall x \exists y\, A(x, y)$ и $\exists y \forall x\, A(x, y)$ не равносильны. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83; §4, с. 85–87.)*

Интерпретация и формализация. Между формулой и интерпретацией существует двойственная связь. Переход от содержательного математического утверждения к формуле языка называется *формализацией*; обратный переход — от формулы к утверждению о конкретных объектах — называется *интерпретацией*. В логике предикатов эти два процесса разделены более чётко, чем в логике высказываний: одна и та же формула допускает много интерпретаций, и её истинность определяется отдельно в каждой из них. *(Источник: Колмогоров, Драгалин, вводная часть, с. 50–51; гл. II, §3, с. 77–83.)*

Итог пункта. Понятие интерпретации складывается из трёх компонент: носителя D (множества значений переменных), приписывания значений символам сигнатуры (константам — элементы, функциональным символам — функции, предикатным — отношения) и оценки свободных переменных. Значение терма и истинность формулы определяются индуктивно; кванторы интерпретируются как минимум и максимум

по носителю. Одна и та же формула может быть истинной в одной интерпретации и ложной в другой, что делает понятие интерпретации ключевым для семантики логики предикатов и подготавливает определения общезначимости, выполнимости и модели. *(Обобщение по: Колмогоров, Драгалин, гл. II, §3–4, с. 77–87; вводная часть, с. 50–51.)*

1.2.3 Выполнимость и общезначимость формулы первого порядка

Основные источники: Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. II, §5, с. 87–91; вводная часть, §9, с. 50–51; Гл. III, §2, с. 107.

Истинность при интерпретации. Пусть зафиксированы язык первого порядка Ω, его интерпретация на носителе D и оценка переменных $\theta \in \Theta_D$. Напомним (см. п. 1.2.2), что каждой формуле A языка Ω приписывается значение истинности

$$|A|_\theta^D \in \{0, 1\}.$$

Если $|A|_\theta^D = 1$, говорят, что формула A *истинна в данной интерпретации при данной оценке*. Если A замкнута (не содержит свободных переменных), её значение не зависит от θ, и пишут $|A|^D$ или $M \models A$. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)*

Понятия выполнимости, общезначимости и противоречивости получаются из этого базового отношения истинности квантификацией по интерпретациям и оценкам.

Общезначимость. Формула A языка Ω называется *общезначимой* (или *логическим законом*, или *тождеством*), если она истинна при любой интерпретации и любой оценке, то есть

$$|A|_\theta^D = 1$$

для всех носителей D, всех интерпретаций символов Ω на D и всех $\theta \in \Theta_D$. Обозначение: $\models A$. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 87–88; вводная часть, с. 50.)*

Иными словами, общезначимая формула истинна «при всех обстоятельствах»: как бы мы ни выбирали носитель, как бы ни интерпретировали константы, функции и предикаты и как бы ни оценивали свободные переменные, формула будет истинна. Это аналог тавтологии в логике высказываний, но в более богатом языке: к пропозициональным связкам добавляются кванторы, и множество общезначимых формул оказывается значительно шире множества формул, получающихся из тавтологий подстановкой. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 87–88.)*

Противоречие. Формула A называется *противоречием* (или *тождественно ложной*), если она ложна при любой интерпретации и любой оценке:

$$|A|_\theta^D = 0$$

для всех D, всех интерпретаций и всех $\theta \in \Theta_D$. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 87–88.)*

Выполнимость. Формула A называется *выполнимой*, если существует хотя бы одна интерпретация и хотя бы одна оценка, при которых она истинна:

$$\text{существуют } D, \text{ интерпретация и } \theta \text{ такие, что } |A|_\theta^D = 1.$$

(Источник: Колмогоров, Драгалин, гл. II, §5, с. 87–88; вводная часть, с. 50–51.)

Формула называется *опровержимой*, если она ложна хотя бы в одной интерпретации при некоторой оценке. Таким образом, формула опровержима тогда и только тогда, когда она не общезначима, и выполнима тогда и только тогда, когда её отрицание не общезначимо. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 87–88.)*

Четыре категории формул. Каждая формула первого порядка попадает в одну из четырёх категорий:

1) *общезначимые* (тождественно истинные) — истинны во всех интерпретациях;
2) *противоречия* (тождественно ложные) — ложны во всех интерпретациях;
3) *выполнимые, но не общезначимые* — истинны в некоторых интерпретациях и ложны в других;
4) *опровержимые, но не противоречивые* — совпадают с категорией 3.

Иными словами, формула либо общезначима, либо противоречива, либо ни то ни другое (тогда она и выполнима, и опровержима одновременно). *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 87–88.)*

Связь общезначимости и выполнимости. Справедливо следующее простое соотношение:

$$\models A \quad \Longleftrightarrow \quad \neg A \text{ невыполнима.}$$

Аналогично,

$$A \text{ выполнима} \quad \Longleftrightarrow \quad \neg A \text{ не общезначима.}$$

Эти равносильности позволяют сводить задачу проверки общезначимости к задаче проверки невыполнимости отрицания и обратно. *(Источник: Колмогоров, Драгалин, вводная часть, с. 50–51; гл. II, §5, с. 87–88.)*

Примеры общезначимых формул. Приведём важнейшие логические законы языка первого порядка. Знак $\sim$ ниже обозначает логическую эквивалентность: $A \sim B$ означает, что формула $A \equiv B$ (то есть $A \leftrightarrow B$) общезначима. Эквивалентность $A \equiv B$ понимается как сокращение для $(A \supset B) \wedge (B \supset A)$. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 88.)*

Законы де Моргана для кванторов:

$$\neg \forall x\, A(x) \sim \exists x\, \neg A(x), \qquad \neg \exists x\, A(x) \sim \forall x\, \neg A(x).$$

Они утверждают, что отрицание квантора общности эквивалентно квантору существования от отрицания, и наоборот. В конечных моделях это сводится к законам де Моргана для конъюнкции и дизъюнкции: $\neg(A_1 \wedge \cdots \wedge A_n) \sim \neg A_1 \vee \cdots \vee \neg A_n$. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 88.)*

Закон контрапозиции:

$$A \supset B \sim \neg B \supset \neg A.$$

Закон двойного отрицания:

$$\neg\neg A \sim A.$$

Законы пронесения кванторов через логические связки (при условии, что переменная x не входит свободно в формулу A):

$$\begin{gathered}
A \wedge \forall x\, B(x) \sim \forall x\, (A \wedge B(x)),\\
A \vee \forall x\, B(x) \sim \forall x\, (A \vee B(x)),\\
A \wedge \exists x\, B(x) \sim \exists x\, (A \wedge B(x)),\\
A \vee \exists x\, B(x) \sim \exists x\, (A \vee B(x)),\\
\forall x\, B(x) \supset A \sim \exists x\, (B(x) \supset A),\\
A \supset \forall x\, B(x) \sim \forall x\, (A \supset B(x)),\\
A \supset \exists x\, B(x) \sim \exists x\, (A \supset B(x)),\\
\exists x\, B(x) \supset A \sim \forall x\, (B(x) \supset A).
\end{gathered}$$

Законы переименования связанных переменных (при условии, что переменная y не входит свободно в A):

$$\forall x\, A(x) \sim \forall y\, A(y), \qquad \exists x\, A(x) \sim \exists y\, A(y).$$

(Источник: Колмогоров, Драгалин, гл. II, §5, с. 88–91.)

Обратим внимание на условие «переменная x не входит свободно в A». Без него пронесение квантора через связку становится неверным. Например, формула

$$A \vee \forall x\, B(x) \supset \forall x\, (A \vee B(x))$$

общезначима, а обратная импликация

$$\forall x\, (A \vee B(x)) \supset A \vee \forall x\, B(x)$$

таковой не является: если A — это $x = 0$, а $B(x)$ — это $x \neq 0$ на носителе $\mathbb{N}$, то посылка $\forall x\, (x = 0 \vee x \neq 0)$ истинна, а заключение $x = 0 \vee \forall x\, (x \neq 0)$ ложно при $x = 1$. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 88–91.)*

Пример общезначимой формулы с кванторами. Классический пример — закон

$$\forall x\, A(x) \supset \exists x\, A(x)$$

при условии непустоты носителя. Если носитель непуст, а формула $\forall x\, A(x)$ истинна, то $A(a)$ истинно для любого a из носителя; выбирая произвольный a (что возможно, так как носитель непуст), получаем $\exists x\, A(x)$. В логике предикатов с пустыми носителями этот закон перестаёт быть общезначимым, поэтому непустота носителя является частью определения интерпретации. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77; §5, с. 88–91.)*

Пример необщезначимой, но выполнимой формулы. Формула

$$\exists y \forall x\,(x < y)$$

в языке с двуместным предикатным символом $<$ выполнима (она истинна на конечном носителе: достаточно взять носитель $\{0,1\}$ и положить $<^D = \{(0,1)\}$, тогда $y = 1$ и для всех $x \in \{0,1\}$ выполнено $x < y$ при $x = 0$, но не при $x = 1$; нужен другой пример — на носителе $\{0\}$ с пустым отношением $<$ формула $\exists y \forall x\,(x < y)$ истинна, так как $y = 0$ и $\forall x\,(x < 0)$ ложно на пустом множестве x... впрочем, для одноэлементного носителя формула $\exists y \forall x\,(x < y)$ ложна, поскольку $\forall x\,(x < 0)$ требует $0 < 0$, что ложно). Более удачный пример: формула

$$\forall x \exists y\,(x < y)$$

выполнима на $\mathbb{N}$ (для каждого x есть большее y), но не общезначима: на конечном носителе $\{0, 1, \ldots, n\}$ она ложна при $x = n$, так как нет $y > n$. Это показывает, что выполнимость на одном носителе не влечёт общезначимости. *(Источник: Колмогоров, Драгалин, гл. II, §4, с. 85–87; §5, с. 88–91.)*

Замечание о разрешимости. В логике высказываний проблема общезначимости разрешима: существует алгоритм (построение таблицы истинности), который по любой формуле определяет, является ли она тавтологией. В логике предикатов первого порядка ситуация иная: проблема общезначимости *неразрешима* — не существует алгоритма, который по произвольной формуле первого порядка определял бы, общезначима она или нет. Этот результат доказан А. Чёрчем в 1936 г. (теорема Чёрча). *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Тем не менее класс общезначимых формул *рекурсивно перечислим*: существует алгоритм, который перечисляет все общезначимые формулы одну за другой. Этот факт следует из теоремы Гёделя о полноте исчисления предикатов: множество доказуемых формул рекурсивно перечислимо (мы можем перечислять все выводы), а по теореме о полноте оно совпадает с множеством общезначимых формул. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Общезначимость и логическое следование. Понятие общезначимости тесно связано с понятием логического следования. Именно,

$$A_1, \ldots, A_n \models B$$

означает, что во всякой интерпретации, в которой истинны все A_i, истинна и B. Справедливо соотношение

$$A_1, \ldots, A_n \models B \quad \Longleftrightarrow \quad \models (A_1 \wedge \cdots \wedge A_n \supset B).$$

Таким образом, проверка логического следования сводится к проверке общезначимости одной формулы. Это позволяет переносить все результаты об общезначимости на следование и обратно. *(Источник: Колмогоров, Драгалин, гл. II, §5, с. 88–91.)*

Связь с семантикой: теорема о корректности. Всякая выводимая формула исчисления предикатов общезначима:

$$\vdash A \quad \Longrightarrow \quad \models A.$$

Это теорема о корректности исчисления предикатов (иногда называемая теоремой о непротиворечивости относительно семантики). Доказательство проводится индукцией по длине вывода: все аксиомы общезначимы, а правила MP и обобщения сохраняют общезначимость. Обратное утверждение — что всякая общезначимая формула выводима — есть теорема Гёделя о полноте (см. п. 1.2.5). *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 95–96; §2, с. 107.)*

Итог пункта. Понятия выполнимости, общезначимости и противоречивости получаются квантификацией отношения истинности по всем интерпретациям и оценкам. Общезначимая формула истинна всегда, противоречие ложно всегда, выполнимая формула истинна хотя бы в одной интерпретации. Каждая формула попадает в одну из четырёх категорий: общезначимая, противоречие, выполнимая-но-не-общезначимая, опровержимая-но-не-противоречивая. Приведены важнейшие общезначимые формулы: законы де Моргана для кванторов, контрапозиция, двойное отрицание, законы пронесения кванторов через связки, переименование связанных переменных. Проблема общезначимости в логике предикатов неразрешима, но класс общезначимых формул рекурсивно перечислим. Понятие общезначимости связано с логическим следованием: следование сводится к общезначимости импликации. Всякая выводимая формула общезначима (корректность); обратное — теорема Гёделя о полноте. *(Обобщение по: Колмогоров, Драгалин, гл. II, §5, с. 87–91; вводная часть, с. 50–51; гл. III, §2, с. 107.)*

1.2.4 Понятие модели

Основные источники: Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. II, §3–4, с. 77–87; Гл. III, §3, с. 108–111.

Модель языка. Понятие интерпретации, введённое в п. 1.2.2, естественно приводит к понятию модели. *Моделью* языка первого порядка Ω (или *интерпретацией* языка Ω) называется пара

$$M = (D,\, I),$$

где D — носитель (функция, сопоставляющая каждому сорту непустое множество), а I — интерпретация всех символов сигнатуры Ω на носителе D: константам приписаны элементы, функциональным символам — функции, предикатным — отношения. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77–83; §4, с. 83–87.)*

Иными словами, модель — это «мир», в котором «живут» значения переменных и в котором символы языка получают конкретное значение. Одна и та же сигнатура Ω допускает, вообще говоря, много моделей: достаточно по-разному выбрать носитель или по-разному проинтерпретировать хотя бы один символ. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 77–83; §4, с. 83–87.)*

Истинность в модели. Пусть A — формула языка Ω. Если A замкнута (не имеет свободных переменных), то её значение истинности в модели M не зависит от оценки и обозначается $|A|^M \in \{0,1\}$. Говорят, что A *истинна в модели* M, и пишут

$$M \models A,$$

если $|A|^M = 1$. Если $|A|^M = 0$, пишут $M \not\models A$ и говорят, что A *ложна в* M. *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)*

Для формулы A со свободными переменными $x_1, \ldots, x_n$ истинность в модели M определяется при каждой оценке θ: пишут $M \models A[\theta]$, если $|A|^M_\theta = 1$. В дальнейшем, если это не вызывает недоразумений, ограничиваются рассмотрением замкнутых формул (предложений). *(Источник: Колмогоров, Драгалин, гл. II, §3, с. 80–83.)*

Модель формальной теории. Пусть $T = (\Omega, X)$ — формальная аксиоматическая теория: Ω — язык, X — множество нелогических аксиом теории T. *Моделью теории* T называется модель M языка Ω, в которой истинны все формулы из X:

$$M \models A \quad \text{для всех } A \in X.$$

Иными словами, модель теории — это такая интерпретация её языка, в которой все её нелогические аксиомы истинны. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 108–109.)*

Если теория T имеет хотя бы одну модель, то она *непротиворечива*: в ней нельзя вывести одновременно A и $\neg A$ для некоторой формулы A. Это следует из теоремы о корректности (см. ниже). *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 108–109.)*

Теорема о корректности. Справедлива следующая фундаментальная теорема, связывающая синтаксис (выводимость) и семантику (истинность в модели).

Теорема (о корректности). Если M — модель теории T и $T \vdash B$, то $M \models B$. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 109.)*

Иными словами, всякая формула, выводимая в теории T, истинна во всех моделях этой теории. Доказательство проводится индукцией по длине вывода: все логические аксиомы истинны в любой модели, все нелогические аксиомы истинны в модели M по определению, а правила вывода (modus ponens и обобщение) сохраняют истинность. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 109.)*

Следствие: если теория T имеет модель, то она непротиворечива. Действительно, если бы в T выводились A и $\neg A$, то в любой модели M теории T были бы истинны A и $\neg A$, что невозможно. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 109.)*

Примеры моделей. Рассмотрим несколько примеров моделей для знакомых теорий.

1. Стандартная модель арифметики ω. Язык арифметики Ar содержит константу 0, двуместные функциональные символы $+$ и $\cdot$, одноместный S и двуместный предикатный символ $=$. Стандартная модель ω имеет носитель $\mathbb{N}$; константе 0 сопоставлено число 0, символам $+$, $\cdot$, S — сложение, умножение и прибавление единицы, символу $=$ — равенство натуральных чисел. В этой модели истинны все аксиомы Пеано, и она является моделью формальной арифметики Ar. *(Источник: Колмогоров, Драгалин, гл. II, §4, с. 83–85; гл. III, §3, с. 108–109.)*

2. Модель R действительных чисел. Тот же язык Ar можно проинтерпретировать на носителе $\mathbb{R}$: константе 0 сопоставлено $0 \in \mathbb{R}$, символам $+$, $\cdot$, S — сложение, умножение и прибавление единицы в $\mathbb{R}$, символу $=$ — равенство действительных чисел. Эта модель не является моделью арифметики Пеано (аксиома $Sx \neq 0$ в ней не выполняется: $S(-1) = 0$), но является моделью многих других теорий, например теории упорядоченного поля. *(Источник: Колмогоров, Драгалин, гл. II, §4, с. 85–87.)*

3. Модель Z целых чисел. Носитель — $\mathbb{Z}$; S — прибавление единицы, $+, \cdot$ — сложение и умножение целых чисел, $=$ — равенство. Эта модель также не является моделью арифметики Пеано (аксиома $Sx \neq 0$ не выполняется при $x = -1$), но служит моделью теории колец. *(Источник: Колмогоров, Драгалин, гл. II, §4, с. 85–87.)*

4. Модели теории групп. Язык теории групп содержит константу e, двуместный функциональный символ $\cdot$ (умножение) и, возможно, символ $^{-1}$ (обращение). Модель этой теории — любая конкретная группа: носитель — множество элементов группы, e — единица, $\cdot$ — групповая операция, $^{-1}$ — обращение. Моделями являются, например, аддитивная группа $\mathbb{Z}$, группа перестановок, группа матриц и т. д. *(Источник: Колмогоров, Драгалин, гл. I, §5, с. 43–44; гл. III, §3, с. 108–109.)*

5. Модели теории векторных пространств. Язык теории n-мерных векторных пространств содержит константу 0, двуместный функциональный символ $+$ (сложение векторов) и двуместный функциональный символ $\cdot$ (умножение вектора на скаляр). Модель этой теории — любое конкретное n-мерное векторное пространство над полем действительных чисел. Стандартная модель — $\mathbb{R}^n$ с обычными операциями. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 108–109.)*

Нестандартные модели. Понятие модели позволяет обнаружить важное явление: одна и та же теория может иметь неизоморфные модели. Например, арифметика Пеано, кроме стандартной модели ω, имеет так называемые *нестандартные модели*, в которых существуют «бесконечные» натуральные числа, не являющиеся значениями никакого терма $S \ldots S0$. Существование таких моделей было установлено Сколемом в 1930-х гг. и является следствием теорем Гёделя о полноте и о неполноте. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110–111.)*

Таким образом, модель теории не определяется однозначно самой теорией: одна и та же теория может описывать разные «миры», и вопрос о том, какая из моделей «настоящая», лежит вне самой теории. Теория описывает то общее, что есть у всех её моделей, — класс всех её моделей. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110–111.)*

Полнота теории относительно модели. Пусть T — теория в языке Ω, M — её модель. Множество всех предложений языка Ω, истинных в M, обозначается Th_M (теория модели M). Через $[T]$ обозначается множество всех предложений, выводимых в T (логическое замыкание T). *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110.)*

Теория T называется *полной относительно модели* M, если

$$[T] = \mathrm{Th}_M.$$

Иными словами, в T выводимы в точности те предложения, которые истинны в M. Теория T называется *просто полной*, если для любого предложения A языка Ω выполнено $T \vdash A$ или $T \vdash \neg A$. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110.)*

Справедливы следующие утверждения.

1) Если теория T полна относительно некоторой модели M, то T полна.

2) Если теория T полна, то она полна относительно любой своей модели.

(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110.)

Например, элементарная арифметика Ar неполна: существует предложение A, которое не выводится и не опровергается в Ar. Следовательно, Ar не полна ни относительно стандартной модели ω, ни относительно какой-либо другой модели. Напротив, теория действительных чисел R (в языке Ar) полна: она полна относительно стандартной модели R. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110–111.)*

Разрешимость теории. Теория T называется *разрешимой*, если множество $[T]$ рекурсивно, то есть существует алгоритм, позволяющий по любому предложению A языка Ω узнать, выводима ли A в T. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110.)*

Известно, что:

- элементарная арифметика Ar неразрешима (теорема Чёрча);
- теория действительных чисел R разрешима (теорема Тарского, 1948);
- элементарная геометрия плоскости разрешима (также Тарский);
- теория векторных пространств разрешима;
- теория абелевых групп разрешима;
- теория групп неразрешима.

Разрешимость и полнота — разные свойства: полная теория может быть неразрешимой (например, теория Th_ω, полная по определению, неразрешима), а разрешимая теория может быть неполной (например, теория полей). *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 110–111; вводная часть, с. 9–10.)*

Пример: теория R. В книге подробно разбирается теория действительных чисел R. Её язык — тот же, что у Ar; нелогические аксиомы R включают аксиомы равенства, аксиомы поля, аксиомы порядка и аксиому непрерывности (в виде схемы). Эта теория:

- непротиворечива (модель R существует);
- полна относительно стандартной модели R;
- разрешима (теорема Тарского).

Таким образом, теория R — пример полной и разрешимой теории, описывающей достаточно богатую математическую структуру. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 111–114.)*

Связь с теоремой о полноте исчисления предикатов. Понятие модели позволяет сформулировать теорему Гёделя о полноте иначе: для любого множества формул Γ и любой формулы A

$$\Gamma \vdash A \quad \Longleftrightarrow \quad \Gamma \models A,$$

где $\Gamma \models A$ означает, что A истинна во всех моделях, в которых истинны все формулы из Γ. Эквивалентная формулировка: всякое непротиворечивое множество формул имеет модель. Эти формулировки принадлежат А. И. Мальцеву (1936) и являются следствиями теоремы о полноте. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107; ср. Мендельсон, гл. 2, §5.)*

Итог пункта. Модель языка — это носитель вместе с интерпретацией символов сигнатуры; модель теории — это модель языка, в которой истинны все нелогические аксиомы теории. Отношение $M \models A$ (истинность в модели) — центральное понятие семантики. Теорема о корректности утверждает, что всякая выводимая формула истинна во всех моделях. Одна и та же теория может иметь много моделей, в том числе неизоморфных (нестандартные модели арифметики). Теория называется полной относительно модели, если в ней выводимы в точности те предложения, которые истинны в этой модели. Понятие модели позволяет связать синтаксические и семантические свойства теорий, сформулировать теорему Гёделя о полноте в форме «непротиворечивое множество формул имеет модель» и изучать разрешимость и полноту конкретных математических теорий. *(Обобщение по: Колмогоров, Драгалин, гл. II, §3–4, с. 77–87; гл. III, §3, с. 108–111.)*

1.2.5 Теорема о полноте исчисления предикатов первого порядка

Основные источники: Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. III, §2, с. 107 (формулировка); дополнительно для доказательства: Мендельсон Э. Введение в математическую логику. М.: Наука, 1971; Клини С.К. Математическая логика. М.: Мир, 1973; Шенфилд Дж. Математическая логика. М.: Наука, 1975.

Постановка вопроса. В п. 1.2.1 было введено исчисление предикатов первого порядка как формальная аксиоматическая теория: четырнадцать схем аксиом и два правила вывода (modus ponens и обобщение). В п. 1.2.3 было введено понятие общезначимой формулы: формула общезначима, если она истинна во всех интерпретациях при всех оценках. Естественно возникает вопрос о соотношении этих двух понятий: совпадает ли множество доказуемых формул с множеством общезначимых?

Ответ на этот вопрос даёт фундаментальная теорема, доказанная К. Гёделем в 1930 г. для счётного языка и А. И. Мальцевым в 1936 г. в общем случае. Она называется *теоремой о полноте исчисления предикатов первого порядка* (или *теоремой Гёделя о полноте*). *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Формулировка теоремы. Теорема (о полноте исчисления предикатов первого порядка). Для любой формулы A языка первого порядка Ω

$$\vdash A \quad \Longleftrightarrow \quad \models A.$$

Иными словами, формула доказуема в исчислении предикатов тогда и только тогда, когда она общезначима. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107 — теорема сформулирована; доказательство в этой книге не приводится.)*

Теорема утверждает совпадение двух множеств: множества формул, выводимых из аксиом исчисления предикатов, и множества формул, истинных во всех интерпретациях. В этом состоит «полнота» исчисления: его аксиом и правил вывода достаточно, чтобы получить все логические законы, и в то же время оно не выводит ничего лишнего (последнее обеспечивается корректностью, см. п. 1.2.4). *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Две части теоремы. Теорема о полноте естественно распадается на две части.

1. Корректность (непротиворечивость относительно семантики):

$$\vdash A \quad \Longrightarrow \quad \models A.$$

Всякая доказуемая формула общезначима. Эта часть доказывается сравнительно просто индукцией по длине вывода: все аксиомы общезначимы, а правила MP и обобщения сохраняют общезначимость. Это утверждение уже отмечалось в пп. 1.2.1 и 1.2.4. *(Источник: Колмогоров, Драгалин, гл. III, §1, с. 95–96; §3, с. 109.)*

2. Собственно полнота:

$$\models A \quad \Longrightarrow \quad \vdash A.$$

Всякая общезначимая формула доказуема. Именно эта часть составляет основное содержание теоремы и является значительно более трудной: нужно показать, что аксиом и правил вывода достаточно для вывода любой формулы, истинной во всех интерпретациях. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Эквивалентные формулировки. Теорема о полноте имеет несколько эквивалентных формулировок, полезных в разных контекстах.

Формулировка через модели. Для любого множества формул Γ и любой формулы A

$$\Gamma \vdash A \quad \Longleftrightarrow \quad \Gamma \models A,$$

где $\Gamma \models A$ означает, что A истинна во всех моделях, в которых истинны все формулы из Γ. Эта форма теоремы о полноте принадлежит А. И. Мальцеву (1936). *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107; ср. Мендельсон, гл. 2, §5.)*

Формулировка через существование модели. Всякое непротиворечивое множество формул первого порядка имеет модель. Иными словами, если из Γ нельзя вывести противоречие, то существует интерпретация, в которой все формулы из Γ истинны. Эта формулировка часто оказывается наиболее удобной в приложениях. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107; ср. Мендельсон, гл. 2, §5.)*

Эквивалентность трёх формулировок устанавливается стандартными рассуждениями: из $\Gamma \models A$ следует, что $\Gamma \cup \{\neg A\}$ не имеет модели, значит, по второй формулировке, $\Gamma \cup \{\neg A\}$ противоречиво, то есть $\Gamma \vdash A$; обратное аналогично. *(Источник: Мендельсон, гл. 2, §5; Клини, гл. IV, §3–4.)*

Исторический контекст. Теорема о полноте была доказана К. Гёделем в 1930 г. в статье «Die Vollständigkeit der Axiome des logischen Funktionenkalküls» для счётного языка. Обобщение на языки произвольной мощности было получено А. И. Мальцевым в 1936 г. (теорема компактности) и независимо от него Л. Лёвенгеймом и Т. Сколемом в других формулировках. Теорема о полноте вместе с теоремой о неполноте (1931) того же автора составляет фундамент современной математической логики. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107; вводная часть, с. 9–10.)*

Следствия из теоремы о полноте. Следствие 1 (перечислимость логических законов). Множество общезначимых формул первого порядка рекурсивно перечислимо. Действительно, множество доказуемых формул перечислимо (мы можем систематически перечислять все выводы), а по теореме о полноте оно совпадает с множеством общезначимых формул. Это — важный результат теории алгоритмов: хотя проблема общезначимости неразрешима (см. ниже), логические законы можно перечислять один за другим. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Следствие 2 (непротиворечивость исчисления предикатов). Исчисление предикатов непротиворечиво: в нём нельзя вывести одновременно A и $\neg A$ для некоторой формулы A. Действительно, если бы это было возможно, то по корректности (см. п. 1.2.4) в любой модели были бы истинны A и $\neg A$, что невозможно. *(Источник: Колмогоров, Драгалин, гл. III, §3, с. 109.)*

Следствие 3 (компактность). Если всякое конечное подмножество множества формул Γ имеет модель, то и всё Γ имеет модель. Это — теорема компактности Мальцева, важное средство построения нестандартных моделей. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107; ср. Мендельсон, гл. 2, §5.)*

Следствие 4 (связь с теоремой Чёрча). Хотя множество общезначимых формул перечислимо, оно неразрешимо: по теореме Чёрча (1936) не существует алгоритма, который по произвольной формуле первого порядка определял бы, общезначима она или нет. Таким образом, теорема о полноте и теорема Чёрча совместно дают точную характеристику сложности проблемы общезначимости: она рекурсивно перечислима, но неразрешима. *(Источник: Колмогоров, Драгалин, гл. III, §2, с. 107.)*

Связь с теоремой о неполноте. Важно отличать теорему о полноте *исчисления предикатов* от теоремы о неполноте *формальной арифметики* (вторая теорема Гёделя, 1931).

Теорема о полноте утверждает, что в исчислении предикатов доказуемы *все* общезначимые формулы. Это — свойство самого логического исчисления, не зависящее от конкретной содержательной теории.

Теорема о неполноте утверждает, что всякая достаточно богатая формальная теория (например, арифметика Пеано) содержит предложение A, которое в ней не выводимо и не опровержимо. Это — свойство *содержательной теории*, а не логики.

Обе теоремы не противоречат друг другу: исчисление предикатов полно как логическая система, но конкретные теории, построенные на его основе, могут быть неполными. *(Источник: Колмогоров, Драгалин, вводная часть, с. 9–10; гл. III, §3, с. 110–111.)*

Метод доказательства (заглушка). Полное доказательство теоремы о полноте исчисления предикатов первого порядка в настоящем конспекте **не приводится**. Отметим лишь основную идею и укажем литературу.

Доказательство обычно проводится *методом Хенкина*: строится так называемая *каноническая модель*, носителем которой служат классы эквивалентности термов, а истинность формулы определяется через выводимость. Схема рассуждения такова:

1) предполагается, что $\nvdash A$, то есть A не доказуема;

2) тогда множество $\Gamma = \{\neg A\}$ непротиворечиво;

3) множество Γ расширяется до максимального непротиворечивого множества Γ^*, содержащего свидетелей для всех экзистенциальных формул (константы Хенкина);

4) по Γ^* строится каноническая модель M^*, в которой истинны все формулы из Γ^*, в частности $\neg A$;

5) следовательно, A не общезначима: $\not\models A$.

Тем самым доказано, что $\models A \Rightarrow \vdash A$.

Полное доказательство со всеми деталями можно найти в следующих учебниках:

- Мендельсон Э. Введение в математическую логику. М.: Наука, 1971. Гл. 2, §5;
- Клини С.К. Математическая логика. М.: Мир, 1973. Гл. IV, §3–4;
- Шенфилд Дж. Математическая логика. М.: Наука, 1975. Гл. 2, §4;
- Ершов Ю.Л. Математическая логика. М.: Наука, 1975 (и последующие издания);
- Гудстейн Р.Л. Математическая логика. М.: ИЛ, 1961.

(Источник: указание на отсутствие доказательства в основном источнике — Колмогоров, Драгалин, гл. III, §2, с. 107; список дополнительной литературы — там же, с. 120; метод Хенкина описан в: Мендельсон, гл. 2, §5; Клини, гл. IV, §3–4.)

Значение теоремы. Теорема о полноте имеет фундаментальное значение для математической логики и её приложений.

1. Обоснование аксиоматического метода. Теорема показывает, что аксиоматическое исчисление предикатов адекватно семантике: все логические законы, истинные во всех интерпретациях, выводимы из аксиом. Это обосновывает использование аксиоматического метода в математике: если утверждение истинно во всех моделях, его можно доказать чисто формально, не обращаясь к содержательным рассуждениям.

2. Связь синтаксиса и семантики. Теорема устанавливает точное соответствие между синтаксическими (выводимость) и семантическими (общезначимость) понятиями. Это позволяет переходить от одних к другим и использовать наиболее удобные средства в каждом конкретном случае.

3. Метод построения моделей. Формулировка «непротиворечивое множество формул имеет модель» (Мальцев) даёт мощный метод построения моделей: достаточно убедиться в непротиворечивости теории, чтобы заключить о существовании её модели. Это особенно полезно для нестандартных моделей (например, нестандартной модели арифметики).

4. Теорема компактности. Как следствие получается теорема компактности, играющая важную роль в теории моделей: если каждое конечное подмножество аксиом имеет модель, то и всё множество имеет модель. Это позволяет строить модели, описывающие «идеальные» объекты (например, бесконечно большие натуральные числа).

5. Ограничения формальных теорий. Вместе с теоремой о неполноте теорема о полноте очерчивает границы применимости аксиоматического метода: логика полна, но содержательные теории могут быть неполными. Это одна из центральных тем оснований математики XX века. *(Источник: Колмогоров, Драгалин, вводная часть, с. 9–10; гл. III, §2, с. 107; §3, с. 110–111.)*

Итог пункта. Теорема о полноте исчисления предикатов первого порядка утверждает, что формула доказуема тогда и только тогда, когда она общезначима: $\vdash A \Leftrightarrow \models A$. Теорема доказана К. Гёделем (1930) и А. И. Мальцевым (1936). Она распадается на две части: корректность ($\vdash \Rightarrow \models$) и собственно полноту ($\models \Rightarrow \vdash$). Эквивалентные формулировки: $\Gamma \vdash A \Leftrightarrow \Gamma \models A$; всякое непротиворечивое множество формул имеет модель. Следствия: перечислимость логических законов, непротиворечивость исчисления, теорема компактности. Теорема о полноте не противоречит теореме о неполноте: первая говорит о полноте логики, вторая — о неполноте содержательных теорий. Доказательство (метод Хенкина) в конспекте не приводится; оно имеется в учебниках Мендельсона, Клини, Шенфилда, Ершова и Гудстейна. *(Обобщение по: Колмогоров, Драгалин, гл. III, §2, с. 107; §3, с. 110–111; вводная часть, с. 9–10; Мендельсон, гл. 2, §5; Клини, гл. IV, §3–4.)*

1.2.6 Краткое резюме

Исчисление предикатов первого порядка строится как формальная аксиоматическая теория: фиксируется язык первого порядка (сигнатура, термы, формулы), задаются четырнадцать схем аксиом и два правила вывода (modus ponens и обобщение). Понятие вывода и выводимости $\Gamma \vdash A$ определяется стандартным образом; теорема о дедукции позволяет сводить вывод импликации к выводу из гипотезы. Содержательный смысл формулам придаёт интерпретация: носитель, приписывание значений константам, функциям и предикатам, оценка переменных. Формула общезначима, если истинна во всех интерпретациях; выполнима, если истинна хотя бы в одной; противоречива, если ложна во всех. Модель языка (и модели формальной теории) — это носитель вместе с интерпретацией; модель теории — та интерпретация, в которой истинны все её нелогические аксиомы. Теорема Гёделя о полноте устанавливает эквивалентность доказуемости и общезначимости: $\vdash A \Leftrightarrow \models A$. Доказательство теоремы в конспекте опущено; его можно найти в учебниках Мендельсона, Клини, Шенфилда или Ершова. *(Обобщение по: Колмогоров, Драгалин, гл. II, §1–5, с. 52–91; гл. III, §1–3, с. 95–114; указание на отсутствие доказательства теоремы о полноте — с. 107.)*

Источники к разделу:

- Колмогоров А.Н., Драгалин А.Г. Введение в математическую логику. М.: Изд-во Моск. ун-та, 1982. Гл. II, §1–5, с. 52–91; Гл. III, §1–3, с. 95–114.
- Мендельсон Э. Введение в математическую логику. М.: Наука, 1971 (для доказательства теоремы о полноте).

- Клини С.К. Математическая логика. М.: Мир, 1973 (для доказательства теоремы о полноте).

1.3 Элементы универсальной алгебры: отношения, функции, фактормножества, упорядоченные множества и решётки

Основной источник: Кон П.М. Универсальная алгебра. М.: Мир, 1968 (P.M. Cohn, Universal Algebra, 1965). Гл. I, §1–4, с. 13–41; гл. II, §1, §4, с. 55–61, 77–93; гл. V, §2, с. 207–215. Ссылки на PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 21–49, 63–69, 85–101; для §V.2 — по оглавлению, с. 207–215.*

1.3.1 Отношения, соответствия и функции

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. I, §1, с. 13–21; §2, с. 22–24; §3, с. 24–31. PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 21–39.*

Классы, множества и упорядоченные пары. В книге Кона теория множеств строится с различением *классов* и *множеств*: класс — это произвольная совокупность объектов, а множество — это такой класс, который сам может быть элементом другого класса. Такое различение нужно, чтобы избежать парадокса Рассела. В дальнейшем, если не оговорено противное, можно считать, что все рассматриваемые объекты принадлежат фиксированному универсальному множеству U; тогда классы, являющиеся элементами U, суть множества. *(Источник: Кон, гл. I, §1, с. 13–14; PDF с. 21–22.)*

Если A и B — множества, то *упорядоченная пара* (A, B) определяется равенством

$$(A, B) = \big\{\{A\}, \{A, B\}\big\}.$$

При этом $(A, B) = (A', B')$ тогда и только тогда, когда $A = A'$ и $B = B'$. Это определение позволяет рассматривать пару как объект и строить декартово произведение. *(Источник: Кон, гл. I, §1, с. 15; PDF с. 23.)*

Декартовым произведением классов A и B называется класс

$$A \times B = \{(X, Y) \mid X \in A,\ Y \in B\}.$$

Если A и B — множества, то $A \times B$ также является множеством. *(Источник: Кон, гл. I, §1, с. 16; PDF с. 24.)*

Функции. *Функцией из A в B* называется такой подкласс $F \subseteq A \times B$, что для каждого $X \in A$ существует единственный $Y \in B$, для которого $(X, Y) \in F$. Класс A называется *областью определения* функции F, а класс

$$\{Y \in B \mid (X, Y) \in F \text{ для некоторого } X \in A\}$$

называется её *областью значений. (Источник: Кон, гл. I, §1, с. 16; PDF с. 24.)*

Если A — класс, а I — множество, то функция из I в A часто называется *семейством* элементов из A, индексированным множеством I. Если x_i — элемент класса A, соответствующий индексу $i \in I$, то семейство обозначается $(x_i)_{i \in I}$; при этом x_i называется i-й *координатой*, i — *индексом*, а I — *множеством индексов*. В частности, само множество A можно рассматривать как семейство $(a)_{a \in A}$, то есть как тождественную функцию на A. *(Источник: Кон, гл. I, §1, с. 16; PDF с. 24.)*

Соответствия. Пусть A и B — множества. *Соответствием из A в B* называется подмножество декартова произведения $A \times B$. Таким образом, соответствие — это некоторое множество пар (x, y), где $x \in A$, $y \in B$. Соответствия обычно обозначаются прописными греческими буквами. *(Источник: Кон, гл. I, §2, с. 22; PDF с. 30.)*

Если Φ — соответствие из A в B и $A' \subseteq A$, то через $A'\Phi$ обозначается множество

$$A'\Phi = \{y \in B \mid (x, y) \in \Phi \text{ для некоторого } x \in A'\}.$$

В случае $B = A$ соответствие Φ называется *соответствием в A*. Диагональю в A называется соответствие

$$\Delta_A = \{(x, x) \mid x \in A\}.$$

(Источник: Кон, гл. I, §2, с. 22; PDF с. 30.)

Для любого соответствия Φ определяется *обратное соответствие*

$$\Phi^{-1} = \{(x, y) \mid (y, x) \in \Phi\}.$$

Если $\Phi \subseteq A \times B$, то $\Phi^{-1} \subseteq B \times A$. Соответствие Φ в A называется *симметричным*, если $\Phi^{-1} = \Phi$; *антисимметричным*, если $\Phi \cap \Phi^{-1} \subseteq \Delta_A$; *рефлексивным*, если $\Phi \supseteq \Delta_A$. *(Источник: Кон, гл. I, §2, с. 22; PDF с. 30.)*

Для соответствий Φ и Ψ определяется *произведение*

$$\Phi \circ \Psi = \{(x, y) \mid (x, z) \in \Phi \text{ и } (z, y) \in \Psi \text{ для некоторого } z\}.$$

Если $\Phi \subseteq A \times B$ и $\Psi \subseteq C \times D$, то $\Phi \circ \Psi \subseteq A \times D$; в частности, $\Phi \circ \Psi = \varnothing$, если $B \cap C = \varnothing$. *(Источник: Кон, гл. I, §2, с. 22; PDF с. 30.)*

Справедливы тождества

$$\begin{aligned} \Phi \circ (\Psi \circ \Theta) &= (\Phi \circ \Psi) \circ \Theta, \\ (\Phi \circ \Psi)^{-1} &= \Psi^{-1} \circ \Phi^{-1}, \\ (\Phi^{-1})^{-1} &= \Phi. \end{aligned}$$

Кроме того, если Φ — соответствие из A в B, то

$$\Phi \circ \Delta_B = \Delta_A \circ \Phi = \Phi.$$

(Источник: Кон, гл. I, §2, с. 22–23; PDF с. 30–31.)

Типы соответствий. Соответствие Φ в A называется *транзитивным*, если

$$\Phi \circ \Phi \subseteq \Phi.$$

Транзитивное рефлексивное соответствие Φ в A называется *предупорядоченностью* в A. В этом случае Φ^{-1} также является предупорядоченностью; она называется *противоположной* к Φ. *(Источник: Кон, гл. I, §2, с. 23; PDF с. 31.)*

Антисимметричная предупорядоченность в A называется *упорядоченностью* в A (или *частичной упорядоченностью*). Если, кроме того, любые два элемента сравнимы, то есть

$$\Phi \cup \Phi^{-1} = A^2,$$

то упорядоченность называется *линейной*. Множество вместе с заданной на нём упорядоченностью называется *упорядоченным множеством*. *(Источник: Кон, гл. I, §2, с. 23; PDF с. 31.)*

Функции как частный случай соответствий. Соответствие Φ из A в B является функцией из A в B тогда и только тогда, когда

$$\Phi \circ \Phi^{-1} \supseteq \Delta_A, \qquad \Phi^{-1} \circ \Phi \subseteq \Delta_B.$$

Первое условие означает, что для каждого $x \in A$ существует хотя бы один $y \in B$ с $(x, y) \in \Phi$; второе — что этот y единствен. Таким образом, понятие функции согласуется с общим понятием соответствия. *(Источник: Кон, гл. I, §2, с. 23; PDF с. 31.)*

Для функций f и g их произведение $f \circ g$ снова является функцией. Это утверждение сформулировано в предложении 2.1. Заметим, что $f \circ g$ может оказаться пустым множеством, если область значений f не пересекается с областью определения g. Для функций часто пишут fg вместо $f \circ g$, а через xf обозначают единственный элемент y, для которого $(x, y) \in f$. *(Источник: Кон, гл. I, §2, предложение 2.1, с. 23; PDF с. 31.)*

Соответствие Φ из A в B называется *биекцией* (или взаимно однозначным соответствием), если Φ есть функция из A в B и Φ^{-1} есть функция из B в A. Два множества называются *равномощными*, если между ними существует биекция; обозначение $A \leftrightarrow B$. *(Источник: Кон, гл. I, §2, с. 23; PDF с. 31.)*

n-арные отношения. Под *n-арным отношением* (или *n-местным отношением*), где n — положительное число, понимается множество A вместе с подмножеством $\Phi \subseteq A^n$. Часто само Φ называют *отношением в A*. В частности, бинарное отношение ($n = 2$) есть просто соответствие в фиксированном множестве A, а унарное отношение ($n = 1$) есть подмножество множества A. *(Источник: Кон, гл. I, §2, с. 23–24; PDF с. 31–32.)*

Замечание о терминологии и роли понятий. В книге Кона термин «соответствие» используется для произвольного подмножества $A \times B$, а «функция» — для соответствия специального вида. Такое разделение удобно: оно позволяет сначала изучить общие операции над соответствиями (обращение, композицию, диагональ), а затем выделить функции как соответствия, удовлетворяющие условиям однозначности и всюду определённости. В дальнейшем эти понятия используются при определении гомоморфизмов, конгруэнций и факторалгебр: гомоморфизм Ω-алгебр — это функция, согласованная со всеми операциями, а конгруэнция — это эквивалентность, являющаяся подалгеброй квадрата алгебры. *(Источник: Кон, гл. I, §2, с. 22–24; гл. II, §3, с. 71–72; PDF с. 30–32, 79–80.)*

Итог пункта. В п. 1.3.1 введены базовые теоретико-множественные понятия, на которых строится универсальная алгебра: упорядоченная пара, декартово произведение, функция, семейство, соответствие, обратное соответствие, композиция, диагональ, рефлексивность, симметричность, антисимметричность, транзитивность, предупорядоченность, упорядоченность, биекция и n-арное отношение. Показано, что функция является частным случаем соответствия, а бинарное отношение — частным случаем соответствия в фиксированном множестве. Эти понятия служат основой для определения эквивалентностей, фактормножеств, упорядоченных множеств и решёток в следующих пунктах. *(Обобщение по: Кон, гл. I, §1–2, с. 13–24; PDF с. 21–32.)*

1.3.2 Отношение эквивалентности, разбиения и фактормножество

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. I, §3, с. 24–31. PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 32–39.*

Определение эквивалентности. Пусть A — множество. *Эквивалентностью* на A называется соответствие $\mathfrak{q} \subseteq A \times A$, обладающее тремя свойствами:

(1) *рефлексивность*: $\mathfrak{q} \supseteq \Delta_A$, то есть $(x, x) \in \mathfrak{q}$ для всех $x \in A$;

(2) *симметричность*: $\mathfrak{q}^{-1} = \mathfrak{q}$, то есть $(x, y) \in \mathfrak{q}$ влечёт $(y, x) \in \mathfrak{q}$;

(3) *транзитивность*: $\mathfrak{q} \circ \mathfrak{q} \subseteq \mathfrak{q}$, то есть из $(x, y) \in \mathfrak{q}$ и $(y, z) \in \mathfrak{q}$ следует $(x, z) \in \mathfrak{q}$.

Иными словами, эквивалентность — это рефлексивное, симметричное и транзитивное соответствие в A. В книге Кона эквивалентности обозначаются строчными готическими буквами: $\mathfrak{q}, \mathfrak{r}, \mathfrak{s}, \ldots$ *(Источник: Кон, гл. I, §3, с. 27; PDF с. 35.)*

Классы эквивалентности. Если $\mathfrak{q}$ — эквивалентность на A, то для каждого $x \in A$ определяется $\mathfrak{q}$-*класс* элемента x:

$$x^{\mathfrak{q}} = \{y \in A \mid (x, y) \in \mathfrak{q}\}.$$

Иными словами, $x^{\mathfrak{q}}$ — это множество всех элементов A, эквивалентных x по отношению $\mathfrak{q}$. Вместо $(x, y) \in \mathfrak{q}$ часто пишут $x \equiv y \pmod{\mathfrak{q}}$ и говорят, что x и y *сравнимы по модулю* $\mathfrak{q}$ (или *эквивалентны относительно* $\mathfrak{q}$). *(Источник: Кон, гл. I, §3, с. 27; PDF с. 35.)*

Из свойств эквивалентности непосредственно вытекают следующие утверждения:

(1) $x \in x^{\mathfrak{q}}$ для всех $x \in A$ (рефлексивность);

(2) $x \equiv y \pmod{\mathfrak{q}}$ тогда и только тогда, когда $x^{\mathfrak{q}} = y^{\mathfrak{q}}$;

(3) если $x^{\mathfrak{q}} \neq y^{\mathfrak{q}}$, то $x^{\mathfrak{q}} \cap y^{\mathfrak{q}} = \varnothing$.

Таким образом, $\mathfrak{q}$-классы образуют *разбиение* множества A: они попарно не пересекаются, непусты и их объединение совпадает с A. *(Источник: Кон, гл. I, §3, с. 27; PDF с. 35.)*

Разбиения. *Разбиением* множества A называется представление A в виде объединения

$$A = \bigcup_{i \in I} A_i$$

попарно непересекающихся непустых подмножеств A_i. Элементы A_i называются *классами* (или *блоками*) разбиения. Между эквивалентностями на A и разбиениями множества A существует взаимно однозначное соответствие: каждая эквивалентность $\mathfrak{q}$ определяет разбиение на $\mathfrak{q}$-классы, и, обратно, каждое разбиение $A = \bigcup A_i$ определяет эквивалентность

$$(x, y) \in \mathfrak{q} \quad \Longleftrightarrow \quad x, y \in A_i \text{ для некоторого } i.$$

Это утверждение сформулировано в следствии 3.2. *(Источник: Кон, гл. I, §3, следствие 3.2, с. 28; PDF с. 36.)*

Фактормножество. Множество всех $\mathfrak{q}$-классов обозначается $A/\mathfrak{q}$ и называется *фактормножеством* множества A по эквивалентности $\mathfrak{q}$:

$$A/\mathfrak{q} = \{x^{\mathfrak{q}} \mid x \in A\}.$$

Таким образом, элементами фактормножества являются не элементы исходного множества A, а классы эквивалентности. Если $\mathfrak{q}$ — отношение равенства Δ_A, то A/Δ_A находится в естественном взаимно однозначном соответствии с A; если же $\mathfrak{q} = A^2$ — универсальное отношение, то $A/\mathfrak{q}$ состоит из одного элемента. *(Источник: Кон, гл. I, §3, с. 27; PDF с. 35.)*

Естественное отображение. С каждой эквивалентностью $\mathfrak{q}$ на A связано отображение

$$\operatorname{nat}\mathfrak{q}\colon A \to A/\mathfrak{q}, \qquad x \mapsto x^{\mathfrak{q}},$$

которое называется *естественным отображением* (или *отождествлением*), связанным с $\mathfrak{q}$. Из определения ясно, что $\operatorname{nat}\mathfrak{q}$ является отображением на $A/\mathfrak{q}$ (сюръекцией). *(Источник: Кон, гл. I, §3, с. 27; PDF с. 35.)*

Ядро отображения. Пусть $f\colon A \to B$ — произвольное отображение. *Ядром* отображения f называется эквивалентность

$$\ker f = f \circ f^{-1}.$$

Иными словами, $(x, y) \in \ker f$ тогда и только тогда, когда $xf = yf$, то есть когда x и y имеют одинаковый образ при отображении f. Ядро f является эквивалентностью на A (это устанавливается в теореме 3.1). *(Источник: Кон, гл. I, §3, с. 28; PDF с. 36.)*

Теорема о разложении отображения. Справедлива следующая фундаментальная теорема, связывающая произвольное отображение с его ядром и фактормножеством.

Теорема 3.1. Пусть $f\colon A \to B$ — произвольное отображение и $\mathfrak{q} = f \circ f^{-1} = \ker f$. Тогда $\mathfrak{q}$ является эквивалентностью на A, множество Af является подмножеством B, и для f существует разложение

$$f = \varepsilon f' \mu,$$

где $\varepsilon = \operatorname{nat}\mathfrak{q}$ — естественное отображение $A \to A/\mathfrak{q}$, $f'\colon A/\mathfrak{q} \to Af$ — взаимно однозначное соответствие, а $\mu\colon Af \to B$ — вложение. Иными словами, диаграмма

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ \downarrow{\scriptstyle \operatorname{nat}\mathfrak{q}} & & \uparrow{\scriptstyle \mu} \\ A/\mathfrak{q} & \xrightarrow{f'} & Af \end{array}$$

коммутативна. *(Источник: Кон, гл. I, §3, теорема 3.1, с. 28; PDF с. 36.)*

Доказательство. По определению $f \circ f^{-1} \supseteq \Delta_A$, и так как $(f \circ f^{-1})^{-1} = f \circ f^{-1}$, а $f^{-1} \circ f \subseteq \Delta_B$, имеем

$$f \circ f^{-1} \circ f \circ f^{-1} \subseteq f \circ \Delta_B \circ f^{-1} = f \circ f^{-1}.$$

Следовательно, $\mathfrak{q} = f \circ f^{-1}$ — эквивалентность на A. Ясно, что $Af \subseteq B$, а из определения $\mathfrak{q}$ следует, что $(x, y) \in \mathfrak{q}$ тогда и только тогда, когда $xf = yf$, то есть когда $x^{\mathfrak{q}} f' = xf$. Поэтому отображение $f' \colon A/\mathfrak{q} \to Af$, определяемое равенством $x^{\mathfrak{q}} f' = xf$, является взаимно однозначным соответствием. Теперь равенство $f = \varepsilon f' \mu$ непосредственно вытекает из определений, если в качестве ε взять $\operatorname{nat}\mathfrak{q}$, а в качестве μ — вложение $Af \to B$. *(Источник: Кон, гл. I, §3, с. 28; PDF с. 36.)*

Следствие о разбиениях. Из теоремы 3.1 вытекает, что каждое разбиение множества A однозначно определяет некоторую эквивалентность и, обратно, каждая эквивалентность определяет разбиение. Это утверждение сформулировано в следствии 3.2.

Следствие 3.2. Пусть $A = \bigcup A_i$ — разбиение множества A. Тогда существует одна и только одна эквивалентность, которая определяет это разбиение. *(Источник: Кон, гл. I, §3, следствие 3.2, с. 28; PDF с. 36.)*

Доказательство. Определим отображение $f \colon A \to I$, сопоставив каждому $x \in A$ тот единственный индекс $i \in I$, при котором $x \in A_i$. Классами эквивалентности $\ker f$ являются в точности множества A_i; следовательно, $\ker f$ — искомая эквивалентность. Единственность очевидна. *(Источник: Кон, гл. I, §3, с. 28; PDF с. 36.)*

Теорема о фактормножествах. Пусть заданы отображение $f \colon A \to B$ и эквивалентность $\mathfrak{q}$ на A. Если $\mathfrak{q} \subseteq \ker f$, то отображение f можно «профакторизовать» по $\mathfrak{q}$.

Теорема 3.3. Пусть $f \colon A \to B$ — отображение и $\mathfrak{q}$ — эквивалентность на A. Если $\mathfrak{q} \subseteq \ker f$, то существует единственное отображение $\overline{f} \colon A/\mathfrak{q} \to B$, такое, что диаграмма

$$\begin{array}{ccc} A & \xrightarrow{\operatorname{nat}\mathfrak{q}} & A/\mathfrak{q} \\ & \searrow f & \downarrow \overline{f} \\ & & B \end{array}$$

коммутативна. Иными словами, $f = (\operatorname{nat}\mathfrak{q})\overline{f}$. *(Источник: Кон, гл. I, §3, теорема 3.3, с. 29; PDF с. 37.)*

Доказательство. Если отображение $\overline{f}$, удовлетворяющее условию теоремы, существует, то для каждого $x \in A$ должно выполняться равенство

$$(x^{\mathfrak{q}})\overline{f} = xf.$$

Поэтому $\overline{f}$ определено однозначно. С другой стороны, так как $\mathfrak{q} \subseteq \ker f$, из $x^{\mathfrak{q}} = y^{\mathfrak{q}}$ следует $xf = yf$; значит, значение xf зависит только от класса $x^{\mathfrak{q}}$, а не от выбора представителя x. Следовательно, формула $(x^{\mathfrak{q}})\overline{f} = xf$ действительно определяет отображение $\overline{f} \colon A/\mathfrak{q} \to B$, и это отображение единственно. *(Источник: Кон, гл. I, §3, с. 29; PDF с. 37.)*

Вложенные эквивалентности. Пусть $\mathfrak{q}$ и $\mathfrak{r}$ — две эквивалентности на A, причём $\mathfrak{q} \subseteq \mathfrak{r}$. Тогда каждый $\mathfrak{q}$-класс целиком содержится в некотором $\mathfrak{r}$-классе, и можно рассмотреть фактормножество $A/\mathfrak{q}$ и на нём — эквивалентность, индуцированную $\mathfrak{r}$. Это приводит к следующей теореме.

Теорема 3.4. Пусть $\mathfrak{q}, \mathfrak{r}$ — такие эквивалентности на множестве A, что $\mathfrak{q} \subseteq \mathfrak{r}$. Тогда существует единственное отображение $\theta \colon A/\mathfrak{q} \to A/\mathfrak{r}$, при котором

$$(\operatorname{nat}\mathfrak{q})\theta = \operatorname{nat}\mathfrak{r}.$$

Если $\ker\theta$ обозначить через $\mathfrak{r}/\mathfrak{q}$, то $\mathfrak{r}/\mathfrak{q}$ будет эквивалентностью на $A/\mathfrak{q}$, а θ индуцирует взаимно однозначное соответствие

$$\theta' \colon (A/\mathfrak{q})/(\mathfrak{r}/\mathfrak{q}) \to A/\mathfrak{r},$$

так что диаграмма

$$\begin{array}{ccccc} A & \xrightarrow{\operatorname{nat}\mathfrak{q}} & A/\mathfrak{q} & \xrightarrow{\operatorname{nat}(\mathfrak{r}/\mathfrak{q})} & (A/\mathfrak{q})/(\mathfrak{r}/\mathfrak{q}) \\ & \searrow_{\operatorname{nat}\mathfrak{r}} & & & \downarrow \theta' \\ & & A/\mathfrak{r} & \to & A/\mathfrak{r} \end{array}$$

коммутативна. *(Источник: Кон, гл. I, §3, теорема 3.4, с. 30; PDF с. 38.)*

Доказательство. Чтобы получить θ, применим теорему 3.3 к $B = A/\mathfrak{r}$ и $f = \operatorname{nat}\mathfrak{r}$. Поскольку $\operatorname{nat}\mathfrak{q}$ и $\operatorname{nat}\mathfrak{r}$ — отображения на соответствующие фактормножества, отсюда следует, что θ — отображение на $A/\mathfrak{r}$. Если теперь воспользоваться теоремой о разложении 3.1, то получим последнее утверждение теоремы. *(Источник: Кон, гл. I, §3, с. 30; PDF с. 38.)*

Замечание о фактормножествах и частичном порядке. В книге отмечается, что фактормножества данного множества A находятся в двойственном соответствии с некоторыми подмножествами булеана $\mathcal{B}(A)$, но это соответствие не является полным. Например, отношение «B есть подмножество A» транзитивно, тогда как отношение «B есть фактормножество A» не транзитивно. Если через $\mathcal{G}(A)$ обозначить множество фактормножеств A, то $\mathcal{G}(A)$ можно упорядочить, полагая $X \geq Y$, если естественное отображение $A \to X$ содержится в естественном отображении $A \to Y$. Применяя теорему о фактормножествах, можно показать, что в этом случае существует отображение X на Y, которое естественно назвать «естественным». На этом пути получается замена отсутствующей транзитивности. *(Источник: Кон, гл. I, §3, с. 29–30; PDF с. 37–38.)*

Итог пункта. В п. 1.3.2 введены понятия эквивалентности, класса эквивалентности, разбиения и фактормножества. Установлена взаимно однозначная связь между эквивалентностями и разбиениями. Доказана теорема о разложении произвольного отображения в композицию естественного отображения, биекции и вложения (теорема 3.1). Сформулированы теорема о фактормножествах (теорема 3.3) и теорема о вложенных эквивалентностях (теорема 3.4). Эти результаты являются основой для определения конгруэнций и факторалгебр в универсальной алгебре. *(Обобщение по: Кон, гл. I, §3, с. 24–31; PDF с. 32–39.)*

1.3.3 Отношения частичного порядка

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. I, §4, с. 31–41. PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 39–49.*

Упорядоченность. Пусть A — множество. *Упорядоченностью* на A называется бинарное отношение $\leq$, удовлетворяющее трём аксиомам:

О.1. Если $x \leq y$ и $y \leq z$, то $x \leq z$ (транзитивность);
О.2. $x \leq x$ (рефлексивность);
О.3. Если $x \leq y$ и $y \leq x$, то $x = y$ (антисимметричность).

Как обычно, запись $x < y$ означает, что $x \leq y$ и $x \neq y$; запись $x \geq y$ означает $y \leq x$; запись $x > y$ означает $y < x$. Множество A вместе с заданной на нём упорядоченностью называется *упорядоченным множеством. (Источник: Кон, гл. I, §4, с. 31; PDF с. 39.)*

Линейная упорядоченность. Упорядоченность называется *линейной*, если любые два элемента сравнимы:

$$\text{О.4. } x \leq y \text{ или } y \leq x \text{ для всех } x, y \in A.$$

Линейно упорядоченное множество называется также *цепью*. В другом крайнем случае абстрактное множество можно считать *вполне неупорядоченным*: в нём $x \leq y$ истинно лишь при $x = y$, так что никакие два различных элемента не сравнимы. *(Источник: Кон, гл. I, §4, с. 31; PDF с. 39.)*

Подмножества и индуцированная упорядоченность. Если $B \subseteq A$, то упорядоченность на A, ограниченная множеством B, будет упорядоченностью на B. В этом смысле каждое подмножество упорядоченного множества считается упорядоченным множеством. Если эта упорядоченность на B линейна, то B называется *цепью* в A. *(Источник: Кон, гл. I, §4, с. 32; PDF с. 40.)*

Верхние и нижние грани. Пусть B — непустое подмножество упорядоченного множества A. Элемент $a \in A$, обладающий тем свойством, что

$$x \leq a \quad \text{для всех } x \in B,$$

называется *верхней границей* (или *верхней гранью*) множества B в A. Аналогично определяются *нижние грани* множества B: элемент $a \in A$ называется нижней гранью, если $a \leq x$ для всех $x \in B$. *(Источник: Кон, гл. I, §4, с. 32; PDF с. 40.)*

Если у множества B существуют верхние грани, то говорят, что B *ограничено сверху* в A. Упорядоченное множество, в котором каждое конечное подмножество имеет верхнюю грань, называется *направленным вверх*. Индукцией по числу элементов легко показать, что достаточно потребовать, чтобы любая пара элементов имела верхнюю грань. Аналогично определяется множество, *направленное вниз*; если не оговорено противное, то «направленное» всегда означает «направленное вверх». *(Источник: Кон, гл. I, §4, с. 32; PDF с. 40.)*

Наибольший и максимальный элементы. Если упорядоченное множество A само обладает верхней границей a, то есть $x \leq a$ для всех $x \in A$, то элемент a называется *наибольшим* элементом A. Если элемент $a \in A$ таков, что ни один из верхних границ a в A не превосходит a, то a называется *максимальным* элементом A. Иными словами,

$$a < x \quad \text{не имеет места для всех } x \in A.$$

(Источник: Кон, гл. I, §4, с. 32; PDF с. 40.)

Конечно, A может иметь более одного максимального элемента, но если A обладает наибольшим элементом, то он будет единственным максимальным элементом. Обратное, вообще говоря, неверно, однако оно имеет место в цепях и в направленных множествах: максимальный элемент в направленном множестве является также наибольшим. Понятия *минимального* и *наименьшего* элементов определяются аналогично. *(Источник: Кон, гл. I, §4, с. 32; PDF с. 40.)*

Точные грани. Если подмножество B имеет наименьшую верхнюю грань, то она называется *точной верхней гранью* множества B и обозначается $\sup B$. Аналогично, наибольшая нижняя грань, если она существует, называется *точной нижней гранью* и обозначается $\inf B$. Все эти определения полностью применимы и в том случае, когда множество A является лишь предупорядоченным. Точная верхняя грань пары $\{a, b\}$ часто обозначается $a \vee b$, точная нижняя — $a \wedge b$. *(Источник: Кон, гл. I, §4, с. 32; PDF с. 40.)*

Лемма Цорна и аксиома выбора. В книге формулируется следующая аксиома:

А.11 (Лемма Цорна). Непустое упорядоченное множество, в котором каждая цепь обладает верх

Вместо неё часто постулируется *аксиома выбора*: для любого множества A существует такая функция f из $\mathcal{B}'(A) = \mathcal{B}(A) \setminus \{\varnothing\}$ в A, что $Bf \in B$, если только $B \in \mathcal{B}'(A)$. Обычно аксиому А.11 доказывают с помощью аксиомы выбора; но поскольку аксиому выбора можно вывести из А.11, то несущественно, предполагается ли А.11 или аксиома выбора. *(Источник: Кон, гл. I, §4, с. 32–33; PDF с. 40–41.)*

Условие минимальности и полная упорядоченность. Говорят, что упорядоченное множество удовлетворяет *условию минимальности*, если каждое его непустое подмножество обладает минимальным элементом. Если выполняется это условие и, кроме того, число минимальных элементов любого подмножества конечно, то множество называется *частично вполне упорядоченным*; если каждое непустое подмножество обладает единственным минимальным элементом, то множество называется *вполне упорядоченным*. Легко видеть, что частично вполне упорядоченное множество будет вполне упорядоченным тогда и только тогда, когда оно линейно упорядочено. *(Источник: Кон, гл. I, §4, с. 33; PDF с. 41.)*

Обобщённый принцип индукции. Для множеств с условием минимальности имеет место следующее утверждение.

Обобщённый принцип индукции (нетерова индукция). Пусть A — упорядоченное множество с условием минимальности и $B \subseteq A$ — такое подмножество, что оно содержит всякий элемент $a \in A$, как только оно содержит все элементы $x \in A$ с $x < a$. Тогда $B = A$. *(Источник: Кон, гл. I, §4, с. 33; PDF с. 41.)*

Действительно, дополнение B в A не имеет минимального элемента и поэтому должно быть пустым. Частный случай этого принципа, когда A вполне упорядочено, известен как *принцип трансфинитной индукции*. Последний сам является обобщением принципа индукции для натуральных чисел, так как натуральные числа $\mathbb{N}$ образуют вполне упорядоченное множество относительно отношения $\leq$. *(Источник: Кон, гл. I, §4, с. 33; PDF с. 41.)*

Аналогично приведённому выше принципу доказательства по индукции существует *принцип определения по индукции*, который также применяется к произвольным упорядоченным множествам с условием минимальности. *(Источник: Кон, гл. I, §4, с. 33; PDF с. 41.)*

Структуры (решётки). Упорядоченное множество, в котором каждая пара элементов a, b имеет точную верхнюю грань $a \vee b$ и точную нижнюю грань $a \wedge b$, называется *структурой* (в других учебниках — *решёткой*). По индукции получаем, что в

структуре каждое непустое конечное подмножество имеет точную верхнюю и точную нижнюю грань. Если это верно для любого подмножества, то структура называется *полной*. В частности, полная структура L всегда обладает наибольшим элементом ($\inf \varnothing = \sup L$) и наименьшим элементом ($\sup \varnothing = \inf L$). *(Источник: Кон, гл. I, §4, с. 33–34; PDF с. 41–42.)*

Ясно, что всякая структура направлена; кроме того, каждое линейно упорядоченное множество будет структурой (хотя и не обязательно полной). С другой стороны, конечная структура всегда полна. Примером полной структуры является булеан $\mathcal{B}(A)$, где A — произвольное множество. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

Левые и правые отрезки. Пусть A — упорядоченное множество. Подмножество $X \subseteq A$ называется *левым отрезком* множества A, если из $x \in X$ и $y \leq x$ следует $y \in X$. Аналогично X называется *правым отрезком*, если из $x \in X$ и $y \geq x$ следует $y \in X$. Легко проверить, что множество $\mathcal{P}(A)$ всех левых отрезков множества A, упорядоченное по включению, будет полной структурой. Отметим, что любое абстрактное множество A можно считать вполне неупорядоченным; в этом случае $\mathcal{P}(A)$ совпадает с $\mathcal{B}(A)$. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

При проверке того, что данное упорядоченное множество является полной структурой, полезен следующий критерий.

Предложение 4.1. Если A — такое упорядоченное множество, в котором каждое подмножество обладает точной нижней гранью, то A является полной структурой. *(Источник: Кон, гл. I, §4, предложение 4.1, с. 34; PDF с. 42.)*

Доказательство. Рассмотрим $X \subseteq A$ и Y — множество всех верхних граней X в A. Положим $y = \inf Y$. Тогда любой элемент из X будет нижней границей Y и, следовательно, $x \leq y$ для любого $x \in X$; если же $x \leq z$ для любого $x \in X$, то $z \in Y$ и, следовательно, $y \leq z$. Поэтому $y = \sup X$. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

К этому предложению нужно относиться с осторожностью: если, например, A — подмножество полной структуры L и каждое подмножество A имеет точную нижнюю грань в A, то A снова будет полной структурой, но точные верхние грани подмножеств из A будут, вообще говоря, различными в зависимости от того, берутся ли они в A или в L; точнее, для $X \subseteq A$ имеем $\sup_L X \leq \sup_A X$, и равенство выполняется не обязательно. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

Монотонные и структурные гомоморфизмы. Пусть A и B — упорядоченные множества. Отображение $f\colon A \to B$ называется *монотонным гомоморфизмом*, если

$$x \leq y \text{ влечёт } xf \leq yf \text{ для всех } x, y \in A.$$

Если $f\colon A \to B$ — монотонный гомоморфизм, а f^{-1} также является отображением и, более того, монотонным гомоморфизмом (из B в A), то f называется *монотонным изоморфизмом* между A и B; говорят также, что A и B монотонно изоморфны. *(Источник: Кон, гл. I, §4, с. 35; PDF с. 43.)*

Легко видеть, что если A направлено или линейно упорядочено, то его образ при монотонном гомоморфизме обладает тем же свойством. Однако если f — монотонный гомоморфизм между структурами, то точные верхние и нижние грани могут не сохраняться; вообще говоря, равенства

$$(x \vee y)f = xf \vee yf, \qquad (x \wedge y)f = xf \wedge yf \quad \text{для всех } x, y \in A$$

не справедливы. Если же это условие выполняется, то f называется *структурным гомоморфизмом*. Соответственно *структурным изоморфизмом* называется такое взаимно однозначное соответствие f, что оба отображения f и f^{-1} являются структурными гомоморфизмами. Каждый монотонный изоморфизм между структурами на самом деле является структурным изоморфизмом, хотя соответствующее утверждение для гомоморфизмов неверно. *(Источник: Кон, гл. I, §4, с. 35; PDF с. 43.)*

Неподвижные точки монотонных гомоморфизмов. Для полных структур справедливо следующее важное утверждение.

Предложение 4.2. Каждый монотонный гомоморфизм полной структуры в себя имеет неподвижную точку. *(Источник: Кон, гл. I, §4, предложение 4.2, с. 35; PDF с. 43.)*

Доказательство. Пусть L — структура и $f\colon L \to L$ — монотонный гомоморфизм. Положим

$$I = \{x \in L \mid x \leq xf\}, \qquad a = \sup I.$$

Для любого $x \in I$ имеем $x \leq xf$ и $x \leq a$; поэтому $xf \leq af$, откуда $x \leq af$ для всех $x \in I$. Таким образом, af — верхняя грань множества I; по определению a это означает, что

$$a \leq af,$$

откуда следует, что $a \in I$. В силу (1) $af \leq af^2$, то есть $af \in I$. Поэтому $af \leq a$, и вместе с (1) это даёт $af = a$. *(Источник: Кон, гл. I, §4, с. 35; PDF с. 43.)*

Теорема Кантора–Бернштейна–Шрёдера для упорядоченных множеств. Следующая теорема обобщает классическую теорему Шрёдера–Бернштейна на упорядоченные множества.

Теорема 4.3. Пусть A и B — такие упорядоченные множества, что A монотонно изоморфно левому отрезку множества B, а B монотонно изоморфно правому отрезку множества A. Тогда существует взаимно однозначное соответствие $f\colon A \to B$, сохраняющее упорядоченность в следующем ослабленном смысле:

$$x < y \text{ влечёт } xf \neq yf \text{ для всех } x, y \in A.$$

(Источник: Кон, гл. I, §4, теорема 4.3, с. 35–36; PDF с. 43–44.)

Идея доказательства. Пусть $g\colon A \to B_0$ и $h\colon B \to A_0$ — данные монотонные изоморфизмы между множеством A и левым отрезком B_0 множества B и между множеством B и правым отрезком A_0 множества A. Для любого подмножества $X \subseteq A$ обозначим через X' его дополнение в A и введём аналогичные обозначения для подмножеств B. Определим отображение θ множества $\mathcal{P}(A)$ в себя правилом

$$X\theta = ((Xg)'h)'.$$

Так как g и h сохраняют упорядоченность, а переход к дополнению меняет её на обратную, то θ сохраняет упорядоченность, то есть является монотонным гомоморфизмом; поскольку структура $\mathcal{P}(A)$ полна, θ имеет неподвижную точку (предложение 4.2), то есть существует такой левый отрезок $A_1 \subseteq A$, что $((A_1g)'h)' = A_1$. Тогда, полагая $B_1 = A_1g$, получаем $B_1'h = A_1'$. Теперь определим $f\colon A \to B$:

$$xf = \begin{cases} xg, & \text{если } x \in A_1, \\ xh^{-1}, & \text{если } x \in A_1'. \end{cases}$$

Оба отображения g, h взаимно однозначны, поэтому f также взаимно однозначно; проверка условия $x < y \Rightarrow xf \neq yf$ проводится разбором случаев (см. книгу). *(Источник: Кон, гл. I, §4, с. 36; PDF с. 44.)*

Следствие Шрёдера–Бернштейна. Если применить эту теорему к вполне неупорядоченным (то есть абстрактным) множествам, то получим классическую теорему.

Следствие 4.4 (теорема Шрёдера–Бернштейна). Если A и B — произвольные множества, $g\colon A \to B$ и $h\colon B \to A$ — некоторые взаимно однозначные отображения (инъекции), то существует взаимно однозначное соответствие между A и B. *(Источник: Кон, гл. I, §4, следствие 4.4, с. 37; PDF с. 45.)*

Следствие для линейно упорядоченных множеств. Для линейно упорядоченных множеств из условия (2) вытекает, что f — монотонный изоморфизм, и мы получаем следующее утверждение.

Следствие 4.5. Пусть A и B — такие упорядоченные множества, что A монотонно изоморфно левому отрезку множества B, а B монотонно изоморфно правому отрезку множества A. Если хотя бы одно из множеств A или B линейно упорядочено, то A монотонно изоморфно B. *(Источник: Кон, гл. I, §4, следствие 4.5, с. 37; PDF с. 45.)*

Для произвольных упорядоченных множеств следствие 4.5 неверно (см. упр. 7). *(Источник: Кон, гл. I, §4, с. 37; PDF с. 45.)*

Теорема Дилуорса–Глисона. Каждому элементу a упорядоченного множества A можно сопоставить левый отрезок $S_a = \{x \in A \mid x < a\}$. Нетрудно проверить, что отображение $a \mapsto S_a$ будет монотонным гомоморфизмом множества A в $\mathcal{P}(A)$. Так как это отображение взаимно однозначно, отсюда вытекает, что A всегда монотонно изоморфно подмножеству множества $\mathcal{P}(A)$. Следующий результат показывает, что A никогда не может быть изоморфно всему $\mathcal{P}(A)$.

Теорема 4.6. Пусть A — упорядоченное множество и $f\colon A_0 \to \mathcal{P}(A)$ — монотонный гомоморфизм подмножества $A_0 \subseteq A$ в $\mathcal{P}(A)$. Тогда f не является отображением на $\mathcal{P}(A)$. *(Источник: Кон, гл. I, §4, теорема 4.6, с. 37; PDF с. 45.)*

Доказательство. Предположим, что f — отображение на $\mathcal{P}(A)$. Положим $B = \{x \in A_0 \mid x \notin xf\}$, и пусть $\bar{B}$ — левый отрезок, порождённый подмножеством B, то есть множество всех $y \in A$, для которых $y \leq x$ при некотором $x \in B$. По предположению $\bar{B} = bf$ для некоторого $b \in A_0$. Если $b \notin \bar{B}$, то $b \notin bf$, значит, $b \in B \subseteq \bar{B}$ — противоречие. Следовательно, $b \in \bar{B}$, то есть $b \leq x$ для некоторого $x \in B$; значит, $\bar{B} = bf \subseteq xf$. Но $x \in B \subseteq \bar{B} \subseteq xf$, а по определению B это означает, что $x \notin B$, — снова противоречие. *(Источник: Кон, гл. I, §4, с. 37; PDF с. 45.)*

Это доказательство обобщает известный диагональный процесс Кантора, с помощью которого обычно показывают, что A и $\mathcal{B}(A)$ не равномощны. Если считать A вполне неупорядоченным, то любое отображение A (или подмножества A) в $\mathcal{P}(A)$ будет монотонным гомоморфизмом, и мы получаем:

Следствие 4.7. Ни для какого множества A не существует взаимно однозначного соответствия между A и $\mathcal{B}(A)$. *(Источник: Кон, гл. I, §4, следствие 4.7, с. 38; PDF с. 46.)*

Следствие 4.8. Ни для какого упорядоченного множества A не существует монотонного изоморфизма A на $\mathcal{P}(A)$. *(Источник: Кон, гл. I, §4, следствие 4.8, с. 38; PDF с. 46.)*

Графы упорядоченных множеств. В книге вводится представление упорядоченных множеств с помощью ориентированных графов. Ориентированный граф состоит из точек (вершин), рёбер, соединяющих некоторые пары вершин, и ориентации каждого ребра. Коммутативные диаграммы §1.3 являются примерами ориентированного графа. Если T — произвольный ориентированный граф, то множество $V(T)$ его вершин можно сделать предупорядоченным, полагая $a \leq b$ тогда и только тогда, когда можно пройти от a к b вдоль пути в этом графе. Обратно, каждому предупорядоченному множеству A можно сопоставить ориентированный граф $T(A)$, считая его элементы вершинами, а рёбра проводя от a к b, если $a \leq b$. Этот граф называется *графом множества A. (Источник: Кон, гл. I, §4, с. 38; PDF с. 46.)*

Граф называется *связным*, если любые две его вершины можно соединить путём. В произвольном графе T отношение «a связано с b» будет эквивалентностью на $V(T)$; классы этой эквивалентности называются *связными компонентами* графа T. *(Источник: Кон, гл. I, §4, с. 38; PDF с. 46.)*

Теорема редукции Ньюмена. Следующая теорема будет использована в § III.9 при рассмотрении нормальной формы элементов.

Теорема 4.9. Пусть A — предупорядоченное множество, и предположим, что

(i) для каждого $a \in A$ существует такое положительное число $k = k(a)$, что любая убывающая цепь, начинающаяся с a,

$$a = a_0 \geq a_1 \geq \dots,$$

где $a_{i-1} \neq a_i$, имеет не более k членов, и

(ii) если a покрывает b_1 и b_2, то множество $\{b_1, b_2\}$ ограничено снизу в A.

Тогда существует взаимно однозначное соответствие между связными компонентами графа множества A и минимальными элементами этого множества:

$$\Gamma_i \leftrightarrow a_i \qquad (i \in I).$$

Если A_i — множество вершин из Γ_i, то $a_i \in A_i$ и на самом деле a_i является наименьшим элементом множества A_i. *(Источник: Кон, гл. I, §4, теорема 4.9, с. 39; PDF с. 47.)*

Идея доказательства. Из условия (i) следует, что A на самом деле упорядочено (а не только предупорядочено) и что каждое A_i содержит минимальный элемент. Если мы сможем показать, что A_i содержит только один минимальный элемент, то, так как каждый элемент больше или равен некоторому минимальному, отсюда будет следовать, что этот единственный минимальный элемент является наименьшим в A_i. Пусть a, b — минимальные элементы A_i. По определению A_i существует последовательность $a_0 = a, a_1, \dots, a_n = b$, в которой соседние элементы сравнимы. Опуская повторения и вставляя при необходимости дополнительные члены, можно считать, что каждый элемент покрывает предыдущий. Затем применяется двойная индукция по максимуму длин убывающих цепей, начинающихся с a_i, и по числу элементов, на которых этот максимум достигается; условие (ii) позволяет «склеить» две покрывающие цепочки и получить противоречие с индуктивным предположением, если $a \neq b$. *(Источник: Кон, гл. I, §4, с. 39–40; PDF с. 47–48.)*

Итог пункта. В п. 1.3.3 введены понятия упорядоченности, линейной упорядоченности, цепи, верхних и нижних граней, направленных множеств, наибольшего и максимального элементов, точных граней, условия минимальности, частичной и полной упорядоченности. Сформулированы лемма Цорна и аксиома выбора, обобщённый принцип индукции (нетерова индукция) и принцип трансфинитной индукции. Даны определения структуры (решётки), полной структуры, левых и правых отрезков, монотонных и структурных гомоморфизмов. Доказано предложение о неподвижной точке монотонного гомоморфизма полной структуры (предложение 4.2) и теорема Кантора–Бернштейна–Шрёдера для упорядоченных множеств (теорема 4.3) вместе со следствиями. Приведены теорема Дилуорса–Глисона (теорема 4.6) и теорема редукции Ньюмена (теорема 4.9). Эти результаты являются основой для определения решёток, полных решёток и систем замыканий в следующих пунктах. *(Обобщение по: Кон, гл. I, §4, с. 31–41; PDF с. 39–49.)*

1.3.4 Решётки: теоретико-множественное и алгебраическое определения

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. I, §4, с. 33–34; гл. II, §4, с. 77–79. PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 41–42, 85–87.*

Терминологическое замечание. В книге Кона вместо термина «решётка» используется термин *структура*. Поэтому ниже слова «решётка» и «структура» употребляются как синонимы; в цитатах и ссылках сохраняется терминология автора.

Определение через порядок. Пусть L — упорядоченное множество. Оно называется *структурой* (решёткой), если каждая пара элементов $a, b \in L$ имеет точную верхнюю грань $a \vee b$ и точную нижнюю грань $a \wedge b$. Иными словами, для любых $a, b \in L$ существуют элементы $a \vee b$ и $a \wedge b$, удовлетворяющие условиям:

$$\begin{aligned} &a \le a \vee b, \qquad b \le a \vee b, \\ &\text{если } a \le c \text{ и } b \le c, \text{ то } a \vee b \le c; \\ &a \wedge b \le a, \qquad a \wedge b \le b, \\ &\text{если } c \le a \text{ и } c \le b, \text{ то } c \le a \wedge b. \end{aligned}$$

Таким образом, $a \vee b = \sup\{a, b\}$, $a \wedge b = \inf\{a, b\}$. *(Источник: Кон, гл. I, §4, с. 33; PDF с. 42.)*

По индукции получаем, что в структуре каждое непустое конечное подмножество имеет точную верхнюю и точную нижнюю грань. Если это верно для любого подмножества, то структура называется *полной*. В частности, полная структура L всегда обладает наибольшим элементом $\sup L = \inf \varnothing$ и наименьшим элементом $\inf L = \sup \varnothing$. *(Источник: Кон, гл. I, §4, с. 33–34; PDF с. 41–42.)*

Примеры структур.

(1) Всякое линейно упорядоченное множество является структурой: $a \vee b = \max(a, b)$, $a \wedge b = \min(a, b)$. Оно не обязано быть полным (например, $\mathbb{Q}$ с обычным порядком).

(2) Булеан $\mathcal{B}(A)$ произвольного множества A, упорядоченный по включению, является полной структурой: $X \vee Y = X \cup Y$, $X \wedge Y = X \cap Y$. Наибольший элемент — A, наименьший — $\varnothing$.

(3) Множество $\mathcal{P}(A)$ всех левых отрезков упорядоченного множества A, упорядоченное по включению, является полной структурой.

(4) Структура подгрупп группы, структура подмодулей модуля, структура нормальных делителей группы (последняя модулярна, но не дистрибутивна в общем случае).

(Источник: Кон, гл. I, §4, с. 34; гл. II, §4, с. 79–80; PDF с. 42, 87–88.)

Алгебраическое определение. Структуру можно рассматривать как алгебру с двумя бинарными операциями $\vee$ и $\wedge$. Именно, справедливо следующее предложение.

Предложение 4.1. Пусть L — произвольная структура; тогда для любых $a, b, c \in L$ выполняются тождества:

$$\begin{aligned} a \vee (b \vee c) = (a \vee b) \vee c, \quad & a \wedge (b \wedge c) = (a \wedge b) \wedge c && \text{(ассоциативность);} \\ a \vee b = b \vee a, \quad & a \wedge b = b \wedge a && \text{(коммутативность);} \\ a \wedge (a \vee b) = a, \quad & a \vee (a \wedge b) = a && \text{(поглощение).} \end{aligned}$$

Обратно, если L — алгебра с двумя бинарными операторами $\vee$ и $\wedge$, удовлетворяющими этим тождествам, то на L можно определить упорядоченность по правилу

$$a \leq b \quad \Longleftrightarrow \quad a \vee b = b,$$

и относительно этой упорядоченности L будет структурой, причём

$$\sup(a, b) = a \vee b, \qquad \inf(a, b) = a \wedge b.$$

(Источник: Кон, гл. II, §4, предложение 4.1, с. 78; PDF с. 86.)

Доказательство прямой части. Если L — структура в смысле порядка, то $a \vee b$ и $a \wedge b$ определены как точные грани. Ассоциативность и коммутативность следуют из того, что точная верхняя грань множества $\{a, b, c\}$ может быть записана как $a \vee (b \vee c)$ или как $(a \vee b) \vee c$, а точная нижняя — аналогично. Закон поглощения вытекает из того, что $a \leq a \vee b$, поэтому $a \wedge (a \vee b) = a$. Двойственно $a \vee (a \wedge b) = a$. *(Источник: Кон, гл. II, §4, с. 78; PDF с. 86.)*

Доказательство обратной части. Пусть L — алгебра с операциями $\vee, \wedge$, удовлетворяющими законам ассоциативности, коммутативности и поглощения. Определим отношение $\leq$ по правилу $a \leq b \Leftrightarrow a \vee b = b$. Покажем, что это упорядоченность.

(1) *Рефлексивность.* Из закона поглощения $a \wedge (a \vee a) = a$ и коммутативности получаем $a \vee a = a \vee (a \wedge (a \vee a)) = a$, то есть $a \leq a$.

(2) *Антисимметричность.* Если $a \leq b$ и $b \leq a$, то $a \vee b = b$ и $b \vee a = a$; по коммутативности $a = b$.

(3) *Транзитивность.* Если $a \le b$ и $b \le c$, то $a \vee b = b$ и $b \vee c = c$; тогда по ассоциативности

$$a \vee c = a \vee (b \vee c) = (a \vee b) \vee c = b \vee c = c,$$

то есть $a \le c$.

Далее, $a \vee b$ является верхней гранью для $\{a, b\}$, так как $a \vee (a \vee b) = (a \vee a) \vee b = a \vee b$, то есть $a \le a \vee b$; аналогично $b \le a \vee b$. Если c — другая верхняя грань, то $a \vee c = c$ и $b \vee c = c$, откуда

$$(a \vee b) \vee c = a \vee (b \vee c) = a \vee c = c,$$

то есть $a \vee b \le c$. Значит, $a \vee b = \sup\{a, b\}$. Аналогично проверяется, что $a \wedge b = \inf\{a, b\}$ (с использованием двойственных равенств). *(Источник: Кон, гл. II, §4, с. 78; PDF с. 86.)*

Тем самым теоретико-множественное и алгебраическое определения решётки эквивалентны: всякая структура в смысле порядка даёт алгебру с операциями $\vee, \wedge$, удовлетворяющую указанным тождествам, и обратно, всякая такая алгебра определяет структуру в смысле порядка.

Следствие: структура как алгебра. Из предложения 4.1 вытекает, что структуры можно рассматривать как алгебры сигнатуры $\{\vee, \wedge\}$ (обе операции бинарны). Гомоморфизмы таких алгебр — это в точности структурные гомоморфизмы, то есть отображения, сохраняющие обе операции:

$$(x \vee y)f = xf \vee yf, \qquad (x \wedge y)f = xf \wedge yf.$$

Монотонные гомоморфизмы (сохраняющие только порядок) в общем случае не обязаны сохранять $\vee$ и $\wedge$; они образуют более широкий класс отображений. *(Источник: Кон, гл. I, §4, с. 35; гл. II, §4, с. 78–79; PDF с. 43, 86–87.)*

Принцип двойственности. Аксиомы структуры (ассоциативность, коммутативность, поглощение) симметричны относительно замены $\vee \leftrightarrow \wedge$. Поэтому если некоторое утверждение, сформулированное в терминах $\vee, \wedge, \le$, истинно во всех структурах, то двойственное утверждение (получающееся заменой $\vee$ на $\wedge$, $\wedge$ на $\vee$, $\le$ на $\ge$) также истинно во всех структурах. Этот принцип двойственности позволяет сокращать доказательства вдвое. *(Источник: Кон, гл. II, §4, с. 79; PDF с. 87.)*

Итог пункта. В п. 1.3.4 даны два эквивалентных определения решётки (структуры): порядковое (через существование точных граней у каждой пары элементов) и алгебраическое (через операции $\vee, \wedge$, удовлетворяющие законам ассоциативности, коммутативности и поглощения). Доказано предложение 4.1, устанавливающее equivalence этих определений. Показано, что структуры образуют алгебры сигнатуры $\{\vee, \wedge\}$, а их гомоморфизмы — это структурные гомоморфизмы. Отмечен принцип двойственности. Эти результаты служат основой для изучения специальных классов решёток (модулярных, дистрибутивных, булевых, полных) в следующих пунктах. *(Обобщение по: Кон, гл. I, §4, с. 33–34; гл. II, §4, с. 77–79; PDF с. 41–42, 85–87.)*

1.3.5 Свойства решёток

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. II, §4, с. 79–93. PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 87–101.*

Терминологическое замечание. В книге Кона вместо термина «решётка» используется термин *структура*. Поэтому ниже слова «решётка» и «структура» употребляются как синонимы; в ссылках сохраняется терминология автора.

Интервалы и проекции. Пусть L — структура и $a, b \in L$, причём $a \le b$. *Интервалом* с концами a, b называется множество

$$[a, b] = \{x \in L \mid a \le x \le b\}.$$

Такой интервал не обязан быть подструктурой L, но всегда является структурой с наибольшим элементом b и наименьшим элементом a. Каждому интервалу $I = [a, b]$ сопоставляются два отображения $L \to I$:

$$\lambda\colon x \mapsto (x \wedge b) \vee a, \qquad \rho\colon x \mapsto (x \vee a) \wedge b.$$

Эти отображения называются *операторами проектирования*. Нетрудно проверить, что следующие условия эквивалентны:

$$\text{(i) } x \in I; \qquad \text{(ii) } x\lambda = x; \qquad \text{(iii) } x\rho = x.$$

Кроме того,

$$\lambda\rho = \lambda^2 = \lambda, \qquad \rho\lambda = \rho^2 = \rho.$$

Всегда выполняется неравенство

$$(x \wedge b) \vee a \le (x \vee a) \wedge b \quad \text{для всех } x \in L.$$

Если это неравенство обращается в равенство при всех $x \in L$, то интервал I называется *модулярным. (Источник: Кон, гл. II, §4, с. 79; PDF с. 87.)*

Модулярные структуры. Структура L называется *модулярной* (или *дедекиндовой*), если все её интервалы модулярны. Это равносильно выполнению *закона модулярности*:

$$(c \vee a) \wedge b = (c \wedge b) \vee a$$

для всех таких $a, b, c \in L$, что $a \le b$. В силу уже отмеченного неравенства условие модулярности эквивалентно равенству

$$(c \vee a) \wedge b = (c \wedge b) \vee a \quad \text{для всех } a, b, c \in L \text{ с } a \le b.$$

(Источник: Кон, гл. II, §4, с. 79–80; PDF с. 87–88.)

Важнейшим примером модулярной структуры является структура нормальных делителей группы G, упорядоченная по включению. В ней

$$A \wedge B = A \cap B, \qquad A \vee B = AB = \{ab \mid a \in A,\ b \in B\}.$$

Модулярность этой структуры равносильна утверждению

$$CA \cap B \subseteq (C \cap B)A, \quad \text{если } A \subseteq B.$$

Это же рассуждение проходит для групп с унарными операторами; в частности, структура всех подмодулей R-модуля модулярна. *(Источник: Кон, гл. II, §4, с. 80; PDF с. 88.)*

Критерий модулярности. В структуре L с наименьшим элементом 0 и наибольшим элементом 1 элементы x, y называются *дополнительными*, если

$$x \wedge y = 0, \qquad x \vee y = 1.$$

Элемент, дополнительный к x, называется *дополнением* x. Два элемента, обладающие общим дополнением x, называются *x-связанными* (или просто *связанными*). Это отношение симметрично, но, вообще говоря, не транзитивно и даже не рефлексивно. *(Источник: Кон, гл. II, §4, с. 80; PDF с. 88.)*

Справедлив следующий критерий.

Предложение 4.2. Структура L модулярна тогда и только тогда, когда в каждом интервале I структуры L любые два сравнимых и связанных в I элемента равны. *(Источник: Кон, гл. II, §4, предложение 4.2, с. 80; PDF с. 88.)*

Иными словами, немодулярность проявляется в существовании интервала, в котором два различных сравнимых элемента имеют общее дополнение. Эквивалентно, структура немодулярна тогда и только тогда, когда она содержит подструктуру, изоморфную пятиэлементной структуре-«пентагону» N_5. Этот критерий часто используется на практике. *(Источник: Кон, гл. II, §4, с. 80–81; PDF с. 88–89.)*

Дистрибутивные структуры. Структура L называется *дистрибутивной*, если в ней выполняются законы дистрибутивности:

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c), \qquad a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$$

для всех $a, b, c \in L$. В силу принципа двойственности достаточно требовать выполнения одного из этих законов. *(Источник: Кон, гл. II, §4, с. 81–82; PDF с. 89–90.)*

Всякая дистрибутивная структура модулярна. Обратное неверно: существуют модулярные, но не дистрибутивные структуры. Критерий дистрибутивности формулируется следующим образом.

Предложение 4.5. Структура L дистрибутивна тогда и только тогда, когда в каждом интервале I структуры L любые два связанных в I элемента равны. *(Источник: Кон, гл. II, §4, предложение 4.5, с. 82; PDF с. 90.)*

Отсюда вытекает важное следствие.

Следствие 4.6. Структура дистрибутивна тогда и только тогда, когда она модулярна и не содержит подструктуры, изоморфной пятиэлементной структуре, изображённой на рис. 4 (ромб M_3). *(Источник: Кон, гл. II, §4, следствие 4.6, с. 83; PDF с. 91.)*

Таким образом, две классические запрещённые подструктуры — пентагон N_5 и ромб M_3 — характеризуют соответственно модулярность и дистрибутивность: структура модулярна тогда и только тогда, когда она не содержит N_5, и дистрибутивна тогда и только тогда, когда она не содержит ни N_5, ни M_3. *(Источник: Кон, гл. II, §4, с. 80–83; PDF с. 88–91.)*

Перспективные и проективные интервалы. Пусть L — модулярная структура и $a, b \in L$. Между интервалами

$$I = [a \wedge b, a], \qquad J = [b, a \vee b]$$

существует изоморфизм, который задаётся отображениями

$$x \mapsto x \vee b \quad (x \in I), \qquad y \mapsto y \wedge a \quad (y \in J).$$

Эти отображения обратны друг другу и сохраняют упорядоченность; следовательно, они устанавливают структурный изоморфизм между I и J. Два интервала, связанные указанным способом, называются *перспективными*. В более общем случае интервалы I и J называются *проективными*, если существует цепь интервалов

$$I_0 = I,\ I_1,\ \ldots,\ I_n = J,$$

в которой I_{i-1} и I_i перспективны. Так как любые два перспективных интервала изоморфны, любые два проективных интервала также изоморфны. *(Источник: Кон, гл. II, §4, с. 83–84; PDF с. 91–92.)*

Теорема об уплотнениях цепей. Две цепи

$$e = a_0 \le a_1 \le \cdots \le a_m = a, \qquad e = b_0 \le b_1 \le \cdots \le b_n = a$$

между одними и теми же элементами e, a называются *изоморфными*, если $m = n$ и существует такая перестановка π чисел $1, \ldots, n$, что интервал $[a_{i-1}, a_i]$ изоморфен интервалу $[b_{\pi(i)-1}, b_{\pi(i)}]$. Если в цепь вставить дополнительные члены, полученная цепь называется *уплотнением* исходной. *(Источник: Кон, гл. II, §4, с. 84; PDF с. 92.)*

Теорема 4.7 (теорема об уплотнениях цепей). В модулярной структуре любые две цепи между одними и теми же двумя точками обладают изоморфными уплотнениями. *(Источник: Кон, гл. II, §4, теорема 4.7, с. 84; PDF с. 92.)*

Доказательство использует элементы

$$a_{ij} = (a_i \wedge b_j) \vee b_{j-1}, \qquad b_{ji} = (b_j \wedge a_i) \vee a_{i-1},$$

и устанавливает, что интервалы $[b_{j-1,i}, b_{ji}]$ и $[a_{i-1,j}, a_{ij}]$ перспективны, а значит, изоморфны. Это позволяет построить изоморфные уплотнения обеих цепей. *(Источник: Кон, гл. II, §4, с. 84–85; PDF с. 92–93.)*

Теорема Жордана–Гёльдера для структур. Из теоремы об уплотнениях вытекает структурный аналог теоремы Жордана–Гёльдера.

Следствие 4.8. В модулярной структуре конечной длины любую цепь можно уплотнить до максимальной цепи, и любые две максимальные цепи между данными конечными точками имеют одну и ту же длину. *(Источник: Кон, гл. II, §4, следствие 4.8, с. 85; PDF с. 93.)*

Длина и формула длины. Пусть L — модулярная структура конечной длины. Для $a \in L$ через $l(a)$ обозначается длина интервала $[0, a]$. Длина самой структуры L равна $l(1)$. Ясно, что $l(a) \le l(1)$, и равенство выполняется тогда и только тогда, когда $a = 1$. Для любых $a, b \in L$ справедлива формула

$$l(a \vee b) + l(a \wedge b) = l(a) + l(b).$$

(Источник: Кон, гл. II, §4, с. 85; PDF с. 93.)

Независимые множества и прямые объединения. Пусть L — модулярная структура с наименьшим элементом 0. Конечное подмножество $\{a_1, \dots, a_n\}$ структуры L называется *независимым*, если

$$a_i \wedge (a_1 \vee \cdots \vee a_{i-1} \vee a_{i+1} \vee \cdots \vee a_n) = 0 \quad (i = 1, \dots, n).$$

Если элемент a представлен в виде объединения

$$a = a_1 \vee \cdots \vee a_n$$

и элементы $a_1, \dots, a_n$ независимы, то говорят, что a является *прямым объединением* элементов $a_1, \dots, a_n$, и пишут

$$a = a_1 \times \cdots \times a_n.$$

(Источник: Кон, гл. II, §4, с. 86; PDF с. 94.)

Предложение 4.9. В модулярной структуре конечной длины всякое подмножество $\{a_1, \dots, a_n\}$ удовлетворяет неравенству

$$l(a_1 \vee \cdots \vee a_n) \le \sum_{i=1}^{n} l(a_i),$$

причём равенство имеет место тогда и только тогда, когда элементы $a_1, \dots, a_n$ независимы. *(Источник: Кон, гл. II, §4, предложение 4.9, с. 87; PDF с. 95.)*

Лемма о замене сомножителя. Часто используется следующее утверждение.

Лемма 4.10. Если в модулярной структуре с 0 и 1

$$1 = a \times a' \quad \text{и} \quad a \le b,$$

то

$$b = a \times (b \wedge a').$$

(Источник: Кон, гл. II, §4, лемма 4.10, с. 87; PDF с. 95.)

Неразложимые элементы и полные разложения. Элемент a называется *неразложимым*, если $a \neq 0$ и не существует прямого разложения $a = b \times c$, в котором $b \neq a$ и $c \neq a$. Если a представлен в виде прямого объединения неразложимых элементов, то говорят о *полном разложении* элемента a. *(Источник: Кон, гл. II, §4, с. 88; PDF с. 96.)*

Теорема Крулля–Шмидта для структур. Теорема 4.11. Если в модулярной структуре L конечной длины даны два полных разложения наибольшего элемента 1:

$$1 = a_1 \times \cdots \times a_m, \qquad 1 = b_1 \times \cdots \times b_n,$$

то каждое a_i связано с некоторым b_j. *(Источник: Кон, гл. II, §4, теорема 4.11, с. 88; PDF с. 96.)*

Напомним, что элементы a, b называются связанными, если у них есть общее дополнение x, то есть

$$a \wedge x = b \wedge x = 0, \qquad a \vee x = b \vee x = 1.$$

Доказательство теоремы 4.11 использует лемму 4.10 и индукцию по длине структуры; оно разбирает два случая: когда $a_1 \vee b'_j < 1$ для некоторого j и когда $a_1 \vee b'_j = 1$ для всех j. *(Источник: Кон, гл. II, §4, с. 88–90; PDF с. 96–98.)*

Следствие 4.12. Если даны два полных разложения наибольшего элемента 1 в модулярной структуре конечной длины, то $m = n$ и для некоторой перестановки π чисел $1, \dots, n$ интервал $[0, a_i]$ проективен интервалу $[0, b_{\pi(i)}]$. *(Источник: Кон, гл. II, §4, следствие 4.12, с. 90; PDF с. 98.)*

В дистрибутивной структуре связанные элементы равны, поэтому имеет место более сильное утверждение.

Следствие 4.13. В дистрибутивной структуре конечной длины сомножители в полном разложении наибольшего элемента 1 определены однозначно с точностью до их порядка. *(Источник: Кон, гл. II, §4, следствие 4.13, с. 90; PDF с. 98.)*

Неприводимые элементы и теорема Куроша–Оре. Элемент a_i называется *неприводимым*, если его нельзя представить в виде объединения двух элементов, каждый из которых отличен от a_i. Рассматриваются несократимые представления наибольшего элемента 1 в виде объединения неприводимых элементов:

$$1 = p_1 \vee \dots \vee p_r, \qquad 1 = q_1 \vee \dots \vee q_s.$$

Несократимость означает, что ни один из членов нельзя отбросить без потери равенства. *(Источник: Кон, гл. II, §4, с. 90–91; PDF с. 98–99.)*

Теорема 4.14 (теорема Куроша–Оре). Пусть в модулярной структуре L даны два представления наибольшего элемента 1 в виде несократимого объединения неприводимых элементов:

$$1 = p_1 \vee \dots \vee p_r, \qquad 1 = q_1 \vee \dots \vee q_s.$$

Тогда $r = s$ и каждый элемент p_i может быть заменён на некоторый элемент q_j, то есть после подходящей замены уцелевших элементов $q_1, \dots, q_s$ получаем

$$1 = q_1 \vee \dots \vee q_i \vee p_{i+1} \vee \dots \vee p_r \quad (i = 1, \dots, r).$$

(Источник: Кон, гл. II, §4, теорема 4.14, с. 91; PDF с. 99.)

В дистрибутивном случае имеет место сильная единственность.

Предложение 4.15. В дистрибутивной структуре каждое несократимое представление наибольшего элемента 1 в виде объединения неприводимых элементов

$$1 = p_1 \vee \dots \vee p_r$$

единственно с точностью до порядка членов. *(Источник: Кон, гл. II, §4, предложение 4.15, с. 92; PDF с. 100.)*

Дополнения и атомы. В структуре с наименьшим элементом 0 элемент a называется *атомом*, если интервал $[0, a]$ простой. В модулярной структуре, в которой 1 является прямым объединением двух атомов a и b, структура разлагается в прямое произведение интервалов $[0, a]$ и $[0, b]$ тогда и только тогда, когда она дистрибутивна. Эти факты рассматриваются в упражнениях к §II.4 и используются при изучении структур с дополнениями. *(Источник: Кон, гл. II, §4, упражнения 5, 7, с. 92–93; PDF с. 100–101.)*

Итог пункта. В п. 1.3.5 рассмотрены основные свойства решёток (структур): интервалы и операторы проектирования, модулярность и её критерий через связанные элементы, дистрибутивность и её критерий через отсутствие ромба M_3, перспективные и проективные интервалы, теорема об уплотнениях цепей и следствие Жордана–Гёльдера для структур, длина структуры и формула длины, независимые множества и прямые объединения, теорема Крулля–Шмидта для структур, теорема Куроша–Оре и единственность разложения в дистрибутивном случае. Эти результаты являются основой для дальнейшего изучения булевых решёток, полных решёток и систем замыканий. *(Обобщение по: Кон, гл. II, §4, с. 79–93; PDF с. 87–101.)*

1.3.6 Полные решётки и системы замыканий

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. I, §4, с. 34; гл. II, §1, с. 55–61; гл. II, §5, с. 93–99. PDF-файл `universalnaya_algebra_cohn.pdf`*: с. 42, 63–69, 101–107.*

Терминологическое замечание. В книге Кона вместо термина «решётка» используется термин *структура*. Поэтому ниже слова «решётка» и «структура» употребляются как синонимы; в ссылках сохраняется терминология автора.

Определение полной решётки. Структура L называется *полной*, если каждое её подмножество (а не только конечное) имеет точную верхнюю и точную нижнюю грань. Иными словами, для любого $X \subseteq L$ существуют $\sup X$ и $\inf X$ в L. В частности, полная структура L всегда обладает наибольшим элементом $\sup L = \inf \varnothing$ и наименьшим элементом $\inf L = \sup \varnothing$. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

Всякая конечная структура полна, так как в ней каждое подмножество конечно. Однако бесконечная структура может не быть полной: например, множество рациональных чисел $\mathbb{Q}$ с обычным порядком является линейно упорядоченным множеством, но не полной структурой — подмножество $\{q \in \mathbb{Q} \mid q^2 < 2\}$ не имеет точной верхней грани в $\mathbb{Q}$. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

Критерий полноты. Для проверки полноты структуры достаточно проверить существование точных нижних граней у всех подмножеств.

Предложение 4.1. Если A — такое упорядоченное множество, в котором каждое подмножество обладает точной нижней гранью, то A является полной структурой. *(Источник: Кон, гл. I, §4, предложение 4.1, с. 34; PDF с. 42.)*

Доказательство. Рассмотрим $X \subseteq A$ и Y — множество всех верхних граней X в A. Положим $y = \inf Y$. Тогда любой элемент из X будет нижней границей Y и, следовательно, $x \leq y$ для любого $x \in X$; если же $x \leq z$ для любого $x \in X$, то $z \in Y$ и, следовательно, $y \leq z$. Поэтому $y = \sup X$. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

К этому предложению нужно относиться с осторожностью. Если $A \subseteq L$, где L — полная структура, и каждое подмножество A имеет точную нижнюю грань в A, то A снова будет полной структурой, но точные верхние грани подмножеств из A будут, вообще говоря, различными в зависимости от того, берутся ли они в A или в L; точнее, для $X \subseteq A$ имеем $\sup_L X \leq \sup_A X$, и равенство выполняется не обязательно. *(Источник: Кон, гл. I, §4, с. 34; PDF с. 42.)*

Примеры полных решёток.

(1) Булеан $\mathcal{B}(A)$ произвольного множества A, упорядоченный по включению, является полной структурой: $\sup \mathcal{X} = \bigcup \mathcal{X}$, $\inf \mathcal{X} = \bigcap \mathcal{X}$ для любого семейства $\mathcal{X} \subseteq \mathcal{B}(A)$. Наибольший элемент — A, наименьший — $\varnothing$.

(2) Множество $\mathcal{P}(A)$ всех левых отрезков упорядоченного множества A, упорядоченное по включению, является полной структурой. В частности, при вполне неупорядоченном A имеем $\mathcal{P}(A) = \mathcal{B}(A)$.

(3) Структура подгрупп группы G, упорядоченная по включению, является полной: пересечение любого семейства подгрупп есть подгруппа, а точная верхняя грань — подгруппа, порождённая объединением.

(4) Структура подмодулей модуля M над кольцом R полна по той же причине.

(5) Структура нормальных делителей группы G полна: пересечение нормальных делителей нормально, а произведение нормальных делителей даёт нормальный делитель.

(6) Структура идеалов кольца R, упорядоченная по включению, полна.

(Источник: Кон, гл. I, §4, с. 34; гл. II, §4, с. 79–80; гл. II, §5, с. 93–94; PDF с. 42, 87–88, 101–102.)

Системы замыканий. Пусть A — произвольное множество и $\mathcal{B}(A)$ — его булеан, то есть множество всех подмножеств. *Системой замыканий* на A называется система $\mathcal{G} \subseteq \mathcal{B}(A)$, замкнутая относительно произвольных пересечений:

$$\bigcap \mathcal{G}' \in \mathcal{G} \quad \text{для любой подсистемы } \mathcal{G}' \subseteq \mathcal{G}.$$

В частности, взяв $\mathcal{G}' = \varnothing$, получаем, что $A \in \mathcal{G}$ (пересечение пустого семейства по определению равно A). Элементы системы $\mathcal{G}$ называются *замкнутыми множествами* множества A. *(Источник: Кон, гл. II, §1, с. 55; PDF с. 63.)*

Так как система замыканий замкнута относительно произвольных пересечений, из предложения 4.1 (см. п. 1.3.3) следует, что система замыканий является полной структурой относительно упорядоченности по включению. Однако это не обязательно подструктура в $\mathcal{B}(A)$, так как операция объединения в $\mathcal{G}$, вообще говоря, отличается от операции объединения в $\mathcal{B}(A)$. *(Источник: Кон, гл. II, §1, с. 55; PDF с. 63.)*

Примеры систем замыканий.

(1) Система всех подгрупп группы G является системой замыканий: пересечение любого семейства подгрупп есть подгруппа.

(2) Система всех подмодулей модуля над кольцом является системой замыканий.

(3) Система всех идеалов кольца является системой замыканий.

(4) Если X — топологическое пространство и $\mathcal{T}$ — система его замкнутых подмножеств, то $\mathcal{T}$ будет системой замыканий с дополнительным свойством:

$$A \cup B \in \mathcal{T} \quad \text{для любых } A, B \in \mathcal{T}.$$

Такая система замыканий называется *топологической*.

(Источник: Кон, гл. II, §1, с. 56; PDF с. 64.)

Операторы замыкания. *Оператором замыкания* на множестве A называется отображение $J\colon \mathcal{B}(A) \to \mathcal{B}(A)$, обладающее свойствами:

J.1. Если $X \subseteq Y$, то $J(X) \subseteq J(Y)$	(монотонность);
J.2. $X \subseteq J(X)$	(экстенсивность);
J.3. $J(J(X)) = J(X)$	(идемпотентность).

(Источник: Кон, гл. II, §1, с. 56; PDF с. 64.)

Для каждой системы замыканий $\mathcal{G}$ можно определить оператор замыкания J равенством

$$J(X) = \bigcap \{Y \in \mathcal{G} \mid Y \supseteq X\}.$$

Этот оператор удовлетворяет условиям J.1–J.3 по определению. Имеем

$$J(X) = X \quad \text{тогда и только тогда, когда } X \in \mathcal{G}.$$

Обратно, если задан оператор замыкания J, удовлетворяющий J.1–J.3, то система

$$\mathcal{G} = \{X \subseteq A \mid J(X) = X\}$$

является системой замыканий. *(Источник: Кон, гл. II, §1, с. 56; PDF с. 64.)*

Справедлива следующая фундаментальная теорема.

Теорема 1.1. Каждая система замыканий $\mathcal{G}$ на множестве A определяет оператор замыкания J на A по правилу

$$J(X) = \bigcap \{Y \in \mathcal{G} \mid Y \supseteq X\}.$$

Обратно, каждый оператор замыкания J на A определяет систему замыканий

$$\mathcal{G} = \{X \subseteq A \mid J(X) = X\},$$

и так определённое соответствие $\mathcal{G} \leftrightarrow J$ между системами замыканий и операторами замыкания взаимно однозначно. *(Источник: Кон, гл. II, §1, теорема 1.1, с. 57; PDF с. 65.)*

Доказательство. Прямая часть уже установлена. Обратно, пусть J — оператор замыкания. Покажем, что определённое формулой $\mathcal{G} = \{X \mid J(X) = X\}$ семейство является системой замыканий и что соответствующее ему замыкание совпадает с J. Если $(X_i)_{i \in I}$ — произвольное семейство в $\mathcal{G}$ и $X = \bigcap X_i$, то $X \subseteq X_i$, следовательно, по J.1 $J(X) \subseteq J(X_i) = X_i$ для всех i, и потому $J(X) \subseteq X$. Вместе с J.2 это даёт $J(X) = X$, то есть $X \in \mathcal{G}$. Значит, $\mathcal{G}$ — система замыканий. Пусть J' — оператор, определённый системой $\mathcal{G}$ по формуле $J'(X) = \bigcap \{Y \in \mathcal{G} \mid Y \supseteq X\}$. Тогда $J(X) = X$ тогда и только тогда, когда $J'(X) = X$. В силу J.3 $J(J(X)) = J(X)$; поэтому $J'(J(X)) = J(X)$. Но $X \subseteq J(X)$, и, применяя J', получаем $J'(X) \subseteq J'J(X) = J(X)$, а обратное включение следует из соображений симметрии. Значит, $J' = J$. *(Источник: Кон, гл. II, §1, с. 57; PDF с. 65.)*

Системы замыканий как полные решётки. Так как система замыканий замкнута относительно произвольных пересечений, она является полной структурой относительно включения. Точнее, если задано некоторое семейство $(X_i)_{i\in I}$ в $\mathcal{G}$, то множество

$$\bigcap X_i$$

будет наибольшим замкнутым множеством, содержащимся во всех X_i, а

$$\bigcap\{Y \in \mathcal{G} \mid Y \supseteq X_i \text{ для всех } i \in I\}$$

— наименьшим замкнутым множеством, содержащим все X_i. Таким образом, точная нижняя грань в $\mathcal{G}$ совпадает с теоретико-множественным пересечением, а точная верхняя грань — с замыканием объединения. *(Источник: Кон, гл. II, §1, с. 57; PDF с. 65.)*

Алгебраические операторы замыкания. Оператор замыкания J на множестве A называется *алгебраическим*, если для любых $X \subseteq A$ и $a \in A$

$$a \in J(X) \quad \text{влечёт} \quad a \in J(X_f)$$

для некоторого конечного подмножества $X_f \subseteq X$. Система замыканий называется *алгебраической*, если соответствующий ей оператор замыкания алгебраический. *(Источник: Кон, гл. II, §1, с. 59; PDF с. 67.)*

Индуктивные системы замыканий. Непустая система $\mathcal{G}$ подмножеств множества A называется *индуктивной*, если каждая цепь в $\mathcal{G}$ обладает точной верхней гранью в $\mathcal{G}$. В силу предложения 1.5.9 слово «цепь» здесь можно заменить словами «направленное множество». Справедлива следующая характеризация алгебраических систем замыканий, принадлежащая Ю. Шмидту.

Теорема 1.2. Система замыканий является алгебраической тогда и только тогда, когда она индуктивна. *(Источник: Кон, гл. II, §1, теорема 1.2, с. 59; PDF с. 67.)*

Доказательство. Пусть $\mathcal{G}$ — алгебраическая система замыканий на множестве A, $\mathcal{C}$ — цепь в $\mathcal{G}$ и $K = \sup\mathcal{C}$. Для доказательства включения $K \in \mathcal{G}$ достаточно проверить, что $J(K_f) \subseteq K$ для каждого конечного $K_f \subseteq K$. Пусть $K_f = \{x_1, \ldots, x_n\}$; каждое x_i принадлежит некоторому члену цепи $\mathcal{C}$, и так как $\mathcal{C}$ — цепь, можно найти член $L \in \mathcal{C}$, содержащий все x_i. Тогда $K_f \subseteq L \subseteq K$ и $L \in \mathcal{G}$; следовательно, $J(K_f) \subseteq J(L) = L \subseteq K$, то есть $J(K_f) \subseteq K$. Обратно, пусть $\mathcal{G}$ — индуктивная система замыканий на A и J — соответствующий оператор замыкания. Нужно показать, что для любого $X \subseteq A$

$$J(X) = \sup\{J(X_f) \mid X_f \subseteq X,\ X_f \text{ конечно}\}.$$

Положим $\mathcal{C} = \{J(X_f) \mid X_f \subseteq X,\ X_f \text{ конечно}\}$. Для любых $Y, Z \in \mathcal{C}$ имеем $J(Y) \cup J(Z) \subseteq J(Y \cup Z)$, и если Y, Z — конечные подмножества X, то $Y \cup Z$ также конечно; это показывает, что $\mathcal{C}$ направлено. Следовательно, $\sup\mathcal{C} \in \mathcal{C}$, откуда вытекает требуемое равенство. *(Источник: Кон, гл. II, §1, с. 59–60; PDF с. 67–68.)*

Следствие о направленных подсистемах. Из теоремы 1.2 и предложения 1.5.9 вытекает:

Следствие 1.3. Если $\mathcal{G}$ — алгебраическая система замыканий на A и $\mathcal{C}$ — направленная подсистема системы $\mathcal{G}$, то $\sup\mathcal{C} \in \mathcal{G}$. *(Источник: Кон, гл. II, §1, следствие 1.3, с. 60; PDF с. 68.)*

Теорема о максимальном элементе. Из леммы Цорна вытекает, что каждая непустая индуктивная система подмножеств множества A содержит максимальное подмножество. Это приводит к следующей теореме, в которой содержатся важнейшие применения леммы Цорна к алгебре.

Теорема 1.4. Пусть $\mathcal{G}$ — алгебраическая система замыканий в A, и пусть A_0, A_1, B — такие подмножества A, что $B \in \mathcal{G}$ и $B \cap A_1 = A_0$. Тогда $\mathcal{G}$ содержит элемент C, который является максимальным в $\mathcal{G}$ относительно свойств $C \subseteq B$, $C \cap A_1 = A_0$. *(Источник: Кон, гл. II, §1, теорема 1.4, с. 60; PDF с. 68.)*

Доказательство. Возьмём систему $\mathcal{G}'$ всех таких $X \in \mathcal{G}$, что $X \subseteq B$ и $X \cap A_1 = A_0$, и покажем, что $\mathcal{G}'$ обладает максимальным элементом. Во-первых, $\mathcal{G}' \neq \varnothing$, так как $B \in \mathcal{G}'$. Пусть (X_i) — некоторая цепь в $\mathcal{G}'$ и $X = \sup X_i$. Тогда $X \in \mathcal{G}$, так как система $\mathcal{G}$ индуктивна. Далее, $X \subseteq B$ и $X \cap A_1 = A_0$; поэтому $X \in \mathcal{G}'$. Таким образом, система $\mathcal{G}'$ индуктивна, и по лемме Цорна $\mathcal{G}'$ обладает максимальным элементом. *(Источник: Кон, гл. II, §1, с. 60; PDF с. 68.)*

Связь с алгебрами: системы подалгебр. Пусть A — некоторая Ω-алгебра и $\mathcal{B}_\Omega(A)$ — множество всех её подалгебр. Тогда $\mathcal{B}_\Omega(A)$ является системой замыканий на A и, следовательно, полной структурой. Действительно, если $(A_\lambda)_{\lambda \in \Lambda}$ — семейство подалгебр алгебры A, то для каждого $\omega \in \Omega$ каждая подалгебра A_λ замкнута относительно ω, и поэтому то же самое верно для $\bigcap A_\lambda$. Соответствующий оператор замыкания обозначается J_Ω; таким образом, $J_\Omega(X)$ — пересечение всех подалгебр, содержащих X. *(Источник: Кон, гл. II, §5, с. 93–94; PDF с. 101–102.)*

Явное построение $J_\Omega(X)$ даётся в предложении 5.1: если $X_0 = X$ и

$$X_{k+1} = \{x \in A \mid x \in X_k \text{ или } x = a_1 \cdots a_n \omega \text{ для некоторого } \omega \in \Omega(n),\ a_i \in X_k\},$$

то $\bigcup_{k=0}^{\infty} X_k = J_\Omega(X)$. *(Источник: Кон, гл. II, §5, предложение 5.1, с. 94; PDF с. 102.)*

Из этого построения вытекает, что оператор J_Ω является алгебраическим. Действительно, если $a \in J_\Omega(X)$, то $a \in X_k$ для некоторого k; выбирая наименьшее такое k и применяя индукцию, получаем, что $a \in J_\Omega(X')$ для некоторого конечного $X' \subseteq X$. Обратно, если $\mathcal{G}$ — произвольная алгебраическая система замыканий на множестве A, то для подходящей области операторов Ω существует такая структура Ω-алгебры на A, что $\mathcal{B}_\Omega(A) = \mathcal{G}$. *(Источник: Кон, гл. II, §5, теорема 5.2, с. 95; PDF с. 103.)*

Теорема 5.2. Система $\mathcal{B}_\Omega(A)$ подалгебр Ω-алгебры A является алгебраической системой замыканий. Обратно, если дана алгебраическая система замыканий $\mathcal{G}$ на множестве A, то для подходящей области операторов Ω можно определить такую структуру Ω-алгебры на A, что $\mathcal{B}_\Omega(A) = \mathcal{G}$. *(Источник: Кон, гл. II, §5, теорема 5.2, с. 95; PDF с. 103.)*

Топологические системы замыканий. Система замыканий $\mathcal{G}$ на A называется *топологической*, если она замкнута относительно конечных объединений:

$$A \cup B \in \mathcal{G} \quad \text{для любых } A, B \in \mathcal{G}.$$

Такая система соответствует топологическому пространству: её элементы — замкнутые множества. Топологическая система замыканий является подструктурой булеана $\mathcal{B}(A)$, но не обязательно полной подструктурой: объединение бесконечного семейства замкнутых множеств может не быть замкнутым. *(Источник: Кон, гл. II, §1, с. 56; PDF с. 64.)*

Справедлива следующая характеризация.

Теорема 5.6. Пусть $\mathcal{G}$ — алгебраическая система замыканий на множестве A. Тогда следующие три условия эквивалентны:

(i) $\mathcal{G}$ — подструктура структуры $\mathcal{B}(A)$;

(ii) $\mathcal{G}$ — топологическая система замыканий;

(iii) существует такая область операторов Ω, арность операторов которой не больше единицы (то есть все они унарны или 0-арны), и такая структура Ω-алгебры на A, что $\mathcal{B}_\Omega(A) = \mathcal{G}$.

(Источник: Кон, гл. II, §5, теорема 5.6, с. 98; PDF с. 106.)

Системы конечного характера. Система $\mathcal{G}$ подмножеств множества A называется *системой конечного характера*, если существует такая система $\mathcal{F}$ конечных подмножеств множества A, что каждое $X \in \mathcal{G}$ однозначно определяется своими пересечениями $F \cap X$ для всех $F \in \mathcal{F}$. Оказывается, что система замыканий алгебраична тогда и только тогда, когда она является системой конечного характера. Это даёт ещё один способ распознавать алгебраические системы замыканий. *(Источник: Кон, гл. II, §1, упражнение 9, с. 61; PDF с. 69.)*

Соответствия Галуа. Системы замыканий тесно связаны с соответствиями Галуа. Пусть A и B — множества и $\Phi \subseteq A \times B$ — соответствие. Для $X \subseteq A$ и $Y \subseteq B$ определим

$$X^* = \{y \in B \mid (x, y) \in \Phi \text{ для всех } x \in X\}, \qquad Y^* = \{x \in A \mid (x, y) \in \Phi \text{ для всех } y \in Y\}.$$

Отображения $X \mapsto X^{**}$ и $Y \mapsto Y^{**}$ являются операторами замыкания на A и B соответственно, а отображения $X \mapsto X^*$, $Y \mapsto Y^*$ устанавливают взаимно однозначное соответствие между соответствующими системами замыканий. Такая пара отображений называется *соответствием Галуа. (Источник: Кон, гл. II, §1, с. 58–59; PDF с. 66–67.)*

Итог пункта. В п. 1.3.6 рассмотрены полные решётки (структуры, в которых каждое подмножество имеет точные грани), критерий полноты (достаточно существования точных нижних граней), примеры полных решёток (булеан, левые отрезки, подгруппы, подмодули, нормальные делители). Введены системы замыканий, операторы замыкания и доказана теорема 1.1 о взаимно однозначном соответствии между ними. Рассмотрены алгебраические операторы замыкания и их характеризация через индуктивность (теорема 1.2). Доказана теорема 1.4 о максимальном элементе в алгебраической системе замыканий. Показано, что система подалгебр Ω-алгебры является алгебраической системой замыканий и, обратно, всякая алгебраическая система замыканий реализуется как система подалгебр подходящей Ω-алгебры (теорема 5.2). Рассмотрены топологические системы замыканий, системы конечного характера и соответствия Галуа. *(Обобщение по: Кон, гл. I, §4, с. 34; гл. II, §1, с. 55–61; гл. II, §5, с. 93–99; PDF с. 42, 63–69, 101–107.)*

1.3.7 Булевы решётки

Основные источники: Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. V, §2, с. 207–215 (по оглавлению); гл. II, §4, с. 92; гл. I, §4, с. 34. PDF-файл `universalnaya_algebra_cohn.p` *оглавление (с. 1–2 PDF), предметный указатель (с. 2–12 PDF); страницы §V.2 в предоставленном фрагменте не имеют текстового слоя.*

Терминологическое замечание. В книге Кона вместо термина «булева решётка» используется термин *булева алгебра*. В предметном указателе книги присутствуют термины «булева алгебра» (с. 208), «полная булева алгебра» (с. 210), «структура с дополнениями» (с. 92, 207), «тождества де Моргана» (с. 210). Поэтому ниже слова «булева решётка» и «булева алгебра» употребляются как синонимы; в ссылках сохраняется терминология автора.

Структуры с дополнениями. Напомним (см. п. 1.3.5), что в структуре L с наименьшим элементом 0 и наибольшим элементом 1 элементы x, y называются *дополнительными*, если

$$x \wedge y = 0, \qquad x \vee y = 1.$$

Элемент, дополнительный к x, называется *дополнением* x. Структура, в которой каждый элемент обладает хотя бы одним дополнением, называется *структурой с дополнениями. (Источник: Кон, гл. II, §4, с. 92; гл. V, §2, с. 207; PDF с. 100 и оглавление.)*

Определение булевой алгебры. *Булевой алгеброй* называется дистрибутивная структура с дополнениями, то есть структура B с наименьшим элементом 0 и наибольшим элементом 1, в которой:

(i) выполняются законы дистрибутивности

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c), \qquad a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c);$$

(ii) для каждого $a \in B$ существует элемент a', называемый *дополнением* a, такой, что

$$a \wedge a' = 0, \qquad a \vee a' = 1.$$

В силу дистрибутивности дополнение каждого элемента единственно: если a' и a'' — два дополнения a, то

$$a' = a' \wedge 1 = a' \wedge (a \vee a'') = (a' \wedge a) \vee (a' \wedge a'') = 0 \vee (a' \wedge a'') = a' \wedge a'',$$

и аналогично $a'' = a' \wedge a''$, откуда $a' = a''$. *(Источник: Кон, гл. V, §2, с. 207–208; гл. II, §4, с. 92; PDF: оглавление и с. 100.)*

Аксиомы булевой алгебры. Эквивалентно, булеву алгебру можно определить как алгебру сигнатуры $\{\vee, \wedge, ', 0, 1\}$ с двумя бинарными операциями $\vee, \wedge$, одной унар-

ной операцией $'$ и двумя 0-арными операциями $0, 1$, удовлетворяющую тождествам:

$a \vee (b \vee c) = (a \vee b) \vee c$,	$a \wedge (b \wedge c) = (a \wedge b) \wedge c$	(ассоциативность);
$a \vee b = b \vee a$,	$a \wedge b = b \wedge a$	(коммутативность);
$a \vee (a \wedge b) = a$,	$a \wedge (a \vee b) = a$	(поглощение);
$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$,	$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$	(дистрибутивность);
$a \vee a' = 1$,	$a \wedge a' = 0$	(дополнение);
$a \vee 0 = a$,	$a \wedge 1 = a$	(нейтральные элементы).

Такое алгебраическое определение удобно тем, что позволяет рассматривать булевы алгебры как объекты категории Ω-алгебр и применять к ним общие теоремы о многообразиях. *(Источник: Кон, гл. V, §2, с. 208; PDF: оглавление.)*

Примеры булевых алгебр.

(1) *Двухэлементная булева алгебра* $\mathbf{2} = \{0, 1\}$ с обычными операциями $\vee = \max$, $\wedge = \min$, $0' = 1$, $1' = 0$. Это наименьшая нетривиальная булева алгебра; она играет роль, аналогичную роли поля из двух элементов в теории полей.

(2) *Булеан* $\mathcal{B}(A)$ произвольного множества A с операциями $\cup, \cap$, дополнением $X' = A \setminus X$, $0 = \varnothing$, $1 = A$. Это классический пример булевой алгебры; в некотором смысле он является «универсальным» (см. теорему Стоуна ниже).

(3) *Алгебра Линденбаума* пропозициональной логики: классы эквивалентности формул по отношению логической эквивалентности с операциями, индуцированными $\vee, \wedge, \neg$. Эта алгебра является булевой и служит основой алгебраической семантики классической логики высказываний.

(4) *Алгебра множеств*: любое поле подмножеств множества A, то есть семейство $\mathcal{F} \subseteq \mathcal{B}(A)$, замкнутое относительно конечных объединений, пересечений и дополнений в A.

(5) *Алгебра регулярных открытых множеств* топологического пространства X: регулярные открытые множества образуют булеву алгебру, но не являются подалгеброй булеана X.

(Источник: Кон, гл. V, §2, с. 208–210; PDF: оглавление; общие сведения о примерах булевых алгебр.)

Законы де Моргана. В булевой алгебре выполняются *тождества де Моргана*:

$$(a \vee b)' = a' \wedge b', \qquad (a \wedge b)' = a' \vee b'.$$

Кроме того, справедливы тождества

$$a'' = a, \qquad 0' = 1, \qquad 1' = 0, \qquad a \vee 1 = 1, \qquad a \wedge 0 = 0.$$

Эти законы вместе с аксиомами булевой алгебры позволяют проводить равносильные преобразования булевых термов, аналогичные преобразованиям формул алгебры логики (см. п. 1.1.1). *(Источник: Кон, гл. V, §2, с. 210; предметный указатель, с. 2 PDF: «Тождества де Моргана 210».)*

Связь с булевыми кольцами. Булеву алгебру можно эквивалентно определить как *булево кольцо* — ассоциативное кольцо с единицей, в котором каждый элемент идемпотентен: $x^2 = x$. Переход между этими определениями задаётся равенствами

$$a + b = (a \wedge b') \vee (a' \wedge b), \qquad a \cdot b = a \wedge b, \qquad a' = 1 + a,$$

где в левой части операции понимаются в кольце, а в правой — в булевой алгебре. Таким образом, категория булевых алгебр эквивалентна категории булевых колец; это один из классических примеров эквивалентности категорий. *(Источник: Кон, гл. V, §2, с. 208–209; PDF: оглавление; стандартный факт теории булевых алгебр.)*

Полные булевы алгебры. Булева алгебра B называется *полной*, если она полна как структура, то есть каждое подмножество $X \subseteq B$ имеет точную верхнюю и точную нижнюю грань. В полной булевой алгебре для любого семейства $\{a_i\}_{i \in I}$ определены $\bigvee_i a_i$ и $\bigwedge_i a_i$. Примером полной булевой алгебры является булеан $\mathcal{B}(A)$; напротив, алгебра Линденбаума пропозициональной логики, вообще говоря, не полна. *(Источник: Кон, гл. V, §2, с. 210; предметный указатель, с. 2 PDF: «Полная булева алгебра 210».)*

В полной булевой алгебре выполняются обобщённые законы де Моргана:

$$\Big(\bigvee_{i \in I} a_i\Big)' = \bigwedge_{i \in I} a_i', \qquad \Big(\bigwedge_{i \in I} a_i\Big)' = \bigvee_{i \in I} a_i',$$

а также обобщённые законы дистрибутивности:

$$a \wedge \Big(\bigvee_{i \in I} b_i\Big) = \bigvee_{i \in I} (a \wedge b_i), \qquad a \vee \Big(\bigwedge_{i \in I} b_i\Big) = \bigwedge_{i \in I} (a \vee b_i).$$

Эти законы важны в теории меры, топологии и квантовой логики. *(Источник: Кон, гл. V, §2, с. 210; PDF: оглавление.)*

Нормальные формы. Аналогично тому, как в алгебре логики вводятся ДНФ и КНФ (см. п. 1.1.3), в произвольной булевой алгебре можно определить *конъюнктивную* и *дизъюнктивную нормальные формы*. В предметном указателе книги присутствуют термины «конъюнктивная нормальная форма» (с. 210) и «нормальная форма» (с. 210). В общем случае нормальная форма строится по конечному множеству образующих $a_1, \dots, a_n$ булевой алгебры: элементы вида

$$a_1^{\varepsilon_1} \wedge \dots \wedge a_n^{\varepsilon_n}, \qquad \varepsilon_i \in \{0, 1\},$$

где $a_i^1 = a_i$, $a_i^0 = a_i'$, называются *минтермами* (или *конституентами единицы*), а их дизъюнкции — дизъюнктивными нормальными формами. Аналогично определяются макстермы и конъюнктивные нормальные формы. В отличие от алгебры логики, где каждая булева функция имеет единственную СДНФ, в произвольной булевой алгебре элементы могут не иметь единственного представления в виде нормальной формы; однако в свободной булевой алгебре с n образующими нормальные формы дают единственное представление. *(Источник: Кон, гл. V, §2, с. 210; предметный указатель, с. 2 PDF: «Конъюнктивная нормальная форма 210», «Нормальная форма 210».)*

Идеалы и фильтры. Пусть B — булева алгебра. *Идеалом* называется непустое подмножество $I \subseteq B$, замкнутое относительно конечных объединений и удовлетворяющее условию: если $a \in I$ и $b \leq a$, то $b \in I$. Иными словами, идеал — это левый отрезок, замкнутый относительно $\vee$. *Фильтром* называется непустое подмножество $F \subseteq B$, замкнутое относительно конечных пересечений и удовлетворяющее условию: если $a \in F$ и $b \geq a$, то $b \in F$. Идеалы и фильтры двойственны друг другу относительно операции дополнения: I — идеал тогда и только тогда, когда $I' = \{a' \mid a \in I\}$ — фильтр. *(Источник: Кон, гл. V, §2, с. 212; предметный указатель, с. 2 PDF: «Идеал 212», «Фильтр 212».)*

Идеал I называется *собственным*, если $I \neq B$; *простым*, если I собственный и для любых $a, b \in B$ из $a \wedge b \in I$ следует $a \in I$ или $b \in I$. Аналогично определяется простой фильтр (ультрафильтр). В предметном указателе книги присутствуют термины «простой идеал» (с. 85, 212), «дуальный идеал» (с. 212), «главный фильтр» (с. 214), «минимальный фильтр» (с. 213), «фильтр Фреше» (с. 213), «ультрафильтр» (с. 212). *(Источник: Кон, гл. V, §2, с. 212–214; предметный указатель, с. 2 PDF.)*

Ультрафильтры и теорема Стоуна. *Ультрафильтром* называется максимальный по включению собственный фильтр. Эквивалентно, фильтр F является ультрафильтром тогда и только тогда, когда для каждого $a \in B$ ровно один из элементов a, a' принадлежит F. Ультрафильтр F называется *главным*, если он порождён одним элементом: $F = \{b \mid b \geq a\}$ для некоторого $a \in B$; в противном случае он называется *неглавным*. В булеане $\mathcal{B}(A)$ главные ультрафильтры имеют вид $\{X \subseteq A \mid x \in X\}$ для фиксированного $x \in A$; существование неглавных ультрафильтров следует из аксиомы выбора. *(Источник: Кон, гл. V, §2, с. 212–214; предметный указатель, с. 2 PDF.)*

Центральный результат теории булевых алгебр — *теорема Стоуна о представлении*: каждая булева алгебра изоморфна некоторой подалгебре булеана $\mathcal{B}(A)$ для подходящего множества A. В качестве A берётся множество всех ультрафильтров алгебры B; отображение

$$a \mapsto \{F \in A \mid a \in F\}$$

является мономорфизмом булевых алгебр. Таким образом, с точностью до изоморфизма булевы алгебры — это в точности поля множеств. Теорема Стоуна позволяет переносить теоретико-множественные рассуждения на произвольные булевы алгебры. *(Источник: Кон, гл. V, §2, с. 212–215; PDF: оглавление; стандартный факт теории булевых алгебр.)*

Булевы алгебры и логика высказываний. Булева алгебра является алгебраической моделью классической логики высказываний. Каждой пропозициональной переменной сопоставляется элемент булевой алгебры; связкам $\neg, \wedge, \vee$ соответствуют операции $', \wedge, \vee$; тавтологиям соответствуют элементы, равные 1 при любой оценке. Алгебра Линденбаума Lind пропозициональной логики (см. пример выше) является свободной булевой алгеброй над множеством пропозициональных переменных; это позволяет применять теоремы о свободных алгебрах (см. п. 1.3.1 и §III.5 книги Кона) к логике высказываний. В предметном указателе книги присутствует термин «Линденбаума алгебра» (с. 224), однако подробное изложение этого вопроса выходит за рамки §V.2 и относится к гл. V, §4 (с. 221–226). *(Источник: Кон, гл. V, §4, с. 221–226; предметный указатель, с. 2 PDF: «Линденбаума алгебра 224».)*

Булевы алгебры как многообразие. Класс всех булевых алгебр является многообразием Ω-алгебр (см. §IV.1 книги Кона), поскольку он задаётся тождествами, перечисленными выше. Следовательно, к нему применимы общие теоремы о многообразиях: существование свободных алгебр, теорема Биркгофа о представлении, теорема о том, что всякая булева алгебра является гомоморфным образом свободной, и т. д. В частности, свободная булева алгебра с n образующими конечна и имеет 2^{2^n} элементов — столько же, сколько булевых функций от n переменных (см. п. 1.1.2); это не случайное совпадение, а проявление того, что свободная булева алгебра с n образующими изоморфна алгебре булевых функций от n переменных. *(Источник: Кон, гл. IV, §1, с. 176; гл. V, §2, с. 208–210; PDF: оглавление.)*

Итог пункта. В п. 1.3.7 рассмотрены булевы решётки (булевы алгебры): дано определение через дистрибутивность и дополнения, приведены эквивалентные аксиомы, примеры (двухэлементная алгебра, булеан, алгебра Линденбаума, алгебры множеств, регулярные открытые множества). Сформулированы законы де Моргана и связь с булевыми кольцами. Введены полные булевы алгебры и обобщённые законы дистрибутивности. Рассмотрены нормальные формы, идеалы и фильтры, ультрафильтры, теорема Стоуна о представлении. Отмечена связь с логикой высказываний (алгебра Линденбаума) и с теорией многообразий (булевы алгебры образуют многообразие; свободная булева алгебра с n образующими имеет 2^{2^n} элементов). В предоставленном PDF-файле страницы §V.2 не имеют текстового слоя, поэтому ссылки даны по оглавлению и предметному указателю книги; содержательное изложение опирается на стандартную теорию булевых алгебр в объёме, соответствующем книге Кона. *(Обобщение по: Кон, гл. V, §2, с. 207–215; гл. V, §4, с. 221–226; гл. IV, §1, с. 176; гл. II, §4, с. 92; PDF: оглавление, предметный указатель, с. 100.)*

1.3.8 Краткое резюме

В п. 1.3 собраны базовые понятия универсальной алгебры по книге Кона: функции и соответствия, n-арные отношения, отношения эквивалентности, разбиения и фактормножества, отношения частичного порядка, решётки (структуры) в порядковой и алгебраической форме, их эквивалентность, основные свойства решёток, полные решётки и системы замыканий, а также булевы решётки (булевы алгебры). В книге Кона термин «решётка» заменён термином «структура»; это следует учитывать при сопоставлении с другими учебниками. *(Обобщение по: Кон, гл. I, §1–4, с. 13–41; гл. II, §1, §4, с. 55–61, 77–93; гл. V, §2, с. 207–215; PDF с. 21–49, 63–69, 85–101.)*
Источники к пункту 1.3:

- Кон П.М. Универсальная алгебра. М.: Мир, 1968. Гл. I, §1–4, с. 13–41; гл. II, §1, §4, с. 55–61, 77–93; гл. V, §2, с. 207–215.
- PDF-файл `universalnaya_algebra_cohn.pdf`: с. 21–49, 63–69, 85–101; для §V.2 — оглавление и предметный указатель.

1.4 Формальные языки и способы их описания. Классификация формальных грамматик. Их использование в лексическом и синтаксическом анализе

Основной источник: Гагарина Л.Г., Кокорева Е.В. Введение в теорию алгоритмических языков и компиляторов. М.: ИД «ФОРУМ»: ИНФРА-М, 2022. Гл. 1, §1.2–1.6, с. 10–32; гл. 2, §2.1–2.4, с. 33–54; гл. 3, §3.1–3.5, с. 55–99.

1.4.1 Формальные языки: алфавит, цепочка, язык

Алфавит. В теории формальных языков *алфавит* определяется как конечное множество символов. Символы алфавита рассматриваются как неделимые далее знаки, из которых строятся цепочки. Алфавит может содержать как односимвольные, так и многосимвольные элементы; например, алфавит

$$X = \{a, b, c, +, -, *, /\}$$

содержит семь символов, а алфавит

$$Y = \{00, 01, 10, 11\}$$

содержит четыре символа, каждый из которых сам состоит из двух символов. Таким образом, «символ» алфавита не обязательно совпадает с отдельным знаком в привычном смысле: важно лишь то, что алфавит конечен и его элементы различаются между собой. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 10–11.)*

Цепочка символов. *Цепочкой символов* в алфавите V называется любая конечная последовательность символов этого алфавита. Например, если $V = \{a, b\}$, то цепочками являются a, b, ab, ba, aab, bba и т. д. Цепочка характеризуется порядком символов: последовательности ab и ba — разные цепочки. В пособии подчёркивается, что элементы лексики языка программирования представляют собой именно регулярные линейные последовательности символов; это одна из причин, по которой для описания лексики оказываются достаточны регулярные грамматики и конечные автоматы. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 10–12.)*

Пустая цепочка и длина. Цепочка, которая не содержит ни одного символа, называется *пустой цепочкой*. Для её обозначения используется символ λ. Длина пустой цепочки равна нулю:

$$|\lambda| = 0.$$

Длина цепочки — это число составляющих её символов. Например, если $\alpha = abcdefg$, то

$$|\alpha| = 7.$$

Длину цепочки α принято обозначать $|\alpha|$. Понятие длины позволяет различать цепочки с одинаковым составом символов, но разным их числом, а также формулировать многие свойства языков и грамматик. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 10–12.)*

Индуктивное определение цепочки. Более формально множество цепочек в алфавите V определяется индуктивно:

1) λ есть цепочка в алфавите V;

2) если α — цепочка в алфавите V и a — символ этого алфавита, то $a\alpha$ или αa есть цепочка в алфавите V;

3) никаких других цепочек в алфавите V, кроме получающихся по пунктам 1 и 2, нет.

Это определение фиксирует два важных обстоятельства: пустая цепочка является допустимым объектом теории, а любая непустая цепочка может быть получена последовательным добавлением символов алфавита слева или справа. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12.)*

Конкатенация цепочек. Если α и β — цепочки, то цепочка $\alpha\beta$ называется *конкатенацией* (или *сцеплением*) цепочек α и β. Конкатенация состоит в приписывании символов цепочки β непосредственно после символов цепочки α. Например, если

$$\alpha = ab, \qquad \beta = cd,$$

то

$$\alpha\beta = abcd.$$

Для любой цепочки α справедливы равенства

$$\alpha\lambda = \lambda\alpha = \alpha.$$

То есть пустая цепочка играет роль нейтрального элемента относительно конкатенации. В общем случае конкатенация некоммутативна: $\alpha\beta$, вообще говоря, не равно $\beta\alpha$. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12.)*

Степень цепочки. n-й *степенью* цепочки α называется конкатенация n копий этой цепочки. Обозначение: α^n. По определению

$$\alpha^0 = \lambda, \qquad \alpha^n = \alpha\alpha^{n-1} = \alpha^{n-1}\alpha.$$

Например,

$$a^0 = \lambda, \qquad a^1 = a, \qquad a^2 = aa, \qquad a^3 = aaa.$$

Для цепочки ab имеем

$$(ab)^0 = \lambda, \qquad (ab)^1 = ab, \qquad (ab)^2 = abab.$$

Понятие степени цепочки часто используется при описании языков, в которых цепочки имеют повторяющуюся структуру. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12.)*

Обращение цепочки. *Обращением* (или *реверсом*) цепочки α называется цепочка, символы которой записаны в обратном порядке. Обращение цепочки α обозначается α^R. Например, если

$$\alpha = abcdef,$$

то

$$\alpha^R = fedcba.$$

Для пустой цепочки выполняется равенство

$$\lambda^R = \lambda.$$

Операция обращения полезна при описании языков, в которых порядок символов существен, а также при формулировке свойств симметрии цепочек. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12.)*

Язык в алфавите. *Язык* в алфавите V — это подмножество множества всех цепочек конечной длины в этом алфавите. Иными словами, язык задаётся некоторым условием, которому удовлетворяют одни цепочки и не удовлетворяют другие. Например, можно рассматривать язык всех цепочек, начинающихся с символа a, язык всех цепочек чётной длины, язык всех идентификаторов, язык всех арифметических выражений и т. п. В теории формальных языков язык не обязан быть конечным; напротив, большинство содержательных языков программирования бесконечны. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12.)*

Множества V^* и V^+. Множество, содержащее все цепочки в алфавите V, включая пустую цепочку λ, обозначается V^*. Например, если $V = \{a, b\}$, то

$$V^* = \{\lambda, a, b, ab, aa, ba, bb, aba, aab, abb, \ldots\}.$$

Множество, содержащее все цепочки в алфавите V, исключая пустую цепочку λ, обозначается V^+. Справедливо равенство

$$V^* = V^+ \cup \{\lambda\}.$$

Таким образом, каждый язык в алфавите V является подмножеством множества V^*:

$$L \subseteq V^*.$$

Это соотношение является исходным для формального определения языка и для дальнейшей классификации языков по способу их задания. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12.)*

Примеры языков. Приведём простейшие примеры языков в алфавите $V = \{a, b\}$:

1) $L_1 = \{\lambda\}$ — язык, состоящий только из пустой цепочки;

2) $L_2 = \{a, ab, abb, abbb, \ldots\}$ — язык цепочек, начинающихся с a и продолжающихся произвольным числом символов b;

3) $L_3 = \{a^n b^n \mid n \geq 0\}$ — язык цепочек, состоящих из одинакового числа символов a и b;

4) $L_4 = V^*$ — язык всех цепочек в алфавите V.

Эти примеры показывают, что один и тот же алфавит может служить основой для разных языков, а задание языка сводится к точному описанию того, какие цепочки включаются в язык, а какие нет. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 12; примеры конкретизируют общее определение языка.)*

Роль базовых понятий. Понятия алфавита, цепочки, длины, конкатенации, степени, обращения и языка являются исходными для всей теории формальных языков. На них строятся определения порождающей грамматики, вывода, сентенциальной формы, языка, порождаемого грамматикой, а также классификация грамматик и языков по Хомскому. В частности, правила грамматики записываются в виде $\alpha \to \beta$, где α и β — цепочки в объединённом алфавите терминалов и нетерминалов; язык, порождаемый грамматикой, определяется как множество терминальных цепочек, выводимых из начального символа. Поэтому корректное понимание операций над цепочками необходимо для дальнейшего изучения лексического и синтаксического анализа. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 10–14.)*

1.4.2 Порождающие грамматики и способы их задания

Общее понятие порождающей грамматики. Для построения трансляторов необходимо однозначное и точное задание входного и выходного языков. Такое задание предполагает определение правил построения допустимых конструкций выражений языка. Существует целый ряд математических формализмов для задания языков; наиболее распространённым механизмом являются *порождающие грамматики*, которые задают все цепочки языка с помощью некоторых правил. Одно из достоинств грамматик заключается в том, что существует множество систем, которые по заданной грамматике генерируют программу, проверяющую соответствие входной цепочки определяемому языку. Грамматика представляет собой математическую систему, порождающую язык; строки языка строятся способом, точно определённым правилами грамматики. *(Источник: Гагарина, Кокорева, гл. 1, вводная часть, с. 6; §1.1, с. 6; §1.2, с. 10.)*

Определение порождающей грамматики. *Порождающая грамматика* — это четвёрка объектов

$$G = (T, N, P, S),$$

где:

1) T — алфавит *терминальных символов* (терминалов);

2) N — алфавит *нетерминальных символов* (нетерминалов);

3) P — множество *правил вывода* (продукций) грамматики вида

$$\alpha \to \beta, \quad \text{где } \alpha \in (N \cup T)^*,\ \beta \in (N \cup T)^*;$$

4) S — *начальный символ* (цель) грамматики, $S \in N$.

При этом $N \cup T = V$ называется *словарём* грамматики G. Дополнительное условие $N \cap T = \varnothing$ означает, что ни один символ не может быть одновременно терминальным и нетерминальным. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 13.)*

Терминалы и нетерминалы. *Терминалами* называют символы грамматики, имеющие конкретное неизменяемое значение, из которых образуются слова языка. В формальных языках, используемых на компьютере, в качестве терминалов применяют все или часть стандартных символов ASCII — буквы, цифры, специальные символы и т. д. Для обозначения терминальных символов используются строчные буквы латинского алфавита, цифры, знаки препинания, знаки арифметических действий и другие символы, кроме прописных букв латинского алфавита. *Нетерминалами* называют объекты, обозначающие какие-либо сущности языка (формулы, арифметические выражения, команды) и не имеющие конкретного символьного значения. Для обозначения нетерминальных символов используются прописные буквы латинского алфавита. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 13.)*

Форма записи правил и альтернативы. Для записи правил вывода с одинаковыми левыми частями

$$\alpha \to \beta_1, \quad \alpha \to \beta_2, \quad \ldots, \quad \alpha \to \beta_n$$

можно использовать сокращённую запись:

$$\alpha \to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n.$$

Каждую цепочку β_i $(i = 1, 2, \ldots, n)$ будем называть *альтернативой* правила вывода из цепочки α. Такая форма записи особенно удобна при описании грамматик языков программирования, где для одного нетерминала часто требуется несколько альтернативных правил. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, с. 13.)*

Пример грамматики. Рассмотрим грамматику

$$G_1 = (\{a, b\}, \{S, A, B\}, P, S),$$

где множество P состоит из правил:

$$S \to A + B \mid B + A; \qquad A \to a; \qquad B \to b.$$

Эта грамматика порождает цепочки $a + b$ и $b + a$. Альтернативами правила для S являются цепочки $A + B$ и $B + A$. Терминалами здесь являются символы a, b, $+$; нетерминалами — S, A, B; начальным символом — S. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, пример 1.6, с. 13–14.)*

Непосредственная выводимость. Цепочка $\beta \in (T \cup N)^*$ *непосредственно выводима* из цепочки $\alpha \in (T \cup N)^+$ в грамматике $G = (T, N, P, S)$, если

$$\alpha = \mu\gamma\tau, \qquad \beta = \mu\delta\tau,$$

где $\mu, \tau, \delta \in (T \cup N)^*$, $\gamma \in (T \cup N)^+$ и правило вывода $\gamma \to \delta$ содержится в P. Непосредственный вывод цепочки β из цепочки α обозначается

$$\alpha \Rightarrow \beta.$$

Иными словами, непосредственная выводимость означает, что в цепочке α некоторая подцепочка γ, являющаяся левой частью правила $\gamma \to \delta$, заменяется на правую часть δ; остальные части цепочки (μ и τ) остаются без изменения. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, определение 1.12, с. 14.)*

Пример непосредственной выводимости. Цепочка $00A11$ непосредственно выводима из $0A1$ в грамматике

$$G_2 = (\{0,1\}, \{A, S\}, P, S)$$

с правилами

$$P: \quad S \to 0A1; \qquad 0A \to 00A1; \qquad A \to \lambda.$$

Действительно, в цепочке $0A1$ подцепочка $0A$ заменяется на $00A1$ по правилу $0A \to 00A1$; в результате получается $00A11$. Здесь $\mu = \lambda$, $\gamma = 0A$, $\delta = 00A1$, $\tau = 1$. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, пример 1.7, с. 14.)*

Выводимость и вывод. Цепочка $\beta \in (T \cup N)^*$ *выводима* из цепочки $\alpha \in (T \cup N)^+$ в грамматике $G = (T, N, P, S)$, если существуют цепочки $\gamma_0, \gamma_1, \ldots, \gamma_n$ $(n \geq 0)$, такие, что

$$\alpha \Rightarrow \gamma_0 \Rightarrow \gamma_1 \Rightarrow \ldots \Rightarrow \gamma_n \Rightarrow \beta.$$

Последовательность $\gamma_0, \gamma_1, \ldots, \gamma_n$ называется *выводом длины* n. Вывод цепочки β из цепочки α обозначается

$$\alpha \Rightarrow^* \beta.$$

Вывод цепочки β из цепочки α за один или более шагов обозначается $\alpha \Rightarrow^+ \beta$. Таким образом, выводимость есть транзитивное замыкание отношения непосредственной выводимости. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, определения 1.13–1.14, с. 14.)*

Пример вывода. Цепочка $000A111$ выводима из S ($S \Rightarrow^* 000A111$) в грамматике G_2 (см. пример 1.7), так как существует вывод

$$S \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111.$$

Длина вывода равна 3. На каждом шаге применялось правило $0A \to 00A1$, увеличивающее число нулей и единиц вокруг нетерминала A. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, пример 1.8, с. 14.)*

Язык, порождаемый грамматикой. *Языком, порождаемым грамматикой* $G = (T, N, P, S)$, называется множество всех цепочек из алфавита терминальных символов T^*, выводимых из S с помощью правил данной грамматики:

$$L(G) = \{\alpha \in T^* \mid S \Rightarrow^* \alpha\}.$$

Иными словами, язык $L(G)$ состоит в точности из тех терминальных цепочек, которые можно получить из начального символа S за конечное число шагов применения правил вывода. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, определение 1.15, с. 14.)*

Примеры языков, порождаемых грамматиками. Для грамматики G_1 из примера 1.6

$$L(G_1) = \{a + b,\ b + a\}.$$

Для грамматики G_2 из примера 1.7

$$L(G_2) = \{0^n 1^n \mid n > 0\}.$$

В последнем случае цепочки имеют вид 01, 0011, 000111 и т. д.; нетерминал A играет роль «счётчика», обеспечивающего одинаковое число нулей и единиц. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, примеры 1.9–1.10, с. 14.)*

Сентенциальные формы. Если цепочка $\alpha \in (T \cup N)^*$ является выводимой из начального символа S ($S \Rightarrow^* \alpha$) в грамматике $G = (T, N, P, S)$, то её называют *сентенциальной формой*. Таким образом, язык, порождаемый грамматикой, можно определить как множество терминальных сентенциальных форм. Сентенциальные формы могут содержать как терминальные, так и нетерминальные символы; терминальные сентенциальные формы образуют язык $L(G)$. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, определение 1.16, с. 14–15.)*

Эквивалентность грамматик. Грамматики G_1 и G_2 называются *эквивалентными*, если они порождают один и тот же язык:

$$L(G_1) = L(G_2).$$

Например, грамматики G_3 и G_4 с правилами

$$\begin{aligned} P_3: &\quad S \to AB; \quad A \to a \mid Aa; \quad B \to b \mid Bb; \\ P_4: &\quad S \to AS \mid SB \mid AB; \quad A \to a; \quad B \to b \end{aligned}$$

являются эквивалентными, так как обе порождают язык

$$L(G_3) = L(G_4) = \{a^n b^m \mid n, m > 0\}.$$

Эквивалентность грамматик — центральное понятие теории формальных грамматик: одна и та же языковая конструкция может описываться разными грамматиками, и выбор конкретной грамматики определяется удобством построения распознавателя. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, определение 1.17 и пример 1.11, с. 15.)*

Почти эквивалентные грамматики. Грамматики G_1 и G_2 называются *почти эквивалентными*, если языки, ими порождаемые, отличаются не более чем на λ:

$$L(G_1) = L(G_2) \cup \{\lambda\}.$$

Например, грамматики

$$G_5 = (\{0,1\}, \{A, S\}, P_5, S), \qquad G_6 = (\{0,1\}, \{S\}, P_6, S),$$

где

$$\begin{aligned} P_5: &\quad S \to 0A1; \quad 0A \to 00A1; \quad A \to \lambda; \\ P_6: &\quad S \to 0S1 \mid \lambda, \end{aligned}$$

почти эквивалентны, так как

$$L(G_5) = \{0^n 1^n \mid n > 0\}, \qquad L(G_6) = \{0^n 1^n \mid n \geq 0\},$$

то есть $L(G_6) = L(G_5) \cup \{\lambda\}$. Понятие почти эквивалентности оказывается полезным, когда требуется учесть или, наоборот, исключить пустую цепочку. *(Источник: Гагарина, Кокорева, гл. 1, §1.2, определение 1.18 и пример 1.12, с. 15.)*

Способы задания схем грамматик. Схема грамматики содержит правила вывода, определяющие структуру цепочек порождаемого языка. Для задания правил используются различные формы описания: символическая, форма Бэкуса — Наура, итерационная форма и синтаксические диаграммы. При рассмотрении общих свойств грамматик обычно применяют символическую форму задания правил. В практических применениях вместо символического представления схем грамматик обычно используют форму Бэкуса — Наура или синтаксические диаграммы. Выбор формы задания зависит от цели: символическая форма удобна для теоретических доказательств, БНФ и синтаксические диаграммы — для описания реальных языков программирования и построения анализаторов. *(Источник: Гагарина, Кокорева, гл. 1, §1.3, с. 15.)*

Форма Бэкуса — Наура. При описании синтаксиса конкретных языков программирования приходится вводить большое число нетерминальных символов, и символическая форма записи теряет свою наглядность. Для того чтобы этого избежать, применяют *форму Бэкуса — Наура* (БНФ), в которой нетерминальные символы представляют собой слова естественного языка, заключённые в угловые скобки, а разделителем служит специальный знак — два двоеточия и равенство (`::=`). БНФ-конструкция содержит несколько предложений следующего вида:

```
<символ> ::= <цепочка1> | <цепочка2> | ... | <цепочкаn>.
```

Например, заданные в символической форме правила грамматики $L \to EL$, $L \to E$, где символу L соответствует понятие «список», а символу E — понятие «элемент списка», можно заменить следующей БНФ-конструкцией:

```
<список> ::= <элемент списка><список>

<список> ::= <элемент списка>
```

или сокращённой БНФ-конструкцией:

```
<список> ::= <элемент списка><список> | <элемент списка>.
```

Следующая БНФ-конструкция служит для описания целого числа:

```
<целое> ::= <целое б/зн.> | +<целое б/зн.> | -<целое б/зн.>

<целое б/зн.> ::= <цифра> | <целое б/зн.><цифра>

<цифра> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9.
```

БНФ широко используется в стандартах языков программирования и в документации компиляторов, поскольку позволяет описывать синтаксис в наглядной и компактной форме. *(Источник: Гагарина, Кокорева, гл. 1, §1.3.1, с. 15–16.)*

Итерационная форма. Для получения более компактных описаний синтаксиса применяют *итерационную форму*. Она предполагает введение специальной операции, которая называется *итерацией* и обозначается парой фигурных скобок со звёздочкой. Итерация вида $\{a\}^*$ определяется как множество, включающее цепочки всевозможной длины, построенные с использованием символа a, и пустую цепочку:

$$\{a\}^* = \{\lambda, a, aa, aaa, aaaa, \ldots\}.$$

Используя итерацию в символической форме описания списков, получим:

$$L \to E\{E\}^*.$$

Описание множества цепочек, каждая из которых должна начинаться с буквы a и может состоять из произвольного числа букв b и c, может быть представлено в итерационной форме так:

$$I \to a\{b \mid c\}^*.$$

Кроме того, в итерационных формах наряду с фигурными скобками применяют квадратные скобки, чтобы показать, что цепочка, заключённая в них, может быть опущена. С помощью таких скобок правила

$$A \to xAyBz; \qquad A \to xBz$$

могут быть записаны следующим образом:

$$A \to x[Ay]Bz.$$

Итерационная форма позволяет сократить число правил и сделать описание синтаксиса более компактным, однако она требует дополнительных соглашений о семантике фигурных и квадратных скобок. *(Источник: Гагарина, Кокорева, гл. 1, §1.3.2, с. 15–16.)*

Синтаксические диаграммы. Представление правил грамматики в графической форме в виде *синтаксических диаграмм* служит для облегчения понимания сложных синтаксических конструкций посредством зрительного восприятия. Правила построения синтаксических диаграмм можно сформулировать следующим образом:

1) Правилу вида $A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_k$ ставится в соответствие диаграмма, структура которой определяется правой частью правила.

2) Каждое появление терминального символа x в цепочке α_i изображается на диаграмме дугой, помеченной этим символом x, заключённым в кружок.

3) Каждому появлению нетерминального символа $\langle A \rangle$ в цепочке α_i ставится в соответствие на диаграмме дуга, помеченная символом, заключённым в квадрат.

4) Правило грамматики вида $A \to \alpha_1\alpha_2 \ldots \alpha_n$ изображается на диаграмме как последовательность дуг.

5) Правило грамматики, имеющее вид $A \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$, изображается на диаграмме как разветвление (альтернатива).

6) Если порождающее правило задано в виде итерации $A \to \{\alpha\}^*$, то ему соответствует синтаксическая диаграмма с обратной связью.

Количество правил порождающей грамматики определяет число построенных для неё синтаксических диаграмм. Для того чтобы сократить это число, диаграммы объединяют, заменяя нетерминальные символы построенными для них диаграммами. На рисунке 1.6 пособия приведена синтаксическая диаграмма правила грамматики для описания идентификаторов; БНФ-форма представлена ниже:

```
<идентификатор> ::= <буква><продолжить> | <_><продолжить>
```

```
<продолжить> ::= <буква><продолжить> | <цифра><продолжить> | <_><продолжить> | <пусто>
```

Синтаксические диаграммы особенно удобны при ручной разработке анализаторов, поскольку непосредственно подсказывают структуру рекурсивных процедур. *(Источник: Гагарина, Кокорева, гл. 1, §1.3.3, с. 16–18.)*

Связь формы задания с построением анализатора. Выбор формы задания грамматики непосредственно влияет на метод построения распознавателя. Символическая форма удобна для доказательства свойств грамматик (например, эквивалентности, однозначности, принадлежности к классу LL(k) или LR(k)), но менее наглядна при описании больших языков. БНФ и итерационная форма позволяют компактно описывать синтаксис реальных языков программирования и часто используются в стандартах. Синтаксические диаграммы дают графическое представление, которое хорошо соответствует структуре рекурсивного спуска: каждой диаграмме соответствует процедура анализа, а альтернативам и итерациям — ветвления и циклы в программе. В любом случае грамматика остаётся математическим объектом $G = (T, N, P, S)$, а форма записи правил является лишь способом её представления. *(Источник: Гагарина, Кокорева, гл. 1, §1.3, с. 15–18; §3.1, с. 56–57.)*

1.4.3 Классификация грамматик и языков по Хомскому

Иерархия Хомского. Иерархия Хомского — это классификация формальных языков и формальных грамматик, согласно которой они делятся на четыре типа в соответствии с их условной сложностью. Каждый тип характеризуется ограничениями, накладываемыми на вид правил вывода. Чем меньше ограничений, тем шире класс языков, порождаемых грамматиками данного типа. Иерархия была предложена Ноамом Хомским в 1950-х годах и с тех пор является фундаментальной основой теории формальных языков и теории компиляторов. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.1, с. 18.)*

Тип 0: неограниченные грамматики. Грамматика $G = (T, N, P, S)$ называется *неограниченной грамматикой*, если на правила вывода не накладывается никаких ограничений (кроме тех, которые указаны в определении грамматики). Это означает, что правила могут иметь произвольный вид $\alpha \to \beta$, где $\alpha \in (T \cup N)^*$, $\beta \in (T \cup N)^*$; в частности, левая часть может быть пустой, а длина правой части может быть меньше длины левой. Неограниченные грамматики порождают класс рекурсивно перечислимых языков, которые распознаются машинами Тьюринга. Практического применения в компиляторах такие грамматики не имеют из-за сложности распознавания, однако они важны для теоретических результатов. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.1, с. 18.)*

Тип 1: контекстно-зависимые (неукорачивающие) грамматики. Грамматика $G = (T, N, P, S)$ называется *контекстно-зависимой* (КЗ), или *неукорачивающей* грамматикой, если каждое правило из P имеет вид $\alpha \to \beta$, где $\alpha \in (T \cup N)^+$, $\beta \in (T \cup N)^+$ и $|\alpha| \leq |\beta|$. Иными словами, длина правой части правила не меньше длины левой части; это исключает «стирание» символов. Название «контекстно-зависимая» связано с тем, что в общем случае правила могут учитывать контекст, то есть заменять символ только в определённом окружении. КЗ-грамматики порождают контекстно-зависимые языки, которые распознаются линейно ограниченными

автоматами (машинами Тьюринга с линейно ограниченной памятью). В компиляторах КЗ-грамматики применяются редко из-за экспоненциальной сложности разбора; они используются, например, для описания некоторых синтаксических конструкций естественных языков. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.1, с. 18.)*

Тип 2: контекстно-свободные грамматики. Грамматика $G = (T, N, P, S)$ называется *контекстно-свободной* (КС), если каждое правило из P имеет вид $A \to \alpha$, где $A \in N$ — нетерминал, а $\alpha \in (T \cup N)^*$ — строка из нетерминалов и терминалов или пустая строка. В КС-грамматиках левая часть правила всегда состоит из одного нетерминала, поэтому замена не зависит от контекста, в котором этот нетерминал встречается. КС-грамматики порождают контекстно-свободные языки, которые распознаются магазинными автоматами (МП-автоматами). Это наиболее важный класс грамматик для языков программирования: синтаксис практически всех языков программирования описывается КС-грамматиками или их подклассами. Для КС-грамматик существует множество эффективных методов разбора (нисходящие и восходящие, LL(k), LR(k), предшествование и др.). *(Источник: Гагарина, Кокорева, гл. 1, §1.4.1, с. 18.)*

Тип 3: регулярные грамматики. Грамматика $G = (T, N, P, S)$ называется *праволинейной*, если каждое правило из P имеет вид $A \to aB$ либо $A \to a$, где $A \in N$, $B \in N$, $a \in T$. Грамматика $G = (T, N, P, S)$ называется *леволинейной*, если каждое правило из P имеет вид $A \to Ba$ либо $A \to a$, где $A \in N$, $B \in N$, $a \in T$. Леволинейные и праволинейные грамматики называются *регулярными* грамматиками. В регулярных грамматиках правая часть правила содержит не более одного нетерминала, и этот нетерминал (если он есть) стоит либо в конце (праволинейные), либо в начале (леволинейные) правой части. Регулярные грамматики порождают регулярные языки, которые распознаются конечными автоматами. Регулярные языки используются в лексическом анализе для описания токенов (идентификаторов, чисел, ключевых слов и т. п.). Регулярные грамматики обладают свойством линейной сложности разбора, что делает их особенно удобными для сканеров. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.1, с. 18.)*

Соотношения между типами грамматик. Формальные языки классифицируются по типу порождающих их грамматик. Заметим, что один и тот же язык может порождаться грамматиками различного типа. Поэтому, когда говорят о типе языка, имеют в виду максимально возможный его номер (то есть наиболее узкий класс, к которому язык принадлежит). Между типами формальных грамматик существуют следующие соотношения:

1) Любая регулярная грамматика является КС-грамматикой. Это следует из того, что правила регулярной грамматики $A \to aB$, $A \to Ba$, $A \to a$ являются частными случаями правил КС-грамматики $A \to \alpha$.

2) Любая КС-грамматика, кроме укорачивающей (в правой части правил которой есть пустая цепочка), является КЗ-грамматикой (неукорачивающей грамматикой). Действительно, в КС-грамматике левая часть правила — один нетерминал, длина которого равна 1; если правая часть непуста, то её длина не меньше 1, что удовлетворяет условию неукорачивания. Исключение составляет правило $A \to \lambda$, которое делает грамматику укорачивающей.

3) Любая КЗ-грамматика является грамматикой типа 0. Это очевидно, поскольку КЗ-грамматика является частным случаем неограниченной грамматики.

Таким образом, имеет место включение классов грамматик:

$$\text{регулярные} \subset \text{КС} \subset \text{КЗ} \subset \text{тип 0}.$$

(Источник: Гагарина, Кокорева, гл. 1, §1.4.2, с. 19.)

Соотношения между типами языков. Аналогичные соотношения имеют место и для языков:

1) Каждый регулярный язык является КС-языком, но существуют КС-языки, которые не являются регулярными. Классический пример — язык $L = \{a^n b^n \mid n > 0\}$, который порождается КС-грамматикой $S \to aSb \mid ab$, но не может быть порождён никакой регулярной грамматикой. Это следует из того, что для распознавания цепочек вида $a^n b^n$ требуется «счётчик», то есть память, а конечный автомат имеет лишь конечное число состояний.

2) Каждый КС-язык, кроме укорачивающего (содержащего пустую цепочку), является КЗ-языком, но существуют КЗ-языки, которые не являются КС-языками. Пример — язык $L = \{a^n b^n c^n \mid n > 0\}$, который не порождается никакой КС-грамматикой (это можно доказать с помощью леммы о накачке для КС-языков), но порождается КЗ-грамматикой.

3) Каждый КЗ-язык является языком типа 0. Это непосредственно следует из включения классов грамматик.

Таким образом, классы языков образуют иерархию:

$$\text{регулярные} \subset \text{КС} \subset \text{КЗ} \subset \text{тип 0}.$$

(Источник: Гагарина, Кокорева, гл. 1, §1.4.2, с. 19–20.)

Пример грамматики типа 0. В пособии приведён пример языка типа 0:

$$L = \{a^{2^n} \mid n \geq 1\}.$$

Этот язык порождается грамматикой со следующими правилами:

$$S \to aaCFD;$$
$$F \to AFB \mid AB;$$
$$AB \to bBA;$$
$$Ab \to bA;$$
$$AD \to D;$$
$$Cb \to bC;$$
$$CB \to C;$$
$$bCD \to \lambda.$$

Грамматика не является неукорачивающей, поскольку правило $bCD \to \lambda$ уменьшает длину цепочки. Она порождает язык, состоящий из цепочек символов a, длина которых является степенью двойки. Этот пример показывает, что неограниченные грамматики могут порождать языки, не являющиеся КЗ-языками. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.3, с. 20.)*

Пример грамматики типа 1. Язык типа 1:

$$L = \{\text{цепочки из 0 и 1 с одинаковым числом 0 и 1}\}.$$

Грамматика:

$$S \to ASB \mid AB;$$
$$AB \to BA;$$
$$A \to 0;$$
$$B \to 1.$$

Все правила неукорачивающие: длина правой части не меньше длины левой. Грамматика порождает все цепочки, в которых число нулей равно числу единиц, но порядок символов может быть произвольным. Это пример КЗ-языка, который не является КС-языком (поскольку для КС-языков существуют ограничения на «согласование» числа символов, но язык с равным числом нулей и единиц без ограничения на порядок является КС-языком? На самом деле язык $\{w \mid \#_0(w) = \#_1(w)\}$ является КС-языком, так как порождается грамматикой $S \to 0S1S \mid 1S0S \mid \lambda$. Однако в пособии приведён именно этот пример как пример грамматики типа 1, возможно, для иллюстрации, а не как пример языка, не являющегося КС. В пособии не утверждается, что этот язык не является КС; просто приведён пример грамматики типа 1.) *(Источник: Гагарина, Кокорева, гл. 1, §1.4.3, с. 20.)*

Пример грамматики типа 2. Язык типа 2:

$$L = \{(ac)^n(cb)^n \mid n > 0\}.$$

Грамматика:

$$S \to aQb \mid accb;$$
$$Q \to cSc.$$

Этот язык является КС-языком, но не является регулярным. Грамматика содержит только правила вида $A \to \alpha$, где A — нетерминал. Цепочки языка имеют вид $accb$, $acaccbcb$, $acacaccbcbcb$ и т. д. Язык демонстрирует «вложенную» структуру, характерную для КС-языков. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.3, с. 20.)*

Пример грамматики типа 3. Язык типа 3:

$$L = \{\omega \perp \mid \omega \in \{a, b\}^+, \text{ где нет рядом стоящих } a\}.$$

Грамматика:

$$S \to A \perp \mid B \perp;$$
$$A \to a \mid Ba;$$
$$B \to b \mid Bb \mid Ab.$$

Здесь символ $\perp$ обозначает конец цепочки. Грамматика является праволинейной (все правила имеют вид $A \to aB$ или $A \to a$) и порождает регулярный язык. Она описывает цепочки, состоящие из символов a и b, в которых нет двух подряд идущих символов a, и завершающиеся символом $\perp$. Этот пример иллюстрирует, как регулярные грамматики могут описывать достаточно сложные ограничения на последовательности символов, но не могут «считать» неограниченное число символов. *(Источник: Гагарина, Кокорева, гл. 1, §1.4.3, с. 20–21.)*

Практическое значение классификации. Классификация Хомского имеет фундаментальное значение для теории компиляторов. Она позволяет выбрать адекватный формализм для описания различных частей языка программирования. Лексика языка (идентификаторы, константы, ключевые слова, разделители) обычно описывается регулярными грамматиками, поскольку для распознавания токенов достаточно конечных автоматов, обладающих линейной сложностью. Синтаксис языка (выражения, операторы, определения функций) описывается КС-грамматиками, для которых существуют эффективные методы разбора (нисходящие и восходящие). Контекстно-зависимые и неограниченные грамматики в компиляторах практически не используются из-за высокой сложности распознавания, хотя они важны для теоретических исследований. Таким образом, иерархия Хомского служит теоретической основой для разделения фаз лексического и синтаксического анализа. *(Источник: Гагарина, Кокорева, гл. 1, §1.4, с. 18–21; гл. 1, §1.6.3, с. 30–32.)*

1.4.4 Разбор цепочек и неоднозначность грамматик

Задача разбора. Одной из основных задач анализа является решение вопроса о принадлежности цепочки данному языку. Если цепочка принадлежит языку, порождаемому грамматикой, то существует её вывод из целевого нетерминала этой грамматики. Процесс построения такого вывода (а следовательно, и определения принадлежности цепочки языку) называется *разбором*. С практической точки зрения наибольший интерес представляют контекстно-свободные грамматики, которые используются в качестве порождающих грамматик для большинства синтаксических структур языков программирования. Существует множество хорошо известных и отработанных способов решения задачи разбора для КС-грамматик; в дальнейшем методы разбора рассматриваются для различных подклассов КС-грамматик. *(Источник: Гагарина, Кокорева, гл. 1, §1.5, с. 21.)*

Левосторонний и правосторонний вывод. Рассмотрим некоторые понятия, связанные с разбором по КС-грамматике. Вывод цепочки $\alpha \in T^*$ из $S \in N$ в КС-грамматике $G = (T, N, P, S)$ называется *левым* (левосторонним), если в этом выводе каждая очередная сентенциальная форма получается из предыдущей заменой самого левого нетерминала. Вывод называется *правым* (правосторонним), если в этом выводе каждая очередная сентенциальная форма получается из предыдущей заменой самого правого нетерминала. Иными словами, левосторонний вывод всегда заменяет самый левый нетерминал в текущей сентенциальной форме, а правосторонний — самый правый. Для одной и той же цепочки может существовать несколько выводов. Выводы являются *эквивалентными*, если в них в одних и тех же местах применяются одни и те же правила вывода в различном порядке. Эквивалентные выводы соответствуют одному и тому же дереву вывода. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.1, определения 1.19–1.20, с. 21–22.)*

Пример различных выводов. Для цепочки $a + b + a$ в грамматике

$$G = (\{a, b\}, \{S, T\}, \{S \to T \mid T + S;\ T \to a \mid b\}, S)$$

можно построить выводы:

(1) $S \Rightarrow T + S \Rightarrow a + S \Rightarrow a + T + S \Rightarrow a + b + S \Rightarrow a + b + T \Rightarrow a + b + a$;
(2) $S \Rightarrow T + S \Rightarrow T + T + S \Rightarrow T + T + T \Rightarrow T + T + a \Rightarrow T + b + a \Rightarrow a + b + a$;
(3) $S \Rightarrow T + S \Rightarrow T + T + S \Rightarrow T + T + T \Rightarrow a + T + T \Rightarrow a + b + T \Rightarrow a + b + a$.

Здесь (1) — левосторонний вывод, (2) — правосторонний, а (3) не является ни левосторонним, ни правосторонним, но все эти выводы — эквивалентные в указанном выше смысле. Эквивалентность выводов означает, что они приводят к одному и тому же дереву вывода, хотя порядок применения правил различен. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.1, пример 1.19, с. 22–23.)*

Дерево вывода. Для КС-грамматик можно ввести удобное графическое представление вывода, называемое *деревом вывода*, причём для всех эквивалентных выводов деревья вывода совпадают. Дерево является деревом вывода для контекстно-свободной грамматики $G = (N, T, P, S)$, если оно удовлетворяет следующим условиям:

1) каждый узел имеет метку — символ из алфавита $V = N \cup T$;
2) метка корня — S;
3) если узел имеет по крайней мере одного потомка, то его метка должна быть нетерминалом;
4) если узлы $n_1, n_2, \ldots, n_k$ — прямые потомки узла n, перечисленные слева направо, с метками $A_1, A_2, \ldots, A_k$ соответственно, а метка узла n есть A, то грамматика G должна содержать правило

$$A \to A_1 A_2 \ldots A_k \in P.$$

Таким образом, дерево вывода полностью определяется грамматикой и выводимой цепочкой; оно не зависит от порядка применения правил. Каждому левостороннему выводу соответствует единственное дерево вывода, и наоборот, каждому дереву вывода можно сопоставить левосторонний вывод (и правосторонний вывод). *(Источник: Гагарина, Кокорева, гл. 1, §1.5.2, определение 1.21, с. 23.)*

Пример дерева вывода. Рассмотрим КС-грамматику

$$G = (\{a, b\}, \{S, A\}, P, S),$$

где

$$P = \{S \to aAS,\ S \to a,\ A \to SbA,\ A \to ba,\ A \to SS\}.$$

На рис. 1.7 пособия изображено дерево, представляющее вывод:

$$S \Rightarrow aAS \Rightarrow aSbAS \Rightarrow aabAS \Rightarrow aabbaAS \Rightarrow aabbaa.$$

Корень дерева помечен символом S; его потомки — a, A, S; далее узел A раскрывается в S, b, A и т. д. Листья дерева образуют выводимую цепочку $aabbaa$. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.2, пример 1.20, с. 23–24.)*

Нисходящий и восходящий разбор. Дерево вывода может быть построено нисходящим или восходящим способом разбора. При *нисходящем разборе* дерево вывода строится от корня к листьям. На каждом шаге метка вершины — нетерминал из левой части правила — заменяется правой частью правила, которое позволяет получить символы исходной цепочки. Иными словами, нисходящий разбор начинает с начального символа S и последовательно применяет правила, пока не получит входную цепочку. При *восходящем разборе* исходную цепочку пытаются «свернуть» к начальному символу S. На каждом шаге ищут подцепочку, совпадающую с правой частью какого-либо правила вывода, и заменяют её нетерминалом из левой части этого правила. Восходящий разбор начинает с входной цепочки и последовательно сворачивает её к S. Оба способа соответствуют различным стратегиям построения дерева вывода: сверху вниз и снизу вверх. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.2, с. 23–24.)*

Неоднозначность грамматик. Может возникнуть ситуация, когда в одной и той же грамматике для цепочки можно сформировать не одно, а два и более различных деревьев вывода. Контекстно-свободная грамматика G называется *неоднозначной*, если в языке $L(G)$ существует хотя бы одна цепочка, для которой может быть построено два или более различных деревьев вывода. В противном случае грамматика называется *однозначной*. Это также означает, что в однозначной грамматике существует не более одного левостороннего (правостороннего) вывода цепочки. Язык, порождаемый грамматикой, называется *неоднозначным*, если он не может быть порождён никакой однозначной грамматикой. С точки зрения формального языка, заданного грамматикой, не имеет значения, какая цепочка вывода и какое дерево вывода из возможных вариантов будут построены. Однако для языков программирования, которые не являются чисто формальными языками и несут некоторую смысловую нагрузку, это важно. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, определения 1.22–1.23, с. 23–24.)*

Пример неоднозначной грамматики. Формальная грамматика

$$G = (\{+, -, *, /, (,), a, b\}, \{S\}, P, S),$$

$$P: \quad S \to S + S \mid S - S \mid S * S \mid S/S \mid (S) \mid a \mid b$$

определяет язык арифметических операций. Примерами цепочек языка, заданного такой грамматикой, являются $a * b - a$, $a * (a + b)$, $a * b + a * a$ и т. д. Построим левосторонний вывод для цепочки $a * b + a$. Получится два варианта:

$$a) \quad S \Rightarrow S + S \Rightarrow S * S + S \Rightarrow a * S + S \Rightarrow a * b + S \Rightarrow a * b + a;$$
$$b) \quad S \Rightarrow S * S \Rightarrow a * S \Rightarrow a * S + S \Rightarrow a * b + S \Rightarrow a * b + a.$$

Каждому из этих вариантов будет соответствовать своё дерево вывода (рис. 1.8 пособия). Рассмотренная грамматика явно неоднозначна. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, пример 1.21, с. 24–25.)*

Смысл неоднозначности для языков программирования. Поскольку рассмотренная выше грамматика определяет язык арифметических выражений, порядок построения дерева в ней соответствует порядку выполнения арифметических

операций. Но в данной грамматике этот порядок не определён, и поэтому все операции равноправны, что неправильно с точки зрения арифметики, где операции умножения и деления обладают более высоким приоритетом по сравнению с операциями сложения и вычитания. И хотя синтаксическая структура полученных выражений правильная, не определён порядок выполнения операций слева направо. Такая ситуация связана с неоднозначностью грамматики. Таким образом, неоднозначность грамматики может приводить к неоднозначности смысла программы, что недопустимо для языков программирования. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, с. 25–26.)*

Свойства неоднозначности. Неоднозначность — это свойство грамматики, а не языка. Для некоторых языков, заданных неоднозначными грамматиками, иногда удаётся построить эквивалентную однозначную грамматику (задающую тот же язык). Для того чтобы убедиться в том, что некоторая грамматика является неоднозначной, согласно определению достаточно найти в заданном ею языке хотя бы одну цепочку, которая допускала бы более чем один левосторонний или правосторонний вывод. Однако не всегда удаётся легко обнаружить такую цепочку символов, поскольку перебрать все цепочки языка, как правило, невозможно — их может быть бесконечно много, так же как невозможно доказать и однозначность грамматики. Данная проблема в общем случае алгоритмически неразрешима. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, с. 26.)*

Виды правил, приводящих к неоднозначности. Можно указать некоторые виды правил вывода, которые приводят к неоднозначности:

- $A \to AA \mid a$;
- $A \to A\alpha A \mid \beta$;
- $A \to \alpha A \mid AB \mid \gamma$;
- $A \to \alpha A \mid \alpha AB \mid \gamma$.

Алгоритмически неразрешимой является в общем случае проблема проверки эквивалентности двух грамматик так же, как и проблема преобразования произвольной неоднозначной грамматики в эквивалентную ей однозначную. Однако в большинстве случаев эта задача разрешима. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, с. 26.)*

Пример устранения неоднозначности. Для рассмотренной в примере 1.21 неоднозначной грамматики арифметических выражений над операндами a и b существует эквивалентная ей однозначная грамматика следующего вида:

$$G' = (\{+, -, *, /, (,), a, b\}, \{S, T, E\}, P', S),$$

$$\begin{aligned} P' : \quad & S \to S + T \mid S - T \mid T; \\ & T \to T * E \mid T/E \mid E; \\ & E \to (S) \mid a \mid b. \end{aligned}$$

В этой грамматике для цепочки символов языка $a * b + a$ возможен только один левосторонний вывод:

$$S \Rightarrow S + T \Rightarrow T + T \Rightarrow T * E + T \Rightarrow E * E + T \Rightarrow a * E + T \Rightarrow$$
$$\Rightarrow a * b + T \Rightarrow a * b + E \Rightarrow a * b + a.$$

Дерево вывода приведено на рис. 1.9 пособия. Приоритет арифметических операций соответствует порядку построения дерева разбора в однозначной грамматике. Таким образом, для построения трансляторов и языков программирования, которые не являются чисто формальными, а несут некоторую смысловую нагрузку, необходимо использовать только однозначные грамматики. *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, пример 1.22, с. 26–27.)*

Практические выводы. Из изложенного следует, что при разработке грамматики языка программирования необходимо стремиться к её однозначности. Неоднозначная грамматика может приводить к различным деревьям разбора для одной и той же цепочки, что, в свою очередь, может приводить к различной семантике программы. Для устранения неоднозначности часто вводят дополнительные нетерминалы, соответствующие уровням приоритета операций, как это показано в примере с грамматикой G'. Однако не всегда удаётся автоматически преобразовать неоднозначную грамматику в однозначную; эта задача в общем случае алгоритмически неразрешима. Поэтому разработчик грамматики должен проектировать её так, чтобы избежать неоднозначности, либо использовать специальные методы разрешения конфликтов (например, задание приоритетов и ассоциативности операций в инструментах генерации синтаксических анализаторов). *(Источник: Гагарина, Кокорева, гл. 1, §1.5.3, с. 26–27.)*

1.4.5 Распознаватели и их связь с типами языков

Понятие распознавателя. Один из методов описания языков базируется на использовании распознавателей. *Распознаватель* — это специальный алгоритм, который позволяет определить принадлежность цепочки символов некоторому языку. Таким образом, задача разбора в общем виде заключается в следующем: на основе имеющейся грамматики некоторого языка построить распознаватель для этого языка. В отношении исходной программы человек, написавший её, выступает в качестве генератора цепочек, а компилятор — в качестве распознавателя. Задача распознавателя состоит в том, чтобы на основе исходной цепочки дать ответ, принадлежит она заданному языку или нет. Распознаватели, как было сказано выше, представляют собой один из способов определения языка. *(Источник: Гагарина, Кокорева, гл. 1, §1.6, определение 1.24, с. 26; §1.6.1, с. 26–27.)*

Общая схема распознавателя. В общем виде распознаватель можно отобразить в виде условной схемы (автомата), состоящей из трёх частей: входной ленты, устройства управления с конечной памятью и внешней (рабочей) памяти. Условной эта схема является потому, что распознаватель, входящий в состав компилятора, представляет собой часть программного обеспечения, а не часть устройства компьютера. Входная лента — линейная последовательность ячеек, каждая из которых содержит один входной символ из конечного входного алфавита. Могут присутствовать левый и правый концевые маркеры, может присутствовать только один концевой маркер

(левый или правый), могут отсутствовать оба маркера. Считывающая головка в каждый момент читает одну входную ячейку. За один шаг входная головка может сдвинуться на одну ячейку влево, вправо и остаться неподвижной. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.1, с. 27.)*

Устройство управления и внешняя память. Внешняя память хранит информацию, построенную только из символов конечного алфавита памяти. Она может иметь различную структуру: очередь, стек (магазин) и т. д. Можно читать из так называемой вспомогательной памяти и писать в неё. Для стека и очереди используются специфические операции (вталкивание, выталкивание). Устройство управления с конечной памятью (УУ) — программа, управляющая поведением распознавателя. Оно может являться аналогом конечного автомата и определяет перемещение входной головки и работу с памятью на каждом шаге (такте). Переходит за шаг из одного состояния в другое; состояние сохраняется в памяти УУ. Распознаватель работает с символами своего алфавита, который является конечным. Алфавит распознавателя включает все допустимые символы входных цепочек, а также некоторый дополнительный алфавит символов, которые могут обрабатываться УУ и храниться в рабочей памяти распознавателя. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.1, с. 27–28.)*

Конфигурация и работа распознавателя. Вся работа распознавателя состоит из последовательности тактов. В начале каждого такта состояние распознавателя определяется его конфигурацией. В процессе работы конфигурация распознавателя меняется. Конфигурация распознавателя определяется следующими параметрами: содержимым входной цепочки символов и положением считывающей головки в ней; состоянием УУ; содержимым внешней памяти. Начальная конфигурация — устройство управления находится в заданном начальном состоянии, входная головка читает самый левый символ на входной ленте, память имеет заранее установленное начальное содержимое. Заключительная конфигурация — устройство управления находится в одном из состояний, принадлежащем заранее выделенному множеству заключительных состояний, входная головка обозревает правый концевой маркер или, если маркер отсутствует, сошла с конца входной ленты. Иногда требуется, чтобы заключительная конфигурация памяти удовлетворяла некоторым условиям. Распознаватель допускает входную цепочку символов, если, находясь в начальной конфигурации и получив на вход эту цепочку, он может проделать последовательность шагов, заканчивающуюся одной из его конечных конфигураций. Язык, определяемый распознавателем, — это множество всех цепочек, которые допускает распознаватель. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.1, с. 28.)*

Виды распознавателей по считывающему устройству. По видам считывающего устройства распознаватели могут быть двусторонние и односторонние. У односторонних распознавателей считывающая головка перемещается по ленте входных символов только в одном направлении — слева направо, так же, как читается входная программа. Когда говорят об односторонних распознавателях, то имеют в виду левосторонние, которые читают исходную цепочку слева направо без возвращения к уже прочитанной части цепочки. Двусторонние распознаватели допускают перемещение считывающей головки в обоих направлениях: как вперёд, от начала ленты к концу, так и назад, возвращаясь к уже прочитанным символам. Создавать двусторонние

распознаватели сложнее, чем односторонние. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.2, с. 28–30.)*

Детерминированные и недетерминированные распознаватели. По видам устройства управления распознаватели бывают детерминированные и недетерминированные. Детерминированным называется распознаватель, у которого для каждой допустимой на некотором шаге работы существует единственно возможная конфигурация, в которую распознаватель перейдёт на следующем шаге. У недетерминированного распознавателя существует хотя бы одна допустимая на некотором шаге конфигурация, из которой возможен переход во множество возможных на следующем шаге конфигураций. Недетерминированные распознаватели по сложности выше детерминированных. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.2, с. 28–30.)*

Виды распознавателей по внешней памяти. По видам внешней памяти распознаватели бывают следующих типов: без внешней памяти; с ограниченной внешней памятью; с неограниченной внешней памятью. У распознавателей без внешней памяти используется только конечная память устройства управления. Для распознавателей с ограниченной внешней памятью размер внешней памяти ограничен в зависимости от длины исходной цепочки символов. Эти ограничения могут накладываться некоторой зависимостью объёма памяти от длины цепочки — линейной, полиномиальной, экспоненциальной и т. д. Кроме того, для таких распознавателей может быть указан способ организации внешней памяти — стек, очередь, список и т. п. Распознавателям с неограниченной внешней памятью для работы может потребоваться внешняя память неограниченного объёма (как правило, вне зависимости от длины входной цепочки). Для означенных распознавателей характерна память с произвольным доступом. Чем выше в данной классификации распознаватель, тем сложнее создавать алгоритм, обеспечивающий его работу. Зависимость затрат на создание алгоритма от типа внешней памяти также очевидна. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.2, с. 28–30.)*

Связь сложности распознавателя с типом языка. Сложность распознавателя также напрямую связана с типом языка, входные цепочки которого может принимать (допускать) распознаватель. Выше было определено четыре основных типа языков. Доказано, что для каждого из этих типов языков существует свой тип распознавателя с определённым составом компонентов и, следовательно, с заданной сложностью алгоритма работы. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.3, с. 30.)*

Языки типа 0. Для языков с фразовой структурой (тип 0) необходим распознаватель, равномощный машине Тьюринга, — недетерминированный двусторонний автомат, имеющий неограниченную внешнюю память. Поэтому для языков данного типа нельзя гарантировать, что за ограниченное время на ограниченных вычислительных ресурсах распознаватель завершит работу и примет решение о том, принадлежит или нет входная цепочка заданному языку. Отсюда можно заключить, что практического применения языки с фразовой структурой не имеют, а потому далее они не рассматриваются. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.3, с. 30–31.)*

Языки типа 1. Для контекстно-зависимых языков (тип 1) распознавателями являются двусторонние недетерминированные автоматы с линейно ограниченной внеш-

ней памятью. Алгоритм работы такого автомата в общем случае имеет экспоненциальную сложность — количество шагов (тактов), необходимых автомату для распознавания входной цепочки, экспоненциально зависит от длины этой цепочки. Следовательно, и время, необходимое на разбор входной цепочки по заданному алгоритму, экспоненциально зависит от длины входной цепочки символов. Как правило, такие распознаватели применяются для автоматизированного перевода и анализа текстов на естественных языках, когда временные ограничения на разбор текста несущественны. В компиляторах контекстно-зависимые распознаватели не применяются, поскольку скорость работы компилятора имеет существенное значение, а синтаксический разбор текста программы можно выполнять в рамках более простого, контекстно-свободного типа языков. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.3, с. 31.)*

Языки типа 2. Для контекстно-свободных языков (тип 2) распознавателями являются односторонние недетерминированные автоматы с магазинной (стековой) внешней памятью — МП-автоматы. При простейшей реализации алгоритма работы такого автомата он имеет экспоненциальную сложность, однако путём некоторых усовершенствований алгоритма можно добиться полиномиальной (кубической) зависимости времени, необходимого на разбор входной цепочки, от длины этой цепочки. Следовательно, можно говорить о полиномиальной сложности распознавателя для КС-языков. Среди всех КС-языков можно выделить класс детерминированных КС-языков, распознавателями для которых являются детерминированные автоматы с магазинной (стековой) внешней памятью — ДМП-автоматы. Эти языки обладают свойством однозначности — доказано, что для любого детерминированного КС-языка всегда можно построить однозначную грамматику. Кроме того, для таких языков существует алгоритм работы распознавателя с квадратичной сложностью. Поскольку эти языки являются однозначными, именно они представляют наибольший интерес для построения компиляторов. Более того, среди всех детерминированных КС-языков существуют такие классы языков, для которых можно построить линейный распознаватель — распознаватель, у которого время принятия решения о принадлежности цепочки языку имеет линейную зависимость от длины цепочки. Синтаксические конструкции практически всех существующих языков программирования могут быть отнесены к одному из таких классов языков. Это обстоятельство очень важно для разработки современных быстродействующих компиляторов. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.3, с. 31.)*

Языки типа 3. Для регулярных языков (тип 3) распознавателями являются детерминированные автоматы без внешней памяти — конечные автоматы (КА). Это очень простой тип распознавателя, который всегда предполагает линейную зависимость времени на разбор входной цепочки от её длины. Кроме того, конечные автоматы имеют важную особенность: любой недетерминированный КА всегда может быть преобразован в детерминированный. Это обстоятельство существенно упрощает разработку программного обеспечения, обеспечивающего функционирование распознавателя. Простота и высокая скорость работы распознавателей определяют широкую область применения регулярных языков. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.3, с. 31–32.)*

Практическое значение классификации распознавателей. Классификация распознавателей по типам языков непосредственно объясняет разделение фаз трансляции. Для лексического анализа, где требуется выделять из входного потока символы идентификаторов, констант, ключевых слов и разделителей, достаточно регулярных грамматик и конечных автоматов; они обеспечивают линейную сложность и просты в реализации. Для синтаксического анализа, где необходимо учитывать вложенную структуру выражений, операторов и определений, используются КС-грамматики и МП-автоматы; среди них наиболее важны детерминированные КС-языки, для которых существуют однозначные грамматики и эффективные распознаватели. Контекстно-зависимые и неограниченные грамматики в компиляторах, как правило, не применяются из-за высокой сложности распознавания. Таким образом, иерархия Хомского и соответствующая ей классификация распознавателей служат теоретической основой для выбора формализма при описании лексики и синтаксиса языка программирования и для построения соответствующих анализаторов. *(Источник: Гагарина, Кокорева, гл. 1, §1.6.3, с. 30–32; гл. 2, §2.1, с. 33; гл. 3, вводная часть, с. 55–56.)*

1.4.6 Использование формальных грамматик в лексическом анализе

Место лексического анализа в процессе трансляции. Лексический анализ — первая наиболее простая фаза трансляции, выполняемая программой — лексическим анализатором (сканером). При этом сканер считывает поток входных символов и выделяет элементарные конструкции языка — лексемы, которые затем передаёт процедуре синтаксического разбора. Как правило, для лексического анализа служат конечные автоматы и регулярные грамматики. Работу транслятора можно представить состоящей из нескольких фаз; первая фаза собственно трансляции называется лексическим анализом, а программа, выполняющая её, — лексическим анализатором (иногда сканером). Лексика языка программирования представляет собой правила составления слов программы (идентификаторы, константы, служебные слова, комментарии) из символов языка. Особенность любой лексики заключается в том, что её элементами являются регулярные линейные последовательности символов. Например, идентификатор — это произвольная последовательность букв, цифр и символа подчёркивания, начинающаяся с буквы или символа подчёркивания. *(Источник: Гагарина, Кокорева, гл. 2, вводная часть, с. 33; §2.1, с. 33.)*

Лексический анализатор как конечный автомат. В компьютерной технике широко применяется такой математический аппарат, как конечные автоматы (КА). Например, КА используются при разработке управляющей схемы, реализующей заданный алгоритм, для реализации аппаратных устройств. Программные КА являются формальной основой лексического анализатора. Конечный автомат — это один из самых простых механизмов, используемых при работе с языками. У этого распознавателя есть только фиксированный набор ячеек памяти, а управляющее устройство может только сдвигаться вправо по входной ленте и изменять состояния, хранящиеся в его памяти. На вход лексического анализатора подаётся последовательность символов входного языка, в которой анализатор выделяет простейшие конструкции языка, называемые лексическими единицами, и заменяет их внутренним представлением — лексемами. *(Источник: Гагарина, Кокорева, гл. 2, §2.1, с. 33–34.)*

Диаграмма состояний-переходов конечного автомата. Конечный автомат представляет собой систему, которая в каждый момент времени может находиться в одном из заданных состояний конечного множества. При нахождении КА в определённом состоянии и поступлении на вход одного из множества входных сигналов автомат переходит в однозначно определённое состояние и вырабатывает определённый выходной сигнал. Иными словами, находясь в состоянии A, при получении сигнала a автомат переходит в состояние B и выполняет действие d. Каждый такой переход называют шагом работы конечного автомата. Обычно поведение конечного автомата представляют в виде его диаграммы состояний-переходов. На диаграмме каждое состояние автомата изображается в виде кружка с подписью — «именем» состояния, каждый переход обозначается дугой со стрелкой, надпись над которой соответствует входному воздействию. На рис. 2.2 пособия показан пример диаграммы состояний-переходов конечного автомата, распознающего идентификатор на входе или константу. *(Источник: Гагарина, Кокорева, гл. 2, §2.1.1, с. 34.)*

Условия применимости КА в качестве лексического анализатора. Для того чтобы конечный автомат мог использоваться в качестве лексического анализатора, должны выполняться несколько условий: автомат должен иметь некоторое начальное состояние, из которого он начинает работу; автомат должен иметь конечное число состояний; в каждом состоянии не могут осуществляться переходы в разные состояния по одному и тому же условию (входному сигналу), то есть автомат должен быть детерминированным. Эти условия обеспечивают однозначность разбора и конечность времени работы сканера. *(Источник: Гагарина, Кокорева, гл. 2, §2.1.1, с. 35.)*

Пример КА для распознавания комментариев. В пособии приведён пример программы, которая распознаёт в строке комментарии типа `/* ... */`, реализованной в виде конечного автомата. Определяются состояния КА: состояние 0 — распознавание обычного текста; состояние 1 — обнаружен символ «/»; состояние 2 — обнаружено начало комментария «/*»; состояние 3 — в комментарии обнаружен символ «*». Программа представляет собой цикл, завершающийся окончанием входной строки символов; на каждом шаге цикла анализируется текущий символ строки, являющийся входным воздействием, по которому происходит переход из состояния в состояние. Текущее состояние автомата хранится в переменной `state`. Этот пример показывает, как диаграмма состояний-переходов непосредственно преобразуется в программу-сканер. *(Источник: Гагарина, Кокорева, гл. 2, §2.1.1, пример 2.1, с. 35–36.)*

Формальное определение конечного автомата. Конечный автомат — это пятёрка

$$A = (Q, T, \delta, q_0, F),$$

где Q — конечное множество состояний автомата; T — непустое конечное множество допустимых входных символов; δ — отображение множества $Q \times T \Rightarrow R(Q)$, определяющее поведение управляющего устройства (эту функцию часто называют функцией переходов). Запись $\delta(A, a) = B$ означает, что из состояния A по входному символу a происходит переход в состояние B. Другой вид записи $\delta = (q_1, a, q_2)$ также обозначает переход из состояния q_1 в состояние q_2 по входному символу a;

$q_0 \in Q$ — начальное состояние управляющего устройства; $F \in Q$ — непустое множество заключительных состояний автомата. Диаграмма состояний-переходов определяет конечный автомат, построенный по регулярной грамматике, который допускает множество цепочек языка, определяемого этой грамматикой. Состояния и дуги диаграммы — это графическое изображение функции переходов конечного автомата из состояния в состояние при условии, что очередной анализируемый символ совпадает с символом-меткой дуги. Среди всех состояний выделяется начальное (считается, что в начальный момент работы автомат находится в этом состоянии) и конечное (если работа завершается переходом в это состояние, то анализируемая цепочка автоматом допускается). *(Источник: Гагарина, Кокорева, гл. 2, §2.1.2, определение 2.1, с. 36–37.)*

Допуск цепочки конечным автоматом. Конечный автомат допускает цепочку $a_1 a_2 \dots a_n$, если

$$\delta(q_0, a_1) = A_1;\ \delta(A_1, a_2) = A_2;\ \dots;\ \delta(A_{n-2}, a_{n-1}) = A_{n-1};\ \delta(A_{n-1}, a_n) = S,$$

где $a_i \in T$, $A_j \in Q$, $j = 1, 2, \dots, n-1$; $i = 1, 2, \dots, n$; q_0 — начальное состояние, $S \in F$ — одно из заключительных состояний. Множество цепочек, допускаемых конечным автоматом, составляет определяемый им язык. *(Источник: Гагарина, Кокорева, гл. 2, §2.1.2, определения 2.2–2.3, с. 37.)*

Алгоритм разбора по диаграмме состояний КА. Одной из задач лексического анализа является разбор цепочек, то есть определение того, принадлежит ли входная цепочка данному языку $L(G)$. Разбор осуществляется следующим образом: объявляем текущим начальное состояние; далее многократно (до тех пор, пока не считаем признак конца цепочки) выполняем шаги: считываем очередной символ исходной цепочки и переходим из текущего состояния в другое по дуге, помеченной этим символом; состояние, в которое мы при этом попадаем, становится текущим. При работе алгоритма возможны следующие ситуации: прочитана вся цепочка; на каждом шаге находилась единственная дуга, помеченная очередным символом анализируемой цепочки; в результате последнего перехода оказались в состоянии S — это означает, что исходная цепочка принадлежит $L(G)$; прочитана вся цепочка; на каждом шаге находилась единственная «нужная» дуга; в результате последнего шага автомат перешёл в состояние, отличное от S — цепочка не принадлежит $L(G)$; на некотором шаге не нашлось дуги, выходящей из текущего состояния и помеченной очередным анализируемым символом — цепочка не принадлежит $L(G)$; на некотором шаге работы алгоритма оказалось, что есть несколько дуг, выходящих из текущего состояния, помеченных очередным анализируемым символом, но ведущих в разные состояния — это говорит о недетерминированности разбора. *(Источник: Гагарина, Кокорева, гл. 2, §2.1.3, с. 37.)*

Регулярные грамматики и языки. В дальнейшем, говоря о регулярных грамматиках, будем иметь в виду леволинейные грамматики, которые используются в методах восходящего разбора. Это обусловлено тем, что все языки программирования определяют нотацию записи «слева направо». В той же нотации работают и компиляторы. На основе леволинейных грамматик строятся КА, которые служат для лексического анализа. Напомним, что в леволинейной грамматике $G = (T, N, P, S)$

все правила имеют вид $A \to Ba$ либо $A \to a$, где $A \in N$, $B \in N$, $a \in T$. *(Источник: Гагарина, Кокорева, гл. 2, §2.2, с. 38.)*

Алгоритм разбора цепочки для леволинейной грамматики. Для удобства предположим, что анализируемая цепочка заканчивается специальным символом $\perp$ — признаком конца цепочки. Алгоритм разбора будет следующим: первый символ исходной цепочки $a_1 a_2 \dots a_n \perp$ заменяем нетерминалом A, для которого в грамматике есть правило вывода $A \to a_1$ (другими словами, производим свёртку терминала a_1 к нетерминалу A); затем многократно (пока не считан признак конца цепочки) выполняем шаги: полученный на предыдущем шаге нетерминал A и расположенный непосредственно справа от него очередной терминал a_i исходной цепочки заменяем нетерминалом B (производим свёртку), для которого в грамматике действует правило вывода $B \to Aa_i$ $(i = 2, 3, \dots, n)$. Таким образом, дерево разбора строится снизу вверх, от листьев к корню, пока не достигнем целевого нетерминала грамматики S. При работе этого алгоритма возможны следующие ситуации: прочитана вся цепочка; на каждом шаге находилась единственная нужная свёртка; на последнем шаге свёртка произошла к символу S — исходная цепочка принадлежит $L(G)$; прочитана вся цепочка; на каждом шаге находилась единственная нужная свёртка; на последнем шаге свёртка произошла к символу, отличному от S — цепочка не принадлежит $L(G)$; на некотором шаге не нашлось нужной свёртки, то есть для полученного на предыдущем шаге нетерминала A и расположенного непосредственно справа от него очередного терминала a исходной цепочки не нашлось нетерминала B, для которого в грамматике действовало бы правило вывода $B \to Aa$ — цепочка не принадлежит $L(G)$. Если на некотором шаге нашлось несколько свёрток, это говорит о недетерминированности разбора. *(Источник: Гагарина, Кокорева, гл. 2, §2.2.1, с. 38–40.)*

Таблица детерминированного разбора. Для детерминированного разбора все возможные свёртки удобно представить в виде таблицы, строки которой помечены нетерминальными символами, а столбцы — терминальными. Значение в ячейке таблицы представляет собой нетерминальный символ, к которому сворачивается пара нетерминал—терминал в соответствующих строке и столбце. Если такой свёртки не существует, в таблице ставится «—». В пособии приведён пример грамматики $G = (\{a, b, \perp\}, \{S, A, B, C\}, P, S)$ с правилами $S \to C \perp$, $C \to Ab \mid Ba$, $A \to a \mid Ca$, $B \to b \mid Cb$, для которой построена таблица детерминированного разбора. *(Источник: Гагарина, Кокорева, гл. 2, §2.2.1, пример 2.3, с. 40.)*

Построение конечного автомата на основе регулярной грамматики. Для того чтобы на основе леволинейной грамматики $G = (T, N, P, S)$ построить конечный автомат $A = (Q, T, \delta, q_0, F)$, необходимо выполнить алгоритм: определить множество состояний автомата Q. При этом каждому состоянию КА будет соответствовать один нетерминальный символ из множества N грамматики G. Кроме того, к множеству состояний автомата добавляется ещё одно дополнительное состояние — H, то есть $Q = N \cup \{H\}$. Входным алфавитом автомата A является множество терминальных символов T грамматики G. Далее формируется функция переходов КА: просматривается всё множество правил P исходной грамматики; если встречается правило вида $A \to a$, где $A \in N$, $a \in T$, то добавляется функция переходов $\delta(H, a) = A$; если встречается правило вида $A \to Bb$, где $A, B \in N$, $b \in T$, то добавляется функция переходов $\delta(B, b) = A$. Начальным состоянием автомата A является состояние H:

$q_0 = H$. Множество конечных состояний автомата A представляет собой одно состояние, соответствующее целевому символу грамматики G: $F = \{S\}$. Процесс создания диаграммы состояний-переходов КА: строим вершины графа, соответствующие нетерминалам грамматики, и добавляем ещё одну вершину, помеченную символом H; вершины графа являются состояниями конечного автомата, а символ H — начальное состояние; соединяем эти состояния дугами по правилам: для каждого правила грамматики вида $N \to t$ соединяем направленной дугой состояния H и N и помечаем дугу символом t; для каждого правила $N \to Mt$ соединяем дугой состояния N и M, стрелка направлена от M к N, помечаем дугу символом t. *(Источник: Гагарина, Кокорева, гл. 2, §2.2.2, с. 40–41.)*

Построение леволинейной грамматики на основе конечного автомата. Для того чтобы построить на основе конечного автомата $A = (Q, T, \delta, q_0, F)$ регулярную грамматику $G = (T, N, P, S)$, необходимо выполнить алгоритм: множество терминальных символов грамматики G строится из алфавита входных символов T автомата A; множество нетерминальных символов грамматики G строится из множества состояний автомата A, исключая начальное состояние q_0: $N = Q \setminus \{q_0\}$, причём каждому нетерминалу соответствует одно состояние автомата; для построения правил грамматики просматривается функция переходов автомата A для всех возможных состояний из множества Q и всех возможных входных символов из множества T. При этом для каждого состояния A и входного символа t справедливо: если $\delta(A, t) = \varnothing$, то ничего не выполняем; если $\delta(A, t) = \{B_1, B_2, \ldots, B_n\}$, $n > 0$, $A, B_i \in Q$, $i = 1, 2, \ldots, n$, $t \in T$, тогда для всех состояний B_i выполняем следующее: во множество P правил грамматики G добавляем правило $B_i \to t$, если $A = q_0$; во множество P правил грамматики G добавляем правило $B_i \to At$, если $A \neq q_0$. Если множество конечных состояний F автомата A содержит только одно состояние $F = \{F_0\}$, то целевым символом S грамматики G становится символ множества N, соответствующий этому состоянию: $S = F_0$; иначе, если множество конечных состояний F автомата A содержит более одного состояния $F = \{F_1, F_2, \ldots, F_n\}$, $n > 1$, тогда во множество нетерминальных символов N грамматики G добавляется новый нетерминальный символ S: $N = N \cup \{S\}$, а во множество правил P грамматики G добавляются правила: $S \to F_1 \mid F_2 \mid \ldots \mid F_n$. На этом построение грамматики заканчивается. *(Источник: Гагарина, Кокорева, гл. 2, §2.2.3, с. 41–42.)*

Преобразование недетерминированного конечного автомата в детерминированный. Для лексического анализа можно использовать только детерминированные конечные автоматы, поэтому нас интересует возможность преобразования недетерминированного конечного автомата (НКА) в детерминированный (ДКА). Один из наиболее важных результатов теории конечных автоматов состоит в том, что классы языков, определяемых недетерминированными и детерминированными конечными автоматами, совпадают. Это означает, что для любого НКА всегда можно построить ДКА, определяющий тот же язык. Некоторые регулярные грамматики содержат правила с одинаковыми правыми частями для разных нетерминалов в левой части. В этом случае непонятно, к какому нетерминалу делать свёртку, то есть в диаграмме состояний-переходов из одного состояния выходит несколько дуг, ведущих в разные состояния, но помеченных одним и тем же входным символом. Функция перехода такого КА $\delta(A, t) = \{B_1, B_2, \ldots, B_n\}$ означает, что из состояния A по входному символу t можно осуществить переход в любое из состояний B_i, $i = 1, 2, \ldots, n$. Реализовать

разбор цепочки с помощью такого КА практически невозможно, поскольку придётся перебирать множество различных путей для нахождения правильного, поэтому встаёт задача преобразовать НКА в ДКА. *(Источник: Гагарина, Кокорева, гл. 2, §2.3, §2.3.1, с. 42–43.)*

Алгоритм детерминирования. Процесс преобразования недетерминированного конечного автомата $A = (Q, T, \delta, H, S)$ в детерминированный конечный автомат $A' = (Q', T, \delta', H', S')$ с тем же самым языком заключается в следующем: формируем множество состояний Q', которое содержит все подмножества множества Q; каждое состояние из Q' будем обозначать $[A_1, A_2, \ldots, A_n]$, где $A_i \in Q$; функцию δ' определим как $\delta'([A_1, A_2, \ldots, A_n], t) = [B_1, B_2, \ldots, B_m]$, где для каждого $1 \leq j \leq m$ $\delta(A_j, t) = B_j$ для каких-либо $1 \leq i \leq n$; если $H = \{H_1, H_2, \ldots, H_k\}$, тогда $H' = [H_1, H_2, \ldots, H_k]$; если $S = \{S_1, S_2, \ldots, S_p\}$, тогда S' — все состояния из Q', имеющие вид $[\ldots S_i \ldots]$, $S_i \in S$ для какого-либо $1 \leq i \leq p$. Состояния из Q', не достижимые из начального состояния, можно исключить. В пособии приведён пример 2.6, в котором для НКА $A = (\{H, A, B, S\}, \{0, 1\}, \delta, \{H\}, \{S\})$ с переходами $\delta(H, 1) = B$, $\delta(B, 0) = A$, $\delta(A, 1) = S$ строится соответствующий ДКА; после удаления недостижимых состояний остаются состояния $[H]$, $[B]$, $[A]$, $[BS]$, и ДКА имеет вид $A' = (\{[H], [B], [A], [BS]\}, \{0, 1\}, \delta', H, \{[BS]\})$. *(Источник: Гагарина, Кокорева, гл. 2, §2.3.2, пример 2.6, с. 43–45.)*

Минимизация конечного автомата. Одной из задач преобразования КА является удаление из него избыточных состояний или минимизация. Алгоритм минимизации включает следующие шаги: исключаем из автомата все состояния, не достижимые из начального; разбиваем все состояния КА на классы эквивалентности следующим способом: 0-эквивалентное разбиение — в первый класс относим все конечные состояния, а во второй — все остальные; 1-эквивалентное разбиение — выделяем состояния, из которых по одинаковым символам происходит переход в 0-эквивалентные состояния; последовательно строим $n + 1$-эквивалентные разбиения по n-эквивалентным, при этом увеличивая число классов эквивалентности до тех пор, пока $n + 1$-эквивалентное разбиение не совпадёт с n-эквивалентным; полученные классы эквивалентности представляют собой состояния минимизированного КА, эквивалентного исходному. В пособии приведён пример 2.7, в котором из автомата на рис. 2.6, а удаляются недостижимые состояния F и G, а затем строится минимизированный автомат на рис. 2.6, б. *(Источник: Гагарина, Кокорева, гл. 2, §2.3.3, пример 2.7, с. 45–47.)*

Токены, шаблоны и лексемы. В лексическом анализе употребляются термины «токен», «шаблон» и «лексема». При этом во входном потоке могут встречаться строки, для которых в качестве выхода получается один и тот же токен. Означенные строки описываются правилом, именуемым шаблоном (pattern), связанным с токеном. О шаблоне говорят, что он соответствует (match) каждой строке в наборе. Лексема, представляя собой последовательность символов исходной программы, в свою очередь, соответствует шаблону токена. В большинстве языков программирования в качестве токенов выступают ключевые слова, операторы, идентификаторы, константы, строки литералов и символы пунктуации, например скобки, запятые и точки с запятой. Возврат токена чаще всего реализуется путём передачи целого числа, соответствующего токену. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.3, с. 48.)*

Таблица токенов. В пособии приведена таблица 2.3, иллюстрирующая терминологию лексического анализа. В ней для каждого токена указан пример лексем и неформальное описание шаблона: токен `const` — лексема `const` — шаблон «const»; токен `if` — лексема `if` — шаблон «if»; токен `relation` — лексемы `<`, `<=`, `==`, `!=`, `>`, `>=` — шаблон «< или <= или == или != или > или >=»; токен `identifier` — лексемы `count`, `i`, `D2`, `pi` — шаблон «буква, за которой следуют буквы и цифры»; токен `number` — лексемы `3.1415`, `0`, `6e-4` — шаблон «любая числовая константа»; токен `literal` — лексема `<core dumped>` — шаблон «любые символы между двумя парными кавычками, исключая сами кавычки». Шаблон — правило, описывающее набор лексем, которые могут представлять определённый токен в исходной программе. Так, шаблон токена `const` представляет собой просто строку `const`, являющуюся ключевым словом. Шаблон токена `relation` — набор всех шести операторов сравнения в языке C. Во многих языках ряд строк является зарезервированным, то есть их смысл предопределён и не может быть изменён пользователем. Если ключевые слова не зарезервированы, лексический анализатор должен уметь определить, с чем он имеет дело — ключевым словом или идентификатором, введённым пользователем. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.3, с. 48–49.)*

Атрибуты токенов. Если шаблону соответствует несколько лексем, лексический анализатор должен обеспечить дополнительную информацию о лексемах для последующих фаз компиляции. Например, шаблон `number` соответствует как строке `0`, так и строке `77.7`, и при генерации кода крайне важно знать, какая именно строка соответствует токену. Лексический анализатор хранит информацию о токенах в связанных с ними атрибутах. Токены определяют работу синтаксического анализатора; атрибуты — трансляцию токенов. На практике токены обычно имеют единственный атрибут — указатель на запись в таблице символов, в которой хранится информация о токене. Для диагностических целей могут понадобиться как лексемы идентификаторов, так и номера строк, в которых они впервые встретились в программе. Вся эта и другая информация может храниться в записях в таблице символов. Следует отметить, что для некоторых токенов (например, для токена присваивание) нет необходимости хранить какие-либо атрибуты. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.4, с. 49–50.)*

Лексические ошибки. На уровне лексического анализатора определяются только некоторые ошибки, поскольку лексический анализатор рассматривает исходный текст программы в ограниченном контексте. Если в программе на языке C строка `fi` впервые встретится в контексте `fi(a==f(x))`, лексический анализатор не сможет определить, что именно представляет собой `fi` — неверно записанное слово `if` или необъявленный идентификатор функции. Поскольку буквосочетание `fi` является корректным идентификатором, лексический анализатор должен вернуть токен для идентификатора и предоставить обработку ошибки другой части компилятора. Представим теперь, что лексический анализатор не способен продолжать работу, поскольку ни один из шаблонов не соответствует оставшейся части входного потока. Простейшим в этой ситуации будет восстановление в «режиме паники». При восстановлении корректного входного потока могут выполняться различные преобразования; самая простая стратегия состоит в проверке, не может ли начало оставшейся части входного потока быть заменено корректной лексемой путём единственного преобразования. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.5, с. 50.)*

Определение токенов с помощью регулярных выражений. В определении токенов важную роль играют регулярные выражения. Каждый шаблон соответствует множеству строк, так что регулярные выражения можно рассматривать как имена таких множеств. В языке Pascal идентификатор представляет собой букву, за которой следует нуль или несколько букв или цифр. С помощью регулярных выражений можно определить идентификатор, например, так:

```
letter (letter | digit)*
```

Вертикальная черта означает союз «или», скобки используются для группировки подвыражений, а непосредственное соседство символа `letter` с оставшейся частью выражения означает конкатенацию. Регулярное выражение строится из более простых регулярных выражений с использованием набора правил. Предположим, что r и s — регулярные выражения. Тогда: $(r) \mid (s)$ означает либо r, либо s; $(r)(s)$ означает конкатенацию; $(r)^*$ — 0 или несколько конкатенаций выражения r; (r) эквивалентна r. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.6, с. 50–51.)*

Примеры регулярных выражений. Пусть дан алфавит символов $\{a, b\}$. Тогда: регулярное выражение $a \mid b$ обозначает множество $\{a, b\}$; регулярное выражение $(a \mid b)(a \mid b)$ обозначает множество $\{aa, ab, ba, bb\}$ (другое регулярное выражение для того же множества — $aa \mid ab \mid ba \mid bb$); регулярное выражение a^* — множество всех строк из 0 и более a; регулярное выражение $(a \mid b)^*$ обозначает множество всех строк, содержащих 0 или несколько экземпляров a и b, то есть множество всех строк из a и b (другое регулярное выражение для этого множества — $(a^*b^*)^*$); регулярное выражение $a \mid a^*b$ — множество, содержащее строку a и все строки, состоящие из 0 или нескольких a, за которыми следует b. Для удобства записи регулярным выражениям можно давать имена и определять регулярные выражения с использованием этих имён так, как если бы это были символы. Некоторые конструкции встречаются в регулярных выражениях настолько часто, что для них введены специальные сокращения: для обозначения одного или нескольких экземпляров используется унарный постфиксный оператор $+$, например $(r)^+$; для обозначения нуля или одного экземпляра используется унарный постфиксный оператор ?, например $(r)^?$; запись $[abc]$, где a, b, c — символы алфавита, означает регулярное выражение $a \mid b \mid c$; сокращённый класс символов типа $[a - z]$ означает регулярное выражение $a \mid b \mid \ldots \mid z$. С использованием классов символов можно описать идентификаторы как строки, заданные регулярным выражением `[A-Za-z][A-Za-z0-9]*`. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.6, пример 2.8, с. 51.)*

Распознавание токенов: пример. В пособии рассматривается язык, порождаемый грамматикой

`stmt` → `if expr then stmt` | `if expr then stmt else stmt` | `EPS`

`expr` → `term relop term` | `term`

`term` → `id` | `num`,

где нетерминалы `if`, `then`, `else`, `relop`, `id` и `num` порождают множества строк, задаваемых регулярными определениями: `if` → `if`; `then` → `then`; `else` → `else`; `relop` → `<` | `<=` | `=` | `<>` | `>` | `>=`; `id` → `letter (letter | digit)*`; `num` → `digit+ (.digit+)? (E(+ | -)? digit+)?`; `letter` → `[A-Za-z]`; `digit` → `[0-9]`. Для этого фрагмента

языка лексический анализатор будет распознавать ключевые слова `if`, `then`, `else`, а также лексемы `relop`, `id` и `num`. Для простоты будем считать, что ключевые слова зарезервированы и, таким образом, не могут использоваться в качестве идентификаторов. Предположим также, что лексемы разделены «пустым пространством», состоящим из непустых последовательностей пробелов, символов табуляций и новой строки. Лексический анализатор будет отбрасывать эти символы путём сравнения строки с регулярным определением `ws`, имеющим регулярное определение `delim` → `space` | `tab` | `newline`, `ws` → `delim+`. Если находится соответствие нетерминалу `ws`, лексический анализатор не возвращает токен синтаксическому анализатору; вместо этого происходит поиск нового токена, следующего за пробелами, и передача его синтаксическому анализатору. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.7, с. 52–53.)*

Таблица пар «атрибут—значение». В пособии приведена таблица 2.4, в которой каждому регулярному выражению сопоставлены токен и атрибут-значение: `ws` — токен не возвращается; `if` — токен `if`; `then` — токен `then`; `else` — токен `else`; `id` — токен `id`, атрибут — указатель на запись в таблице; `num` — токен `num`, атрибут — указатель на запись в таблице; < — токен `relop`, атрибут `LT`; <= — токен `relop`, атрибут `LE`; = — токен `relop`, атрибут `EQ`; <> — токен `relop`, атрибут `NE`; > — токен `relop`, атрибут `GT`; >= — токен `relop`, атрибут `GE`. Поскольку ключевые слова есть не что иное, как последовательности букв, они представляют собой исключение из правила, гласящего, что последовательность букв и цифр, начинающаяся с буквы, является идентификатором. Но вместо кодирования исключений можно рассматривать ключевые слова как специальные идентификаторы. Тогда после выделения токена из потока символов выполняется некоторый код, который определяет, чем является лексема — ключевым словом или идентификатором. Таким образом, при добавлении нового ключевого слова оно просто заносится в таблицу ключевых слов, но код при этом не меняется. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.7, с. 53–54.)*

Построение распознавателя для чисел. Построение распознавателя беззнаковых чисел, задаваемых регулярным определением

$$\texttt{num} \rightarrow \texttt{digit+ (.digit+)? (E(+ | -)? digit+)?},$$

имеет ряд особенностей. Данное определение имеет вид `digit+ fraction? exponent?`, где `fraction` и `exponent` — необязательные части. При построении распознавателя необходимо предусмотреть обработку всех трёх частей и корректное завершение токена при встрече символа, не являющегося цифрой или допустимым символом числа. *(Источник: Гагарина, Кокорева, гл. 2, §2.4.7, с. 54.)*

1.4.7 Использование формальных грамматик в синтаксическом анализе

Место синтаксического анализа в процессе трансляции. Синтаксический анализ — более сложная по сравнению с лексическим анализом фаза трансляции, которая служит для построения внутреннего представления программы (промежуточного кода). Синтаксический анализатор получает строку токенов от лексического анализатора и проверяет, может ли эта строка порождаться грамматикой исходного языка. Он также сообщает обо всех выявленных ошибках, причём достаточно внятно и полно. Кроме того, он должен уметь обрабатывать обычные, часто встречающиеся

ошибки и продолжать работу с оставшейся частью программы. Синтаксические анализаторы всегда строятся на основе контекстно-свободных грамматик. *(Источник: Гагарина, Кокорева, гл. 3, вводная часть, с. 55–56.)*

Типы синтаксических анализаторов. Имеются три основных типа синтаксических анализаторов грамматик: нисходящие; восходящие; универсальные. Универсальные методы разбора, такие как алгоритмы Кока — Янгера — Касами или Эрли, могут работать с любой грамматикой. Однако эти методы слишком неэффективны для использования в промышленных компиляторах. Методы, обычно применяемые в компиляторах, классифицируются как нисходящие (top-down) и восходящие (bottom-up). Нисходящие синтаксические анализаторы строят дерево разбора сверху (от корня) вниз (к листьям), тогда как восходящие начинают с листьев и идут к корню. В обоих случаях входной поток синтаксического анализатора сканируется посимвольно слева направо. Наиболее эффективные нисходящие и восходящие методы работают только с подклассами грамматик, однако некоторые из этих подклассов, такие как LL- и LR-грамматики, достаточно выразительны для описания большинства синтаксических конструкций языков программирования. Реализованные вручную синтаксические анализаторы чаще работают с LL-грамматиками. Для описания синтаксиса языка программирования используют однозначные приведённые КС-грамматики. *(Источник: Гагарина, Кокорева, гл. 3, вводная часть, с. 55–56.)*

Приведённые грамматики. Приведённые грамматики — это КС-грамматики, которые не содержат недостижимых и бесплодных символов, циклов и λ-правил («пустых» правил). Приведённые грамматики называют также КС-грамматиками в каноническом виде. Для того чтобы преобразовать произвольную КС-грамматику к приведённому виду, необходимо удалить: все бесплодные символы; все недостижимые символы; λ-правила; цепные правила. Следует подчеркнуть, что шаги преобразования должны выполняться строго в указанном порядке. *(Источник: Гагарина, Кокорева, гл. 3, §3.1, с. 56.)*

Удаление бесплодных символов. Бесплодные нетерминальные символы не участвуют в порождении цепочек языка, поэтому могут быть удалены из грамматики. Символ $A \in N$ называется *бесплодным* в грамматике $G = (T, N, P, S)$, если множество $\{\alpha \mid \alpha \in T^*, A \Rightarrow^* \alpha\} = \varnothing$. Метод удаления бесплодных символов состоит в рекурсивном построении множеств $N_0, N_1, \ldots, N_n, \ldots$: полагаем $N_0 = \varnothing$, $i = 1$; строим $N_i = \{A \mid (A \to \alpha) \in P \text{ и } \alpha \in (N_{i-1} \cup T)^*\} \cup N_{i-1}$; если $N_i \neq N_{i-1}$, то присваиваем $i = i + 1$ и переходим к предыдущему шагу, иначе $N' = N_i$; множество правил P' состоит из правил множества P, содержащих только символы из $N' \cup T$; $G' = (T, N', P', S)$. *(Источник: Гагарина, Кокорева, гл. 3, §3.1.1, определение 3.1, с. 56–57.)*

Удаление недостижимых символов. Символ $X \in (T \cup N)$ называется *недостижимым* в грамматике $G = (T, N, P, S)$, если он не появляется ни в одной сентенциальной форме этой грамматики. Метод удаления недостижимых символов заключается в построении множеств $V_0, V_1, \ldots, V_n, \ldots$: полагаем $V_0 = \{S\}$, $i = 1$; строим $V_i = \{X \mid X \in (T \cup N), (A \to \alpha X \beta) \in P \text{ и } A \in V_{i-1}\} \cup V_{i-1}$; если $V_i \neq V_{i-1}$, то $i = i + 1$ и переходим к предыдущему шагу, иначе $N' = V_i \cap N$; $T' = V_i \cap T$; P' состоит из пра-

вил множества P, содержащих только символы из V_i; $G' = (T', N', P', S)$. *(Источник: Гагарина, Кокорева, гл. 3, §3.1.2, определение 3.2, с. 57.)*

Исключение цепных правил. Правило грамматики вида $A \to B$, где $A, B \in N$, называется *цепным*. Для КС-грамматики G, содержащей цепные правила, всегда можно построить эквивалентную ей грамматику G' без цепных правил. Правила грамматики G вида $A \to B$, $B \to C$, $C \to aX$ могут быть заменены одним правилом $A \to aX$, потому что вывод $A \Rightarrow B \Rightarrow C \Rightarrow aX$ цепочки aX в грамматике G может быть получен в грамматике G' с помощью правила $A \to aX$. Метод исключения цепных правил: разобьём множество правил P грамматики G на два подмножества P_1 и P_2, включая в P_1 все правила вида $A \to B$; построим множество правил $R(A)$ на множестве P_1 по следующему принципу: если существует вывод символа A_1 из A и в P_2 есть правило $A_1 \to \alpha$, где α — цепочка словаря $(N \cup T)^*$, то правило $A_1 \to \alpha$ помещается в множество $R(A_1)$; построим новое множество правил P' путём объединения правил P_2 и всех построенных множеств $R(A)$. В результате получим грамматику $G' = (T, N, P', S)$, которая эквивалентна заданной и не содержит правил вида $A \to B$ (цепных). *(Источник: Гагарина, Кокорева, гл. 3, §3.1.3, определение 3.3, с. 57–58.)*

Исключение аннулирующих правил. Грамматика, не содержащая пустых, или аннулирующих, правил, называется *неукорачивающей*. Неукорачивающей является также грамматика, которая содержит только одно правило вида $S \to \lambda$, где S — начальный символ грамматики и он не встречается в правых частях остальных правил грамматики. Правило вида $A \to \lambda$ называется *пустым* (аннулирующим) правилом. Для каждой КС-грамматики G', содержащей аннулирующие правила, можно построить эквивалентную ей неукорачивающую грамматику G, такую, что $L(G') = L(G)$. Исключение из грамматики пустых правил приведёт к построению дополнительных правил и к «разрастанию» грамматики. Для того чтобы построить дополнительные правила, необходимо выполнить все возможные подстановки пустой цепочки вместо аннулирующего нетерминала во все правила грамматики. Если же в грамматике есть правило вида $S \to \lambda$, где S — начальный символ грамматики, и символ S входит в правые части других правил грамматики, то следует ввести новый начальный символ I и заменить правило $S \to \lambda$ двумя новыми правилами: $I \to \lambda$ и $I \to S$. *(Источник: Гагарина, Кокорева, гл. 3, §3.1.4, определение 3.4, с. 58–60.)*

Нормальная форма Хомского. Любая приведённая КС-грамматика может быть преобразована в нормальную форму Хомского. КС-грамматика $G = (T, N, P, S)$ называется грамматикой в *нормальной форме Хомского*, если в её множестве правил P присутствуют только правила следующего вида: $A \to BC$, где $A, B, C \in N$; $A \to a$, где $A \in N$ и $a \in T$; $S \to \lambda$, если $\lambda \in L(G)$, причём начальный символ грамматики S не должен встречаться в правых частях других правил. Название «бинарная нормальная форма» происходит от того, что на каждом шаге вывода в такой грамматике один нетерминальный символ может быть заменён только на два других нетерминальных символа. Поэтому в дереве вывода грамматики в нормальной форме Хомского каждая вершина либо распадается на две другие вершины (в соответствии с первым видом правил), либо содержит один последующий лист с терминальным символом (в соответствии со вторым видом правил). Третий вид правил введён для того, чтобы к нормальной форме Хомского можно было преобразовывать грамматики КС-языков,

содержащих пустые цепочки символов. *(Источник: Гагарина, Кокорева, гл. 3, §3.2, §3.2.1, определение 3.5, с. 60–61.)*

Алгоритм преобразования к нормальной форме Хомского. Алгоритм преобразования грамматики G в эквивалентную грамматику G' в нормальной форме Хомского: преобразовать исходную грамматику G к приведённому виду, то есть исключить из неё бесплодные и недостижимые символы, цепные и λ-правила; множество нетерминальных символов N' результирующей грамматики G' строится на основе множества нетерминальных символов N исходной грамматики G: $N' = N$; далее просматриваются правила из множества P исходной грамматики G и в зависимости от вида каждого правила строится множество правил P' результирующей грамматики G' и дополняется множество нетерминальных символов этой грамматики N': если встречается правило вида $A \to a$, где $A \in N$ и $a \in T$, то оно переносится во множество P' без изменений; если встречается правило вида $A \to BC$, где $A, B, C \in N$, то оно переносится во множество P' без изменений; если встречается правило вида $S \to \lambda$, где S — целевой символ грамматики G, то оно переносится во множество P' без изменений; если встречается правило вида $A \to aB$, где $A, B \in N$ и $a \in T$, то во множество правил P' включаются правила $A \to \langle D\rangle B$ и $\langle D\rangle \to a$, и новый символ $\langle D\rangle$ добавляется во множество нетерминальных символов N' грамматики G'; если встречается правило вида $A \to Ba$, где $A, B \in N$ и $a \in T$, то во множество правил P' включаются правила $A \to B\langle D\rangle$ и $\langle D\rangle \to a$, и новый символ $\langle D\rangle$ добавляется во множество нетерминальных символов N' грамматики G'; если встречается правило вида $A \to ab$, где $A \in N$ и $a, b \in T$, то во множество правил P' включаются правила $A \to \langle D\rangle\langle E\rangle$, $\langle D\rangle \to a$ и $\langle E\rangle \to b$, новые символы $\langle D\rangle$ и $\langle E\rangle$ добавляются во множество нетерминальных символов N' грамматики G'; если встречается правило вида $A \to X_1 \dots X_k$, $k > 2$, где $A \in N$ и $X_i \in T \cup N$, $i = 1, 2, \dots, k$, то во множество правил P' включаются правила $A \to \langle X_1'\rangle\langle X_2 \dots X_k\rangle$; $\langle X_2 \dots X_k\rangle \to \langle X_3'\rangle\langle X_3 \dots X_k\rangle$; …; $\langle X_{k-1}X_k\rangle \to \langle X_{k-1}'\rangle\langle X_k\rangle$, новые нетерминальные символы $\langle X_2 \dots X_k\rangle, \langle X_3 \dots X_k\rangle, \dots, \langle X_{k-1}X_k\rangle$ включаются во множество нетерминальных символов N' грамматики G', кроме того, если $X_i \in N$, то $\langle X_i'\rangle \equiv X_i$, иначе (если $X_i \in T$) $\langle X_i'\rangle$ — это новый нетерминальный символ, он добавляется во множество N', а во множество правил P' грамматики G' добавляется правило $\langle X_i'\rangle \to X_i$, $i = 1, 2, \dots, k$. Целевым символом результирующей грамматики G' является целевой символ исходной грамматики G. Как видно из примера, в результате преобразований количество правил грамматики увеличилось, но следует помнить, что целью построения грамматики в бинарной нормальной форме является не упрощение самой грамматики, а упрощение построения распознавателя на её основе. *(Источник: Гагарина, Кокорева, гл. 3, §3.2.1, с. 61–63.)*

Нормальная форма Грейбах. КС-грамматика $G = (T, N, P, S)$ называется грамматикой в *нормальной форме Грейбах*, если она не является леворекурсивной и в её множестве правил P присутствуют только правила следующего вида: $A \to a\alpha$, где $a \in T$ и $\alpha \in N^*$; $S \to \lambda$, если $\lambda \in L(G)$, причём символ S не должен встречаться в правых частях других правил. Правило вида $A \to \alpha A$, где $A \in N$, $\alpha \in (T \cup N)^*$, называется *праворекурсивным*, а правило вида $A \to A\alpha$ — *леворекурсивным*. Для каждой КС-грамматики G, содержащей леворекурсивные правила, можно построить эквивалентную грамматику G', не содержащую леворекурсивных правил. Алгоритм устранения левой рекурсии: допустим, что исходная грамматика G содержит

правила

$$A \to A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n,$$

где ни одна цепочка β_i не начинается с символа A и $\alpha_i, \beta_i \in (T \cup N)^*$. Для того чтобы исключить левую рекурсию, необходимо ввести новый нетерминал $\langle B \rangle$ и преобразовать правила так:

$$A \to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n \mid \beta_1\langle B\rangle \mid \beta_2\langle B\rangle \mid \ldots \mid \beta_n\langle B\rangle,$$
$$\langle B\rangle \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_m \mid \alpha_1\langle B\rangle \mid \alpha_2\langle B\rangle \mid \ldots \mid \alpha_m\langle B\rangle.$$

Нормальная форма Грейбах является удобной формой представления грамматик для построения нисходящих левосторонних распознавателей, в которых недопустима левая рекурсия. *(Источник: Гагарина, Кокорева, гл. 3, §3.2.2, определение 3.6, с. 62–64.)*

Отношения между символами формальной грамматики. Для построения распознавателей используются следующие функции, определяющие множества символов грамматики. $\mathrm{FIRST}(U)$ определяет множество терминальных символов, с которых может начинаться цепочка, выводимая из нетерминального символа U. $\mathrm{FIRST}(\alpha)$ определяет множество терминальных символов, с которых может начинаться цепочка, выводимая из цепочки α. $\mathrm{FOLLOW}(U)$ определяет множество терминалов a, которые могут появиться непосредственно справа от нетерминала U в сентенциальных формах грамматики, то есть существует вывод $S \Rightarrow^* \alpha U a \beta$, где $\alpha, \beta \in (N \cup T)$. $\mathrm{LAST}(U)$ определяет множество символов, завершающих все возможные цепочки, выводимые из заданного нетерминала. Функция $\mathrm{SEL}(p)$ определяет множество выбирающих символов для правил грамматики: если продукция грамматики имеет вид $U \to a\alpha \in P$, где $a \in T$, $\alpha \in (T \cup N)^*$, то $\mathrm{SEL}(U \to a\alpha) = a$; если продукция имеет вид $U \to \lambda$, то $\mathrm{SEL}(U \to \lambda) = \mathrm{FOLLOW}(U)$. Множество выбирающих символов определяет, какое из правил грамматики будет использоваться на следующем шаге разбора цепочки. *(Источник: Гагарина, Кокорева, гл. 3, §3.3.1, определения 3.7–3.11, с. 64–67.)*

Алгоритм вычисления FIRST. Алгоритм вычисления множества FIRST для нетерминалов грамматики G: положить значение функции $\mathrm{FIRST}(U) = \varnothing$; добавить к множеству $\mathrm{FIRST}(U)$ все символы, с которых начинаются цепочки, непосредственно выводимые из символа U: $\mathrm{FIRST}(U) = \mathrm{FIRST}(U) \cup \{X \mid U \to X\alpha \in P, X \in (T \cup N), \alpha \in (T \cup N)^*\}$; добавить к множеству $\mathrm{FIRST}(U)$ все символы, с которых начинаются цепочки, непосредственно выводимые из нетерминалов, вошедших в $\mathrm{FIRST}(U)$: $\mathrm{FIRST}(U) = \mathrm{FIRST}(U) \cup \mathrm{FIRST}(V)$, $V \in (\mathrm{FIRST}(U) \cap N)$; исключить из множества $\mathrm{FIRST}(U)$ все нетерминальные символы: $\mathrm{FIRST}(U) = \mathrm{FIRST}(U) \setminus N$. Построение множества $\mathrm{FIRST}(\alpha)$ для цепочек вида $\alpha = X_1 X_2 \ldots X_n$ выполняется так: вычислить значения функций $\mathrm{FIRST}(X_i)$ для $i = 1, 2, \ldots, n$; вычислить значение функции $\mathrm{FIRST}(\alpha)$, включив в него все элементы множеств $\mathrm{FIRST}(X_i)$, кроме пустых цепочек: $\mathrm{FIRST}(\alpha) = (\mathrm{FIRST}(X_1) \setminus \{\lambda\}) \cup (\mathrm{FIRST}(X_2) \setminus \{\lambda\}) \cup \ldots \cup (\mathrm{FIRST}(X_n) \setminus \{\lambda\})$; включить в множество $\mathrm{FIRST}(\alpha)$ пустую цепочку, если она входит во все множества $\mathrm{FIRST}(X_i)$: $\mathrm{FIRST}(\alpha) = \mathrm{FIRST}(X_1) \cup \{\lambda\}$, если $\lambda \in \mathrm{FIRST}(X_i)$ для всех i. Множество $\mathrm{LAST}(U)$ строится аналогично множеству $\mathrm{FIRST}(U)$, только для завершающих символов цепочки. *(Источник: Гагарина, Кокорева, гл. 3, §3.3.1, с. 65–67.)*

Алгоритм вычисления FOLLOW. Множество FOLLOW(U) связано с использованием в формальных грамматиках аннулирующих правил, которые заменяют нетерминал левой части на пустую цепочку, завершая тем самым цикл повторения либо исключая необязательные элементы. В этом случае принципиально важным является ответ на вопрос, в каком окружении может находиться такой нетерминал, точнее, каков последующий за нетерминалом символ. Алгоритм вычисления множества FOLLOW для нетерминалов грамматики G: положить множество FOLLOW(U) = $\varnothing$; добавить к множеству FOLLOW(U) все символы, которые в правых частях правил грамматики встречаются непосредственно за U: FOLLOW(U) = FOLLOW(U) $\cup \{X \mid V \to \alpha U X \beta \in P, V \in N, X \in (T \cup N), \alpha, \beta \in (T \cup N)^*\}$; добавить к множеству FOLLOW(S) пустую цепочку, это означает, что в конце разбора за целевым символом S цепочка кончается: FOLLOW(S) = FOLLOW(S) $\cup \{\lambda\}$; добавить к множеству FOLLOW(U) все символы, с которых начинаются цепочки, непосредственно выводимые из нетерминалов, вошедших в FOLLOW(U): FOLLOW(U) = FOLLOW(U) $\cup$ FIRST(V), $V \in$ (FOLLOW(U) $\cap N$); добавить к множеству FOLLOW(U) все символы, которые следуют за нетерминалами, вошедшими в FOLLOW(U), для которых есть аннулирующие правила: FOLLOW(U) = FOLLOW(U) $\cup$ FOLLOW(V), $V \in$ (FOLLOW(U)$\cap N$) и существует $V \to \lambda$; добавить к множеству FOLLOW(U) все символы, которые следуют за нетерминалами, для которых есть правила с нетерминалом U в правой части: FOLLOW(U) = FOLLOW(U)$\cup$FOLLOW(V), если существует $V \to \alpha U$, $\alpha \in (T \cup N)^*$; исключить из FOLLOW(U) все нетерминальные символы: FOLLOW(U) = FOLLOW(U) $\setminus N$. *(Источник: Гагарина, Кокорева, гл. 3, §3.3.1, с. 67–68.)*

S-грамматики. Для S-грамматик должны выполняться следующие условия: правая часть каждого правила грамматики начинается с терминального символа; для двух правил, имеющих одинаковые нетерминалы в левой части, правые части правил должны начинаться с различных терминалов; не допускаются аннулирующие правила. Пример S-грамматики:

$$G = (\{a, b\}, \{S, T\}, P, S), \qquad P: \quad S \to aT \mid bS; \quad T \to aT \mid b.$$

Данная грамматика порождает цепочки вида $aa \ldots aaab$ и $b \ldots baaa \ldots aabb \ldots b$. Разбор цепочек, принадлежащих языку, порождённому S-грамматикой, состоит в следующем: на каждом шаге в полученной цепочке единственный нетерминальный символ должен быть заменён на правую часть одного из правил, в котором он присутствует в левой части; для замены выбирается правило, в котором первый терминальный символ в правой части совпадает с очередным «незакрытым» символом во входной цепочке (выбирающий символ). *(Источник: Гагарина, Кокорева, гл. 3, §3.3.2, с. 67–68.)*

Q-грамматики. Данная грамматика отличается от S-грамматики наличием дополнительных аннулирующих правил вида $A \to \lambda$. При разборе цепочек языка, порождённого Q-грамматикой, необходимо руководствоваться выбирающими символами правил. Пример Q-грамматики:

$$G = (\{a, b\}, \{S, T\}, P, S),$$

$$P: \quad S \to aT \mid bSb; \quad T \to aT \mid \lambda;$$

$\mathrm{SEL}(S \to aT) = a$; $\mathrm{SEL}(S \to bSb) = b$; $\mathrm{SEL}(T \to aT) = a$; $\mathrm{SEL}(T \to \lambda) = \mathrm{FOLLOW}(T) = b$.

Тогда разбор цепочки $bbaabb$ имеет вид

$$S \Rightarrow bSb \Rightarrow bbSb \Rightarrow bbaTbb \Rightarrow bbaaTbb \Rightarrow bbaabb.$$

(Источник: Гагарина, Кокорева, гл. 3, §3.3.3, с. 68–69.)

LL(k)-грамматики. Рассмотрим дерево вывода в процессе получения левого вывода цепочки α. Промежуточная цепочка в процессе вывода состоит из цепочки из терминалов w, самого левого нетерминала A, неразобранной части входной цепочки x. Для продолжения разбора необходимо заменить нетерминал A по одному из правил вида $A \to y$, причём для детерминированного разбора правило следует выбирать специальным способом. Говорят, что грамматика имеет свойство $\mathrm{LL}(k)$, если для выбора правила достаточно рассмотреть только цепочку wAx и первые k символов непросмотренной цепочки u. Формально: грамматика G имеет свойство $\mathrm{LL}(k)$, если из существования двух цепочек левых выводов

$$S \Rightarrow^* wAx \Rightarrow wzx \Rightarrow^* wu; \qquad S \Rightarrow^* wAx \Rightarrow wtx \Rightarrow^* wv$$

и из условия $\mathrm{FIRST}(u) = \mathrm{FIRST}(v)$ следует $z = t$. Грамматика называется $\mathrm{LL}(k)$-грамматикой, если она обладает свойством $\mathrm{LL}(k)$ для некоторого $k > 0$. В названии грамматики первая литера L происходит от слова left и означает, что входная цепочка символов читается слева направо. Вторая литера L также происходит от слова left и означает, что при работе распознавателя используется левосторонний вывод. Вместо символа k в названии класса грамматики стоит некоторое число, которое показывает, сколько символов надо рассмотреть, чтобы сделать однозначный вывод. Так, существуют LL(1)-грамматики, LL(2)-грамматики и другие классы LL-грамматик. Очевидно, что LL(0)-грамматика бессмысленна, так как в этом случае разбор вообще не зависит от входной цепочки. С использованием множества выбирающих символов можно дать следующее определение: КС-грамматика называется LL(1)-грамматикой тогда и только тогда, когда множества выбирающих символов для правил с одинаковой левой частью не пересекаются. Для $\mathrm{LL}(k)$-грамматик известны следующие полезные свойства: всякая $\mathrm{LL}(k)$-грамматика для любого $k > 0$ является однозначной; существует алгоритм, позволяющий проверить, является ли заданная грамматика $\mathrm{LL}(k)$-грамматикой для строго определённого числа k. Есть, однако, неразрешимые проблемы для произвольных КС-грамматик: не существует алгоритма, который мог бы проверить, является ли заданная КС-грамматика $\mathrm{LL}(k)$-грамматикой для некоторого произвольного числа k; не существует алгоритма, который мог бы преобразовать произвольную КС-грамматику к виду $\mathrm{LL}(k)$-грамматики для некоторого k (или доказать, что преобразование невозможно). $\mathrm{LL}(k)$-грамматики служат для построения нисходящих распознавателей без возвратов. *(Источник: Гагарина, Кокорева, гл. 3, §3.3.4, определения 3.12–3.14, с. 69–72.)*

LR(k)-грамматики. Более широкий класс языков по сравнению с $\mathrm{LL}(k)$-грамматиками определяется так называемыми $\mathrm{LR}(k)$-грамматиками. На их основе строятся восходящие распознаватели без возвратов. Если в процессе LR-разбора принять детерминированное решение о замене удаётся, рассматривая только цепочку x и первые k символов непросмотренной части входной цепочки u (эти k символов называют

аванцепочкой), говорят, что грамматика обладает LR(k)-свойством. Для определения LR(k)-грамматики вводится понятие пополненной грамматики. Грамматика G' является пополненной грамматикой, построенной на основании исходной грамматики $G = (T, N, P, S)$, если выполняются следующие условия: грамматика G' совпадает с грамматикой G, если целевой символ S не встречается нигде в правых частях правил грамматики G; грамматика G' строится как грамматика $G' = (T, N \cup \{S'\}, P \cup \{S' \to S\}, S')$, если целевой символ S встречается в правой части хотя бы одного правила из множества P в исходной грамматике G. Грамматика G' имеет свойство LR(k), если из существования двух выводов $S' \Rightarrow^* wAx \Rightarrow wzx \Rightarrow^* wu$ и $S' \Rightarrow^* wAx \Rightarrow wtx \Rightarrow^* wv$ и из условия FIRST(u) = FIRST(v) следует $z = t$. Грамматика называется LR(k)-грамматикой, если она обладает свойством LR(k) для некоторого $k \geq 0$. В названии LR(k)-грамматики первая литера L — left — означает порядок чтения входной цепочки символов слева направо. Вторая литера R происходит от слова right и означает, что в результате работы распознавателя получается правосторонний вывод. Вместо символа k в названии грамматики стоит число, которое показывает, сколько символов входной цепочки надо рассмотреть, чтобы принять решение о действии на каждом шаге работы распознавателя. Доказано, что класс LR-грамматик является более широким, чем класс LL-грамматик, то есть для каждого КС-языка, заданного LL-грамматикой, может быть построена LR-грамматика, задающая тот же язык, но не наоборот. Для LR(k)-грамматик известны следующие полезные свойства: всякая LR(k)-грамматика для любого $k \geq 0$ является однозначной; существует алгоритм, позволяющий проверить, является ли заданная грамматика LR(k)-грамматикой для строго определённого числа k. Есть, однако, неразрешимые проблемы для произвольных КС-грамматик: не существует алгоритма, который мог бы проверить, является ли заданная КС-грамматика LR(k)-грамматикой для некоторого произвольного числа k; не существует алгоритма, который мог бы преобразовать (или доказать, что преобразование невозможно) произвольную КС-грамматику к виду LR(k)-грамматики для некоторого k. *(Источник: Гагарина, Кокорева, гл. 3, §3.3.5, определения 3.15–3.16, с. 72–73.)*

Грамматики предшествования. Грамматики предшествования характеризуются тем, что для каждой упорядоченной пары символов в грамматике устанавливается некоторое отношение, называемое отношением предшествования. Существует несколько видов грамматик предшествования, которые различаются в зависимости от того, какие отношения предшествования в них определены и между какими типами символов (терминальными или нетерминальными) установлены эти отношения. Выделяют следующие виды грамматик предшествования: простого; расширенного; слабого; смешанной стратегии; операторного. Грамматикой простого предшествования называется такая приведённая КС-грамматика $G = (T, N, P, S)$, для которой выполняются следующие условия: для каждой упорядоченной пары терминальных и нетерминальных символов выполняется не более одного из трёх отношений предшествования: $X = Y$ для всех $X, Y \in (T \cup N)$, если и только если существует правило $A \to \alpha XY\beta \in P$, где $A \in N$, $\alpha, \beta \in (T \cup N)^*$; $X < Y$ для всех $X, Y \in (T \cup N)$, если и только если существуют правило $A \to \alpha XD\beta \in P$ и вывод $D \Rightarrow^* Y\gamma$, где $A, D \in N$, $\alpha, \beta, \gamma \in (T \cup N)^*$; $X > Y$ для всех $X, Y \in (T \cup N)$, если и только если существуют правило $A \to \alpha CY\beta \in P$ и вывод $C \Rightarrow^* \gamma X$ или существуют правило $A \to \alpha CD\beta \in P$ и выводы $C \Rightarrow^* \gamma X$ и $D \Rightarrow^* Y\omega$, где $A, C, D \in N$, $\alpha, \beta, \gamma, \omega \in (T \cup N)^*$; разные правила в грамматике не должны иметь одинаковые правые части. Отношения $=$, $<$ и $>$ на-

зывают отношениями предшествования для символов. Отношение предшествования однозначно определено для каждой упорядоченной пары символов. Если два символа не могут находиться рядом ни в одном элементе разбора синтаксически правильной цепочки, то отношение предшествования между ними отсутствует. Отношения предшествования зависят от порядка, в котором стоят символы, и в этом смысле их нельзя путать со знаками математических операций — они не обладают ни свойством коммутативности, ни свойством ассоциативности. Например, если известно, что $X > Y$, то не обязательно выполняется отношение $Y < X$. Для грамматик простого предшествования характерны следующие полезные свойства: каждая из них является однозначной; чтобы проверить, является ли произвольная КС-грамматика грамматикой простого предшествования, достаточно проверить, удовлетворяет ли она описанным выше условиям. Однако для грамматик простого предшествования не существует алгоритма, который мог бы преобразовать произвольную КС-грамматику в грамматику простого предшествования (или доказать, что преобразование невозможно). Для построения распознавателя на основе грамматик предшествования к входной цепочке добавляют два символа: $\downarrow_H$ — символ начала цепочки и $\downarrow_K$ — символ конца цепочки. Эти дополнительные символы вступают в следующие отношения с символами грамматики: $\downarrow_H < X$ для всех $X \in (T \cup N)$, если существует вывод $S \Rightarrow^* Xy$, где $S \in N$, $y \in (T \cup N)^*$; $\downarrow_K > X$ для всех $X \in (T \cup N)$, если существует вывод $S \Rightarrow^* yX$, где $S \in N$, $y \in (T \cup N)^*$. *(Источник: Гагарина, Кокорева, гл. 3, §3.3.6, определение 3.17, с. 73–75.)*

Магазинные автоматы: модель и формальное определение. Если конечные автоматы, рассмотренные в гл. 2, дают возможность построить распознаватели на основе регулярных грамматик, применяемых для лексического анализа, то для распознавателей КС-языков используются магазинные автоматы. Модель магазинного автомата состоит из входной ленты; устройства управления; вспомогательной ленты, называемой магазином или стеком. Входная лента содержит символы входного алфавита; при этом предполагается, что в неиспользуемых клетках входной ленты расположены пустые символы λ. Вспомогательная лента разделена на клетки, в которых могут располагаться символы магазинного алфавита; начало вспомогательной ленты называется дном магазина. Устройство управления перемещает вдоль ленты две головки: головка входной ленты является считывающей и может перемещаться только в одну сторону (вправо) или оставаться на месте; головка вспомогательной ленты может выполнять как чтение, так и запись, но эти операции связаны с перемещением головки определённым образом: при записи головка предварительно сдвигается на одну позицию вверх, а затем символ заносится на ленту; при чтении символ, находящийся под головкой, считывается с ленты, а затем головка сдвигается на одну позицию вниз; таким образом головка всегда установлена против последнего записанного символа. Позицию, находящуюся в рассматриваемый момент времени под головкой, называют вершиной магазина. Магазинный автомат M определяется следующей совокупностью семи объектов: $M = \{S, T, H, \delta, s_0, h_0, F\}$, где S — алфавит состояний; T — входной алфавит; H — алфавит магазинных символов, записываемых на вспомогательную ленту; δ — функция переходов, $\delta : \{S \times T \cup \{\lambda\} \times H\} \to S \times H^*$, если M — автомат детерминированный, и $\delta : \{S \times T \cup \{\lambda\} \times H\} \to 2^{S \times H^*}$, если M — автомат недетерминированный; s_0 — начальное состояние, $s_0 \in S$; h_0 — маркер дна, он всегда записывается на дно магазина, $h_0 \in H$; F — множество конечных состояний, являющееся подмножеством множества S. Функция δ отображает тройки

(p_i, s_j, h_k) в пары (s_i, v_i) для детерминированного автомата или в множество таких пар для недетерминированного автомата, где $v_i \in H^*$, а h_k — символ в вершине магазина. Эта функция описывает изменение состояния магазинного автомата, происходящее при чтении символа с входной ленты и перемещении входной головки. В дальнейшем при построении магазинных автоматов потребуются две разновидности функций переходов: функция переходов с пустым символом в качестве входного символа $\delta^0(s, \lambda, h) = (s', \beta)$, предписывающая прочитать символ h из вершины магазина, изменить состояние автомата на s' и записать цепочку β в магазин независимо от того, какой символ находится под читающей головкой входной ленты; функция переходов с определённым входным символом $\delta(s, a, h) = (s', \beta)$, предписывающая прочитать входной символ a, изменить состояние автомата на s' и заменить верхний символ магазина h цепочкой β. *(Источник: Гагарина, Кокорева, гл. 3, §3.4, §3.4.1, определение 3.18, с. 75–78.)*

Работа магазинного автомата. Для того чтобы описать, как работает автомат, вводится понятие конфигурации. Конфигурацией автомата M называют тройку $(s, \alpha, \gamma) \in S \times T^* \times H^*$, где s — текущее состояние управляющего устройства; α — неиспользованная часть входной цепочки $\alpha \in T^*$, самый левый символ этой цепочки находится под головкой; если $\alpha = \lambda$, то считается, что вся входная цепочка прочитана; γ — цепочка, записанная в магазине, $\gamma \in H^*$, самый правый символ цепочки считается вершиной магазина; если $\gamma = \lambda$, то магазин пуст. Работа автомата может быть представлена как смена конфигураций. Один такт работы автомата заключается в определении новой конфигурации исходя из текущей, что символизирует запись $(s, \alpha, \gamma) \models (s', \alpha, \gamma)$. Такая смена конфигураций возможна, если функция $\delta(s, a, h)$ или $\delta(s, \lambda, h)$ определена и имеет значение (s', β). При этом предполагается, что автомат: читает символ a, находящийся под головкой, или не читает ничего в случае входного символа λ; определяет новое состояние s'; читает символ h, находящийся в вершине магазина, и записывает цепочку β в магазин вместо символа h; если $\beta = \lambda$, то верхний символ оказывается удалённым из магазина. Таким образом, возможны три случая работы автомата: функция $\delta(s, a, h)$ определена и выполняется такт работы; функция $\delta(s, a, h)$ не определена, но определена функция $\delta(s, \lambda, h)$ и выполняется пустой такт (без чтения входной информации); функции $\delta(s, a, h)$ и $\delta(s, \lambda, h)$ не определены, в этом случае дальнейшая работа автомата невозможна. Начальной конфигурацией называется конфигурация (s_0, α, h_0), где s_0 — начальное состояние, α — исходная цепочка, h_0 — маркер дна магазина. Заключительная конфигурация — (s, λ, λ), где $s \in F$. Для обозначения последовательности сменяющих друг друга конфигураций используется знак $\models^*$. *(Источник: Гагарина, Кокорева, гл. 3, §3.4.2, определение 3.19, с. 78–79.)*

Язык, допускаемый магазинным автоматом. Цепочка α называется допустимой для автомата M, если существует последовательность конфигураций, в которой первая конфигурация является начальной с цепочкой α, а последняя — заключительной, то есть $(s_0, \alpha, h_0) \models^* (s_1, \lambda, \lambda)$, где $s_1 \in F$. Множество цепочек, допустимых для автомата M, называется языком, допускаемым или определяемым автоматом M, и обозначается $L(M)$:

$$L(M) = \{\alpha \mid (s_0, \alpha, h_0) \models^* (s', \lambda, \lambda) \text{ и } s' \in F\}.$$

В пособии приведены примеры детерминированного (пример 3.15) и недетерминированного (пример 3.16) магазинных автоматов. В примере 3.15 автомат M_1 с $T = \{a, b\}$, $S = \{s_0, s_1, s_2\}$, $H = \{h_0, a\}$, $F = \{s_0\}$ и переходами $\delta(s_0, a, h_0) = (s_1, h_0a)$, $\delta(s_1, a, a) = (s_1, aa)$, $\delta(s_1, b, a) = (s_2, \lambda)$, $\delta(s_2, b, a) = (s_2, \lambda)$, $\delta(s_2, \lambda, h_0) = (s_0, \lambda)$ допускает цепочку *aabb*, но не допускает *aabbb*. В примере 3.16 автомат M_2 является недетерминированным, поскольку одному и тому же набору аргументов, например (s_0, a, a), соответствуют два значения функции; цепочка *abba* допускается автоматом M_2 при одной последовательности команд и не допускается при другой. *(Источник: Гагарина, Кокорева, гл. 3, §3.4.3, определения 3.20–3.21, примеры 3.15–3.16, с. 79–80.)*

Построение магазинного автомата по КС-грамматике. Пусть задана контекстно-свободная грамматика $G = (N, T, P, S)$. Определим компоненты автомата M следующим образом: $S = \{s_0\}$, $T = T$, $H = N \cup T \cup \{h_0\}$, $F = \{s_0\}$; в качестве начального состояния автомата примем символ s_0 и построим функцию переходов так: для всех $A \in N$ таких, что встречаются в левой части правил $A \to \alpha$, построим команды вида $\delta^0(s_0, \lambda, A) = (s_0, \alpha^R)$, где α^R — реверс цепочки α; для всех $a \in T$ построим команды вида $\delta(s_0, a, a) = (s_0, \lambda)$; для перехода в конечное состояние построим команду $\delta(s_0, \lambda, h_0) = (s_0, \lambda)$; начальную конфигурацию автомата определим в виде (s_0, ω, h_0S), где ω — исходная цепочка, записанная на входной ленте. Автомат, построенный по приведённым выше правилам, работает следующим образом. Если в вершине магазина находится терминал и символ, читаемый с входной ленты, совпадает с ним, то по команде второго типа терминал удаляется из магазина, а входная головка сдвигается. Если в вершине магазина находится нетерминал, то выполняется команда первого типа, которая вместо терминала записывает в магазин цепочку, представляющую собой правую часть правила грамматики. Таким образом, автомат, последовательно заменяя нетерминалы, появляющиеся в вершине магазина, строит в магазине левый вывод входной цепочки, удаляя полученные терминальные символы, совпадающие с символами входной цепочки. Это означает, что каждая цепочка, которая может быть получена с помощью левого вывода в грамматике G, допускается построенным автоматом M. В пособии приведён пример 3.17, в котором для грамматики $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to (E) \mid a$ строится недетерминированный автомат и показывается последовательность тактов, допускающая цепочку $a + a * a$. *(Источник: Гагарина, Кокорева, гл. 3, §3.4.4, пример 3.17, с. 80–83.)*

Распознаватели с возвратом. Магазинные автоматы часто называют распознавателями, поскольку они определяют, является ли цепочка, подаваемая на вход автомата, допустимой, и, следовательно, отвечают на вопрос, принадлежит ли эта цепочка языку, порождаемому грамматикой, использованной для построения автомата. Учитывая характер построения вывода в магазине, автоматы рассмотренного типа называют нисходящими распознавателями. Наиболее простой тип распознавателей для КС-языков — это распознаватели с возвратом, которые создаются на основе недетерминированных МП-автоматов (в общем случае не преобразуемых в детерминированные). На некотором шаге работы недетерминированного автомата может возникнуть неоднозначность — наличие нескольких допустимых конфигураций автомата, следующих за текущей. Возможны два выхода из данной ситуации: либо реализация нескольких параллельных процессов, пока один из них не достигнет заключительной конфигурации (все остальные при этом завершаются), либо последо-

вательный перебор возможных конфигураций в месте неоднозначности с сохранением этого состояния автомата в памяти и возвратом в него, пока какой-либо из путей не приведёт к заключительной конфигурации. Если ни один из вариантов не привёл к заключительной конфигурации, алгоритм завершается с ошибкой. Реализация первого алгоритма связана с управлением параллельными процессами и является достаточно сложной, поэтому большее распространение получил второй вариант — разбор с возвратами. Однако время выполнения второго алгоритма экспоненциально зависит от длины входной цепочки и поэтому его использование возможно только для простых КС-языков с малой длиной входных предложений. Для многих классов КС-языков существуют более эффективные методы разбора. *(Источник: Гагарина, Кокорева, гл. 3, §3.5, §3.5.1, с. 83–85.)*

Нисходящий распознаватель с возвратом и алгоритм «сдвиг—свертка». Нисходящий распознаватель с возвратом представляет собой МП-автомат с одним состоянием: $M = (\{s\}, T, H, \delta, s, h_0, \{s\})$. Автомат распознаёт цепочки КС-языка, заданного КС-грамматикой $G = (T, N, P, S)$. Входной алфавит автомата содержит терминальные символы грамматики: $T = T$, а алфавит магазинных символов строится из терминальных и нетерминальных символов грамматики: $H = T \cup N$. Начальной конфигурацией автомата является (s, a, S) — автомат пребывает в единственном состоянии s, считывающая головка находится в начале входной цепочки символов $a \in T$, в стеке находится символ, соответствующий целевому символу грамматики S. Заключительной конфигурацией автомата является набор (s, λ, λ) — автомат пребывает в своём единственном состоянии s, считывающая головка находится за концом входной цепочки символов, стек пуст. Функция переходов МП-автомата δ строится на основе правил грамматики: $\delta(s, \lambda, A) = (s, \alpha^R)$, $A \in N$, $\alpha \in (T \cup N)^*$, если правило $A \to \alpha$ содержится во множестве правил P грамматики; $\delta(s, a, a) = (s, \lambda)$ для всех $a \in T$. Работа данного МП-автомата происходит следующим образом: если на вершине стека автомата находится нетерминальный символ A, то он заменяется на цепочку символов α, если в грамматике языка есть правило $A \to \alpha$, без сдвига считывающей головки автомата; если на верхушке стека находится терминальный символ a, который совпадает с текущим символом входной цепочки, то этот символ выбрасывается из стека со сдвигом считывающей головки на одну позицию вправо. Если грамматика содержит более одного правила вида $A \to \alpha$, то МП-автомат будет недетерминированным. Данный МП-автомат строит левосторонние выводы для грамматики $G = (T, N, P, S)$ и читает цепочку входных символов слева направо, поэтому естественным для него является построение дерева вывода сверху вниз. Грамматика G при этом не должна содержать левую рекурсию, иначе автомат может войти в бесконечный цикл. Алгоритм «сдвиг—свертка» строится на основе МП-автомата с одним состоянием s: $M = (\{s\}, T, H, \delta, s, h_0, \{s\})$. Автомат распознаёт цепочки КС-языка, заданного грамматикой $G = (T, N, P, S)$. Входной алфавит автомата содержит терминальные символы грамматики: $T = T$, а алфавит магазинных символов состоит из терминальных и нетерминальных символов грамматики: $H = T \cup N$. Начальной конфигурацией автомата является набор (s, α, λ) — автомат пребывает в своём единственном состоянии s, считывающая головка находится в начале входной цепочки символов $\alpha \in T$, стек пуст. Заключительной конфигурацией автомата является набор (s, λ, S) — автомат пребывает в своём единственном состоянии s, считывающая головка находится за концом входной цепочки символов, в стеке находится целевой символ грамматики S. Функция переходов МП-автомата δ строится на основе правил

грамматики: $\delta(s, \lambda, \beta) = (s, B)$, $B \in N$, $\beta \in (T \cup N)^*$, если правило $B \to \beta$ содержится во множестве правил P грамматики G; $\delta(s, b, \lambda) = (s, b)$ для всех $b \in T$. Работа данного МП-автомата происходит следующим образом: если на вершине стека автомата находится цепочка символов β, то проводится её свёртка к нетерминальному символу B, если в грамматике языка существует правило вида $B \to \beta$, без сдвига считывающей головки автомата; если под считывающей головкой находится некоторый символ входной цепочки a, то он помещается в стек со сдвигом головки на одну позицию вправо. Такой алгоритм получил название «сдвиг—свертка» по основным действиям алгоритма. Данный МП-автомат строит правосторонние выводы для грамматики $G = (T, N, P, S)$ и читает цепочку входных символов слева направо, то есть дерево вывода строится снизу вверх. Такой распознаватель называется восходящим. Для моделирования этого автомата необходимо, чтобы грамматика G не содержала λ-правил и цепных правил, иначе алгоритм может войти в бесконечный цикл из свёрток. Рассмотренные алгоритмы обладают двумя преимуществами: универсальностью — возможностью распознавать цепочки языков, заданных любой приведённой КС-грамматикой; простотой реализации. Тем не менее они не получили применения в реальных трансляторах; из-за низкой эффективности их используют только при разборе коротких цепочек, принадлежащих простым КС-языкам. *(Источник: Гагарина, Кокорева, гл. 3, §3.5.1, с. 85–87.)*

Табличные распознаватели для КС-языков. В отличие от распознавателей на основе МП-автоматов табличные распознаватели, получая на вход цепочку символов, сначала создают для неё промежуточное хранилище информации в виде таблицы, а затем с её использованием производят разбор данной цепочки. В связи с этим эффективность таких распознавателей определяется длиной входной цепочки. Табличные распознаватели являются универсальными, то есть они могут использоваться для разбора цепочек, принадлежащих любому КС-языку, и наиболее эффективны с точки зрения вычислительных ресурсов по сравнению с другими универсальными распознавателями. В реальных компиляторах такие распознаватели, как правило, не используются из-за полиномиальной зависимости от длины цепочки. Алгоритм Кока — Янгера — Касами: язык, допускаемый распознавателем, задаётся грамматикой $G = (T, N, P, S)$. Предположим, что на входе цепочка символов $\alpha \in T^*$, $\alpha = a_1 a_2 \dots a_n$, $|\alpha| = n$. В соответствии с алгоритмом Кока — Янгера — Касами для данной цепочки строится таблица R размерностью $n \times n$, такая, что $A \in R_{ij}$, если $A \Rightarrow^+ a_i \dots a_{j-1}$, $A \in N$. В результате каждая ячейка таблицы содержит множество нетерминалов грамматики G. Тогда то, что $S \in R_{1n}$, означает, что цепочка α принадлежит данному языку. В этом случае по таблице R можно проследить разбор цепочки $S \Rightarrow^* \alpha$. Для построения вывода по алгоритму Кока — Янгера — Касами грамматика G должна быть в нормальной форме Хомского. Алгоритм Кока — Янгера — Касами фактически состоит из трёх вложенных циклов, поэтому время его выполнения имеет кубическую зависимость от длины входной цепочки символов. Требуемый для хранения таблицы R размером $n \times n$ объём памяти имеет квадратичную зависимость от длины входной цепочки. Алгоритм Эрли: для заданной КС-грамматики $G = (T, N, P, S)$ и входной цепочки $\omega = a_1 a_2 \dots a_n$, $\omega \in T$, $|\omega| = n$ строится последовательность списков ситуаций $I_0, I_1, \dots, I_n$. Каждая ситуация, входящая в список I_j для входной цепочки ω, представляет собой структуру вида $[A \to X_1 X_2 \dots X_k \bullet X_{k+1} \dots X_m, i]$, $X_k \in (N \cup T)$, где правило $A \to X_1 \dots X_m$ принадлежит множеству правил P грамматики G, и $0 \le i \le n$, $0 \le k \le m$. Символ

• (точка) — это метасимвол, который не входит ни во множество терминальных N, ни во множество нетерминальных T символов грамматики. В ситуации I_j этот символ может стоять в любой позиции, в том числе в начале $(\bullet X_1 \dots X_m)$ или в конце $(X_1 \dots X_m \bullet)$ всей цепочки символов правила $A \to X_1 \dots X_m$. Если цепочка символов правила пустая $(A \to \lambda)$, то ситуация будет выглядеть так: $[A \to \bullet, i]$. Список ситуаций строится таким образом, что для j, $0 \le j \le n$: $[A \to \alpha \bullet \beta, i] \in I_j$ тогда и только тогда, если существует $S \Rightarrow^* \gamma A \delta$, $\gamma \Rightarrow^* a_1 \dots a_i$, и $\alpha \Rightarrow^* a_{i+1} \dots a_j$. Иначе говоря, между вторым компонентом ситуации и номером списка, в котором он появляется, заключена часть входной цепочки, выводимая из правила A. Условия ситуации I_j гарантируют возможность применения правила $A \to \alpha\beta$ в выводе некоторой входной цепочки, совпадающей с заданной цепочкой ω до позиции j. Вывод заданной входной цепочки ω в грамматике G после завершения алгоритма Эрли возможен при условии $[S \to \omega \bullet, 0] \in I_n$. На основе выявленного в результате выполнения алгоритма списка ситуаций можно с помощью специальной процедуры построить всю цепочку вывода и получить номера применяемых правил. Как и все табличные алгоритмы, алгоритм Эрли обладает полиномиальными характеристиками зависимости времени выполнения и объёма памяти от длины входной цепочки. Для произвольной КС-грамматики время выполнения данного алгоритма будет иметь кубическую зависимость от длины входной цепочки, а необходимый объём памяти — квадратичную зависимость от длины входной цепочки. Но для однозначных КС-грамматик алгоритм Эрли имеет лучшие характеристики — время его выполнения в этом случае квадратично зависит от длины входной цепочки. Кроме того, для некоторых классов КС-грамматик время выполнения алгоритма линейно зависит от длины входной цепочки (правда, для этих классов, как правило, существуют более простые алгоритмы распознавания). В целом алгоритм Эрли имеет наилучшие характеристики среди всех универсальных алгоритмов распознавания входных цепочек для произвольных КС-грамматик. По ряду параметров он превосходит алгоритм Кока — Янгера — Касами для однозначных грамматик, хотя и является более сложным в реализации. *(Источник: Гагарина, Кокорева, гл. 3, §3.5.2, с. 86–88.)*

Метод рекурсивного спуска. Метод рекурсивного спуска — легко реализуемый детерминированный метод разбора сверху вниз. С его помощью на основе соответствующей грамматики можно написать синтаксический анализатор. Синтаксический анализатор содержит по одной рекурсивной процедуре для каждого нетерминала N. Каждая такая процедура осуществляет разбор фраз, выводимых из N. Процедуре сообщается, с какого места данной программы следует начинать поиск фразы, выводимой из N. Процедура ищет эту фразу, сравнивая программу в указанном месте с правыми частями правил для N и вызывая по мере необходимости другие процедуры для распознавания промежуточных целей. Для разбора предложения языка может потребоваться много рекурсивных вызовов процедур, составляющих нетерминалы в грамматике. Преимущества написания рекурсивного нисходящего анализатора очевидны. Основное из них — это скорость написания анализатора на основе соответствующей грамматики. Другое преимущество заключается в соответствии между грамматикой и анализатором, благодаря чему увеличивается вероятность того, что анализатор окажется правильным, или по крайней мере того, что ошибки будут простыми. Эффективность алгоритма определяется линейной зависимостью времени его работы от длины входной цепочки. Недостатки метода, хотя и менее очевидны, но весьма реальны. Из-за большого числа вызовов процедур во время синтаксическо-

го анализа анализатор становится относительно медленным. Кроме того, он может быть довольно большим по сравнению с анализаторами, основанными на табличных методах разбора (методы предшествования, LL- и LR-методы). Метод рекурсивного спуска способствует включению в анализатор действий по генерации кода, что неизбежно приводит к смешению разных фаз компиляции. Последнее снижает надёжность компиляторов и усложняет обращение с ними и как бы привносит в анализатор зависимость от машины. Описанный метод можно применять в том случае, если каждое правило грамматики имеет вид: $A \to \alpha$, где $\alpha \in (T \cup N)^*$ является единственным правилом вывода для этого нетерминала; $A \to a_1\alpha_1 \mid a_2\alpha_2 \mid \ldots \mid a_n\alpha_n$, где $a_i \in T$ для всех $i = 1, 2, \ldots, n$; $a_i \neq a_j$ для $i \neq j$; $\alpha_i \in (T \cup N)^*$, то есть если для нетерминала A правил вывода несколько, то они должны начинаться с терминалов, причём все эти терминалы должны быть различными. Если грамматика не удовлетворяет вышеприведённым условиям, то для реализации метода рекурсивного спуска необходимо преобразовать её к требуемому виду, однако не существует алгоритма, который даст ответ на вопрос: можно ли построить эквивалентную произвольной КС-грамматике грамматику, к которой применим метод рекурсивного спуска. Известно также, что если для некоторых нетерминальных символов грамматики $\mathrm{FIRST}(A) \cap \mathrm{FOLLOW}(A) \neq \varnothing$, то метод рекурсивного спуска к данной грамматике не применим. Метод рекурсивного спуска можно применять к достаточно узкому подклассу КС-грамматик. Известны более широкие подклассы КС-грамматик, для которых существуют эффективные анализаторы, обладающие тем же свойством, что и анализатор, написанный методом рекурсивного спуска, — входная цепочка считывается один раз слева направо и процесс разбора полностью детерминирован, то есть время, затраченное на обработку, линейно зависит от длины цепочки. К таким грамматикам относятся LL(k)- и LR(k)-грамматики, грамматики предшествования и некоторые другие виды. В пособии приведён пример 3.18, в котором для грамматики $G = (\{a, b, c, \bot\}, \{S, A, B\}, P, S)$ с правилами $S \to AB\ \bot$, $A \to a \mid cA$, $B \to bA$ строится набор процедур рекурсивного спуска; функции $A()$, $B()$, $S()$ соответствуют нетерминальным символам A, B, S; функция $ERROR()$ обрабатывает ошибки. *(Источник: Гагарина, Кокорева, гл. 3, §3.5.3, пример 3.18, с. 88–91.)*

Нисходящий распознаватель без возвратов на основе LL(k)-грамматик. Класс LL-грамматик широк, но недостаточен для того, чтобы охватить все возможные синтаксические конструкции в языках программирования (к ним относятся все детерминированные КС-языки). Известно, что существуют детерминированные КС-языки, которые не могут быть заданы LL(k)-грамматикой ни для каких k. Однако LL-грамматики удобны для использования, поскольку позволяют построить распознаватели с линейными характеристиками (линейной зависимостью требуемых для работы алгоритма распознавания вычислительных ресурсов от длины входной цепочки символов). Распознаватель LL(k)-грамматик представляет собой МП-автомат с одним состоянием s: $R = (\{s\}, T, H, \delta, s, h_0, \{s\})$, где $H = T \cup N$, а S — целевой символ грамматики G. Функция переходов автомата строится на основе управляющей таблицы M, которая отображает множество $(H \cup \{\lambda\}) \times T^*$ на множество, состоящее из следующих элементов: пар вида $[\beta, i]$, где β — цепочка символов, помещаемая автоматом на вершину стека, а i — номер правила вида $A \to \beta$, $A \in N$, $\beta \in H^*$; «выброса»; «допуска»; «ошибки». Для построения распознавателя используются два важных множества, определяемые следующим образом: $\mathrm{FIRST}(k, \alpha)$ — множество терминальных цепочек, выводимых из строк $\alpha \in (T \cup N)^*$, укороченных

до k символов; очевидно, что если имеется цепочка терминальных символов $\alpha \in T^*$, то $\mathrm{FIRST}(k, \alpha)$ — это первые k символов цепочки α; $\mathrm{FOLLOW}(k, A)$ — множество укороченных до k символов терминальных цепочек, которые могут следовать непосредственно за нетерминалом $A \in N$ в цепочках вывода. Конфигурацию распознавателя можно отобразить в виде конфигурации МП-автомата с дополнением цепочки ω, в которую помещаются номера применённых правил. Если считать, что X — это символ на вершине стека автомата, α — непрочитанная автоматом часть входной цепочки символов, а $v = \mathrm{FIRST}(k, \alpha)$, то работу алгоритма распознавателя можно представить следующим образом: $(\alpha, X\gamma, \omega) \vdash (\alpha, \beta\gamma, \omega i)$, $\gamma \in H^*$, если $M[X, v] = [\beta, i]$; верхний символ магазина X заменяется цепочкой $\beta \in H$, к выходу добавляется i — номер правила; считывающая головка не сдвигается; $(\alpha, X\gamma, \omega) \vdash (\sigma, \gamma, \omega)$, если $X = a \in T$ и $\alpha = a\sigma$, $M[a, v] =$ «выброс»; когда верхний символ магазина совпадает с текущим входным символом, он выбрасывается из магазина и считывающая головка сдвигается на один символ вправо; $(\lambda, \lambda, \omega)$ — завершение работы, при этом $M[\lambda, \lambda] =$ «допуск»; остальные случаи — «ошибка», разбор прекращается. Цепочка $v = \mathrm{FIRST}(k, \alpha)$ при работе автомата называется «аванцепочкой». Таким образом, для создания алгоритма распознавателя языка, заданного произвольной $\mathrm{LL}(k)$-грамматикой, достаточно построить управляющую таблицу M на основе правил исходной грамматики. Алгоритм работы распознавателя для $\mathrm{LL}(1)$-грамматик заключается всего в двух условиях, проверяемых на шаге выбора альтернативы. Исходными данными для этих условий являются символ $a \in T$, обозреваемый считывающей головкой МП-автомата (текущий символ входной цепочки), и символ $A \in N$, находящийся на вершине стека автомата. Эти условия можно сформулировать так: в качестве альтернативы выбирается правило $A \to x$, если символ $a \in \mathrm{FIRST}(x)$; в качестве альтернативы выбирается правило $A \to \lambda$, если символ $a \in \mathrm{FOLLOW}(A)$. Если ни одно из этих условий не выполняется, это значит, что цепочка не принадлежит заданному языку, и МП-автомат должен выдать сообщение об ошибке. В пособии приведён пример 3.19, в котором для грамматики $G = (\{a, b\}, \{S, T\}, P, S)$ с правилами $S \to aTS \mid b$, $T \to a \mid bST$ строится управляющая таблица и показывается разбор цепочки *abbab. (Источник: Гагарина, Кокорева, гл. 3, §3.5.4, пример 3.19, с. 91–93.)*

Восходящий распознаватель без возвратов на основе LR(k)-грамматик. Восходящий детерминированный распознаватель строится согласно алгоритму «сдвиг—свертка», рассмотренному ранее. Необходимо модифицировать этот алгоритм таким образом, чтобы на каждом шаге его работы можно было однозначно ответить на вопросы: что следует выполнять: «сдвиг» (перенос) или «свертку»; какую цепочку символов α выбрать из стека для выполнения «свертки»; какое правило выбрать для выполнения «свертки» (в том случае, если существует несколько правил вида $A_1 \to \alpha$, $A_2 \to \alpha$, …, $A_n \to \alpha$). Для модификации алгоритма служат рассмотренные $\mathrm{LR}(k)$-грамматики. Распознаватель для $\mathrm{LR}(k)$-грамматик функционирует под управлением некоторой таблицы T, которая состоит из столбцов — «действие» и «переход». В строках таблицы содержатся все цепочки символов, находящихся на вершине стека, которые влияют на работу распознавателя. В столбце «действие» распределены все входные цепочки символов длиной не более k (аванцепочки), которые могут следовать за считывающей головкой автомата в процессе выполнения разбора; а в столбце «переход» — все терминальные и нетерминальные символы грамматики, которые могут появляться на вершине стека автомата при выполнении действий (сдвигов или свёрток).

Ячейки таблицы «действие» содержат следующие элементы: «сдвиг» — если в данной ситуации требуется выполнить сдвиг (перенос текущего символа из входной цепочки в стек); «успех» — если возможна свёртка к целевому символу грамматики S и разбор входной цепочки завершён; «свертка» (целое число) — если возможно выполнение свёртки (число обозначает номер правила грамматики, по которому должна выполняться свёртка); «ошибка» — в остальных ситуациях. Ячейки «переход» управляющей таблицы T служат для выяснения номера строки таблицы, которая будет использована для определения выполняемого действия на очередном шаге. Эти клетки содержат следующие данные: целое число — номер строки таблицы T; «ошибка» — в остальных ситуациях. Для удобства работы с распознавателем будем считать, что входная строка начинается с $\downarrow_H$ — символа начала цепочки и заканчивается $\downarrow_K$ — символом конца цепочки. В начальном состоянии символ $\downarrow_H$ находится на вершине стека, а считывающая головка обозревает первый символ входной цепочки. В конечном состоянии в стеке должны находиться символы S (целевой символ грамматики) и $\downarrow_H$, а считывающая головка автомата должна обозревать символ $\downarrow_K$. Алгоритм разбора: поместить в стек символ $\downarrow_H$ и начальную (нулевую) строку управляющей таблицы T; в конец входной цепочки поместить символ $\downarrow_K$; прочитать с вершины стека строку управляющей таблицы T; выбрать из этой строки часть «действие» в соответствии с аванцепочкой, обозреваемой считывающей головкой автомата; в соответствии с типом действия выполнить выбор из четырёх вариантов: «сдвиг» — если входная цепочка не прочитана до конца, прочитать и запомнить как «новый символ» очередной символ из входной цепочки, сдвинуть считывающую головку на одну позицию вправо, в противном случае прервать выполнение алгоритма и сообщить об ошибке; целое число («свертка») — выбрать правило в соответствии с номером, удалить из стека цепочку символов, составляющую правую часть выбранного правила, взять символ из левой части правила и запомнить его как «новый символ»; «ошибка» — прервать выполнение алгоритма, сообщить об ошибке; «успех» — произвести свёртку к целевому символу S, прервать выполнение алгоритма, сообщить об успешном разборе входной цепочки символов, если входная цепочка прочитана до конца, в противном случае сообщить об ошибке; прочитать с вершины стека строку управляющей таблицы T; выбрать из этой строки часть «переход» в соответствии с символом, который был занесён в память как «новый символ»; если часть «переход» содержит вариант «ошибка», прервать выполнение алгоритма и сообщить об ошибке, в противном случае (если там содержится номер строки управляющей таблицы T) — положить в стек «новый символ» и строку таблицы T с выбранным номером; перейти к выполнению шага 2. Здесь «новый символ» — промежуточная переменная для хранения терминалов и нетерминалов, полученных на некоторых шагах работы МП-автомата. Для LR(0)-грамматик решение о сдвиге или свёртке принимается только на основе содержимого стека; текущий символ входной цепочки в расчёт не принимается. В пособии приведён пример 3.20, в котором для грамматики $G = (\{a, b\}, \{S\}, P, S)$ с правилами $S \to aSS \mid b$ строится пополненная грамматика G' с правилами $S' \to S$, $S \to aSS$, $S \to b$, управляющая таблица и показывается разбор цепочки *abababb*. *(Источник: Гагарина, Кокорева, гл. 3, §3.5.5, пример 3.20, с. 93–97.)*

Метод предшествования. Для разбора цепочек, принадлежащих языку, порождённому грамматиками предшествования, используется алгоритм «сдвиг—свертка», адаптированный к данным грамматикам. Этот алгоритм выполняется расширенным МП-автоматом с одним состоянием s: $M = (\{s\}, T, H, \delta, s, h_0, \{s\})$. Отношения пред-

шествования определяют в процессе работы автомата, какое действие — сдвиг или свертка — должно выполняться на каждом шаге алгоритма, и служат для того, чтобы выбрать цепочку для свёртки. Правило при свёртке выбирается однозначно за счёт различия правых частей правил грамматики. В начальной конфигурации автомата считывающая головка обозревает первый символ входной цепочки на входной ленте, в магазине (стеке) МП-автомата находится символ $\downarrow_H$, в конец цепочки помещён символ $\downarrow_K$. Работу автомата можно описать следующим образом: поместить на вершине стека символ $\downarrow_H$; считывающая головка должна находиться в начале входной цепочки символов; сравнить с помощью отношения предшествования символ, находящийся на вершине стека (левый символ отношения), с текущим символом входной цепочки (правый символ отношения); если имеет место отношение $<$ или $=$, то произвести сдвиг (перенос текущего символа из входной цепочки в стек и сдвиг считывающей головки на один шаг вправо) и выполнить шаг 2; в противном случае перейти к выполнению шага 4; если имеет место отношение $>$, то произвести свёртку, для этого надо найти на вершине стека все символы, связанные отношением $=$, и удалить эти символы из стека; затем выбрать из грамматики правило, имеющее правую часть, совпадающую с «основой», и поместить в стек левую часть выбранного правила; если символов, связанных отношением $=$, на верхушке стека нет, то в качестве «основы» используется один, самый верхний символ стека; затем, если разбор не закончен (считывающая головка не дошла до конца цепочки), вернуться к выполнению шага 2; если правило, совпадающее с «основой», найти не удалось, то необходимо прервать выполнение алгоритма и сообщить об ошибке; если не установлено ни одно отношение предшествования между текущим символом входной цепочки и символом на вершине стека, необходимо прервать выполнение алгоритма и сообщить об ошибке. Разбор завершается, если считывающая головка автомата обозревает символ конца цепочки и при этом больше не может быть выполнена свёртка. Решение о принадлежности цепочки языку зависит от содержимого магазина. Цепочка символов считается принадлежащей языку, если в результате завершения алгоритма в магазине находятся начальный символ грамматики S и символ $\downarrow_H$. Если на каком-либо этапе выполнения алгоритма возникает ошибка (невозможно перейти к очередному этапу), он прерывается, и цепочка считается не принадлежащей языку. Для удобства отношения предшествования предварительно заносят в таблицу, строки и столбцы которой помечены терминалами и нетерминалами грамматики, включая символы начала и конца цепочки. *(Источник: Гагарина, Кокорева, гл. 3, §3.5.6, с. 97–99.)*

Обработка синтаксических ошибок. Любая программа потенциально содержит множество ошибок самого разного уровня. Например, ошибки бывают: лексическими, такими, как неверно записанные идентификаторы, ключевые слова или операторы; синтаксическими, например арифметические выражения с несбалансированными скобками; семантическими, такими, как операторы, применяемые к несовместимым с ними операндам; логическими, например бесконечная рекурсия. Основные действия по выявлению ошибок и восстановлению состояния анализатора концентрируются в фазе синтаксического анализа. Одна из причин этого состоит в том, что многие ошибки по своей природе являются синтаксическими или проявляются, когда поток токенов, идущий от лексического анализатора, нарушает определяющие язык программирования грамматические правила. Обработчик ошибок синтаксического анализатора имеет просто формулируемые цели: ясно и однозначно сообщать о наличии ошибок; обеспечивать быстрое восстановление после обнаружения ошибки,

чтобы продолжить поиск следующих ошибок; существенно не замедлять обработку корректной программы. Эффективная реализация этих целей представляет собой весьма сложную задачу. К счастью, типичные ошибки достаточно просты, и для их обработки зачастую реализуются относительно простые механизмы обработки ошибок. В некоторых случаях, однако, ошибка может произойти задолго до момента обнаружения, и определить её природу весьма непросто. Бывают ситуации, когда обработчик ошибок должен догадаться, что имел в виду программист при написании программы. Каким образом обработчик ошибок сообщит о наличии ошибки? Как минимум — указать место в исходной программе, где была выявлена ошибка; как правило, ошибка находится в пределах нескольких предыдущих токенов. Общая стратегия, используемая многими компиляторами, — вывод ошибочной строки с указанием позиции, в которой обнаружена ошибка. Если имеется обоснованное предположение о природе ошибки, вывод включает диагностическое пояснение типа «отсутствие точки с запятой в данной позиции». В чём состоит восстановление синтаксического анализатора после того, как ошибка выявлена? В большинстве случаев окончание анализа после выявления первой ошибки не является корректным решением, так как восстановление и продолжение анализа могли бы выявить ряд последующих ошибок. Обычно после обнаружения ошибки синтаксический анализатор пытается восстановить некоторое своё состояние, в котором продолжается обработка входного потока так, как если бы он был корректным. Неадекватное восстановление после обнаружения ошибки может вызвать настоящую лавину ложных ошибок, которые не были допущены программистом, а появились вследствие изменений состояния синтаксического анализатора в процессе восстановления после ошибки. Аналогично восстановление после синтаксической ошибки может привести к ложным семантическим ошибкам, которые выявятся позже, на стадии семантического анализа или генерации кода. Например, при восстановлении после ошибки синтаксический анализатор может пропустить объявление некоторой переменной. Консервативная стратегия компилятора состоит в запрете сообщений об ошибках, которые возникают во входном потоке слишком близко друг к другу. После обнаружения одной синтаксической ошибки компилятор должен проанализировать несколько входных токенов и только после этого разрешить вывод других сообщений об ошибках. *(Источник: Гагарина, Кокорева, гл. 3, §3.7.2, с. 102–104.)*

Стратегии восстановления состояния анализатора после ошибок. Существуют различные стратегии, которые синтаксический анализатор может использовать для восстановления состояния после синтаксической ошибки. Режим паники: эта стратегия реализуется проще всего и может использоваться большинством методов синтаксического анализа. При обнаружении ошибки синтаксический анализатор поочерёдно пропускает входные символы, пока не будет найден один из специально определённых множества синхронизирующих токенов. Обычно такими токенами являются разделители, например `end` или точка с запятой, роль которых в исходной программе совершенно очевидна. Разработчик компилятора, естественно, должен выбрать синхронизирующие токены исходя из особенностей исходного языка. Несмотря на то что при коррекции часто пропускается значительное количество символов без проверки на наличие ошибок, метод достаточно прост и в отличие от других гарантирует отсутствие зацикливания. В случаях, когда несколько ошибок в одной инструкции встречается очень редко, этот метод работает вполне адекватно. Уровень фразы: при обнаружении ошибки синтаксический анализатор может вы-

полнить локальную коррекцию оставшегося входного потока, то есть заменить префикс остальной части потока некоторой строкой, которая позволит синтаксическому анализатору продолжить работу. Типичная локальная коррекция может состоять в удалении лишней точки с запятой или вставке недостающей. Выбор способа локальной коррекции — прерогатива разработчика компилятора. Естественно, здесь нужно быть предельно осторожным, чтобы не получить, например, бесконечный цикл. Этот тип замещения может скорректировать любую входную строку и используется в ряде компиляторов, исправляющих ошибки. Основной его недостаток заключается в сложности обработки ситуации, когда ошибка реально располагается до точки обнаружения. Продукции ошибок: если известны наиболее распространённые ошибки, можно расширить грамматику языка продукциями, порождающими ошибочные конструкции. Затем на основе такой расширенной грамматики построить синтаксический анализатор. Если последний использует одну из «ошибочных» продукций, можно сгенерировать корректное диагностическое сообщение для распознанной ошибки. *(Источник: Гагарина, Кокорева, гл. 3, §3.7.3, с. 104–105.)*

Нерекурсивный предиктивный анализ. Нерекурсивный предиктивный (предсказывающий) анализ является одним из вариантов синтаксического анализа. Анализатор такого вида строится с помощью явного использования стека. Ключевая проблема предиктивного анализа заключается в определении продукции, которую следует применить к нетерминалу. Нерекурсивный синтаксический анализатор ищет необходимую для анализа продукцию в таблице разбора. Предиктивный синтаксический анализатор под управлением таблицы разбора имеет входной буфер, стек, таблицу разбора и выходной поток. Входной буфер содержит анализируемую строку с символом её правого конца — \$; в стеке находится последовательность символов грамматики с символом \$ на дне. В начальный момент стек содержит стартовый символ грамматики непосредственно над символом \$. Таблица разбора представляет собой матрицу M_{ij}, строки которой помечены $A_i \in N$ — нетерминалами, а столбцы $a_j \in T \cup \{\$\}$ — терминалами грамматики или символом \$. Записью таблицы является продукция грамматики $A \to \alpha \in P$ или сообщение об ошибке. Обозначим ячейки таблицы $M[A, a]$. Программа синтаксического анализа работает следующим образом. Рассматривается X — символ на вершине стека и a — текущий входной символ, которые определяют действия синтаксического анализатора. Возможны следующие варианты: если символ $X = a = \$$, то синтаксический анализатор прекращает работу и сообщает об успешном завершении разбора; если символ $X = a \neq \$$, то синтаксический анализатор выталкивает X из стека и перемещает указатель входного потока к следующему символу; если символ X представляет собой нетерминал $X \in N$, то программа рассматривает запись $M[X, a]$ из таблицы разбора M: при $M[X, a] = \{X \to Y\beta\}$, где $Y, \beta \in (T \cup N)^*$, синтаксический анализатор замещает символ X на вершине стека на βY (Y будет располагаться на вершине стека); при $M[X, a] = \text{error}$ синтаксический анализатор вызывает программу восстановления после ошибки. В пособии приведён алгоритм нерекурсивного предиктивного анализа, использующий указатель ip на первый символ $\omega\$$, и цикл, в котором рассматривается символ X на вершине стека и символ a, на который указывает ip. *(Источник: Гагарина, Кокорева, гл. 3, §3.7.4, с. 105–107.)*

Построение таблиц предиктивного анализа. Заполнение таблицы разбора производится с помощью множеств FIRST и FOLLOW. Приведём алгоритм построения

таблицы. Предположим, что грамматика $A \to \alpha$ представляет собой продукцию, у которой символ $a \in \text{FIRST}(\alpha)$. Тогда синтаксический анализатор заменит грамматику A строкой α при текущем входном символе a. В случае $\alpha = \lambda$ или $\alpha \Rightarrow \lambda$ необходимо снова заменить A на α, если текущий входной символ имеется в множестве FOLLOW(A) или из входного потока получен символ \$, который входит в множество FOLLOW(A). Метод построения таблицы разбора: для каждой продукции грамматики $A \to \alpha$ выполняем следующие шаги; для каждого терминала a из множества FIRST(α) добавляем грамматику $A \to \alpha$ в ячейку $M[A, a]$; если в множество FIRST(α) входит λ, для каждого терминала b из множества FOLLOW(α) добавляем $A \to \alpha$ в ячейку $M[A, b]$; если λ входит в FIRST(α), символ \$ — в множество FOLLOW(α), добавляем $A \to \alpha$ в ячейку $M[A, \$]$; в остальные ячейки добавляем сообщение об ошибке «error». Данный алгоритм может быть применён к любой грамматике G для построения таблицы разбора M. Однако если грамматика G неоднозначная или леворекурсивная, в одной ячейке таблицы может оказаться несколько записей. Устранив левую рекурсию и неоднозначность (это не всегда возможно), можно построить эквивалентную G-грамматику, для которой применим метод нерекурсивного предиктивного анализа. В пособии приведён пример 3.21, в котором для грамматики $G = (T, N, P, S)$ с продукциями $E \to TA$, $A \to +TA \mid \lambda$, $T \to FB$, $B \to *FB \mid \lambda$, $F \to (E) \mid w$ строятся множества FIRST и FOLLOW, таблица предиктивного разбора и показываются перемещения предиктивного синтаксического анализатора при входном потоке $w + w * w$. *(Источник: Гагарина, Кокорева, гл. 3, §3.7.5, пример 3.21, с. 107–109.)*

Восстановление после ошибок в предиктивном анализе. Синтаксический анализатор находит ошибку в тот момент, когда терминал на вершине стека не соответствует очередному входному символу или на вершине стека находится нетерминал A, очередной входной символ a, а ячейка таблицы синтаксического анализа $M[A, a]$ пуста. Восстановление после ошибки в «режиме паники» заключается в том, что символы из входного потока пропускаются до тех пор, пока не будет обнаружен токен из выбранного множества синхронизирующих токенов. Эффективность этого метода зависит от выбора синхронизирующего множества, которое должно быть таким, чтобы синтаксический анализатор быстро восстанавливался после часто встречающихся ошибок. Приведём некоторые эвристические правила. В качестве первого приближения можно поместить в синхронизирующее множество для нетерминала A все символы из множества FOLLOW(A). Множества FOLLOW(A) недостаточно. Например, если инструкция завершается точкой с запятой, то ключевое слово, с которого начинается инструкция, может не оказаться во множестве FOLLOW нетерминалов, порождающих выражения. Отсутствие точки с запятой после присваивания может, таким образом, привести к пропуску ключевого слова, с которого начинается новая инструкция. Зачастую в языке имеется иерархическая структура конструкций — например, выражения появляются в инструкциях, которые находятся в блоках, и т. д. В таком случае к синхронизирующему множеству конструкции нижнего уровня можно добавить символы, с которых начинаются конструкции более высокого уровня, например добавить ключевые слова в самое начало инструкций в синхронизирующих множествах нетерминалов, порождающих выражения. Если добавить символы из множества FIRST(A) в синхронизирующее множество для нетерминала A, то станет возможным продолжать анализ в соответствии с A, когда во входном потоке появится символ из множества FIRST(A). Если нетерминал может порождать пустую

строку, то в качестве продукции по умолчанию используется продукция, порождающая пустую строку λ. Это может отсрочить обнаружение ошибки, зато произойдёт её пропуск и сократится число нетерминалов, которые следует рассмотреть в процессе восстановления после ошибки. Если терминал на вершине стека невозможно сопоставить с входным символом, то данный терминал удаляется из стека, генерируется сообщение о вставке терминала в программу и синтаксический анализ продолжается. По сути, при этом подходе синхронизирующее множество токена состоит из всех остальных токенов. В пособии приведён пример 3.22, в котором для грамматики из примера 3.10 строится таблица синтаксического анализа с синхронизирующими токенами и показываются перемещения предиктивного синтаксического анализатора при разборе ошибочной цепочки $)w*+w$. В общем случае компилятор должен не только обнаруживать ошибки, восстанавливаться после них и продолжать разбор, но и выдавать информационные сообщения об обнаруженных ошибках. Разработчик компилятора должен обеспечить эту возможность. Восстановление на уровне фразы реализуется заполнением пустых ячеек в таблице предиктивного синтаксического анализа указателями на подпрограммы обработки ошибок. Эти подпрограммы могут изменять, вставлять или удалять символы входного потока и выводить соответствующие сообщения об ошибках. Кроме того, они могут удалять элементы из стека. При таком способе восстановления возникает вопрос, можно ли разрешить изменение символов в стеке или разрешить только размещение новых символов, поскольку после этого синтаксический анализатор не обязательно приведёт к порождению хотя бы одного слова языка. В любом случае необходимо предусмотреть отсутствие зацикливания программы. Надёжная защита от зацикливания — проверка того, что каждое действие по восстановлению после ошибки в конечном счете приводит к обработке очередного входного символа (или к удалению элементов из стека, если достигнут конец входного потока). *(Источник: Гагарина, Кокорева, гл. 3, §3.7.6, пример 3.22, с. 110–113.)*

1.4.8 Краткое резюме

Формальные языки описываются через алфавит, цепочки и операции над ними; язык в алфавите V есть подмножество V^*. Порождающая грамматика $G = (T, N, P, S)$ задаёт язык $L(G)$ как множество терминальных цепочек, выводимых из S. Схемы грамматик могут задаваться символически, в форме Бэкуса — Наура, в итерационной форме и синтаксическими диаграммами. Иерархия Хомского делит грамматики и языки на четыре типа: неограниченные (тип 0), контекстно-зависимые (тип 1), контекстно-свободные (тип 2) и регулярные (тип 3); между типами существуют соотношения вложенности. Для распознавания цепочек используются распознаватели: машины Тьюринга для типа 0, автоматы с линейно ограниченной памятью для типа 1, МП-автоматы для типа 2 и конечные автоматы для типа 3. В лексическом анализе применяются регулярные грамматики, регулярные выражения и конечные автоматы; в синтаксическом анализе — КС-грамматики, приведённые грамматики, нормальные формы Хомского и Грейбах, отношения FIRST, FOLLOW, SEL, классы S-, Q-, LL(k)-, LR(k)-грамматик и грамматик предшествования, магазинные автоматы, метод рекурсивного спуска, предиктивный анализ и алгоритм «сдвиг—свертка». *(Обобщение по: Гагарина, Кокорева, гл. 1, §1.2–1.6, с. 10–32; гл. 2, §2.1–2.4, с. 33–54; гл. 3, §3.1–3.5, с. 55–99.)*

Источники к пункту 1.4:

- Гагарина Л.Г., Кокорева Е.В. Введение в теорию алгоритмических языков и компиляторов: учеб. пособие. М.: ИД «ФОРУМ»: ИНФРА-М, 2022. Гл. 1, §1.2–1.6, с. 10–32; гл. 2, §2.1–2.4, с. 33–54; гл. 3, §3.1–3.5, с. 55–99.

1.5 Основы комбинаторного анализа. Метод производящих функций, метод включений и исключений. Примеры применения

Основной источник: Липский В. Комбинаторика для программистов. М.: Мир, 1988. Гл. 1, §1.1–1.13, с. 9–76.

1.5.1 Основные понятия комбинаторики: размещения, перестановки, сочетания

Основные источники: Липский В. Комбинаторика для программистов. М.: Мир, 1988. Гл. 1, §1.2–1.3, с. 14–24; §1.5–1.6, с. 34–45.

Постановка комбинаторных задач о размещениях. Классическая задача комбинаторики состоит в подсчёте числа способов размещения объектов по «ящикам» при заданных ограничениях. Формально даны два конечных множества X и Y, причём

$$|X| = n, \qquad |Y| = m.$$

Элементы X интерпретируются как объекты, элементы Y — как ящики, а произвольная функция $f : X \to Y$ задаёт размещение: объект $x \in X$ попадает в ящик $f(x) \in Y$. Без потери общности можно считать

$$X = \{1, \ldots, n\}, \qquad Y = \{1, \ldots, m\},$$

отождествляя функцию f с последовательностью

$$\langle f(1), \ldots, f(n) \rangle.$$

В зависимости от ограничений на f получаются различные комбинаторные объекты: произвольные функции, инъективные функции (размещения без повторений), перестановки, упорядоченные размещения по ящикам, сочетания и сочетания с повторениями. *(Источник: Липский, гл. 1, §1.2, с. 14–15.)*

Функции и размещения без ограничений. Если на размещение не накладывается никаких ограничений, каждый объект может быть помещён в любой из m ящиков независимо от остальных. Справедлива следующая теорема.

Теорема 1.1 (о числе функций). Если $|X| = n$, $|Y| = m$, то число всех функций $f : X \to Y$ равно

$$m^n.$$

Доказательство. Считая $X = \{1, \ldots, n\}$, сводим задачу к подсчёту числа всех последовательностей $\langle y_1, \ldots, y_n \rangle$ с членами из m-элементного множества Y. Каждый член y_i можно выбрать m способами независимо от остальных. Поэтому число всех последовательностей равно

$$\underbrace{m \cdot m \cdot \ldots \cdot m}_{n \text{ раз}} = m^n.$$

Теорема доказана. *(Источник: Липский, гл. 1, §1.2, с. 15.)*

Инъективные размещения и перестановки. Если каждый ящик содержит не более одного объекта, то различным объектам соответствуют различные ящики, то есть функция $f : X \to Y$ должна быть взаимно однозначной (инъективной). Обозначим через $[m]_n$ число всех инъективных функций из n-элементного множества в m-элементное множество. Полагаем $[m]_0 = 1$.

Теорема 1.2 (о числе инъективных функций). Если $|X| = n$, $|Y| = m$, то число всех взаимно однозначных функций $f : X \to Y$ равно

$$[m]_n = m(m-1)\cdots(m-n+1).$$

При $n > m$ это число равно нулю.

Доказательство. Будем строить инъективную последовательность $\langle y_1, \ldots, y_n \rangle$ с членами из Y. Элемент y_1 можно выбрать m способами. После выбора $y_1, \ldots, y_{i-1}$ элемент y_i можно выбрать любым из

$$m - (i-1) = m - i + 1$$

элементов множества

$$Y \setminus \{y_1, \ldots, y_{i-1}\}.$$

Перемножая возможности для всех $i = 1, \ldots, n$, получаем

$$m(m-1)\cdots(m-n+1) = [m]_n.$$

Если $n > m$, то на некотором шаге свободных ящиков не останется, поэтому инъективных функций нет. *(Источник: Липский, гл. 1, §1.2, с. 15.)*

В частности, если $m = n$, то каждая инъективная функция $f : X \to Y$ является взаимно однозначным отображением X на Y. Такое отображение называется *перестановкой* множества X. Из теоремы 1.2 получаем:

$$[n]_n = n(n-1)\cdots 1 = n!.$$

Таким образом, число перестановок n-элементного множества равно $n!$. *(Источник: Липский, гл. 1, §1.2, с. 16.)*

Пример. Число инъективных последовательностей длины 3 с элементами из множества $\{1, 2, 3, 4\}$ равно

$$[4]_3 = 4 \cdot 3 \cdot 2 = 24.$$

Это все упорядоченные тройки различных чисел из $\{1, 2, 3, 4\}$. *(Источник: Липский, гл. 1, §1.2, с. 16.)*

Упорядоченные размещения по ящикам. Рассмотрим теперь ситуацию, когда n объектов размещаются по m ящикам так, что каждый ящик содержит не множество, а *последовательность* помещённых в него объектов. Два размещения считаются равными, если в каждом ящике содержится одна и та же последовательность объектов. Такие размещения называются *упорядоченными размещениями* n объектов по m ящикам. Число таких размещений обозначается $[m]^n$; полагаем $[m]^0 = 1$.

Теорема 1.4 (о числе упорядоченных размещений). Число упорядоченных размещений n объектов по m ящикам равно

$$[m]^n = m(m+1)\cdots(m+n-1).$$

Доказательство. Будем добавлять объекты по одному. Первый объект можно разместить m способами. Второй объект можно поместить либо в один из $m-1$ пустых ящиков, либо в ящик, содержащий первый объект, — перед ним или после него; всего $m+1$ способ. В общем случае предположим, что уже размещено $i-1$ объектов, причём в k-м ящике находятся r_k объектов. Тогда i-й объект можно добавить в k-й ящик r_k+1 способами. Суммарное число возможностей равно

$$(r_1+1)+\cdots+(r_m+1)=(r_1+\cdots+r_m)+m=(i-1)+m=m+i-1.$$

Перемножая эти числа для $i=1,\dots,n$, получаем

$$m(m+1)\cdots(m+n-1).$$

Теорема доказана. *(Источник: Липский, гл. 1, §1.2, с. 16–17.)*

Пример. При $m=3$, $n=2$ и объектах a,b число упорядоченных размещений равно

$$[3]^2=3\cdot 4=12.$$

Это все способы распределить a и b по трём ящикам с учётом порядка внутри каждого ящика. *(Источник: Липский, гл. 1, §1.2, с. 17.)*

Полезны следующие простые соотношения:

$$[m]_n=(m-n+1)[m]_{n-1},$$

$$[m]_n=\frac{m!}{(m-n)!},$$

$$[m]^n=[m+n-1]_n.$$

Последнее равенство показывает, что упорядоченные размещения n объектов по m ящикам эквивалентны инъективным размещениям n объектов по $m+n-1$ позициям. *(Источник: Липский, гл. 1, §1.2, с. 17.)*

Перестановки: разложение на циклы. Напомним, что перестановкой n-элементного множества X называется произвольная взаимно однозначная функция $f: X\to X$. Обычно перестановка записывается таблицей из двух строк, в которой под элементом x стоит $f(x)$. Например, для $X=\{a,b,c,d\}$ перестановка

$$f=\begin{pmatrix} a & b & c & d\\ d & a & c & b\end{pmatrix}$$

задаёт $f(a)=d$, $f(b)=a$, $f(c)=c$, $f(d)=b$. *(Источник: Липский, гл. 1, §1.3, с. 17–18.)*

Произвольную перестановку $f\in S_n$ можно представить ориентированным графом с вершинами $1,\dots,n$, в котором из x идёт дуга в $f(x)$. Так как из каждой вершины выходит ровно одна дуга и в каждую входит ровно одна дуга, граф распадается на непересекающиеся циклы. Это даёт *разложение перестановки на циклы*:

$$f=[a_0^{(1)}\dots a_{n_1-1}^{(1)}]\cdots[a_0^{(k)}\dots a_{n_k-1}^{(k)}].$$

Цикл длины 1 соответствует неподвижной точке, цикл длины 2 называется *транспозицией*. Перестановка имеет *тип* $1^{\lambda_1}2^{\lambda_2}\dots n^{\lambda_n}$, если в её разложении на циклы ровно λ_i циклов длины i. *(Источник: Липский, гл. 1, §1.3, с. 19–20.)*

Пример. Перестановка

$$f = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 7 & 5 & 1 & 4 & 2 & 3 & 6 \end{pmatrix}$$

раскладывается в циклы

$$f = [1\,7\,6\,3][2\,5][4]$$

и имеет тип $1^1 2^1 4^1$. *(Источник: Липский, гл. 1, §1.3, с. 20.)*

Инверсии и знак перестановки. Пусть перестановка задана последовательностью $\langle a_1, \ldots, a_n \rangle$. Пара $\langle a_i, a_j \rangle$, $i < j$, называется *инверсией*, если $a_i > a_j$. Число инверсий обозначается $I(f)$. *Знак* перестановки определяется равенством

$$\operatorname{sgn}(f) = (-1)^{I(f)}.$$

Перестановка называется *чётной*, если $\operatorname{sgn}(f) = 1$, и *нечётной*, если $\operatorname{sgn}(f) = -1$. *(Источник: Липский, гл. 1, §1.3, с. 20–21.)*

Пример. Перестановка $\langle 2, 4, 3, 5, 1 \rangle$ имеет инверсии

$$\langle 2,1\rangle, \quad \langle 4,3\rangle, \quad \langle 4,1\rangle, \quad \langle 3,1\rangle, \quad \langle 5,1\rangle,$$

поэтому $I(f) = 5$ и $\operatorname{sgn}(f) = -1$; перестановка нечётна. *(Источник: Липский, гл. 1, §1.3, с. 21.)*

Тождественная перестановка $\langle 1, \ldots, n \rangle$ не имеет инверсий и является чётной. Перестановка $\langle n, n-1, \ldots, 1 \rangle$ имеет

$$\frac{n(n-1)}{2}$$

инверсий, поэтому её чётность зависит от n: она чётна при $n = 4k$ или $n = 4k+1$ и нечётна в остальных случаях. *(Источник: Липский, гл. 1, §1.3, с. 21.)*

Справедливы следующие важные свойства.

Лемма 1.5. Произвольную перестановку $f \in S_n$ можно представить в виде суперпозиции $I(f)$ транспозиций соседних элементов.

Лемма 1.6. Для любых перестановок $f, g \in S_n$

$$\operatorname{sgn}(fg) = \operatorname{sgn}(f)\operatorname{sgn}(g).$$

Лемма 1.7. Каждая транспозиция нечётна; знак произвольного цикла длины k равен $(-1)^{k-1}$.

Лемма 1.8. Если перестановка f имеет тип $1^{\lambda_1} 2^{\lambda_2} \ldots n^{\lambda_n}$, то

$$\operatorname{sgn}(f) = (-1)^{\lambda_2 + \lambda_4 + \cdots}.$$

Иными словами, знак перестановки определяется только числом циклов чётной длины. *(Источник: Липский, гл. 1, §1.3, с. 21–23.)*

На лемме 1.8 основан эффективный алгоритм определения знака перестановки за время $O(n)$. Он последовательно просматривает позиции $1, \ldots, n$, выделяет циклы перестановки и меняет знак переменной s при обнаружении цикла чётной длины. По окончании работы $s = \operatorname{sgn}(f)$. *(Источник: Липский, гл. 1, §1.3, алгоритм 1.9, с. 23–24.)*

Подмножества и биномиальные коэффициенты. Число всех подмножеств n-элементного множества X равно 2^n. Это устанавливается взаимно однозначным соответствием между подмножествами $Y \subseteq X$ и бинарными последовательностями $b_1 \dots b_n$, где

$$b_i = \begin{cases} 1, & x_i \in Y, \\ 0, & x_i \notin Y. \end{cases}$$

Число таких последовательностей равно 2^n, поэтому

$$|\mathcal{P}(X)| = 2^n.$$

(Источник: Липский, гл. 1, §1.5, с. 34.)

Число k-элементных подмножеств n-элементного множества обозначается

$$\binom{n}{k}$$

и называется *биномиальным коэффициентом*. Оно входит в формулу бинома Ньютона:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

(Источник: Липский, гл. 1, §1.6, с. 39–40.)

Теорема 1.14 (формула для биномиального коэффициента).

$$\binom{n}{k} = \frac{[n]_k}{k!} = \frac{n!}{k!(n-k)!}.$$

Доказательство. Каждая инъективная последовательность длины k с элементами из n-элементного множества определяет k-элементное подмножество, состоящее из её элементов. Одно и то же подмножество получается ровно из $k!$ таких последовательностей, соответствующих всевозможным перестановкам его элементов. Поэтому

$$\binom{n}{k} = \frac{[n]_k}{k!} = \frac{n!}{k!(n-k)!}.$$

Теорема доказана. *(Источник: Липский, гл. 1, §1.6, с. 42.)*

Для биномиальных коэффициентов справедливы важнейшие тождества:

$$\binom{n}{k} = \binom{n}{n-k},$$

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1},$$

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n,$$

$$\sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}.$$

Первое выражает симметрию выбора k элементов и $n-k$ элементов; второе — правило треугольника Паскаля; третье — подсчёт всех подмножеств; четвёртое получается

подсчётом пар $\langle x, K\rangle$, где $x \in K \subseteq X$, двумя способами. *(Источник: Липский, гл. 1, §1.6, с. 40–41.)*

Справедливо также *тождество Коши*:

$$\binom{m+n}{k} = \sum_{s=0}^{k} \binom{m}{s}\binom{n}{k-s}.$$

Оно получается выбором k человек из m мужчин и n женщин с классификацией по числу выбранных мужчин. В частности, при $m = n = k$

$$\binom{2n}{n} = \sum_{s=0}^{n} \binom{n}{s}^2.$$

(Источник: Липский, гл. 1, §1.6, с. 41–42.)

Полезно тождество

$$\binom{n}{k}\binom{k}{m} = \binom{n}{m}\binom{n-m}{k-m},$$

которое подсчитывает пары $\langle K, M\rangle$ подмножеств $K \subseteq M \subseteq X$ с $|K| = m$, $|M| = k$ двумя способами. *(Источник: Липский, гл. 1, §1.6, с. 42.)*

Биномиальные коэффициенты сначала возрастают, а затем убывают:

$$\binom{n}{0} < \binom{n}{1} < \cdots < \binom{n}{\lfloor n/2\rfloor} = \binom{n}{\lceil n/2\rceil} > \cdots > \binom{n}{n}$$

(при $n > 1$). Это следует из отношения

$$\frac{\binom{n}{i+1}}{\binom{n}{i}} = \frac{n-i}{i+1}.$$

(Источник: Липский, гл. 1, §1.6, с. 43.)

Сочетания с повторениями. Рассмотрим множество с повторениями

$$(k * x_1, \ldots, k * x_n),$$

в котором каждый из n различных элементов может встречаться не более k раз. Спрашивается, сколько существует k-элементных подмножеств такого множества с повторениями. Это число равно числу неубывающих последовательностей длины k с элементами из $\{1, \ldots, n\}$.

Теорема 1.15 (о сочетаниях с повторениями). Число k-элементных подмножеств множества с повторениями

$$(k * x_1, \ldots, k * x_n)$$

равно

$$\binom{n+k-1}{k}.$$

Доказательство. Каждой неубывающей последовательности $\langle a_1, \ldots, a_k\rangle$ с элементами из $\{1, \ldots, n\}$ поставим в соответствие последовательность длины $k+n-1$,

разделив n групп элементов $1, \ldots, n$ с помощью $n-1$ нулей-разделителей. Например, при $n = 5$, $k = 4$ последовательности $\langle 2, 2, 4, 5\rangle$ соответствует

$$\langle 0, 2, 2, 0, 0, 4, 0, 5\rangle.$$

Такая последовательность однозначно определяется расположением $n-1$ нулей среди $k + n - 1$ позиций. Число способов выбрать позиции для нулей равно

$$\binom{k+n-1}{n-1} = \binom{n+k-1}{k}.$$

Теорема доказана. *(Источник: Липский, гл. 1, §1.6, с. 43–44.)*

Эту же формулу можно записать в виде

$$\binom{n+k-1}{k} = \frac{n(n+1)\cdots(n+k-1)}{k!} = \frac{[n]^k}{k!}.$$

Она показывает, что число k-элементных сочетаний с повторениями из n элементов равно числу упорядоченных размещений k объектов по n ящикам, делённому на $k!$. *(Источник: Липский, гл. 1, §1.6, с. 44–45.)*

Пример. При $n = 3$, $k = 4$ число 4-элементных подмножеств множества с повторениями, в котором каждый из трёх элементов может встречаться не более четырёх раз, равно

$$\binom{3+4-1}{4} = \binom{6}{4} = 15.$$

(Источник: Липский, гл. 1, §1.6, с. 43–45.)

Сводка основных формул. Для удобства приведём основные формулы раздела:

все функции $X \to Y$:	m^n;
инъективные функции $X \to Y$:	$[m]_n = m(m-1)\cdots(m-n+1)$;
перестановки n-элементного множества :	$n!$;
упорядоченные размещения по ящикам :	$[m]^n = m(m+1)\cdots(m+n-1)$;
все подмножества :	2^n;
k-элементные подмножества :	$\binom{n}{k} = \frac{n!}{k!(n-k)!}$;
k-элементные подмножества с повторениями :	$\binom{n+k-1}{k}$.

(Обобщение по: Липский, гл. 1, §1.2–1.3, с. 14–24; §1.5–1.6, с. 34–45.)

Итог пункта. В п. 1.5.1 введены базовые комбинаторные объекты: функции и размещения, инъективные размещения, перестановки, упорядоченные размещения по ящикам, подмножества, сочетания и сочетания с повторениями. Для каждого объекта приведены основные формулы, доказательства и примеры. Показано, что число перестановок равно $n!$, число k-элементных подмножеств выражается биномиальным коэффициентом $\binom{n}{k}$, а сочетания с повторениями считаются по формуле $\binom{n+k-1}{k}$. Введены разложение перестановки на циклы, инверсии и знак перестановки; сформулированы леммы о знаке и алгоритм его вычисления за $O(n)$. Приведены

важнейшие тождества для биномиальных коэффициентов: симметрия, правило Паскаля, суммы $\sum \binom{n}{k} = 2^n$, $\sum k\binom{n}{k} = n2^{n-1}$, тождество Коши и тождество для пар подмножеств. Эти результаты служат основой для дальнейшего изучения разбиений множеств и чисел, чисел Стирлинга, производящих функций и метода включений и исключений. *(Обобщение по: Липский, гл. 1, §1.2–1.3, с. 14–24; §1.5–1.6, с. 34–45.)*

1.5.2 Разбиения множеств и чисел, числа Стирлинга

Основные источники: Липский В. Комбинаторика для программистов. М.: Мир, 1988. Гл. 1, §1.8–1.11, с. 48–64.

Разбиения множества. Под *разбиением* n-элементного множества X на k блоков понимается семейство

$$\pi = \{B_1, \ldots, B_k\},$$

такое что

$$B_1 \cup \cdots \cup B_k = X, \qquad B_i \cap B_j = \varnothing \quad (1 \le i < j \le k), \qquad B_i \neq \varnothing \quad (1 \le i \le k).$$

Подмножества $B_1, \ldots, B_k$ называются *блоками* разбиения π. Множество всех разбиений X на k блоков обозначается $\Pi_k(X)$, а множество всех разбиений — через $\Pi(X)$. Очевидно,

$$\Pi(X) = \Pi_1(X) \cup \Pi_2(X) \cup \cdots \cup \Pi_n(X),$$

причём эти классы попарно не пересекаются. *(Источник: Липский, гл. 1, §1.8, с. 48.)*

С каждым разбиением π связано отношение эквивалентности

$$E(\pi) = \bigcup_{B \in \pi} (B \times B),$$

то есть x и y эквивалентны тогда и только тогда, когда они лежат в одном блоке разбиения π. Обратно, каждому отношению эквивалентности E на X соответствует разбиение

$$X/E = \{x/E \mid x \in X\},$$

где $x/E = \{y \in X \mid xEy\}$. Формулы

$$E(\pi) = \bigcup_{B \in \pi} (B \times B), \qquad X/E = \{x/E \mid x \in X\}$$

задают взаимно однозначное соответствие между разбиениями множества X и отношениями эквивалентности на X. *(Источник: Липский, гл. 1, §1.8, с. 48.)*

Если $\pi, \sigma \in \Pi(X)$ и каждый блок $B \in \sigma$ является объединением некоторого числа блоков разбиения π, то говорят, что π есть *измельчение* σ, и пишут $\pi \le \sigma$. Например,

$$(\{1\}, \{2,5\}, \{4,6\}, \{3\}) \le (\{1,2,3,5\}, \{4,6\}).$$

Отношение $\le$ является частичным порядком на $\Pi(X)$. Более того, $\Pi(X)$ относительно измельчения образует решётку: для любых π, σ

$$\pi \wedge \sigma = \{A \cap B \mid A \in \pi,\ B \in \sigma,\ A \cap B \neq \varnothing\},$$

а $\pi \vee \sigma$ определяется как наименьшее разбиение, блоки которого являются объединениями блоков из π и σ. *(Источник: Липский, гл. 1, §1.8, с. 48–50.)*

Числа Стирлинга второго рода. Число разбиений n-элементного множества на k блоков обозначается $S(n,k)$ и называется *числом Стирлинга второго рода*:

$$S(n,k) = |\Pi_k(X)|, \qquad |X| = n.$$

Например, $S(4,2) = 7$, поскольку существуют ровно семь разбиений множества $\{1,2,3,4\}$ на два блока:

$$\begin{array}{lll} \{\{1,2,3\},\{4\}\}, & \{\{1,2,4\},\{3\}\}, & \{\{1,3,4\},\{2\}\}, \\ \{\{1,2\},\{3,4\}\}, & \{\{1,3\},\{2,4\}\}, & \{\{1,4\},\{2,3\}\}, \\ \{\{1\},\{2,3,4\}\}. & & \end{array}$$

Очевидно, $S(n,k) = 0$ при $k > n$. Принимают также $S(0,0) = 1$: пустое семейство блоков является разбиением пустого множества. *(Источник: Липский, гл. 1, §1.9, с. 50–51.)*

Для чисел Стирлинга второго рода справедливы рекуррентные соотношения

$$S(n,k) = S(n-1,k-1) + kS(n-1,k) \quad (0 < k < n),$$
$$S(n,n) = 1 \quad (n \geq 0),$$
$$S(n,0) = 0 \quad (n > 0).$$

Доказательство первого соотношения. Рассмотрим все разбиения множества $\{1,\ldots,n\}$ на k блоков. Они распадаются на два класса: разбиения, содержащие одноэлементный блок $\{n\}$, и разбиения, в которых n входит в блок мощности не меньше двух. Число первых равно $S(n-1,k-1)$: удалив блок $\{n\}$, получаем разбиение множества $\{1,\ldots,n-1\}$ на $k-1$ блоков. Число вторых равно $kS(n-1,k)$: каждому разбиению множества $\{1,\ldots,n-1\}$ на k блоков соответствует ровно k разбиений, полученных добавлением n в один из k блоков. Складывая, получаем требуемое. *(Источник: Липский, гл. 1, §1.9, с. 51.)*

Формулы для $S(n,k)$ удобно вычислять с помощью «треугольника Стирлинга»: каждое значение, кроме крайних единиц, равно сумме числа, стоящего над ним с левой стороны, и числа, стоящего точно над ним, умноженного на k. Например:

$n\backslash k$	0	1	2	3	4	5	6
0	1	0	0	0	0	0	0
1	0	1	0	0	0	0	0
2	0	1	1	0	0	0	0
3	0	1	3	1	0	0	0
4	0	1	7	6	1	0	0
5	0	1	15	25	10	1	0
6	0	1	31	90	65	15	1

(Источник: Липский, гл. 1, §1.9, табл. 1.2, с. 51.)

Справедливо также соотношение

$$S(n,k) = \sum_{i=k-1}^{n-1} \binom{n-1}{i} S(i,k-1) \quad (k \geq 2).$$

Оно получается классификацией разбиений по блоку B, содержащему элемент n. Если $|B| = b$, то блок B можно выбрать $\binom{n-1}{b-1}$ способами, а оставшееся множество разбить на $k-1$ блоков $S(n-b,k-1)$ способами. Полагая $i = n-b$, получаем формулу. *(Источник: Липский, гл. 1, §1.9, с. 52.)*

Числа Белла. Число всех разбиений n-элементного множества называется *числом Белла*:

$$B_n = |\Pi(X)| = \sum_{k=0}^{n} S(n,k).$$

Для чисел Белла справедлива рекуррентная зависимость

$$B_{n+1} = \sum_{i=0}^{n} \binom{n}{i} B_i$$

(принимают $B_0 = 1$). Она получается классификацией разбиений множества $\{1, \ldots, n+1\}$ по блоку, содержащему элемент $n+1$: если из этого блока удалить $n+1$, остаётся разбиение некоторого i-элементного подмножества, а само подмножество можно выбрать $\binom{n}{i}$ способами. *(Источник: Липский, гл. 1, §1.9, с. 52–53.)*

Первые числа Белла:

$$B_0 = 1, \quad B_1 = 1, \quad B_2 = 2, \quad B_3 = 5, \quad B_4 = 15, \quad B_5 = 52, \quad B_6 = 203.$$

(Источник: Липский, гл. 1, §1.9, табл. 1.3, с. 53.)

Связь с сюръективными функциями. Число функций $f : X \to Y$, отображающих n-элементное множество X на k-элементное множество Y, обозначим $s_{n,k}$. Каждой такой функции соответствует её ядро

$$N(f) = \{f^{-1}(y) \mid y \in Y\},$$

являющееся разбиением X на k блоков. Обратно, каждому разбиению $\pi \in \Pi_k(X)$ соответствует ровно $k!$ функций из X на Y, для которых $N(f) = \pi$: достаточно взаимно однозначно сопоставить блоки разбиения π элементам множества Y. Поэтому

$$s_{n,k} = k!\, S(n,k).$$

Отсюда, в частности, следует явная формула

$$S(n,k) = \frac{1}{k!} \sum_{i=0}^{k-1} (-1)^i \binom{k}{i} (k-i)^n,$$

которая будет получена в п. 1.5.4 с помощью принципа включений и исключений. *(Источник: Липский, гл. 1, §1.9, с. 53; §1.13, с. 73–74.)*

Числа Стирлинга второго рода являются коэффициентами перехода от базиса $1, x, x^2, \ldots$ к базису $1, [x]_1, [x]_2, \ldots$, где

$$[x]_k = x(x-1)\cdots(x-k+1).$$

Справедливо тождество

$$x^n = \sum_{k=0}^{n} S(n,k)[x]_k.$$

Оно доказывается подсчётом числа всех функций $f : A \to B$, где $|A| = n$, $|B| = x$, двумя способами: слева это x^n, а справа — классификация функций по множеству $f(A)$ даёт $\sum_k \binom{x}{k} k! S(n,k) = \sum_k [x]_k S(n,k)$. *(Источник: Липский, гл. 1, §1.9, теорема 1.18, с. 54.)*

Числа Стирлинга первого рода. Числа Стирлинга первого рода $s(n,k)$ определяются как коэффициенты в разложении

$$[x]_n = \sum_{k=0}^{n} s(n,k)x^k,$$

где $[x]_n = x(x-1)\cdots(x-n+1)$. Иными словами, $s(n,k)$ позволяют перейти от базиса $1, [x]_1, [x]_2, \ldots$ обратно к базису $1, x, x^2, \ldots$. Очевидно, $s(n,k) = 0$ при $k > n$. *(Источник: Липский, гл. 1, §1.9, с. 54.)*

Для чисел $s(n,k)$ справедливы рекуррентные соотношения

$$s(n,k) = s(n-1,k-1) - (n-1)s(n-1,k),$$

$$s(n,n) = 1,$$

$$s(n,0) = 0 \quad (n > 0).$$

Первое соотношение получается сравнением коэффициентов при x^k в равенстве

$$[x]_n = [x]_{n-1}(x-n+1).$$

Действительно,

$$\sum_{k=0}^{n} s(n,k)x^k = (x-n+1)\sum_{k=0}^{n-1} s(n-1,k)x^k,$$

откуда коэффициент при x^k равен $s(n-1,k-1) - (n-1)s(n-1,k)$. *(Источник: Липский, гл. 1, §1.9, с. 54–55.)*

Первые значения $s(n,k)$:

$n\backslash k$	0	1	2	3	4	5	6
0	1	0	0	0	0	0	0
1	0	1	0	0	0	0	0
2	0	−1	1	0	0	0	0
3	0	2	−3	1	0	0	0
4	0	−6	11	−6	1	0	0
5	0	24	−50	35	−10	1	0
6	0	−120	274	−225	85	−15	1

(Источник: Липский, гл. 1, §1.9, табл. 1.4, с. 55.)

Модуль $|s(n,k)|$ имеет комбинаторный смысл: это число перестановок n-элементного множества, имеющих в разложении на циклы ровно k циклов. Кроме того, справедливо тождество

$$[x]^n = \sum_{k=0}^{n} |s(n,k)|x^k,$$

где

$$[x]^n = x(x+1)\cdots(x+n-1)$$

— возрастающий факториал. Матрицы чисел Стирлинга первого и второго рода взаимно обратны:

$$[s(n,k)]^{-1}_{0\le n,k\le m} = [S(n,k)]_{0\le n,k\le m}.$$

(Источник: Липский, гл. 1, §1.9, с. 54–55; задачи 1.34–1.36, с. 81.)

Разбиения чисел. *Разбиением числа* n на k слагаемых называется представление

$$n = a_1 + \cdots + a_k,$$

где

$$a_1 \geq a_2 \geq \cdots \geq a_k > 0.$$

Число таких разбиений обозначается $P(n,k)$, а число всех разбиений числа n — через $P(n)$:

$$P(n) = \sum_{k=1}^{n} P(n,k) \quad (n > 0).$$

Принимают $P(0,0) = P(0) = 1$. Например, $P(7) = 15$; все разбиения числа 7 имеют вид

$$\begin{aligned}
&7, \quad 6+1, \quad 5+2, \quad 5+1+1, \quad 4+3, \quad 4+2+1,\\
&4+1+1+1, \quad 3+3+1, \quad 3+2+2, \quad 3+2+1+1,\\
&3+1+1+1+1, \quad 2+2+2+1, \quad 2+2+1+1+1,\\
&2+1+1+1+1+1, \quad 1+1+1+1+1+1+1.
\end{aligned}$$

(Источник: Липский, гл. 1, §1.11, с. 60–61.)

Для разбиений чисел часто используется *диаграмма Феррерса*: разбиению $n = a_1 + \cdots + a_k$ сопоставляется таблица из k строк, в i-й строке которой стоит a_i точек. Транспонирование диаграммы (перемена ролей строк и столбцов) даёт *сопряжённое разбиение* числа n. Например, разбиению

$$16 = 6 + 4 + 4 + 2$$

соответствует диаграмма, сопряжённая диаграмме разбиения

$$16 = 4 + 4 + 3 + 3 + 1 + 1.$$

(Источник: Липский, гл. 1, §1.11, с. 60–61.)

Теорема 1.20. Число разбиений числа n на k слагаемых равно числу разбиений числа n с наибольшим слагаемым, равным k.

$$P(n,k) = P(n, \text{наибольшее слагаемое } = k).$$

Доказательство. Транспонирование диаграммы Феррерса устанавливает взаимно однозначное соответствие между разбиениями числа n на k слагаемых и разбиениями числа n с наибольшим слагаемым k. *(Источник: Липский, гл. 1, §1.11, теорема 1.20, с. 60.)*

Теорема 1.21. Число разбиений числа n на попарно различные слагаемые равно числу разбиений числа n на нечётные слагаемые. *(Источник: Липский, гл. 1, §1.11, теорема 1.21, с. 61.)*

Доказательство. Установим взаимно однозначное соответствие между указанными разбиениями. Рассмотрим разбиение числа n на нечётные слагаемые $b_1, \ldots, b_p$, где слагаемое b_i встречается r_i раз. Пусть

$$r_i = 2^{q_1} + 2^{q_2} + \ldots \qquad (q_1 > q_2 > \ldots)$$

— двоичное представление числа r_i. Заменим r_i слагаемых b_i на попарно различные слагаемые

$$b_i 2^{q_1}, \quad b_i 2^{q_2}, \quad \ldots$$

Сумма при этом не меняется. Выполнив такую замену для всех i и упорядочив слагаемые по невозрастанию, получаем разбиение числа n на попарно различные слагаемые. Обратное преобразование возможно потому, что каждое натуральное число единственным образом представляется в виде произведения нечётного числа на степень двойки. Например,

$$26 = 7 + 5 + 5 + 3 + 3 + 1 + 1 + 1 + 1$$

переходит в

$$26 = 10 + 7 + 6 + 2 + 1.$$

Теорема доказана. *(Источник: Липский, гл. 1, §1.11, с. 61–62.)*

Производящая функция для последовательности $P(0), P(1), P(2), \ldots$ имеет вид

$$\sum_{n=0}^{\infty} P(n)x^n = \prod_{h=1}^{\infty}(1 - x^h)^{-1}.$$

Этот факт будет подробно рассмотрен в п. 1.5.3; здесь отметим лишь, что он даёт удобный аналитический инструмент для изучения разбиений чисел. *(Источник: Липский, гл. 1, §1.12, с. 68.)*

Алгоритм генерирования разбиений числа. Для практических приложений важен не только подсчёт, но и перечисление разбиений числа. В пособии приведён алгоритм 1.22, который генерирует все разбиения числа n в порядке, обратном лексикографическому. Разбиение хранится в виде массивов $S[1], \ldots, S[d]$ попарно различных слагаемых и $R[1], \ldots, R[d]$, где $R[i]$ показывает, сколько раз слагаемое $S[i]$ входит в разбиение. Алгоритм находит каждое следующее разбиение за число шагов, ограниченное константой, не зависящей от n. *(Источник: Липский, гл. 1, §1.11, алгоритм 1.22, с. 63.)*

Итог пункта. В п. 1.5.2 рассмотрены разбиения множеств и чисел. Введены числа Стирлинга второго рода $S(n,k)$ как числа разбиений n-элементного множества на k блоков, доказаны рекуррентные соотношения и приведены таблицы. Введены числа Белла B_n, доказана рекуррентная формула для них. Показана связь $S(n,k)$ с сюръективными функциями: $s_{n,k} = k!S(n,k)$, и приведено тождество $x^n = \sum_k S(n,k)[x]_k$. Определены числа Стирлинга первого рода $s(n,k)$ через разложение $[x]_n = \sum_k s(n,k)x^k$, доказаны рекуррентные соотношения, указан комбинаторный смысл $|s(n,k)|$ как числа перестановок с k циклами. Наконец, введены разбиения чисел $P(n,k)$, $P(n)$, диаграммы Феррерса и сопряжённые разбиения; доказаны теоремы о равенстве числа разбиений на k слагаемых и разбиений с наибольшим слагаемым k, а также о равенстве числа разбиений на попарно различные и на нечётные слагаемые. *(Обобщение по: Липский, гл. 1, §1.8–1.11, с. 48–64; §1.12, с. 68.)*

1.5.3 Метод производящих функций

Основные источники: Липский В. Комбинаторика для программистов. М.: Мир, 1988. Гл. 1, §1.12, с. 64–72; задачи 1.43–1.48, с. 82–83.

Определение производящей функции. В комбинаторных задачах на подсчёт числа объектов при наличии ограничений искомым решением часто является последовательность

$$a_0, a_1, a_2, \ldots,$$

где a_k — число искомых объектов «размерности» k. Например, если речь идёт о разбиениях числа, можно положить $a_k = P(k)$; если о подмножествах n-элементного множества — $a_k = \binom{n}{k}$; если о бинарных деревьях — $a_k = c_k$ (числа Каталана) и т. д. В таких случаях удобно сопоставить последовательности формальный ряд

$$A(x) = \sum_{k=0}^{\infty} a_k x^k,$$

который называется *производящей функцией* для данной последовательности. *(Источник: Липский, гл. 1, §1.12, с. 64.)*

Слово «формальный» означает, что ряд трактуется только как удобная запись последовательности коэффициентов; вопрос о сходимости при этом не ставится. Мы никогда не вычисляем значение такого ряда в конкретной точке x; мы выполняем над рядами некоторые алгебраические операции, а затем находим коэффициенты при отдельных степенях x. Такой подход позволяет применять к комбинаторным задачам мощную технику, не заботясь о сходимости. *(Источник: Липский, гл. 1, §1.12, с. 64.)*

Операции над формальными рядами. Для произвольных рядов

$$A(x) = \sum_{k=0}^{\infty} a_k x^k, \qquad B(x) = \sum_{k=0}^{\infty} b_k x^k$$

определяются следующие операции:

1) *сложение*:

$$A(x) + B(x) = \sum_{k=0}^{\infty} (a_k + b_k) x^k;$$

2) *умножение на число* p (действительное или комплексное):

$$pA(x) = \sum_{k=0}^{\infty} p a_k x^k;$$

3) *произведение Коши* (коротко: произведение):

$$A(x) \cdot B(x) = \sum_{k=0}^{\infty} c_k x^k,$$

где

$$c_k = \sum_{i=0}^{k} a_i b_{k-i} = a_0 b_k + a_1 b_{k-1} + \cdots + a_k b_0.$$

Если $a_k = 0$ для всех $k > n$, то ряд $A(x)$ отождествляется с многочленом $a_n x^n + \cdots + a_0$. *(Источник: Липский, гл. 1, §1.12, с. 64–65.)*

Связь с аналитическими функциями. Из математического анализа известно, что если ряд $A(x) = \sum a_k x^k$ сходится в некоторой окрестности нуля, то его сумма является аналитической функцией в этой окрестности и

$$a_k = \frac{A^{(k)}(0)}{k!}, \qquad k = 0, 1, 2, \ldots$$

(ряд $A(x)$ есть не что иное, как ряд Маклорена функции $A(x)$). Более того, если $A(x)$ и $B(x)$ аналитичны в окрестности нуля, то операции сложения, умножения на число и произведения Коши совпадают с обычными операциями над функциями. Это позволяет отождествлять формальный ряд с аналитической функцией в случаях, когда ряд сходится, и писать, например,

$$\sum_{k=0}^{\infty} x^k = (1-x)^{-1}, \qquad \sum_{k=0}^{\infty} \frac{x^k}{k!} = e^x.$$

Такое отождествление удобно тем, что позволяет пользоваться аналитическими тождествами (дифференцированием, разложением в ряд, почленным интегрированием) при работе с формальными рядами. *(Источник: Липский, гл. 1, §1.12, с. 65.)*

Простейшие производящие функции. Приведём несколько базовых примеров.

1) Для биномиальных коэффициентов при фиксированном n:

$$\sum_{k=0}^{\infty} \binom{n}{k} x^k = \sum_{k=0}^{n} \binom{n}{k} x^k = (1+x)^n.$$

   Это непосредственно следует из формулы бинома Ньютона.

2) Для последовательности $1, 2, 4, 8, \ldots, 2^k, \ldots$:

$$\sum_{k=0}^{\infty} 2^k x^k = \sum_{k=0}^{\infty} (2x)^k = (1-2x)^{-1}.$$

3) Для последовательности $0, 1, 2, 3, \ldots$:

$$\sum_{k=0}^{\infty} k x^k = x \sum_{k=0}^{\infty} k x^{k-1} = x \frac{d}{dx} \sum_{k=0}^{\infty} x^k = x \frac{d}{dx} (1-x)^{-1} = x(1-x)^{-2}.$$

   Здесь использовано почленное дифференцирование, допустимое в окрестности нуля.

4) Для последовательности $1, 1, 1, \ldots$:

$$\sum_{k=0}^{\infty} x^k = (1-x)^{-1}.$$

(Источник: Липский, гл. 1, §1.12, с. 65–66.)

Комбинаторная интерпретация произведения. Формула $(1+x)^n$ имеет простую комбинаторную интерпретацию. Каждый множитель $(1+x)$ можно трактовать как соответствующий некоторому элементу a_i множества $X = \{a_1, \ldots, a_n\}$ и как представляющий два возможных числа появлений этого элемента в подмножестве: нуль раз (слагаемое $x^0 = 1$) и один раз (слагаемое $x^1 = x$). Каждое подмножество множества X однозначно определяется указанием числа появлений в нём каждого из элементов, то есть выбором одного из слагаемых из каждого множителя произведения $(1+x)\cdots(1+x)$. Определённое таким образом слагаемое разложения даёт вклад в коэффициент при x^k, где k — мощность подмножества, равный единице. Это рассуждение естественным образом переносится на множества с повторениями. *(Источник: Липский, гл. 1, §1.12, с. 66.)*

Множества с повторениями и производящие функции. Предположим, что $X = (2*a_1, 3*a_2, 1*a_3, 4*a_4)$, и обозначим через c_k число k-элементных подмножеств этого множества с повторениями. В силу рассуждения, аналогичного приведённому выше, производящая функция для последовательности $c_0, c_1, c_2, \ldots$ равна

$$\begin{aligned}\sum_{k=0}^{\infty} c_k x^k &= (1+x+x^2)(1+x+x^2+x^3)(1+x)(1+x+x^2+x^3+x^4)\\ &= 1+4x+9x^2+15x^3+20x^4+22x^5+20x^6\\ &\quad +15x^7+9x^8+4x^9+x^{10}.\end{aligned}$$

Соответствующим подбором i-го множителя можно накладывать произвольные ограничения на число вхождений элемента a_i. Например, множитель имеет вид $1+x^3+x^7$, если i-й элемент может появляться 0, 3 или 7 раз; множитель $1+x^2+x^4+\cdots = (1-x^2)^{-1}$, если этот элемент может появляться произвольное чётное число раз, и т. д. В частности, если не накладывать никаких ограничений на число вхождений элементов a_i, $1 \le i \le n$, то производящая функция имеет вид

$$(1+x+x^2+\ldots)\cdots(1+x+x^2+\ldots) = (1-x)^{-n}.$$

(Источник: Липский, гл. 1, §1.12, с. 66–67.)

Доказательство теоремы о сочетаниях с повторениями. Напомним (см. п. 1.5.1), что число k-элементных подмножеств множества с повторениями $(k*x_1, \ldots, k*x_n)$ равно $\binom{n+k-1}{k}$. Покажем, как этот результат получается с помощью производящих функций.

Имеем

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \binom{n+k-1}{k} x^k.$$

Действительно, k-я производная функции $(1-x)^{-n}$ равна

$$\frac{d^k}{dx^k}(1-x)^{-n} = (-n)(-n-1)\cdots(-n-k+1)(1-x)^{-n-k}(-1)^k = [n]^k(1-x)^{-n-k},$$

где $[n]^k = n(n+1)\cdots(n+k-1)$. Разлагая $(1-x)^{-n}$ в ряд Маклорена, получаем

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \frac{[n]^k}{k!} x^k = \sum_{k=0}^{\infty} \binom{n+k-1}{k} x^k.$$

Это даёт ещё одно независимое доказательство теоремы 1.15. *(Источник: Липский, гл. 1, §1.12, с. 67.)*

Доказательство тождества Коши. Производящие функции удобны и для доказательства тождеств, связанных с биномиальными коэффициентами. Например, тождество Коши

$$\binom{m+n}{k} = \sum_{s=0}^{k} \binom{m}{s}\binom{n}{k-s}$$

получается сравнением коэффициентов в обеих частях равенства

$$\sum_{k=0}^{m+n}\binom{m+n}{k}x^k = (1+x)^{m+n} = (1+x)^m(1+x)^n = \sum_{i=0}^{m}\binom{m}{i}x^i\cdot\sum_{j=0}^{n}\binom{n}{j}x^j = \sum_{k=0}^{m+n}\sum_{s=0}^{k}\binom{m}{s}\binom{n}{k-s}x^k.$$

Здесь использована формула произведения Коши двух многочленов. *(Источник: Липский, гл. 1, §1.12, с. 67.)*

Разбиения чисел и производящие функции. Каждое разбиение $n = a_1+\cdots+a_k$ можно однозначно представить последовательностью $\langle\lambda_1,\ldots,\lambda_n\rangle$, где λ_i обозначает число слагаемых, равных i. Имеем

$$\sum_{i=1}^{n} i\lambda_i = n,$$

и каждая последовательность $\langle\lambda_1,\ldots,\lambda_n\rangle$, $\lambda_i \geq 0$, удовлетворяющая этому условию, однозначно определяет разбиение числа n, содержащее λ_i слагаемых, равных i.

Обозначим через $P_h(n)$ число разбиений числа n на слагаемые, не превышающие h.

Теорема 1.23. Производящая функция для последовательности $P_h(0), P_h(1), P_h(2), \ldots$ равна

$$(1+x+x^2+\ldots)(1+x^2+x^4+\ldots)\cdots(1+x^h+x^{2h}+\ldots) = (1-x)^{-1}(1-x^2)^{-1}\cdots(1-x^h)^{-1}.$$

Доказательство. Согласно определению произведения рядов правая часть является суммой слагаемых вида

$$x^{\lambda_1}x^{2\lambda_2}\cdots x^{h\lambda_h},$$

где λ_i — номер члена, выбранного из i-го ряда. Поэтому коэффициент при x^n равен числу последовательностей $\langle\lambda_1,\ldots,\lambda_h\rangle$, таких что

$$\sum_{i=1}^{h} i\lambda_i = n.$$

Но это в точности число разбиений числа n на слагаемые, не превышающие h, то есть $P_h(n)$. *(Источник: Липский, гл. 1, §1.12, теорема 1.23, с. 67–68.)*

Теорема 1.24. Производящая функция для последовательности $P(0), P(1), P(2), \ldots$ равна

$$\prod_{h=1}^{\infty}(1-x^h)^{-1}.$$

Доказательство. Аналогично предыдущему, но число множителей неограниченно. Формально бесконечное произведение определяется как предел частичных произведений; для коэффициента при x^n достаточно взять первые n множителей. *(Источник: Липский, гл. 1, §1.12, теорема 1.24, с. 68.)*

Доказательство теоремы 1.21 через производящие функции. Напомним теорему 1.21: число разбиений числа n на попарно различные слагаемые равно числу разбиений числа n на нечётные слагаемые. Производящая функция для разбиений на попарно различные слагаемые равна

$$R(x) = (1+x)(1+x^2)(1+x^3)\cdots(1+x^k)\cdots,$$

а производящая функция для разбиений на нечётные слагаемые равна

$$N(x) = (1-x)^{-1}(1-x^3)^{-1}\cdots(1-x^{2k-1})^{-1}\cdots.$$

Пользуясь зависимостью

$$1 + x^k = \frac{1-x^{2k}}{1-x^k},$$

получаем

$$\begin{aligned} R(x) &= \frac{1-x^2}{1-x}\cdot\frac{1-x^4}{1-x^2}\cdot\frac{1-x^6}{1-x^3}\cdot\ldots \\ &= \frac{1}{1-x}\cdot\frac{1}{1-x^3}\cdot\frac{1}{1-x^5}\cdot\ldots = N(x). \end{aligned}$$

Совпадение производящих функций означает совпадение коэффициентов, то есть равенство чисел разбиений. *(Источник: Липский, гл. 1, §1.12, с. 68–69.)*

Числа Фибоначчи. Числа Фибоначчи F_n определяются рекуррентным уравнением

$$F_{n+1} = F_n + F_{n-1}, \qquad n \geq 1,$$

с начальными условиями

$$F_0 = F_1 = 1.$$

Производящая функция $F(x) = \sum_{k=0}^{\infty} F_k x^k$ удовлетворяет уравнению

$$\begin{aligned} F(x) &= \sum_{k=0}^{\infty} F_k x^k = 1 + x + \sum_{k=2}^{\infty}(F_{k-2} + F_{k-1})x^k \\ &= 1 + x + x^2\sum_{k=2}^{\infty} F_{k-2}x^{k-2} + x\sum_{k=2}^{\infty} F_{k-1}x^{k-1} \\ &= 1 + x + x^2F(x) + x(F(x)-1) = 1 + (x+x^2)F(x). \end{aligned}$$

Отсюда

$$F(x) = \frac{1}{1-x-x^2}.$$

(Источник: Липский, гл. 1, §1.12, с. 69.)

Найдя корни уравнения $1 - x - x^2 = 0$, получаем разложение

$$1 - x - x^2 = (1-ax)(1-bx),$$

где

$$a = \frac{1+\sqrt{5}}{2}, \qquad b = \frac{1-\sqrt{5}}{2}.$$

Методом неопределённых коэффициентов находим

$$\frac{1}{(1-ax)(1-bx)} = \frac{A}{1-ax} + \frac{B}{1-bx} = \frac{A+B-(Ab+Ba)x}{(1-ax)(1-bx)}.$$

Сравнивая коэффициенты в числителе, получаем

$$A+B=1, \qquad Ab+(1-A)a=0,$$

откуда

$$A = \frac{a}{a-b}, \qquad B = -\frac{b}{a-b}.$$

Следовательно,

$$\begin{aligned} F(x) &= A(1-ax)^{-1} + B(1-bx)^{-1} \\ &= A\sum_{k=0}^{\infty} a^k x^k + B\sum_{k=0}^{\infty} b^k x^k = \sum_{k=0}^{\infty} \frac{a^{k+1}-b^{k+1}}{a-b} x^k, \end{aligned}$$

и окончательно

$$F_k = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^{k+1} - \left(\frac{1-\sqrt{5}}{2}\right)^{k+1}\right].$$

Это *формула Бине* для чисел Фибоначчи. Она показывает, что F_k растёт как $a^{k+1}/\sqrt{5}$, где $a = (1+\sqrt{5})/2$ — золотое сечение. *(Источник: Липский, гл. 1, §1.12, с. 69–70.)*

Числа Каталана. Под *бинарным деревом* с n вершинами понимается пустое дерево $T = \varnothing$ при $n = 0$ или тройка

$$T = \langle L, r, P\rangle,$$

где r — корень дерева, L — левое поддерево с l вершинами, P — правое поддерево с p вершинами, причём $l + p + 1 = n$. Два бинарных дерева изоморфны, если они одновременно пусты или их левые и правые поддеревья изоморфны. Обозначим через c_k число неизоморфных бинарных деревьев с k вершинами. *(Источник: Липский, гл. 1, §1.12, с. 70.)*

Из рекурсивного определения следует, что $c_0 = 1$ и

$$c_k = \sum_{s=0}^{k-1} c_s c_{k-1-s}, \qquad k > 0.$$

Действительно, дерево с k вершинами однозначно задаётся числом s вершин в левом поддереве; число таких деревьев равно $c_s c_{k-1-s}$, а s может принимать значения $0, 1, \ldots, k-1$. *(Источник: Липский, гл. 1, §1.12, с. 70.)*

Рассмотрим производящую функцию

$$C(x) = \sum_{k=0}^{\infty} c_k x^k.$$

Равенство для c_k напоминает формулу для коэффициентов произведения Коши:

$$C(x)C(x) = C^2(x) = \sum_{k=0}^{\infty}\left(\sum_{s=0}^{k} c_s c_{k-s}\right) x^k.$$

Следовательно,

$$C(x) = 1 + xC^2(x),$$

или

$$xC^2(x) - C(x) + 1 = 0.$$

(Источник: Липский, гл. 1, §1.12, с. 70–71.)

Рассматривая это равенство как квадратное уравнение относительно $C(x)$, получаем

$$C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}.$$

Разложим $\sqrt{1-4x} = (1-4x)^{1/2}$ в ряд Маклорена. Для $k > 0$

$$\begin{aligned}\frac{d^k}{dx^k}(1-4x)^{1/2} &= \frac{1}{2}\left(\frac{1}{2}-1\right)\left(\frac{1}{2}-2\right)\cdots\left(\frac{1}{2}-k+1\right)(1-4x)^{1/2-k}(-4)^k \\ &= 2^k(-1)\cdot 1\cdot 3\cdot 5\cdots(2k-3)(1-4x)^{1/2-k}.\end{aligned}$$

Следовательно,

$$\sqrt{1-4x} = 1 - 2\sum_{k=1}^{\infty}\frac{1}{k}\binom{2k-2}{k-1}x^k.$$

Для получения решения с положительными коэффициентами следует выбрать знак минус:

$$C(x) = \frac{1-\sqrt{1-4x}}{2x} = \sum_{k=0}^{\infty}\frac{1}{k+1}\binom{2k}{k}x^k.$$

Таким образом,

$$c_k = \frac{1}{k+1}\binom{2k}{k}.$$

Числа c_k называются *числами Каталана*. Они возникают во многих комбинаторных задачах: число правильных расстановок скобок в произведении $n+1$ сомножителей, число способов разбиения выпуклого $(n+2)$-угольника на треугольники непересекающимися диагоналями и др. *(Источник: Липский, гл. 1, §1.12, с. 71–72; задачи 1.47–1.48, с. 83.)*

Общий метод решения линейных рекуррентных уравнений. Метод производящих функций применим к произвольным линейным рекуррентным уравнениям с постоянными коэффициентами:

$$C_0 a_n + C_1 a_{n-1} + \cdots + C_r a_{n-r} = 0$$

с начальными условиями, задающими $a_0, a_1, \ldots, a_{r-1}$. Схема решения такова:

1) записать производящую функцию $A(x) = \sum a_n x^n$;
2) умножить рекуррентное соотношение на x^n и просуммировать по $n \geq r$;
3) выразить $A(x)$ через начальные условия, получив рациональную функцию;
4) разложить рациональную функцию на простейшие дроби и разложить каждую дробь в степенной ряд;
5) коэффициент при x^n даёт явную формулу для a_n.

Именно так были получены формулы Бине для чисел Фибоначчи и формула для чисел Каталана. *(Источник: Липский, гл. 1, §1.12, с. 70; задача 1.46, с. 83.)*

Экспоненциальные производящие функции. Наряду с обычными производящими функциями в комбинаторике используются *экспоненциальные производящие функции*

$$C(x) = \sum_{k=0}^{\infty} c_k \frac{x^k}{k!}.$$

Они особенно удобны для последовательностей, в которых важен порядок элементов (например, для перестановок и размещений). Основное свойство экспоненциальных производящих функций состоит в том, что произведению рядов соответствует комбинаторная операция «размеченного произведения»: если c_k — число способов составить объект из k помеченных элементов, а d_k — аналогичная величина для другого типа объектов, то коэффициент при $x^k/k!$ в произведении $C(x)D(x)$ равен $\sum_{s=0}^{k} \binom{k}{s} c_s d_{k-s}$, что соответствует выбору s элементов для объекта первого типа и $k-s$ элементов для объекта второго типа. *(Источник: Липский, гл. 1, §1.12, задача 1.43, с. 82.)*

Пусть c_k — число различных последовательностей длины k с элементами из множества $Y = \{y_1, \dots, y_n\}$, в которых число появлений элемента y_i принадлежит множеству $\{m_{i1}, m_{i2}, \dots\}$. Тогда экспоненциальная производящая функция для последовательности $c_0, c_1, c_2, \dots$ равна

$$C(x) = \left(\frac{x^{m_{11}}}{m_{11}!} + \frac{x^{m_{12}}}{m_{12}!} + \dots\right) \cdots \left(\frac{x^{m_{n1}}}{m_{n1}!} + \frac{x^{m_{n2}}}{m_{n2}!} + \dots\right).$$

Например, если каждый элемент может появляться произвольное число раз, то

$$C(x) = \left(\sum_{m=0}^{\infty} \frac{x^m}{m!}\right)^n = e^{nx},$$

откуда $c_k = n^k$, что совпадает с теоремой 1.1. *(Источник: Липский, гл. 1, §1.12, задача 1.43, с. 82.)*

Пример: число сюръективных функций. Число функций $f : X \to Y$, отображающих n-элементное множество на m-элементное, равно $s_{n,m} = m!S(n,m)$. С помощью экспоненциальных производящих функций можно получить для $s_{n,m}$ явную формулу. Рассмотрим экспоненциальную производящую функцию по n при фиксированном m:

$$\sum_{n=0}^{\infty} s_{n,m} \frac{x^n}{n!} = (e^x - 1)^m.$$

Действительно, каждый из m элементов множества Y должен появиться хотя бы один раз, что даёт множитель $e^x - 1$; элементы независимы, поэтому множители перемножаются. Раскладывая $(e^x - 1)^m$ по формуле бинома Ньютона, получаем

$$(e^x - 1)^m = \sum_{i=0}^{m} (-1)^i \binom{m}{i} e^{(m-i)x} = \sum_{n=0}^{\infty} \left(\sum_{i=0}^{m} (-1)^i \binom{m}{i} (m-i)^n\right) \frac{x^n}{n!}.$$

Сравнивая коэффициенты, находим

$$s_{n,m} = \sum_{i=0}^{m} (-1)^i \binom{m}{i} (m-i)^n.$$

Отсюда

$$S(n,m) = \frac{1}{m!}\sum_{i=0}^{m}(-1)^i\binom{m}{i}(m-i)^n.$$

Это та же формула, которая будет получена в п. 1.5.4 с помощью принципа включений и исключений. *(Источник: Липский, гл. 1, §1.12, с. 67; §1.13, с. 73–74.)*

Итог пункта. В п. 1.5.3 введено понятие производящей функции как формального ряда, сопоставленного последовательности. Определены операции сложения, умножения на число и произведения Коши; установлена связь с аналитическими функциями. Приведены простейшие производящие функции: $(1+x)^n$, $(1-2x)^{-1}$, $x(1-x)^{-2}$, $(1-x)^{-1}$. Показано, как производящие функции применяются к множествам с повторениями, к доказательству теоремы о сочетаниях с повторениями и тождества Коши. С помощью производящих функций доказаны теоремы о разбиениях чисел (теоремы 1.23, 1.24), получены формулы Бине для чисел Фибоначчи и явная формула для чисел Каталана. Описан общий метод решения линейных рекуррентных уравнений. Введены экспоненциальные производящие функции и показано их применение к подсчёту сюръективных функций и чисел Стирлинга второго рода. *(Обобщение по: Липский, гл. 1, §1.12, с. 64–72; задачи 1.43–1.48, с. 82–83.)*

1.5.4 Метод включений и исключений

Основные источники: Липский В. Комбинаторика для программистов. М.: Мир, 1988. Гл. 1, §1.13, с. 72–76.

Постановка задачи. Пусть дано конечное множество X и набор подмножеств $A_1, \ldots, A_n \subseteq X$. Требуется найти мощность объединения $A_1 \cup \cdots \cup A_n$ или, что эквивалентно, мощность множества элементов, не принадлежащих ни одному из A_i. Метод включений и исключений даёт точную формулу для этой мощности через мощности всевозможных пересечений.

Формула включений и исключений. Для любых множеств $A_1, \ldots, A_n$ справедливо равенство

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{i=1}^{n}|A_i| - \sum_{1\le i<j\le n}|A_i\cap A_j| + \sum_{1\le i<j<k\le n}|A_i\cap A_j\cap A_k| - \cdots + (-1)^{n-1}|A_1\cap\cdots\cap A_n|.$$

Эквивалентная формулировка для числа элементов, не обладающих ни одним из свойств A_i:

$$\left|X \setminus \bigcup_{i=1}^{n} A_i\right| = |X| - \sum_{i=1}^{n}|A_i| + \sum_{1\le i<j\le n}|A_i\cap A_j| - \cdots + (-1)^n|A_1\cap\cdots\cap A_n|.$$

Знак перед членом, соответствующим пересечению k множеств, равен $(-1)^{k-1}$ (в первой формуле) или $(-1)^k$ (во второй).

Доказательство (индукция по n). Докажем первую формулу индукцией по числу множеств n. При $n = 1$ формула тривиальна: $|A_1| = |A_1|$. Предположим, что формула верна для $n-1$ множеств. Рассмотрим объединение

$$A_1 \cup \cdots \cup A_n = (A_1 \cup \cdots \cup A_{n-1}) \cup A_n.$$

По формуле для двух множеств:

$$|(A_1 \cup \cdots \cup A_{n-1}) \cup A_n| = |A_1 \cup \cdots \cup A_{n-1}| + |A_n| - |(A_1 \cup \cdots \cup A_{n-1}) \cap A_n|.$$

По индукционному предположению

$$|A_1 \cup \cdots \cup A_{n-1}| = \sum_{i=1}^{n-1} |A_i| - \sum_{1 \le i < j \le n-1} |A_i \cap A_j| + \cdots + (-1)^{n-2} |A_1 \cap \cdots \cap A_{n-1}|.$$

Кроме того,

$$(A_1 \cup \cdots \cup A_{n-1}) \cap A_n = (A_1 \cap A_n) \cup \cdots \cup (A_{n-1} \cap A_n).$$

Применяя к этому объединению $n-1$ множеств индукционное предположение, получаем

$$|(A_1 \cup \cdots \cup A_{n-1}) \cap A_n| = \sum_{i=1}^{n-1} |A_i \cap A_n| - \sum_{1 \le i < j \le n-1} |A_i \cap A_j \cap A_n| + \ldots \\ + (-1)^{n-2} |A_1 \cap \cdots \cap A_{n-1} \cap A_n|.$$

Подставляя эти выражения в формулу для $|(A_1 \cup \cdots \cup A_{n-1}) \cup A_n|$ и группируя члены, получаем требуемое равенство. Доказательство завершено. *(Источник: Липский, гл. 1, §1.13, с. 73.)*

Комбинаторный смысл. Формулу можно интерпретировать следующим образом. Если элемент принадлежит ровно r из множеств A_i ($1 \le r \le n$), то в правой части первой формулы он учитывается

$$\binom{r}{1} - \binom{r}{2} + \binom{r}{3} - \cdots + (-1)^{r-1} \binom{r}{r} = 1$$

раз. Если же элемент не принадлежит ни одному из A_i, он не учитывается. Таким образом, каждый элемент объединения учитывается ровно один раз, а элементы вне объединения — ноль раз. Это даёт альтернативное доказательство. *(Источник: Липский, гл. 1, §1.13, с. 72.)*

Пример: число сюръективных функций. Пусть X — множество из n элементов, Y — множество из m элементов. Требуется найти число функций $f : X \to Y$, отображающих X на Y (сюръективных). Обозначим это число через $s_{n,m}$. Всего функций из X в Y ровно m^n. Для каждого $y \in Y$ пусть A_y — множество функций $f : X \to Y$, в образе которых не содержится y, то есть $y \notin f(X)$. Тогда функция не является сюръективной тогда и только тогда, когда она принадлежит хотя бы одному из множеств A_y. По формуле включений и исключений:

$$s_{n,m} = m^n - \left| \bigcup_{y \in Y} A_y \right|.$$

Для любого подмножества $S \subseteq Y$ мощности i пересечение $\bigcap_{y \in S} A_y$ состоит из функций, не принимающих значений из S, то есть функций из X в $Y \setminus S$. Число таких функций равно $(m-i)^n$. Количество подмножеств S мощности i равно $\binom{m}{i}$. Поэтому

$$\left| \bigcup_{y \in Y} A_y \right| = \sum_{i=1}^{m} (-1)^{i-1} \binom{m}{i} (m-i)^n.$$

Следовательно,

$$s_{n,m} = \sum_{i=0}^{m} (-1)^{i} \binom{m}{i} (m-i)^n.$$

В частности, при $m = n$ получаем число перестановок n-элементного множества, но эта формула даёт также число сюръективных функций для любых n, m. *(Источник: Липский, гл. 1, §1.13, с. 73–74.)*

Числа Стирлинга второго рода. Известно, что $s_{n,m} = m! S(n,m)$, где $S(n,m)$ — число Стирлинга второго рода (число разбиений n-элементного множества на m блоков). Поэтому

$$S(n,m) = \frac{1}{m!} \sum_{i=0}^{m} (-1)^{i} \binom{m}{i} (m-i)^n.$$

Это даёт явную формулу для чисел Стирлинга второго рода. Например, при $m = 2$:

$$S(n,2) = \frac{1}{2} \left(2^n - 2 \cdot 1^n\right) = 2^{n-1} - 1.$$

При $n = 4$: $S(4,2) = 7$, что совпадает с ранее найденным значением. *(Источник: Липский, гл. 1, §1.13, с. 74.)*

Пример: беспорядки (derangements). Беспорядком (или инверсией) на множестве $\{1, \ldots, n\}$ называется перестановка f, не имеющая неподвижных точек: $f(i) \neq i$ для всех i. Обозначим число беспорядков через D_n. Всего перестановок $n!$. Для каждого i пусть A_i — множество перестановок, оставляющих i на месте: $f(i) = i$. Тогда беспорядки — это перестановки, не принадлежащие ни одному из A_i. По формуле включений и исключений:

$$D_n = n! - \sum_{i=1}^{n} |A_i| + \sum_{1 \leq i < j \leq n} |A_i \cap A_j| - \cdots + (-1)^n |A_1 \cap \cdots \cap A_n|.$$

Множество A_i состоит из перестановок, фиксирующих i; таких перестановок $(n-1)!$. Пересечение $A_i \cap A_j$ состоит из перестановок, фиксирующих i и j; таких перестановок $(n-2)!$, и так далее. Пересечение k множеств имеет мощность $(n-k)!$. Количество способов выбрать k индексов равно $\binom{n}{k}$. Поэтому

$$D_n = \sum_{k=0}^{n} (-1)^k \binom{n}{k} (n-k)! = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}.$$

Сумма в скобках является частичной суммой ряда $e^{-1} = \sum_{k=0}^{\infty} (-1)^k / k!$. Поэтому $D_n \approx n!/e$, и доля беспорядков среди всех перестановок стремится к $e^{-1} \approx 0.36788$. Например, $D_4 = 9$, $D_5 = 44$. *(Источник: Липский, гл. 1, §1.13, с. 75–76.)*

Пример: подмножества множества с повторениями с ограничениями. Рассмотрим задачу о числе k-элементных подмножеств множества с повторениями $X = (k_1 * a_1, \ldots, k_n * a_n)$, где кратность элемента a_i не превосходит k_i. Эта задача эквивалентна подсчёту числа целочисленных решений системы

$$x_1 + \cdots + x_n = k, \qquad 0 \le x_i \le k_i \quad (i = 1, \ldots, n),$$

где x_i — кратность элемента a_i в подмножестве. Без ограничений $x_i \le k_i$ число решений равно $\binom{n+k-1}{k}$ (сочетания с повторениями). Чтобы учесть ограничения, применим метод включений и исключений. Пусть A_i — множество решений, в которых $x_i > k_i$. Тогда искомое число равно

$$N = \binom{n+k-1}{k} - \left| \bigcup_{i=1}^{n} A_i \right|.$$

Для подсчёта $|A_i|$ сделаем замену $x_i' = x_i - (k_i + 1) \ge 0$. Тогда уравнение принимает вид

$$x_1 + \cdots + x_{i-1} + x_i' + x_{i+1} + \cdots + x_n = k - (k_i + 1),$$

и число неотрицательных решений равно

$$\binom{n + (k - k_i - 1) - 1}{k - k_i - 1} = \binom{n + k - k_i - 2}{k - k_i - 1},$$

если $k > k_i$, и 0 в противном случае. Аналогично для пересечений $A_i \cap A_j$ делаем замены для двух переменных и получаем $\binom{n+k-k_i-k_j-3}{k-k_i-k_j-2}$ и т.д. Общая формула:

$$N = \sum_{S \subseteq \{1,\ldots,n\}} (-1)^{|S|} \binom{n + k - \sum_{i \in S}(k_i + 1) - 1}{k - \sum_{i \in S}(k_i + 1)},$$

где биномиальный коэффициент считается равным нулю, если верхний индекс меньше нижнего или нижний отрицателен. *(Источник: Липский, гл. 1, §1.13, с. 74–75.)*

Пример: число простых чисел. Хотя этот пример обычно приводится в учебниках по теории чисел, он хорошо иллюстрирует метод. Пусть $\pi(x)$ — количество простых чисел, не превосходящих x. Используя решето Эратосфена, можно выразить $\pi(n) - \pi(\sqrt{n})$ через включения-исключения. Если $p_1, \ldots, p_k$ — все простые числа, не превосходящие $\sqrt{n}$, то число чисел, не превосходящих n и не делящихся ни на одно из p_i, равно

$$n - \sum_i \left\lfloor \frac{n}{p_i} \right\rfloor + \sum_{i<j} \left\lfloor \frac{n}{p_i p_j} \right\rfloor - \cdots + (-1)^k \left\lfloor \frac{n}{p_1 \ldots p_k} \right\rfloor.$$

Это число равно $\pi(n) - \pi(\sqrt{n}) + 1$ (нужно учесть, что 1 не является простым, а сами простые p_i не делятся на другие p_j, поэтому они учитываются). В итоге получается формула

$$\pi(n) = \pi(\sqrt{n}) - 1 + \sum_{S \subseteq \{1,\ldots,k\}} (-1)^{|S|} \left\lfloor \frac{n}{\prod_{i \in S} p_i} \right\rfloor.$$

Эта формула имеет скорее теоретическое значение, так как число слагаемых растёт экспоненциально с k, но она демонстрирует универсальность метода. *(Источник: Липский, гл. 1, §1.13, задача 1.49, с. 83.)*

Вычислительные аспекты. Формула включений и исключений содержит 2^n слагаемых, поэтому при больших n прямое вычисление становится неэффективным. Однако во многих задачах структура пересечений позволяет сгруппировать слагаемые и получить более компактные выражения (как в примерах с сюръективными функциями и беспорядками). Кроме того, часто достаточно вычислить несколько первых членов, чтобы получить хорошее приближение, поскольку знаки чередуются и вклад последующих членов быстро убывает.

Итог пункта. В п. 1.5.4 сформулирован и доказан принцип включений и исключений. Приведены его комбинаторная интерпретация и основные примеры применения: подсчёт числа сюръективных функций, чисел Стирлинга второго рода, беспорядков, подмножеств множества с повторениями с ограничениями, а также числа простых чисел. Показано, что метод даёт явные формулы, удобные для вычислений и анализа, и что он тесно связан с другими комбинаторными понятиями (числами Стирлинга, числами Белла, производящими функциями). *(Обобщение по: Липский, гл. 1, §1.13, с. 72–76; задачи 1.49–1.50, с. 83.)*

1.5.5 Краткое резюме

Комбинаторный анализ изучает способы подсчёта числа объектов, удовлетворяющих заданным ограничениям. Основные понятия — размещения, перестановки, сочетания (с повторениями и без) — выражаются через факториалы и биномиальные коэффициенты. Разбиения множеств приводят к числам Стирлинга и Белла, разбиения чисел — к функции $P(n)$. Метод производящих функций позволяет сводить комбинаторные задачи к алгебраическим операциям над формальными рядами и получать явные формулы (числа Фибоначчи, Каталана и др.). Метод включений и исключений даёт универсальный способ подсчёта мощности объединения множеств и применяется к задачам о сюръективных функциях, беспорядках и ограниченных композициях. *(Обобщение по: Липский, гл. 1, §1.1–1.13, с. 9–76.)*
Источники к пункту 1.4:

- Липский В. Комбинаторика для программистов. М.: Мир, 1988. Гл. 1, §1.1–1.13, с. 9–76.

1.6 Коды с исправлением ошибок. Алфавитное кодирование. Методы сжатия информации

Основной источник: Прокопенко Н.Ю. Теория кодирования: учеб. пособие / Н.Ю. Прокопенко; Нижегор. гос. архитектур.-строит. ун-т. – Н. Новгород: ННГАСУ, 2023. – 94 с.

1.6.1 Алфавитное кодирование и взаимная однозначность

Основные источники: Прокопенко Н.Ю. Теория кодирования: учеб. пособие. Н. Новгород: ННГАСУ, 2023. Разд. 2, с. 17–22; разд. 3, с. 23–28.

Алфавиты, слова и кодирование. Пусть задано множество $A = \{a_1, \ldots, a_n\}$, которое называется *алфавитом сообщений*, и множество $B = \{b_1, \ldots, b_m\}$, называемое *кодирующим алфавитом*. Элементы алфавита называются *буквами*. Конечная

последовательность букв алфавита A называется *словом* в алфавите A. Множество всех слов в алфавите A обозначается A^*, множество непустых слов — A^+. Через λ обозначается пустое слово:

$$A^* = A^+ \cup \{\lambda\}.$$

Длиной слова называется число букв в нём. Если слово α имеет длину k, пишут $|\alpha| = k$. Для пустого слова $|\lambda| = 0$. Соединение слов α и β обозначается $\alpha\beta$; через α^n обозначается соединение n одинаковых слов, причём $\alpha^0 = \lambda$. *(Источник: Прокопенко, разд. 2, с. 17.)*

Кодированием называют отображение

$$F\colon S \to B^*,$$

где $S \subseteq A^*$ — некоторое множество сообщений. Слово $b = F(a)$ называется *кодом* слова a. Обратное отображение F^{-1} (если оно существует) называется *декодированием*. Кодирование F называется *взаимно однозначным* (или *однозначно декодируемым*), если каждый код сообщения является кодом ровно одного сообщения. В этом случае код называется *свободным*. *(Источник: Прокопенко, разд. 2, с. 17–18.)*

Если все кодовые слова имеют одинаковую длину, код называется *равномерным* (или *блочным*). В противном случае код называется *неравномерным*. Например, код $e_4(n)$, сопоставляющий числам $0, 1, 2, \ldots$ их четырёхразрядные двоичные записи, является равномерным; код $e(n)$, в котором числа записываются двоичными словами без фиксированной длины, равномерным не является. *(Источник: Прокопенко, разд. 2, с. 17.)*

Алфавитное (побуквенное) кодирование. Пусть задано отображение Σ букв алфавита A в множество B^*:

$$\Sigma: \quad \begin{cases} a_1 \to B_1, \\ a_2 \to B_2, \\ \ldots \\ a_r \to B_r, \end{cases}$$

где $B_i \in B^*$. *Алфавитным кодированием*, задаваемым схемой Σ, называется кодирование F_Σ, удовлетворяющее двум условиям:

1) $F_\Sigma(a_i) = B_i$, $i = 1, \ldots, r$;

2) $F_\Sigma(a_{i_1} a_{i_2} \ldots a_{i_k}) = B_{i_1} B_{i_2} \ldots B_{i_k}$.

Иными словами, код слова получается последовательным приписыванием кодов его букв. Слова B_i называются *элементарными кодами*. Множество

$$C(\Sigma) = \{B_1, \ldots, B_r\}$$

называется *кодом алфавита* в схеме Σ. *(Источник: Прокопенко, разд. 2, с. 19.)*

Длиной кода называется число знаков в коде без учёта пробелов. *Спектром кода* называется упорядоченная по невозрастанию последовательность длин кодовых слов:

$$L(V) = \langle l_1, l_2, \ldots, l_r \rangle, \qquad l_1 \geq l_2 \geq \cdots \geq l_r.$$

Код называется *разделимым*, если любое слово, составленное из элементарных кодов, единственным образом разлагается на элементарные коды. Иными словами, из равенства

$$B_{i_1} B_{i_2} \dots B_{i_k} = B_{j_1} B_{j_2} \dots B_{j_l}$$

должно следовать $k = l$ и $i_t = j_t$ для всех $t = 1, \dots, k$. *(Источник: Прокопенко, разд. 2, с. 19.)*

Алфавитное кодирование является взаимно однозначным тогда и только тогда, когда оно задаётся с помощью разделимого кода. Из определения разделимого кода следует, что все слова разделимого кода различны и не пусты. *(Источник: Прокопенко, разд. 2, с. 19–20.)*

Примеры.

1) Пусть $A = \{a, b, c, d\}$, $B = \{0, 1\}$, а схема кодирования имеет вид

$$a \to 01, \qquad b \to 100, \qquad c \to 101, \qquad d \to 0.$$

Это кодирование не является взаимно однозначным, а значит, код не является свободным. Например, слово 0100 можно декодировать как *db* или как *add*; слово 0010100 допускает декодирования *ddcdd*, *daadd*, *dadb*.

2) Азбука Морзе не является свободным кодом без дополнительных пауз между буквами и словами. Поэтому при передаче сообщений с помощью азбуки Морзе используют пробелы (паузы), которые неформально отделяют буквы друг от друга.

(Источник: Прокопенко, разд. 2, с. 18.)

Неравенство Макмиллана. Для того чтобы схема алфавитного кодирования была разделимой в q-буквенном кодирующем алфавите, необходимо, чтобы длины элементарных кодов удовлетворяли неравенству Макмиллана:

$$\frac{1}{q^{|B_1|}} + \frac{1}{q^{|B_2|}} + \dots + \frac{1}{q^{|B_r|}} \le 1.$$

В частности, для двоичного кодирующего алфавита $B = \{0, 1\}$:

$$\frac{1}{2^{|B_1|}} + \frac{1}{2^{|B_2|}} + \dots + \frac{1}{2^{|B_r|}} \le 1.$$

(Источник: Прокопенко, разд. 2, с. 21.)

Неравенство Макмиллана является *необходимым* условием разделимости, но не является достаточным. Например, схема

$$a \to 0, \qquad b \to 01$$

является разделимой, хотя и не является префиксной: $1/2 + 1/4 = 3/4 \le 1$. В то же время азбука Морзе не удовлетворяет неравенству Макмиллана, поэтому без дополнительных пауз она не является разделимой. *(Источник: Прокопенко, разд. 2, с. 21–22.)*

Кроме того, справедлива теорема: если для чисел $l_1, \dots, l_n$ выполнено неравенство Макмиллана

$$\frac{1}{q^{l_1}} + \frac{1}{q^{l_2}} + \dots + \frac{1}{q^{l_n}} \le 1,$$

то существует префиксный код $V = \{v_1, \dots, v_n\}$ в алфавите $B = \{b_1, \dots, b_m\}$ со спектром длин $L(V) = \langle l_1, \dots, l_n \rangle$. *(Источник: Прокопенко, разд. 4.2, с. 33–34.)*

Проблема взаимной однозначности. Одним из основных вопросов теории кодирования является проблема взаимной однозначности: по коду β сообщения α требуется однозначно восстановить α. Если решать эту задачу исходя непосредственно из определения, то нужно перебрать бесконечное число слов, так как для каждого кода нужно установить, допускает он однозначное декодирование или нет. Для эффективного решения этой задачи используется алгоритм Маркова, основанный на понятии нетривиального разложения элементарных кодов. *(Источник: Прокопенко, разд. 2, с. 20; разд. 3, с. 23.)*

Нетривиальные разложения. Пусть $C(\Sigma)$ — алфавитный код. Обозначим через P_Σ множество слов β', каждое из которых является *собственным префиксом* некоторого кодового слова B_i. Через S_Σ обозначим множество слов β'', каждое из которых является *собственным суффиксом* некоторого кодового слова B_i.

Определение. Представление элементарного кода B_i в виде

$$B_i = \beta'\omega\beta'',$$

где слово β' не может оканчиваться на элементарный код, а слово β'' не может начинаться с элементарного кода, называется *нетривиальным разложением* элементарного кода. При этом одно из слов β', ω, β'' может быть пустым. Для наглядности слова в разложении отделяют точками:

$$B_i = \beta'.\omega.\beta''.$$

(Источник: Прокопенко, разд. 3, с. 23.)

Теорема Маркова. Алфавитное кодирование со схемой Σ не обладает свойством взаимной однозначности тогда и только тогда, когда граф G_Σ содержит ориентированный цикл (контур), проходящий через вершину λ. Иными словами, алфавитное кодирование является взаимно однозначным (код $C(\Sigma)$ свободен) тогда и только тогда, когда граф G_Σ не содержит ориентированного цикла, проходящего через вершину λ. *(Источник: Прокопенко, разд. 3, с. 23.)*

Если схема Σ не обладает свойством взаимной однозначности, то слово из B^*, допускающее два декодирования, легко находится по графу G_Σ. Для этого нужно рассмотреть ориентированный цикл, проходящий через вершину λ, начиная с λ, и выписать последовательно все слова, приписанные рёбрам и вершинам, входящим в этот цикл. *(Источник: Прокопенко, разд. 3, с. 23.)*

Алгоритм построения графа G_Σ. Пусть $V = P_\Sigma \cap S_\Sigma \cup \{\lambda\}$ — множество вершин графа. Вершины β' и β'' соединяются ориентированным ребром, если для некоторого элементарного кода B_i существует нетривиальное разложение

$$B_i = \beta'.\omega.\beta'',$$

где $\beta' \in P_\Sigma$, $\beta'' \in S_\Sigma$, $\omega \in C^*(\Sigma)$. Ребру приписывается значение ω. Множество всех рёбер обозначается R, множество значений рёбер — $m(R)$. Полученный взвешенный граф обозначается G_Σ.

Алгоритм построения графа:

1) Для каждого элементарного кода выписать все нетривиальные разложения $B_i = \beta'.\omega.\beta''$.

2) Выписать множество P_Σ, состоящее из слов β', которые входят в качестве начал в нетривиальные разложения элементарных кодов.

3) Выписать множество S_Σ, состоящее из всех слов β'', которые являются окончаниями нетривиальных разложений элементарных кодов.

4) Составить множество $V = P_\Sigma \cap S_\Sigma \cup \{\lambda\}$.

5) Выписать все разложения элементарных кодов, связанных с множеством V, то есть все разложения вида
$$B_i = \beta'\beta_{i_1} \dots \beta_{i_k}\beta'',$$
где $\beta', \beta'' \in V$, а k может быть равно 0. Если $k = 0$, то $\beta_{i_1} \dots \beta_{i_k} = \lambda$.

6) По разложениям, полученным в п. 5, построить ориентированный граф G_Σ: вершины графа отождествляются с элементами множества V; пара вершин β' и β'' соединяется ориентированным ребром в том и только в том случае, если существует разложение $B_i = \beta'\beta_{i_1} \dots \beta_{i_k}\beta''$. При этом ребру (β', β'') приписывается слово $\beta_{i_1} \dots \beta_{i_k}$.

7) По полученному графу G_Σ проверить, обладает ли исходная схема кодирования свойством взаимной однозначности, используя теорему Маркова.

(Источник: Прокопенко, разд. 3, с. 23–24.)

Пример 5. Пусть $A = \{a_1, a_2, a_3\}$, $B = \{0, 1\}$, а код задан схемой
$$\Sigma: \quad \begin{cases} a_1 \to 1, \\ a_2 \to 010, \\ a_3 \to 101. \end{cases}$$

Выпишем нетривиальные разложения элементарных кодов:
$$B_1 = 1$$
не имеет нетривиальных разложений;
$$B_2 = 010 = 0.B_1.0 = 01.\lambda.0 = 0.\lambda.10;$$
$$B_3 = 101 = \lambda.B_1.01 = 10.B_1.\lambda = 1.\lambda.01 = 10.\lambda.1.$$
Тогда
$$P_\Sigma = \{0, 01, \lambda, 10, 1\}, \qquad S_\Sigma = \{0, 01, \lambda, 10, 1\},$$
$$V = P_\Sigma \cap S_\Sigma \cup \{\lambda\} = \{0, 1, 01, \lambda, 10\}.$$
Соответствующий граф G_Σ содержит ориентированный цикл, проходящий через вершину λ. Поэтому код не является свободным. По графу можно построить слово, допускающее два декодирования, например слово
$$1010101.$$
Оно допускает две расшифровки:
$$101\,010\,1 = a_3a_2a_1, \qquad 1\,010\,101 = a_1a_2a_3.$$

(Источник: Прокопенко, разд. 3, с. 25–26.)

Пример 6. Пусть $A = \{a_1, a_2, a_3, a_4, a_5\}$, $B = \{0, 1\}$, а код задан схемой

$$\Sigma: \quad \begin{cases} a_1 \to 1, \\ a_2 \to 01, \\ a_3 \to 100, \\ a_4 \to 0100, \\ a_5 \to 0000. \end{cases}$$

Выпишем нетривиальные разложения:

$$B_1 = 1$$

не имеет нетривиальных разложений;

$$B_2 = 01 = 0.B_1.\lambda = 0.\lambda.1;$$

$$B_3 = 100 = \lambda.B_1.00 = 1.\lambda.00 = 10.\lambda.0;$$

$$B_4 = 0100 = 0.B_1.00 = \lambda.B_2.00 = 0.B_3.\lambda = 010.\lambda.0;$$

$$B_5 = 0000 = 0.\lambda.000 = 00.\lambda.00 = 000.\lambda.0.$$

Тогда

$$P_\Sigma = \{0, 1, 01, \lambda, 10, 100, 00, 000\},$$

$$S_\Sigma = \{0, 1, 00, 000, \lambda\},$$

$$V = P_\Sigma \cap S_\Sigma \cup \{\lambda\} = \{0, 1, \lambda, 00, 000\}.$$

Соответствующий граф не содержит ориентированного цикла, проходящего через вершину λ. Следовательно, код $C(\Sigma)$ является взаимно однозначным. *(Источник: Прокопенко, разд. 3, с. 26–27.)*

Пример 7. Пусть $A = \{a_1, a_2, a_3, a_4, a_5\}$, $B = \{b_1, b_2, b_3\}$, а схема кодирования имеет вид

$$\Sigma: \quad \begin{cases} a_1 \to b_1 b_2, \\ a_2 \to b_1 b_3 b_2, \\ a_3 \to b_2 b_3, \\ a_4 \to b_1 b_2 b_1 b_3, \\ a_5 \to b_2 b_1 b_2 b_2 b_3. \end{cases}$$

Сравнивая префиксы и суффиксы кодовых слов, получаем

$$P_\Sigma \cap S_\Sigma \cup \{\lambda\} = \{b_1 b_3, b_2, \lambda\}.$$

Это и есть множество вершин V графа G_Σ. Элементарные коды B_1 и B_3 не имеют нетривиальных разложений. Далее:

$$B_2 = b_1 b_3 b_2 = b_1 b_3.\lambda.b_2,$$

$$B_4 = b_1 b_2 b_1 b_3 = \lambda.B_1.b_1 b_3,$$

$$B_5 = b_2 b_1 b_2 b_2 b_3 = b_2.B_1 B_3.\lambda.$$

Граф содержит ориентированный цикл, проходящий через вершину λ:

$$\lambda \to b_1 b_3 \to b_2 \to \lambda.$$

Выписывая слова, приписанные вершинам и рёбрам этого цикла, получаем слово

$$b_1 b_2 b_1 b_3 b_2 b_1 b_2 b_2 b_3,$$

которое имеет два разложения по элементарным кодам:

$$B_1 B_2 B_1 B_3 = B_4 B_5.$$

Иными словами, слова

$$A_1 = a_4 a_5 \qquad \text{и} \qquad A_2 = a_1 a_2 a_1 a_3$$

кодируются одним и тем же словом. Следовательно, код $C(\Sigma)$ не является свободным. *(Источник: Прокопенко, разд. 3, с. 27–28.)*

Итог. Алфавитное кодирование сопоставляет каждой букве исходного алфавита элементарный код, а слову — конкатенацию кодов его букв. Для однозначной декодируемости код должен быть разделимым; необходимым условием является неравенство Макмиллана. Проверка взаимной однозначности в общем случае сводится к построению графа G_Σ по нетривиальным разложениям элементарных кодов и применению теоремы Маркова: код не является свободным тогда и только тогда, когда в графе есть ориентированный цикл, проходящий через вершину λ. Примеры 5–7 показывают, как по графу устанавливается неоднозначность или однозначность кода и как строится слово, допускающее два декодирования. *(Обобщение по: Прокопенко, разд. 2, с. 17–22; разд. 3, с. 23–28.)*

1.6.2 Префиксные коды и их построение

Основные источники: Прокопенко Н.Ю. Теория кодирования: учеб. пособие. Н. Новгород: ННГАСУ, 2023. Разд. 4, с. 30–38.

Определение префиксного кода. Пусть задано конечное множество $A = \{a_1, \ldots, a_n\}$, называемое алфавитом. Последовательность букв называется словом в данном алфавите. Если $\alpha = \alpha_1 \alpha_2$, то α_1 называется *началом* (или *префиксом*) слова α, а α_2 — *окончанием* (или *постфиксом*) слова α. Если при этом $\alpha_1 \neq \lambda$ (соответственно $\alpha_2 \neq \lambda$), то α_1 (соответственно α_2) называется *собственным началом* (соответственно *собственным окончанием*) слова α. Иными словами, префикс (суффикс) слова называется собственным, если он отличен от пустого слова и от самого слова.

Обозначим через P_α множество всех собственных префиксов слова α, а через S_α — множество всех собственных суффиксов. Например, для $\alpha = 110$ имеем

$$P_\alpha = \{1, 11\}, \qquad S_\alpha = \{0, 10\}.$$

Определение. Схема Σ (код $C(\Sigma)$, кодирование F_Σ) обладает *свойством префикса*, если для любых двух различных кодовых слов B_i и B_j $(i \neq j)$ слово B_i не является префиксом слова B_j. Код $C(\Sigma)$, обладающий свойством префикса, называется *префиксным кодом. (Источник: Прокопенко, разд. 4, с. 30.)*

Формально: для любых $i \neq j$ и любого слова $\beta \in B^*$ выполнено $B_i \neq B_j \beta$. Иными словами, никакое кодовое слово не является началом другого кодового слова.

Связь префиксности и разделимости. Напомним, что код называется *разделимым*, если любое слово, составленное из элементарных кодов, единственным образом разлагается на элементарные коды. Справедлива следующая теорема.

Теорема. Префиксная схема является разделимой. *(Источник: Прокопенко, разд. 4, с. 31.)*

Доказательство. Предположим противное: пусть кодирование со схемой Σ не является разделимым. Тогда существует слово $\beta \in F_\Sigma(A^*)$, допускающее два различных разложения на элементарные коды:

$$\beta = \beta_{i_1} \dots \beta_{i_k} = \beta_{j_1} \dots \beta_{j_l},$$

причём наборы индексов $(i_1, \dots, i_k)$ и $(j_1, \dots, j_l)$ различны. Рассмотрим первое место, где эти разложения отличаются. Пусть это будут кодовые слова β_{i_t} и β_{j_t}. Одно из них является префиксом другого, что противоречит свойству префикса. Следовательно, предположение неверно, и префиксная схема разделима. $\square$

Замечание. Свойство быть префиксной является *достаточным*, но не *необходимым* условием разделимости. Например, код

$$a \to 0, \qquad b \to 01$$

не является префиксным, так как код буквы a является префиксом кода буквы b. Однако этот код разделим: если после символа 0 следует 0, то первый символ кодирует букву a, так как 00 не является началом никакого кодового слова; если после 0 следует 1, то пара 01 является кодом буквы b. Таким образом, существуют разделимые, но не префиксные коды. *(Источник: Прокопенко, разд. 4, с. 31.)*

Представление префиксного кода с помощью дерева. Пусть задан префиксный код $V = \{v_1, \dots, v_n\}$ в двоичном алфавите $B = \{0, 1\}$. Элементарные коды определяют *бинарное кодовое дерево*. Из каждой вершины дерева выходит не более двух рёбер в вершины следующего яруса; левое ребро помечается символом 0, правое — символом 1. Элементарным кодам соответствуют вершины дерева, определяемые путём, идущим от корня. Если код префиксный, то элементарные коды расположены в листьях дерева.

Дерево называется *насыщенным*, если из каждой вершины, не являющейся листом, в следующий ярус выходит ровно два ребра. *(Источник: Прокопенко, разд. 4.1, с. 32.)*

Для префиксных кодов и только для них множество кодовых слов совпадает с множеством концевых вершин (листьев) кодового дерева. Это свойство часто используется при построении и анализе префиксных кодов.

Пример 1. Пусть код задан таблицей:

$$\begin{aligned} b_1 &\to 00, \\ b_2 &\to 111, \\ b_3 &\to 100, \\ b_4 &\to 01, \\ b_5 &\to 101, \\ b_6 &\to 1100, \\ b_7 &\to 1101. \end{aligned}$$

Соответствующее кодовое дерево имеет вид: из корня выходят две ветви (0 и 1). По ветви 0: из вершины 0 идут ветви 0 и 1, приводящие к $b_1 = 00$ и $b_4 = 01$. По ветви 1: из вершины 1 идёт ветвь 0 к $b_3 = 100$ и $b_5 = 101$; ветвь 1 ведёт к вершине 11, из которой идут ветви 0 (к $b_6 = 1100$ и $b_7 = 1101$) и 1 (к $b_2 = 111$). Все кодовые слова расположены в листьях, дерево не является насыщенным, так как, например, из вершины 10 не выходят два ребра (она является листом для b_3 и b_5? На самом деле в этом примере из вершины 10 выходят рёбра к b_3 и b_5, так что она не лист; но есть другие вершины, например, из вершины 0 выходят два ребра, из вершины 1 — два, из вершины 11 — два, из вершины 110 — два; все внутренние вершины имеют по два потомка, значит, дерево насыщенное). В данном примере дерево является насыщенным. *(Источник: Прокопенко, разд. 4.1, с. 32.)*

Построение префиксного кода по набору длин. Пусть задан префиксный код $V = \{v_1, \ldots, v_n\}$ в алфавите $B = \{b_1, \ldots, b_m\}$. Пусть $|v_i| = l_i$ — длины кодовых слов. Упорядоченное множество

$$L(V) = \langle l_1, \ldots, l_n \rangle$$

называется *спектром длин* кода V. Коды v_1 и v_2 называются *спектрально эквивалентными*, если их спектры длин совпадают.

Теорема. Если для чисел $l_1, \ldots, l_n$ выполнено неравенство Макмиллана

$$\frac{1}{q^{l_1}} + \frac{1}{q^{l_2}} + \cdots + \frac{1}{q^{l_n}} \le 1,$$

то существует префиксный код $V = \{v_1, \ldots, v_n\}$ в алфавите $B = \{b_1, \ldots, b_m\}$ со спектром длин $L(V) = \langle l_1, \ldots, l_n \rangle$. *(Источник: Прокопенко, разд. 4.2, с. 33–34.)*

Доказательство. Будем полагать, что $l_1 \le l_2 \le \cdots \le l_n$. Разобьём множество чисел на k непересекающихся классов так, что в каждом классе стоят одинаковые числа. Пусть L_i — i-й класс, s_i — длина кодовых слов в этом классе, $|L_i|$ — число элементов в классе. Тогда неравенство Макмиллана можно переписать в виде

$$\frac{|L_1|}{q^{s_1}} + \frac{|L_2|}{q^{s_2}} + \cdots + \frac{|L_k|}{q^{s_k}} \le 1.$$

Это неравенство равносильно следующему:

$$|L_i| \le q^{s_i} - |L_1| q^{s_i - s_1} - |L_2| q^{s_i - s_2} - \cdots - |L_{i-1}| q^{s_i - s_{i-1}}$$

для каждого $i = 1, \ldots, k$.

Построение префиксного кода (алгоритм 1):

1) Из множества B^{s_1} выберем произвольные $|L_1|$ слов. Обозначим их через $V_1 \subseteq B^{s_1}$. Возможность выбора следует из неравенства для $i = 1$.

2) Из множества B^{s_2} выберем произвольные $|L_2|$ слов так, чтобы ни одно из слов V_2 не являлось бы префиксом выбранных ранее. Обозначим их через V_2. Выбор таких слов возможен в силу выполнимости неравенства для $i = 2$.

3) Пусть построены множества $V_1, V_2, \ldots, V_{i-1}$. На i-м шаге выберем произвольные $|L_i|$ слов из множества

$$B^{s_i} \big/ \big((V_1 \times B^{s_i - s_1}) \cup (V_2 \times B^{s_i - s_2}) \cup \cdots \cup (V_{i-1} \times B^{s_i - s_{i-1}}) \big).$$

Это множество обозначим через V_i.

4) Через k шагов алгоритм заканчивается. Код $V = \bigcup_{i=1}^{k} V_i$ является префиксным по построению.

(Источник: Прокопенко, разд. 4.2, с. 34–35.)

Пример 3. Пусть $L(V) = \langle 1, 3, 3, 4, 4, 4, 4 \rangle$. Проверим неравенство Макмиллана:

$$\frac{1}{2} + \frac{1}{2^3} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^4} + \frac{1}{2^4} + \frac{1}{2^4} = \frac{1}{2} + \frac{2}{8} + \frac{4}{16} = 1 \leq 1.$$

Неравенство выполнено, следовательно, префиксный код с таким спектром существует.

Разобьём спектр на классы: $L_1 = \{1\}$, $L_2 = \{3, 3\}$, $L_3 = \{4, 4, 4, 4\}$. Имеем $s_1 = 1$, $s_2 = 3$, $s_3 = 4$, $|L_1| = 1$, $|L_2| = 2$, $|L_3| = 4$.

Шаг 1. Выбираем $V_1 \subseteq \{0, 1\}$, $|V_1| = 1$. Пусть $V_1 = \{1\}$.

Шаг 2. Выбираем $V_2 \subseteq \{0, 1\}^3$, $|V_2| = 2$, так, чтобы слова не начинались с 1. Пусть $V_2 = \{000, 001\}$.

Шаг 3. Выбираем $V_3 \subseteq \{0, 1\}^4$, $|V_3| = 4$, так, чтобы слова не начинались с 1, а также не начинались с 000 или 001. Пусть $V_3 = \{0100, 0101, 0110, 0111\}$.

Искомый код:

$$V = V_1 \cup V_2 \cup V_3 = \{1,\ 000,\ 001,\ 0100,\ 0101,\ 0110,\ 0111\}.$$

Он является префиксным по построению. Заметим, что можно было построить и другой код с тем же спектром длин; все такие коды спектрально эквивалентны. *(Источник: Прокопенко, разд. 4.2, с. 35.)*

Пример 4. Для спектра $L(V) = \langle 1, 1, 3 \rangle$ построим префиксный код в различных алфавитах.

1) В алфавите $B = \{0, 1\}$ неравенство Макмиллана не выполняется:

$$\frac{1}{2} + \frac{1}{2} + \frac{1}{2^3} = \frac{1}{2} + \frac{1}{2} + \frac{1}{8} > 1.$$

Следовательно, префиксного кода не существует.

2) В алфавите $B = \{0, 1, 2\}$ неравенство Макмиллана выполнено:

$$\frac{1}{3} + \frac{1}{3} + \frac{1}{3^3} = \frac{19}{27} \leq 1.$$

Значит, префиксный код существует. Например:

$$\Sigma = \{B_1 = 1,\ B_2 = 2,\ B_3 = 000\}$$

или

$$\Sigma = \{B_1 = 0,\ B_2 = 2,\ B_3 = 110\}.$$

Построение по алгоритму 1: $V_1 = \{0, 2\}$ (два слова длины 1), $V_2 = \{110\}$ (одно слово длины 3, не начинающееся с 0 или 2). Искомый код $V = V_1 \cup V_2 = \{0, 2, 110\}$.

3) В алфавите $B = \{0, 1, 2, 3\}$ неравенство Макмиллана также выполнено:

$$\frac{1}{4} + \frac{1}{4} + \frac{1}{4^3} = \frac{33}{64} \leq 1.$$

Например, $\Sigma = \{B_1 = 0,\ B_2 = 2,\ B_3 = 100\}$ или $\Sigma = \{B_1 = 2,\ B_2 = 3,\ B_3 = 000\}$.

(Источник: Прокопенко, разд. 4.2, с. 36.)

Алгоритм Шеннона построения префиксного кода. Пусть задан набор длин $l_1, \ldots, l_n$, удовлетворяющих неравенству Макмиллана. В силу доказанной теоремы существует префиксный код с таким спектром длин. Алгоритм Шеннона (алгоритм 2) строит этот код конструктивно.

Построим последовательность чисел $q_1, q_2, \ldots, q_n$ по правилам:

$$q_1 = 0, \qquad q_{i+1} = q_i + 2^{-l_i} \quad (i = 1, \ldots, n-1).$$

Очевидно, $0 \leq q_i < 1$ и каждое q_i имеет единственное представление в виде двоичной дроби с l_i знаками после запятой:

$$q_i = 0, c_1^{(i)} c_2^{(i)} \ldots c_{l_i}^{(i)}.$$

Кодовое слово v_i определяется как последовательность этих l_i цифр:

$$v_i = c_1^{(i)} c_2^{(i)} \ldots c_{l_i}^{(i)}.$$

Так как набор длин упорядочен по неубыванию, при $h > i$ выполняются неравенства $l_h \geq l_i$ и $q_h \geq q_i + 2^{-l_i}$. Поэтому кодовое слово v_h отличается от v_i в первых l_i разрядах. Следовательно, v_i не является префиксом v_h, и построенный код является префиксным. *(Источник: Прокопенко, разд. 4.2, с. 37.)*

Пример 5. Пусть $L(V) = \langle 1, 3, 3, 4, 4, 4, 4 \rangle$. Неравенство Макмиллана проверено в примере 3. Построим последовательность $q_1, \ldots, q_7$:

$$\begin{aligned}
q_1 &= 0{,}0, \\
q_2 &= 0 + 2^{-1} = 0{,}100, \\
q_3 &= 0{,}100 + 2^{-3} = 0{,}100 + 0{,}001 = 0{,}101, \\
q_4 &= 0{,}101 + 2^{-3} = 0{,}101 + 0{,}001 = 0{,}1100, \\
q_5 &= 0{,}1100 + 2^{-4} = 0{,}1100 + 0{,}0001 = 0{,}1101, \\
q_6 &= 0{,}1101 + 2^{-4} = 0{,}1101 + 0{,}0001 = 0{,}1110, \\
q_7 &= 0{,}1110 + 2^{-4} = 0{,}1110 + 0{,}0001 = 0{,}1111.
\end{aligned}$$

Выбирая в качестве B_i дробную часть q_i, получаем код

$$\Sigma = \{B_1 = 0,\ B_2 = 100,\ B_3 = 101,\ B_4 = 1100,\ B_5 = 1101,\ B_6 = 1110,\ B_7 = 1111\}.$$

Этот код является префиксным и спектрально эквивалентен коду, построенному в примере 3 (там был код $\{1, 000, 001, 0100, 0101, 0110, 0111\}$). *(Источник: Прокопенко, разд. 4.2, с. 38.)*

Замечание о выборе алгоритма. Оба алгоритма (алгоритм 1 и алгоритм Шеннона) позволяют построить префиксный код по заданному спектру длин. Алгоритм 1 действует последовательно по классам длин и более нагляден; алгоритм Шеннона использует двоичные дроби и удобен для программирования. Результаты работы алгоритмов могут быть различными, но все построенные коды спектрально эквивалентны, то есть имеют одинаковый набор длин кодовых слов. *(Источник: Прокопенко, разд. 4.2, с. 34–38.)*

Итог. Префиксный код — это код, в котором ни одно кодовое слово не является префиксом другого. Префиксные коды являются разделимыми, и их удобно представлять с помощью кодового дерева, где кодовые слова соответствуют листьям. Основная задача — построение префиксного кода по заданному набору длин. Необходимым условием существования такого кода является неравенство Макмиллана; для префиксных кодов оно оказывается и достаточным. Алгоритм 1 строит код последовательным выбором слов из множеств B^{s_i}, исключая уже занятые префиксы. Алгоритм Шеннона использует двоичные дроби q_i и выбирает первые l_i цифр их двоичного разложения. Оба алгоритма дают префиксный код с заданным спектром длин, что подтверждается примерами. *(Обобщение по: Прокопенко, разд. 4, с. 30–38.)*

1.6.3 Оптимальное кодирование (сжатие информации)

Основные источники: Прокопенко Н.Ю. Теория кодирования: учеб. пособие. Н. Новгород: ННГАСУ, 2023. Разд. 5, с. 39–57.

Постановка задачи сжатия. Вопросы сжатия информации играют важную роль в информатике, что связано с развитием вычислительной техники и средств связи, а также с необходимостью хранения и передачи больших объёмов информации. Одно из направлений теории кодирования, в котором изучаются вопросы сжатия, называется *экономным кодированием*.

Определение. Методы кодирования, которые позволяют построить (без потери информации) коды сообщений, имеющие меньшую длину по сравнению с исходным сообщением, называются *методами сжатия* (или *упаковки*) информации. *(Источник: Прокопенко, разд. 5, с. 39.)*

Качество сжатия обычно определяется *коэффициентом сжатия*, который измеряется в процентах и показывает, насколько сжатое сообщение короче исходного. Например, коэффициент сжатия 0,6 означает, что сжатые данные занимают 60% от исходного размера. Значения, большие 1, говорят о том, что выходной файл больше входного (отрицательное сжатие). Также используется коэффициент сжатия в *bpb* (bit per bit) — число бит сжатого файла на один бит входного. Величина $100 \cdot (1 - k)$, где k — коэффициент сжатия, также отражает качество сжатия: значение 60 означает, что файл занимает на 60% меньше места. *(Источник: Прокопенко, разд. 5, с. 39–40.)*

Возможности сжатия определяются вероятностными и структурными (синтаксическими) свойствами информации. С помощью вероятностей моделируются частоты появления различных букв в сообщениях, частоты фрагментов, частоты самих сообщений. Дополнительные возможности появляются, когда рассматриваются не любые последовательности символов, а только некоторые (например, в текстах русского

языка не встречаются фрагменты «ггг», «аь» и т.п.). Начало математическому исследованию экономного кодирования положила работа К. Шеннона «Математическая теория связи» (1948). Вопросы сжатия с учётом структурных свойств изучал А.А. Марков. *(Источник: Прокопенко, разд. 5, с. 40.)*

Стоимость кодирования. Пусть на буквах алфавита $A = \{a_1, \ldots, a_n\}$ задано распределение вероятностей $P = \{p_1, \ldots, p_n\}$, $p_i \geq 0$, $\sum_{i=1}^{n} p_i = 1$, и пусть $V = \{v_1, \ldots, v_n\}$ — свободный код. *Стоимостью кодирования* f называется величина

$$C_f(P) = \sum_{i=1}^{n} p_i |v_i|,$$

где $|v_i|$ — длина элементарного кода буквы a_i. Стоимость показывает, во сколько раз в среднем увеличивается длина кодируемого сообщения, то есть число букв, приходящееся на одну кодируемую букву. *(Источник: Прокопенко, разд. 5, с. 41.)*

Требование экономичности заключается в минимизации средней длины кодового слова. Положим

$$C^*(P) = \min_f C_f(P).$$

Код V^* со схемой f^*, для которой $C_{f^*}(P) = C^*(P)$, называется *оптимальным* для набора вероятностей P. Оптимальные коды дают в среднем минимальное увеличение длин слов.

Свойства оптимальных кодов. Существует несколько способов получения кодов с малой избыточностью, причём все они обладают следующими свойствами:

1) Более вероятным буквам источника соответствуют более короткие кодовые слова (принцип статистического кодирования).

2) Никакое кодовое слово не является началом другого, более длинного.

3) Все буквы алфавита, используемого для передачи по каналу, приблизительно равновероятны.

4) Символы в последовательности на выходе кодера практически независимы.

Коды с одинаковым спектром длин элементарных кодов имеют одинаковую стоимость. Поэтому оптимальные коды ищут в классе префиксных кодов. *(Источник: Прокопенко, разд. 5, с. 42.)*

Пример 1. Пусть $A = \{a_1, a_2, a_3, a_4\}$, $P = \{0{,}4; 0{,}25; 0{,}2; 0{,}15\}$. Рассмотрим две схемы кодирования:

$$f_1: \begin{cases} a_1 \to 00, \\ a_2 \to 01, \\ a_3 \to 10, \\ a_4 \to 11; \end{cases} \qquad f_2: \begin{cases} a_1 \to 1, \\ a_2 \to 01, \\ a_3 \to 000, \\ a_4 \to 001. \end{cases}$$

Для f_1 стоимость равна

$$C_{f_1}(P) = 0{,}4 \cdot 2 + 0{,}25 \cdot 2 + 0{,}2 \cdot 2 + 0{,}15 \cdot 2 = 2.$$

Для f_2:

$$C_{f_2}(P) = 0{,}4 \cdot 1 + 0{,}25 \cdot 2 + 0{,}2 \cdot 3 + 0{,}15 \cdot 3 = 0{,}4 + 0{,}5 + 0{,}6 + 0{,}45 = 1{,}95.$$

Таким образом, стоимость кодирования может изменяться при переходе от одной схемы к другой. *(Источник: Прокопенко, разд. 5, с. 41–42.)*

Алгоритм Фано. Алгоритм Фано строит код, близкий к оптимальному. Упорядоченный по невозрастанию вероятностей список букв делится на две последовательные части так, чтобы суммы вероятностей входящих в них букв как можно меньше отличались друг от друга. Буквам из первой части приписывается символ 0, буквам из второй — символ 1. Затем точно так же поступают с каждой из полученных частей, если она содержит хотя бы две буквы. Процесс продолжается до тех пор, пока весь список не разобьётся на части, содержащие по одной букве. Каждой букве ставится в соответствие последовательность символов, приписанных данной букве в результате этого процесса. *(Источник: Прокопенко, разд. 5.1, с. 42–43.)*

Пример 2. Для списка сообщений с заданным распределением частот построим код Фано.

a_i	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
p_i	0,08	0,1	0,15	0,15	0,3	0,05	0,12	0,05

Упорядочим по убыванию: $a_5(0{,}3)$, $a_3(0{,}15)$, $a_4(0{,}15)$, $a_7(0{,}12)$, $a_2(0{,}1)$, $a_1(0{,}08)$, $a_6(0{,}05)$, $a_8(0{,}05)$. Разделим на две части: $\{a_5, a_3\}$ (сумма 0,45) и $\{a_4, a_7, a_2, a_1, a_6, a_8\}$ (сумма 0,55). Разница 0,1 минимальна. Припишем первой части 0, второй — 1. В первой части $\{a_5, a_3\}$: a_5 получает 00, a_3 — 01. Во второй части $\{a_4, a_7, a_2, a_1, a_6, a_8\}$: делим на $\{a_4, a_7\}$ (0,27) и $\{a_2, a_1, a_6, a_8\}$ (0,28). Приписываем 0 и 1. Далее: $a_4 \to 100$, $a_7 \to 101$. Для $\{a_2, a_1, a_6, a_8\}$: делим на $\{a_2\}$ (0,1) и $\{a_1, a_6, a_8\}$ (0,18). $a_2 \to 110$. Для $\{a_1, a_6, a_8\}$: делим на $\{a_1\}$ (0,08) и $\{a_6, a_8\}$ (0,1). $a_1 \to 1110$, $a_6 \to 11110$, $a_8 \to 11111$. Итоговый код:

$$a_5 \to 00, \quad a_3 \to 01, \quad a_4 \to 100, \quad a_7 \to 101, \quad a_2 \to 110, \quad a_1 \to 1110, \quad a_6 \to 11110, \quad a_8 \to 11111.$$

Стоимость кодирования:

$$C_\Sigma(P) = 2 \cdot 0{,}3 + 2 \cdot 0{,}15 + 3 \cdot 0{,}15 + 3 \cdot 0{,}12 + 3 \cdot 0{,}1 + 4 \cdot 0{,}08 + 5 \cdot 0{,}05 + 5 \cdot 0{,}05 = 2{,}83.$$

(Источник: Прокопенко, разд. 5.1, с. 43–44.)

Алгоритм Шеннона. Алгоритм Шеннона применим, когда все вероятности $p_i > 0$. Буквы упорядочиваются по невозрастанию вероятностей. Каждой букве a_i ставится в соответствие двоичная дробь q_i, определяемая как сумма вероятностей предыдущих букв:

$$q_1 = 0, \qquad q_{i+1} = q_i + p_i.$$

Кодовое слово v_i — это первые l_i знаков двоичной записи q_i, где $l_i = \lceil -\log_2 p_i \rceil$. Построенный код является префиксным. *(Источник: Прокопенко, разд. 5.2, с. 45–46.)*

Пример. Для того же распределения, что и в примере 2, алгоритм Шеннона даёт код:

$$a_1 \to 1000, \quad a_2 \to 1001, \quad a_3 \to 010, \quad a_4 \to 011, \quad a_5 \to 00, \quad a_6 \to 10110, \quad a_7 \to 1010, \quad a_8 \to 1011$$

Стоимость кодирования:

$$C_\Sigma(P) = 4 \cdot (0{,}08 + 0{,}1) + 3 \cdot (0{,}15 + 0{,}15) + 2 \cdot 0{,}3 + 5 \cdot (0{,}05 + 0{,}05) + 4 \cdot 0{,}12 = 3{,}2.$$

(Источник: Прокопенко, разд. 5.2, с. 45–46.)

Алгоритм Хаффмана. Алгоритм Хаффмана — адаптивный жадный алгоритм оптимального префиксного кодирования алфавита с минимальной избыточностью. Он был разработан в 1952 году Д. Хаффманом. Идея состоит в том, что зная вероятности символов, можно построить коды переменной длины: символам с большей вероятностью ставятся в соответствие более короткие коды. Коды Хаффмана обладают свойством префиксности и являются оптимальными.

Классический алгоритм Хаффмана на входе получает таблицу частот встречаемости символов. Строится дерево кодирования Хаффмана:

1) Символы входного алфавита образуют список свободных узлов. Каждый лист имеет вес, равный вероятности (или количеству вхождений) символа.

2) Выбираются два свободных узла с наименьшими весами.

3) Создаётся их родитель с весом, равным сумме весов.

4) Родитель добавляется в список свободных узлов, а два его потомка удаляются.

5) Одной дуге, выходящей из родителя, ставится в соответствие бит 1, другой — бит 0.

6) Шаги 2–5 повторяются, пока в списке не останется один узел — корень дерева.

7) Прослеживая путь от корня к каждому листу и записывая биты на дугах, получаем кодовые слова.

(Источник: Прокопенко, разд. 5.3, с. 46–48.)

Пример 4. Пусть $A = \{a_1, \ldots, a_8\}$, $P = \{0{,}08; 0{,}1; 0{,}15; 0{,}15; 0{,}3; 0{,}05; 0{,}12; 0{,}05\}$. Упорядочим по убыванию вероятностей. Последовательно складывая две наименьшие вероятности, получаем дерево. Итоговый код:

$$a_5 \to 00, \quad a_3 \to 010, \quad a_4 \to 011, \quad a_7 \to 100, \quad a_2 \to 101, \quad a_1 \to 1100, \quad a_6 \to 11010, \quad a_8 \to 11011.$$

Стоимость кодирования:

$$C_\Sigma(P) = 2 \cdot 0{,}3 + 3 \cdot (0{,}15 + 0{,}15 + 0{,}12 + 0{,}1 + 0{,}08) + 4 \cdot (0{,}05 + 0{,}05) = 2{,}8.$$

(Источник: Прокопенко, разд. 5.3, с. 48–49.)

Оптимальность кода Хаффмана. Коды Хаффмана являются оптимальными и префиксными. Справедлива теорема:

$$H(P) \le C^*(P) \le H(P) + 1,$$

где $H(P) = -\sum_{i=1}^{n} p_i \log_2 p_i$ — энтропия источника. Коэффициент эффективности кода определяется как

$$\gamma = \frac{H(P)}{C_\Sigma(P)}.$$

Для оптимального кода $\gamma = 1$. Избыточность равна $1-\gamma$. При блочном кодировании, выбирая длину блока N достаточно большой, можно сделать стоимость кодирования на одну букву сколь угодно близкой к $H(P)$. *(Источник: Прокопенко, разд. 5.3, с. 49–50, 55.)*

Пример 5. Для распределения $P = \{0{,}4; 0{,}2; 0{,}1; 0{,}1; 0{,}1; 0{,}05; 0{,}05\}$ код Хаффмана имеет стоимость $C_\Sigma(P) = 2{,}5$. Энтропия $H(P) \approx 2{,}42$. Коэффициент эффективности $\gamma \approx 0{,}9885$, избыточность $1 - \gamma \approx 0{,}0115$, что соответствует возможному сокращению длины кода на 1,15%. Код близок к оптимальному, так как для оптимального двоичного кода $H(P) = C_\Sigma(P)$ и $\gamma = 1$. *(Источник: Прокопенко, разд. 5.3, с. 49–50.)*

Сравнение методов. Метод Хаффмана производит идеальное сжатие (сжимает данные до их энтропии), если вероятности символов точно равны отрицательным степеням числа 2. Результаты эффективного кодирования по методу Хаффмана всегда лучше результатов кодирования по методу Шеннона–Фано. Алгоритм Фано строит код, близкий к оптимальному, но не всегда оптимальный. Алгоритм Шеннона также даёт близкий к оптимальному код, но его стоимость может быть выше, чем у Хаффмана. *(Источник: Прокопенко, разд. 5.3, с. 50.)*

Блочное кодирование. Для передачи данных можно перейти от побуквенного кодирования к кодированию «блоков», состоящих из фиксированного числа последовательных букв. Например, при кодировании двухбуквенных комбинаций алфавита $\{,\}$ с вероятностями $P() = 0{,}8$, $P() = 0{,}2$ по методу Фано получаем:

$$\to 0, \quad \to 10, \quad \to 110, \quad \to 111.$$

Цена кода для блока $C_\Sigma(P) = 1 \cdot 0{,}64 + 2 \cdot 0{,}16 + 3 \cdot (0{,}16 + 0{,}04) = 1{,}56$. Стоимость одной буквы $C_\Sigma(P)/2 = 0{,}78$. При кодировании по Хаффману трёхбуквенных комбинаций стоимость одной буквы уменьшается до 0,728, приближаясь к энтропии $H(P) = 0{,}723$. *(Источник: Прокопенко, разд. 5.3, с. 53–55.)*

Итог. Оптимальное кодирование направлено на минимизацию средней длины кодового слова при заданном распределении вероятностей. Стоимость кодирования определяется как математическое ожидание длины кодового слова. Алгоритмы Фано, Шеннона и Хаффмана строят префиксные коды; наилучшим является алгоритм Хаффмана, который всегда даёт оптимальный код. Эффективность кода оценивается через энтропию источника и коэффициент эффективности $\gamma = H(P)/C_\Sigma(P)$. При блочном кодировании стоимость на одну букву приближается к энтропии. Эти методы широко применяются в сжатии изображений, видео, в архиваторах и протоколах передачи данных. *(Обобщение по: Прокопенко, разд. 5, с. 39–57.)*

1.6.4 Помехоустойчивое кодирование

Основные источники: Прокопенко Н.Ю. Теория кодирования: учеб. пособие. Н. Новгород: ННГАСУ, 2023. Разд. 6, с. 59–92; разд. 7, с. 87–90.

Назначение помехоустойчивого кодирования. При передаче информации по каналу связи возникают помехи, ошибки, и небольшая часть информации теряется. Помехоустойчивое кодирование необходимо для устранения ошибок, которые возникают в процессе передачи и хранения информации. Без использования помехоустойчивого кодирования было бы невозможно передавать большие объёмы информации, так как в любой системе передачи и хранения неизбежно возникают ошибки. *(Источник: Прокопенко, разд. 6, с. 59.)*

В зависимости от того, используется в системе обнаружение или исправление ошибок с помощью помехоустойчивого кода, различают следующие варианты:

- *запрос повторной передачи* (Automatic Repeat reQuest, ARQ): с помощью помехоустойчивого кода выполняется только обнаружение ошибок, при их наличии производится запрос на повторную передачу пакета данных;
- *прямое исправление ошибок* (Forward Error Correction, FEC): производится декодирование помехоустойчивого кода, то есть исправление ошибок с его помощью.

Возможен также гибридный вариант: полученный пакет пытаются исправить, и если это не удаётся, отправляют запрос на повторную передачу. *(Источник: Прокопенко, разд. 6, с. 59.)*

Любое помехоустойчивое кодирование добавляет избыточность, за счёт чего появляется возможность восстановить информацию при частичной потере данных. В эффективном оптимальном кодировании избыточность убирали, а в помехоустойчивом кодировании добавляется контролируемая избыточность. *(Источник: Прокопенко, разд. 6, с. 59.)*

Классификация помехоустойчивых кодов. Помехоустойчивые коды делятся на *непрерывные* и *блочные* (блоковые). В непрерывных кодах процесс кодирования и декодирования носит непрерывный характер; сверточный код является частным случаем непрерывного кода. В блочных кодах процесс кодирования и декодирования осуществляется по блокам; с точки зрения понимания работы блочный код проще. *(Источник: Прокопенко, разд. 6.1, с. 60.)*

По используемому алфавиту коды делятся на двоичные, оперирующие битами, и недвоичные.

Блочные коды делятся на:

- *систематические* — отдельно неизменённые информационные символы и отдельно проверочные; выходной блок состоит из информационных и проверочных символов;
- *несистематические* — символы исходного сообщения в явном виде не присутствуют; по выходному блоку нельзя понять, что было на входе.

(Источник: Прокопенко, разд. 6.1, с. 61.)

Параметры помехоустойчивого кодирования. Первый параметр — *скорость кода* R, которая характеризует долю информационных («полезных») данных в сообщении:

$$R = \frac{m}{n} = \frac{m}{m+k},$$

где n — количество символов закодированного сообщения, k — количество проверочных символов, m — количество информационных символов. Параметры n и m часто приводят вместе с наименованием кода: например, код Хэмминга $(7, 4)$ означает, что на вход кодера приходит 4 символа, а на выходе 7 символов. *(Источник: Прокопенко, разд. 6.1, с. 61–62.)*

Второй параметр — *кратность обнаруживаемых ошибок*: количество ошибочных символов, которые код может обнаружить.

Третий параметр — *кратность исправляемых ошибок* t: количество ошибочных символов, которые код может исправить.

Избыточность кода:

$$R = \frac{k}{m}.$$

Основная проблема кодирования — найти коды с максимально возможной скоростью передачи информации при минимальной избыточности. *(Источник: Прокопенко, разд. 6.1, с. 62.)*

Простой бит чётности. Самый простой метод помехоустойчивого кодирования — добавление одного бита чётности. Если в информационном сообщении нечётное количество единиц, добавляется 0; если чётное — добавляется 1. Если принятый бит чётности не совпадает с рассчитанным, считается, что произошла ошибка. Кратность исправляемых ошибок при этом равна 0, так как ошибки нельзя исправить, а кратность обнаруживаемых равна 1. *(Источник: Прокопенко, разд. 6.1, с. 62.)*

Прямоугольный код. Прямоугольный код — код с контролем чётности, позволяющий исправить одну ошибку. Последовательность из 0 и 1 составляется в прямоугольную матрицу (например, 4×4); для каждой строки и каждого столбца вычисляется бит чётности. При передаче и возникновении ошибки проверка показывает, в каком столбце и в какой строке произошла ошибка; чтобы исправить, достаточно инвертировать неверный бит. Этот код исправляет все однобитные ошибки, но не все двух- и трёхбитные. Скорость кода для прямоугольного кода (4×4): $R = 16/24 = 0{,}66$. *(Источник: Прокопенко, разд. 6.1, с. 63.)*

Кодовое расстояние. Важнейшим параметром кода является *кодовое расстояние* d_Σ — минимальное расстояние Хэмминга между двумя различными кодовыми словами:

$$d_\Sigma = \min d(x, y), \qquad x \neq y, \quad x, y \in \Sigma.$$

Вес кодового слова — число ненулевых символов. Для двоичного кода под расстоянием понимается расстояние Хэмминга — число позиций, в которых соответствующие символы двух кодовых слов одинаковой длины различны. *(Источник: Прокопенко, разд. 6.1, с. 63–64.)*

Теорема 1. Код Σ исправляет все комбинации из t или менее ошибок тогда и только тогда, когда кодовое расстояние не меньше $2t + 1$:

$$d \geq 2t + 1.$$

Теорема 2. Код Σ обнаруживает все комбинации из t или менее ошибок тогда и только тогда, когда кодовое расстояние не меньше $t + 1$:

$$d \geq t + 1.$$

(Источник: Прокопенко, разд. 6.1, с. 64.)

Геометрическая иллюстрация: вокруг каждого кодового слова строятся сферы радиуса t. Чтобы каждая ошибка кратности t была исправлена, слово с такой ошибкой должно содержаться внутри сферы, описанной только вокруг одного кодового слова; расстояние между центрами двух сфер должно быть не меньше $2t+1$. Для обнаружения ошибки слово с ошибкой не должно быть кодовым, то есть расстояние между любыми различными кодовыми словами должно быть больше t. *(Источник: Прокопенко, разд. 6.1, с. 64–65.)*

Пример 1. Дана схема кодирования:

$$\Sigma: \quad \begin{cases} a_1 \to 00000, \\ a_2 \to 10101, \\ a_3 \to 01011, \\ a_4 \to 11110. \end{cases}$$

Матрица расстояний:

	a_1	a_2	a_3	a_4
a_1	0	3	3	4
a_2	3	0	4	3
a_3	3	4	0	3
a_4	4	3	3	0

Минимальное расстояние $d_\Sigma = 3$. Тогда $3 \geq t+1$, откуда $t \leq 2$ для обнаружения, и $3 \geq 2t+1$, откуда $t \leq 1$ для исправления. Следовательно, код позволяет обнаруживать все однократные и двукратные ошибки и исправлять любые однократные ошибки. *(Источник: Прокопенко, разд. 6.1, с. 65–66.)*

Код Хэмминга. Код Хэмминга — двоичный код, позволяющий автоматически исправлять одиночные ошибки. Пусть дано слово длины m. В кодовом слове должно быть $n = m+k$ разрядов, где m — количество информационных разрядов, k — количество контрольных разрядов. Количество контрольных разрядов выбирается так, чтобы удовлетворялось неравенство

$$2^k - 1 \leq n < 2^k \qquad \text{или} \qquad m + k \leq 2^k - 1.$$

Минимальные значения k при заданных n приведены в таблице:

n	2	3	$4-7$	$8-13$
$k_{\min}$	1	2	3	4

(Источник: Прокопенко, разд. 6.2, с. 67.)

Для произвольного слова $A = a_1 a_2 \dots a_n$ определяется вектор

$$H(A) = a_1 e_k(1) \oplus a_2 e_k(2) \oplus \dots \oplus a_n e_k(n),$$

где $e_k(i)$ — k-разрядная двоичная запись номера i. **Теорема 3.** Код Хэмминга H_n, состоящий из всех слов $X = x_1 x_2 \dots x_n$, для которых $H(X) = (00\dots0)$, является кодом с исправлением одного замещения. Число элементов кода Хэмминга $|H_n| = 2^{n-k}$, позиции с номерами $2^0, 2^1, \dots, 2^{k-1}$ являются проверочными, остальные — информационными. Таким кодом можно кодировать все слова длины $m = n-k$. *(Источник: Прокопенко, разд. 6.2, с. 68.)*

Пример 3. Для $X = 010101$ и $Y = 110100$, $n = 6$, $k = 3$.

$$H(X) = (010) \oplus (100) \oplus (110) = (000),$$

то есть $X \in H_6$.

$$H(Y) = (001) \oplus (010) \oplus (100) = (111),$$

то есть $Y \notin H_6$. *(Источник: Прокопенко, разд. 6.2, с. 68–69.)*

Пример 4. Пусть в слове $X = 010101 \in H_6$ произошло замещение пятого символа, в результате получилось $T = 010111$. Тогда

$$H(T) = (010) \oplus (100) \oplus (101) \oplus (110) = (101),$$

числовое значение $H(T)$ равно 5, что определяет номер замещённого символа. *(Источник: Прокопенко, разд. 6.2, с. 69.)*

Пример 5. Пусть кодируемое слово $A = 1100$, $m = 4$, $k = 3$, $n = 7$. Закодированное сообщение имеет вид $p_1p_2p_3p_4p_5p_6p_7$, где $p_3 = 1$, $p_5 = 0$, $p_6 = 0$, $p_7 = 0$ — информационные символы, а p_1, p_2, p_4 — проверочные. Проверочные символы вычисляются по формулам:

$$p_1 = p_3 \oplus p_5, \qquad p_2 = p_3 \oplus p_6, \qquad p_4 = p_5 \oplus p_6.$$

В результате $p_1 = 1$, $p_2 = 1$, $p_4 = 0$. Закодированное слово: 1101100. *(Источник: Прокопенко, разд. 6.2, с. 69.)*

Линейно блочные коды. Линейные блочные коды — это класс кодов с контролем чётности, которые можно описать парой чисел (n, m). В процессе кодирования блок из m символов сообщения (вектор сообщения) преобразуется в больший блок из n символов кодового слова (кодовый вектор), образованный с использованием элементов данного алфавита. Если алфавит состоит только из двух элементов (0 и 1), код является двоичным.

Линейным блочным (n, m)-кодом называется множество N последовательностей длины n, называемых кодовыми словами, которое характеризуется тем, что сумма двух кодовых слов является кодовым словом, а произведение любого кодового слова на элемент поля (скаляр) также является кодовым словом. Обычно $N = q^m$, где m — некоторое целое число. *(Источник: Прокопенко, разд. 6.3, с. 75.)*

Линейный (n, m)-код называется высокоскоростным, если отношение m/n близко к 1, и низкоскоростным, если m/n близко к нулю. Благодаря линейности для запоминания или перечисления всех кодовых слов достаточно хранить только те слова, которые образуют базис соответствующего линейного пространства. *(Источник: Прокопенко, разд. 6.3, с. 75.)*

Способы задания линейных кодов.

1) Перечислением кодовых слов.

2) Системой проверочных уравнений:

$$b_j = \sum_{i=1}^{m} a_i h_{ij},$$

где j — номер проверочного символа, i — номер информационного символа, h_{ij} — коэффициенты, принимающие значения 0 или 1.

3) Матричный способ, основанный на построении порождающей и проверочной матриц.

(Источник: Прокопенко, разд. 6.3, с. 76.)

Порождающей матрицей кода называется $G(n, m)$-матрица, строки которой состоят из координат векторов $g_1, \ldots, g_m$ в некотором заданном базисе. Любое кодовое слово является линейной комбинацией строк матрицы G. В систематической (канонической) форме матрица имеет вид

$$G(n, m) = [I_m, Q_{m \times k}],$$

где I_m — единичная матрица, $Q_{m \times k}$ — прямоугольная матрица, составленная из проверочных символов. *(Источник: Прокопенко, разд. 6.3, с. 76–77.)*

Проверочная матрица в систематическом виде имеет вид

$$H(n, m) = [Q^T_{m \times k}, I_k],$$

где I_k — единичная матрица, $Q^T_{m \times k}$ — транспонированная матрица $Q_{m \times k}$ из порождающей матрицы. Справедливо соотношение

$$G(n, m) \cdot H^T(n, m) = 0.$$

(Источник: Прокопенко, разд. 6.3, с. 78.)

Синдром ошибки. k-разрядный двоичный вектор S, определяемый как

$$S = y \cdot H^T,$$

где y — принятое слово, называется *синдромом ошибки*. Если ошибок нет, то синдром равен нулю. Если произошла одиночная ошибка, то синдром равен вектору-столбцу подматрицы Q, соответствующему искажённой позиции. *(Источник: Прокопенко, разд. 6.3, с. 79.)*

Основные свойства линейных кодов:

1) Произведение любого кодового слова v_i на транспонированную проверочную матрицу даёт нулевой вектор: $v_i H^T = 0$.

2) Произведение слова с ошибкой v'_i на H^T называется синдромом: $v'_i H^T = S_i$.

3) Между порождающей и проверочной матрицами в систематическом виде существует однозначное соответствие: $G \cdot H^T = 0$.

4) Кодовое расстояние d_0 равно минимальному числу линейно зависимых столбцов проверочной матрицы.

5) Произведение информационного слова на порождающую матрицу даёт кодовое слово: $s = a \cdot G$.

6) Синдром вычисляется по правилу $c = b \cdot H^T$.

7) Кодовое расстояние любого линейного (n, m)-кода удовлетворяет неравенству $d_0 \leq n - m + 1$ (граница Сингтона).

(Источник: Прокопенко, разд. 6.3, с. 79–80.)

Пример 13. Линейный блочный $(5, 2)$-код задан производящей матрицей

$$G = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

Проверочная матрица:

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

Принят вектор $b = 00110$, известны только одиночные ошибки. Декодирование по минимуму расстояния: расстояния до кодовых слов (00000), (10110), (01011), (11101) равны соответственно 2, 1, 3, 2. Минимальное расстояние 1 до слова 10110, следовательно, передавался вектор 10110, информационная последовательность 10.

Декодирование вычислением синдрома: $c = bH^T = (110)$. Синдром не равен нулю, есть ошибка. Вектору $c = 110$ соответствует вектор ошибки в первом символе $e = 10000$. Суммируя $b \oplus e = 10110$, получаем тот же результат. *(Источник: Прокопенко, разд. 6.3, с. 80–82.)*

Пример 14. Построить код Хэмминга с параметрами $d_\Sigma = 3$, $m = 3$. Проверочная матрица строится из всевозможных 3-разрядных двоичных чисел, исключая нуль:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

После приведения к каноническому виду и составления проверочных уравнений получаем код $(7, 4)$. Принятый вектор $b = 1011001$ имеет синдром $c = bH^T = (001)$. Синдром совпадает с первым столбцом матрицы H, значит, ошибка в первом символе; передавался вектор $a = 0011001$. *(Источник: Прокопенко, разд. 6.3, с. 82–83.)*

Циклические коды (кратко). Циклическим кодом называется линейный блочный (n, m)-код, который характеризуется свойством цикличности: сдвиг влево на один шаг любого разрешённого кодового слова даёт также разрешённое кодовое слово, принадлежащее этому же коду. Множество кодовых слов представляется совокупностью многочленов степени $(n - 1)$ и менее:

$$v(x) = v_{n-1}x^{n-1} + \cdots + v_0x^0.$$

Общее свойство кодовых слов циклического кода — их делимость без остатка на некоторый многочлен $g(x)$, называемый *порождающим*. Результатом деления двучлена $x^n + 1$ на $g(x)$ является *проверочный многочлен* $h(x)$. *(Источник: Прокопенко, разд. 7.1, с. 87.)*

При декодировании циклических кодов используются *многочлен ошибок* $e(x)$ и *синдромный многочлен* $S(x)$:

$$e(x) = v'(x) + v(x), \qquad S(x) = R_{g(x)}[v'(x)],$$

где $v'(x)$ — принятое (с ошибкой) кодовое слово, $v(x)$ — переданное. Ненулевые коэффициенты $e(x)$ занимают позиции, соответствующие ошибкам. В процессе декодирования по принятому слову вычисляется синдром, затем по таблице находится

соответствующий многочлен $e(x)$, суммирование которого с принятым словом даёт исправленное кодовое слово. *(Источник: Прокопенко, разд. 7.1, с. 87–88.)*

Циклический код может быть задан порождающей и проверочной матрицами. Для несистематического кода матрицы строятся циклическим сдвигом порождающего и проверочного многочленов:

$$G(n,m) = \|g(x); xg(x); \ldots; x^{m-1}g(x)\|^T,$$

$$H(n,m) = \|h(x); xh(x); \ldots; x^{k-1}h(x)\|^T.$$

Для систематического циклического кода матрица $G(n,m)$ имеет вид $[I_m, R_{m\times k}]$, где строки $R_{m\times k}$ определяются из выражений $r_i(x) = R_{g(x)}[a_i(x)x^k]$. *(Источник: Прокопенко, разд. 7.2, с. 88–90.)*

Пример 20. Для циклического $(7,4)$-кода с порождающим многочленом $g(x) = x^3 + x + 1$ матрицы G и H имеют вид:

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}, \qquad H = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix},$$

где $h(x) = (x^7+1)/g(x) = x^4+x^2+x+1$. *(Источник: Прокопенко, разд. 7.2, с. 89–90.)*

Итог. Помехоустойчивое кодирование добавляет контролируемую избыточность для обнаружения и исправления ошибок. Основные параметры кода — скорость $R = m/n$, избыточность k/m, кратность обнаруживаемых и исправляемых ошибок. Минимальное кодовое расстояние d_Σ определяет возможности кода: для исправления t ошибок необходимо $d \geq 2t+1$, для обнаружения t ошибок — $d \geq t+1$. Код Хэмминга исправляет одиночные ошибки; для него $n = m + k$, $m + k \leq 2^k - 1$, проверочные разряды располагаются на позициях $2^0, 2^1, \ldots$, а синдром указывает номер искажённого разряда. Линейно блочные (n,m)-коды задаются порождающей и проверочной матрицами, связанными соотношением $GH^T = 0$; декодирование выполняется по синдрому $S = yH^T$. Циклические коды используют аппарат многочленов: кодовые слова делятся на порождающий многочлен $g(x)$, а синдромный многочлен позволяет локализовать ошибки. *(Обобщение по: Прокопенко, разд. 6, с. 59–92; разд. 7, с. 87–90.)*

1.6.5 Краткое резюме

Алфавитное кодирование сопоставляет словам исходного алфавита кодовые слова; для однозначной декодируемости необходимо выполнение неравенства Макмиллана, а достаточным условием является префиксность. Префиксные коды удобно строить по заданному спектру длин (алгоритмы 1 и Шеннона). Для сжатия информации используются оптимальные коды Фано, Шеннона и Хаффмана, минимизирующие среднюю длину кодового слова. Помехоустойчивое кодирование добавляет избыточность для обнаружения и исправления ошибок; минимальное кодовое расстояние определяет кратность обнаруживаемых и исправляемых ошибок. Код Хэмминга исправляет одиночные ошибки, линейно блочные коды задаются порождающей и проверочной

матрицами и декодируются с помощью синдрома, циклические коды используют аппарат многочленов. *(Обобщение по: Прокопенко, разд. 2–7, с. 17–92.)*
Источники к пункту 1.6:

- Прокопенко Н.Ю. Теория кодирования: учеб. пособие. Н. Новгород: ННГАСУ, 2023. 94 с.
- Разд. 2 «Алфавитное кодирование», с. 17–22.
- Разд. 3 «Алгоритм Маркова распознавания взаимной однозначности алфавитного кодирования», с. 23–28.
- Разд. 4 «Построение префиксного кода», с. 30–38.
- Разд. 5 «Оптимальное кодирование», с. 39–57.
- Разд. 6 «Помехоустойчивое кодирование», с. 59–92.
- Разд. 7 «Циклические коды», с. 87–90.